Angular Masterclass

Building production-ready applications with advanced techniques and best practices

Souvik Basu

bpb

Angular Masterclass

Building production-ready applications with advanced techniques and best practices

Souvik Basu

bpb

Angular Masterclass

Building production-ready applications with advanced techniques and best practices

Souvik Basu

www.bpbonline.com

First Edition 2025

Copyright © BPB Publications, India

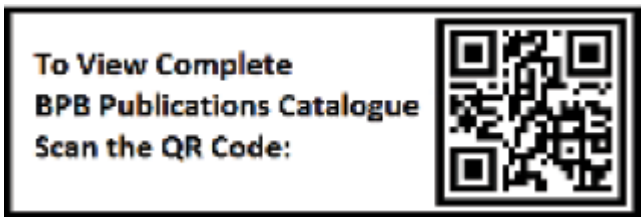
ISBN: 978-93-65897-814

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.



www.bpbonline.com

Dedicated to

My late mother, whose unwavering spirit inspires me to persevere; my father, whose values and principles have shaped my own; and my wife, whose tremendous support and love have been my constant source of strength.

About the Author

Souvik Basu is a seasoned software engineer with over 20 years of experience in software development. He started his career as a software engineer with the Applied AI department of the Center for Development of Advanced Computing (C-DAC), Pune, in 2003. He has worked as a Frontend engineer since 2012 and has spoken at many developer conferences on topics related to JavaScript frameworks and web development. Souvik's association with Angular started in 2015 when he co-created a personal finance management startup, Wallora. In a career spanning over two decades, he has worked with Microsoft, ThoughtWorks, Tesco, Avaya, and Symantec Corporation. He has held pivotal roles at Microsoft, where he led the frontend team for TakeLessons.com and developed an AI-powered vertical for Bing.com. He lives in Vancouver, Canada, and manages a software consultancy firm, Shoonya Technologies, which has clients across the US, Canada, Australia, China, and India. With a strong mentoring and corporate training background, Souvik continues to inspire and lead in the tech community.

About the Reviewer

Alejandro G. Peregrina is a versatile web and cloud-native developer with a strong foundation in front-end and back-end technologies. Skilled in crafting engaging user interfaces using HTML, CSS, and JavaScript/TypeScript, Alejandro leverages frameworks like Eleventy or Hugo for efficient **static site generation** (**SSG**) and builds reusable web components with Lit.

He also has experience in designing and developing robust web applications using the MEAN stack, including Angular and Preact/React (with Next.js for server-side rendering when necessary). Alejandro translates UI/UX designs into code, utilizing Figma and Material Design principles to create visually appealing and user-friendly experiences.

Recently, Alejandro has ventured into the realm of backend microservice development, focusing on Kotlin and a cloud-native architecture powered by Quarkus and orchestrated by AWS CDK.

Beyond one's job, he is a professional musician and DSP programmer (PureData and MaxMSP programming languages) interested in video game development poised to revolutionize the intersection of music and technology by creating interactive musical works.

Acknowledgement

I would like to express my profound gratitude to everyone who played a crucial role in bringing this book to life.

First and foremost, I am deeply grateful to my spiritual guru, Sadhguru, whose grace and wisdom have been guiding lights at every stage of our lives.

I extend my heartfelt appreciation to my family and friends for their unwavering support and encouragement throughout this journey. My wife has been an incredible source of support, enabling me to complete this book despite my professional and personal commitments.

I would also like to extend my sincere appreciation to Pankaj Parkar, with whom I have navigated a multitude of Angular concepts over the years. My colleague Colin Bate at Orbis Investments has also played an instrumental role in shaping my Angular journey. Your invaluable insights and encouragement have been vital in this process.

A special thanks goes to BPB Publications for their exceptional guidance and expertise in bringing this book to fruition. I am grateful to the reviewers, technical experts, and editors who provided invaluable feedback, helping to refine and elevate the content.

Lastly, I wish to convey my heartfelt gratitude to the readers who have shown interest in this work. Your support fuels my passion, and I genuinely hope that this book enriches your skill set and propels your career journey forward.

Thank you to every individual who contributed to making this book a reality. Your impact has been immeasurable.

Preface

This book starts with the basics of web development using Angular and quickly shifts gear to intermediate and advanced concepts to focus on production readiness for any Angular application.

The book is divided into four sections, with an increasing level of complexity added at each step.

The first section is a basic one that introduces the reader to the history of web development and where Angular fits in. It provides step-by-step guidance on setting up the development environment and creating the first Angular app. It briefly describes the structure of an Angular project but does not go into any detail.

The second section is focused on creating a simple read-only website in Angular and deploying it to production. It touches upon the concepts of routing, styling, TypeScript language constructs, lifecycle hooks, data binding, and deployment to cloud services. At the end of this section, the reader can deploy a running web app to the cloud that looks aesthetic and can jump between multiple pages.

The third section is focused on adding user interactions to an Angular app. The reader will learn about Form inputs, reactivity, API calls, user authentication, and other aspects of development, like testing and debugging. This is the most important section to learn about real-world application development.

The final section is an advanced section concentrated on fine-tuning an Angular application to load faster, be responsive, and allow easy debugging of production issues using telemetry.

This book is an excellent resource for programmers who want to enhance their Angular skills. Whether you are a beginner or an experienced developer, this book will guide you through the process of building robust and scalable applications with Angular. It is perfect for anyone who wants to learn how to use Angular effectively and improve their

programming skills. The book contains easy-to-follow sample code that can be cloned from GitHub to try out. The text is written in a clear and concise manner, making it easy for anyone to understand, regardless of their level of expertise. A basic understanding of JavaScript is a prerequisite. If the user has worked on any other frontend frameworks or libraries like React or Vue, that will help to move faster, as this book concentrates mainly on intermediate and advanced concepts in Angular.

Chapter 1: Introduction to Web Apps and Angular

Web development has evolved significantly over the years, from simple read-only HTML pages to dynamic and interactive web applications. Angular is one of the frameworks that pioneered this evolution by providing a declarative and modular way of creating user interfaces. Angular is based on the concept of components that are reusable pieces of UI logic and presentation composed together to form complex applications. In this chapter, we will run through a short history of web development and understand at a high level how Angular renders a component on the browser. This chapter will discuss how a browser fetches a web page from the server and renders it on screen. It will define the various blocks that make up a web page. Lastly, it will give a brief introduction to Single Page Applications and TypeScript language as an alternative to JavaScript.

Chapter 2: Setting Up the Development Environment

In this chapter, we will examine how Node.js and NPM help to build and run an Angular app. Setting up a development environment for Angular is an essential step for any developer looking to build web applications using this popular framework. The most popular IDE for web development using Angular is VS Code. WebStorm and Atom are other popular IDEs. This chapter will discuss the features of WebStorm and VS Code IDE and will set up the development environment. We will also look at essential and recommended IDE extensions that can improve developer experience, speed up web development, and make the code error-free. AI assistance is a much-needed feature now in most IDEs. We will look at how to use GitHub copilot in VS Code.

Chapter 3: Create Your First Angular App

If you are new to Angular, the Angular CLI can be a lifesaver. It is a command-line interface tool that helps you generate and manage Angular

projects with ease. With Angular CLI, creating your first Angular app is a breeze. You do not have to worry about setting up your project structure or configuring your build tools, as Angular CLI takes care of all that for you. In this chapter, I will walk you through the steps of how to use Angular CLI to create your first Angular app. Once you have created your first Angular app with Angular CLI, it is essential to understand its anatomy. An Angular app comprises several files and folders, each serving a specific purpose. By exploring the anatomy of your Angular app, you will be able to better understand how it works. The chapter will explore the most used features of the Angular CLI.

Chapter 4: Building Blocks of Angular App

When developing an Angular application, there are several key elements to consider. The application is broken down into modules at the top level to organize into logical units, while components are the building blocks of these modules. Services provide reusable functionality throughout the application, while directives allow for the manipulation of the DOM. Decorators provide a way to modify the behavior of classes and functions, and pipes allow for the transformation of data before it is displayed to the user. By utilizing these various elements, developers can create powerful and dynamic Angular applications that meet the needs of their users. This chapter sets the foundation of an Angular application that may be referred to many times while reading this book.

Chapter 5: Routing

Routing is a powerful feature in Angular that allows you to navigate between different parts of your application. This chapter discusses the difference in routing of a traditional web page versus a single page application. One of the key aspects of routing is the ability to pass URL parameters, which can be used to customize the content that is displayed on a specific page. By passing URL parameters, you can make your application more dynamic and interactive, providing users with a more personalized experience. Whether you are working on a simple website or a complex web application, Angular's routing capabilities can help you create a seamless user experience that is both efficient and effective. The chapter covers advanced topics in routing, like nested routes, setting redirects, showing active routes, lazy loading, preloading, fragments, and how to debug routes.

Chapter 6: Styling and Theming

Styling in an Angular application is essential to creating a visually appealing and user-friendly interface. With Angular, you have a variety of options for styling your application, from basic CSS to more advanced tools like Sass and Less. In this chapter, we will explore the different ways to style your Angular app and provide tips for creating a cohesive and professional-looking design. We will look at how to create theming in your Angular application for a consistent and customizable look and feel throughout the application. The chapter also introduces Tailwind CSS, which has been widely accepted within the Angular community to style components. Finally, we will discuss styling best practices.

Chapter 7: TypeScript Language Constructs

TypeScript is a powerful programming language that offers a wide range of helpful language constructs. These constructs allow developers to write cleaner and more efficient code, making it easier to maintain and debug. One of the key benefits of TypeScript is its strong type system, which helps catch errors at compile time rather than runtime. Other useful constructs include classes, interfaces, modules, and enums, allowing developers to organize and structure their code better. With TypeScript, developers can write better code faster, making it an ideal language for both small and large projects. This chapter introduces these TypeScript language constructs needed to develop an Angular application. It also discusses the parameters set in the default TypeScript config file in an Angular application.

Chapter 8: Lifecycle Hooks

Angular lifecycle hooks are a powerful feature that allows developers to manage and control the behavior of their components. These hooks provide a way to observe and react to changes in the component's state and can be used to perform tasks such as initializing data, responding to user input, and cleaning up resources. Understanding how to use these hooks effectively can greatly improve the performance and reliability of your Angular applications. This chapter explores the basics of Angular lifecycle hooks and shows you how to use them to build better applications. It covers DestroyRef, which is a construct introduced in Angular v16. It allows you to register a callback function that will be executed when the component or directive that injects it is destroyed. The last section of the chapter discusses the order in which these life cycle hooks are called for a component.

Chapter 9: Data Binding and Events Handling

When working with Angular, data binding and events handling are two essential concepts to understand. Data binding allows you to connect the data in your component with the view so that changes to the data are automatically reflected in the view. The chapter looks at different types of data binding, like interpolation and property binding. On the other hand, event handling allows you to listen for user actions, such as clicks or key presses, and respond to them in your component logic. The chapter also discusses two-way data binding. By mastering these concepts, you can create dynamic and responsive web applications with Angular.

Chapter 10: Deployment

When building web applications with Angular, deployment is a crucial step in getting your app out into the world. Whether you are deploying to an in-premise server or a hosting platform there are a variety of deployment options available to Angular developers. This chapter begins with how to build an Angular application and then lists down some of the hosting solutions available in the market to deploy your Angular application. It explains in detail how to deploy your Angular application using the Azure Static Web App. It also explains how to add a custom domain to the hosted application so your users can access the site using a more readable domain name. This chapter is intended to provide you with a basic deployment option. Chapter 25 explores more advanced deployment topics in the last section of the book.

Chapter 11: User Input Using Forms

When it comes to handling user input in Angular, there are two main approaches: template-driven forms and reactive forms. Template-driven forms are simpler and quicker to set up but may not be as flexible in certain situations. Reactive forms are great for more complex scenarios where you need more control over the form data and validation. Either way, both approaches can be used to create dynamic and interactive forms that make it easy for users to input data. This chapter begins with an introduction to forms and user input event handling. It then explores how to use template-driven and reactive forms in your Angular applications. It also touches upon topics such as nested form groups and form validation.

Chapter 12: Observables and Signals

Reactivity is a critical aspect of modern web application development. It allows for real-time updates and dynamic interactions between various components of an application. One way this is achieved is through the use of Reactive Extensions for JavaScript (RxJS), a powerful library that enables reactive programming. At its core, reactivity revolves around the use of Observables and RxJS, which provides a robust framework for managing data flows and state changes in an application. Angular v16 introduced a powerful feature called Signals, which enables seamless communication between different components of a web application. It is a simple way to establish real-time updates and dynamic interactions. With Signals, developers can easily build complex and reactive applications that offer a seamless user experience.

Chapter 13: API Calls

API calls allow an application to fetch data from remote servers. In Angular, handling API calls is made easy with the help of the built-in HttpClient module. This chapter will look at the intricate details of handling REST-based API calls in an Angular application. This chapter will also introduce GraphQL, which is a query language for APIs and a runtime for executing those queries using a type system. It provides a more efficient, powerful, and flexible alternative to traditional REST APIs. We will also look at how Web Sockets can help with real-time data fetching in apps.

Chapter 14: Authentication and Authorization

Providing user-specific content is one of the major features of a web app that differentiates it from traditional websites. To provide user content, the application needs to identify the user correctly and also ensure that the signed-in user has access to the requested content. Authentication ensures that users are who they claim to be, while authorization determines the actions users are allowed to perform within the application. These processes are essential for maintaining the security and integrity of the application, protecting sensitive data, and providing a personalized experience for each user. This chapter explores ways to authenticate and authorize a user and delves into security best practices.

Chapter 15: Testing

Testing ensures that the application behaves the way it is intended to. A thorough testing of the application features avoids issues discovered by the users in production. A comprehensive test suit also helps to regress an application when new features are added. This provides a level of confidence that adding new features to the application does not break existing functionalities. This chapter explores tools and techniques to test components, services, pipes, and decorators. It also explores end-to-end testing and code coverage topics.

Chapter 16: Debugging

Debugging is the process of identifying and resolving issues, errors, or bugs in a computer program. It involves examining the code, tracing the program's execution, and identifying the root cause of unexpected behavior. Techniques such as using breakpoints, stepping through code, and logging are commonly used to debug software and ensure that it runs as intended. In this chapter, we will explore these debudding techniques and also look at Angular DevTools and IDE extensions used to debug application code.

Chapter 17: Exception Handling

Exception handling is crucial to ensuring an application's stability and reliability. Exceptions can be handled at a local or global level in an Angular application. A try/catch block can help respond to error conditions raised within a code block. Angular also provides a global error handler to gracefully handle error conditions at an application level. This chapter will discuss the two approaches to exception handling and list a few tools available to monitor exceptions in an application.

Chapter 18: Internationalization and Localization

In today's global business, your user base will likely be from around the world. Your Angular application must be accessible and usable to users regardless of their language, culture, and location. Internationalization and localization are two key concepts that enable web applications to be adapted and customized based on the user's language, culture, and location. In this chapter, we will explore the basics of internationalization and localization, the concept of a locale, how it impacts dates, currency, and number formats, and how to handle bidirectional text.

Chapter 19: Accessibility

The users of our websites may have certain disabilities, such as visual, auditory, physical, speech, cognitive, language, learning, and neurological impairments. Accessibility is creating websites and applications that can be used by all people, irrespective of their disability. To make our websites usable by a larger section of the society, it is important to make it accessible. In many countries, there are laws governing accessibility features. The goal is to ensure that everyone has equal access to information and functionalities on the web. In this chapter, we will explore how using ARIA tags can help with screen readers and other accessibility tools. We will look at the features in Angular Material Design and CDK that can help make our websites accessible. Lastly, we will look at testing accessibility features.

Chapter 20: Animation

Using animation, you can add motion and style to your web applications, making them more visually appealing and engaging. This chapter explores how to add animation to Angular components using CSS. We will also look at how to use the built-in Animation module in Angular. It also explores how to reuse animation across components. Sometimes, adding animation and transitions can cause accessibility issues. This chapter discusses ways to disable animation when needed.

Chapter 21: Micro Frontends

Micro frontends is a technique that allows you to break down a large frontend application into smaller, more manageable pieces. Each micro frontend can be developed and deployed independently, making it easier to maintain and scale your application. With the rise of microservices architecture, the concept of micro frontends has become increasingly popular. This chapter demonstrates how to create an application using micro frontends. Micro frontends allow the integration of components created using different frameworks into a single application. We will explore using a React component inside an Angular application and maintaining a common application state between them.

Chapter 22: Charts

Charts are a powerful tool for visualizing data trends and patterns, and many websites incorporate them to make complex data easy to understand. This increases user engagement on the site. This chapter lists the features of a charting library to consider for your Angular application.

It demonstrates how to create a simple bar chart using one of the popular charting libraries.

Chapter 23: Performance and User Experience

When it comes to building a web application, performance, and user experience should always be top of mind. A poorly performing app can lead to frustration and lost users, while a smooth and responsive app can keep users engaged and coming back for more. This chapter begins with an introduction to the RAIL model and explores ways to achieve 60 fps to make user interactions feel responsive and seamless. It then explores tools to measure page performance. The chapter discusses the important concept of how Angular change detection works and strategies to improve it. It explores further techniques for performance improvement and finally adds a section on load testing.

Chapter 24: Handling Offline Mode with Progressive Web App

In today's world, having an offline-capable app is becoming increasingly important. Users expect to be able to access their information and use apps even without an internet connection. The service worker is a powerful tool to achieve offline capability. By leveraging the service worker API, you can create Progressive Web Apps (PWAs) that can be installed on a user's device and run even when they are offline. In this chapter, we will explore how to use the service worker to create PWAs in Angular, including how to cache assets and data for offline use and how to handle updates to your app.

Chapter 25: Advanced Deployment

Chapter 10 covers basic deployment topics. For an enhanced and seamless user experience, the web server must be near the user's device. Content delivery networks (CDN) use edge servers that bring the web pages closer to the user device and significantly reduce page load time. Load balancers distribute traffic to multiple web servers, thereby allowing faster responses and optimal server resource usage. This chapter introduces these concepts. Furthermore, you can capitalize on benefits such as improved security, scalability, and cost reductions by building your app as a static site and employing the JAMstack architecture. The chapter demonstrates creating a sample JAMstack app using a static site generator tool called Scully. The chapter also introduces Firebase, which is a fantastic backend-as-a-service (BaaS) platform that provides

developers with tools for building high-quality mobile and web applications. It's a popular choice for Angular developers because it simplifies the process of setting up a serverless backend.

Chapter 26: Responsiveness

It is essential to ensure that your Angular application looks great on desktop, mobile, and tablet screens. One technique to achieve this is using media queries in CSS, which allow you to adjust the layout and styling of your app based on the screen size. Angular provides utilities to add conditional code logic for different UI breakpoints. Additionally, many libraries like Bootstrap and Angular Material can help you create a mobile-friendly user interface. This chapter provides sample code to create a responsive app using media queries and libraries. It introduces the concept of a mobile-first approach that is essential for getting a higher page rank in search engines as well.

Chapter 27: Design System and Tools

A design system is a set of guidelines, rules, and components used to create consistent and cohesive visual and interactive designs for a website or application. It provides a common language and framework for designers and developers to work together, ensuring that the final product is both aesthetically pleasing and user-friendly. Design tools help designers create screen mockups and share them with the development team. This chapter explores a few features of Figma as a design tool for the frontend team. It also explains how to create themes for an Angular application. It demonstrates using the built-in theme in the Angular Material design system.

Chapter 28: Creating a Mobile App using Angular

There are subtle differences between a responsive web app and a native app. Sometimes, a mobile app might make more business sense than a web app. Mobile apps can improve user engagement as they are easier to open on the mobile device. Also, they provide native access to device features like camera, storage, and multi-media. Cordova and NativeScript are popular frameworks for creating a native mobile app using Angular. This chapter explores creating a mobile app using Cordova. It also covers the topic of accessing native device features like a camera in the mobile app.

Chapter 29: State Management

State management involves managing an application's data and UI state that can be shared across different components. Maintaining a common state avoids passing data through input/output fields in the component hierarchy, which makes the code easy to debug and maintain. This chapter will cover the key concepts of state management in Angular, including how to manage the application state using services, observables, and state management libraries such as NgRx. It also covers how to debug state changes using DevTools.

Chapter 30: Telemetry and Instrumentation

Telemetry refers to the collection and analysis of data related to the application's performance and usage. It is a popular technology used to collect and transmit data from remote sources to a central location for analysis. Instrumentation, on the other hand, involves integrating tools and techniques that enable developers to monitor and diagnose issues in the application. This chapter explores implementing telemetry services in an Angular application using Azure Application Insights. This service allows developers to collect and analyze real-time insights into the app's performance, enabling developers to detect and diagnose issues quickly.

Chapter 31: Server-side Rendering and Search Engine Optimization

SEO, short for search engine optimization, is the process of optimizing a website or application to rank higher on search engine result pages (SERPs). It is crucial for businesses and individuals to have a strong online presence. SEO helps your site appear on the first page of search results, driving traffic and potentially boosting sales. It can be challenging to implement SEO in Angular apps due to the framework's reliance on JavaScript. This is where server-side rendering (SSR) can help. In this chapter, we will cover the various strategies and best practices for optimizing your Angular app for SEO, including server-side rendering, metadata management, and structured data.

Chapter 32: AI Tools and Chatbots

Code generation using AI tools has been a game changer for developers. It has significantly reduced the time and effort required to build complex Angular applications. With the help of AI-powered tools, one can generate a large portion of their code automatically, which not only speeds up the development process but also ensures that the code is error-free and consistent. This chapter explores using Github Copilot for

automatic code and test generation. It also demonstrates how to create an AI Chatbot for an Angular application using OpenAI API.

Chapter 33: Security

Security is a crucial aspect of any application, and it is essential to ensure that measures are in place to protect against potential threats. Hackers do not break into systems, they log into systems. Hence it is necessary to ensure that appropriate actions are taken to avoid unauthorized access into the websites. This chapter provides an overview of security considerations that you should keep in mind while developing an Angular application. It covers various topics, including XSS, CSP, CORS, CSRF, etc., to ensure the application's overall security.

Code Bundle and Coloured Images

Please follow the link to download the
Code Bundle and the ***Coloured Images*** of the book:

https://rebrand.ly/ecbf6e

The code bundle for the book is also hosted on GitHub at **https://github.com/bpbpublications/Angular-Masterclass**. In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at **https://github.com/bpbpublications**. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.bpbonline.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

business@bpbonline.com for more details.

At **www.bpbonline.com**, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at **business@bpbonline.com** with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit **www.bpbonline.com**. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit **www.bpbonline.com**.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Table of Contents

1. Introduction to Web Apps and Angular

- Introduction
- Structure
- Objectives
- Brief history of frontend development
- Evolution of Angular
- Understanding how browsers work
- Building blocks of a web page
- Website versus web app
- Single page applications
- TypeScript
- Understanding how Angular works
- Client-side versus server-side rendering
  - *Before we get started*
- Conclusion
- Points to remember
- Questions
- References

2. Setting Up the Development Environment

- Introduction
- Structure
- Objectives
- Role of Node.js and npm
- Node version manager

Integrated development environment
WebStorm
Visual Studio Code
Installing VS Code and extensions
Quick introduction to VS Code features
AI assisted tools in VS Code
Browser developer tools
Conclusion
Points to remember
Questions
References

3. Create Your First Angular App

Introduction
Structure
Objectives
Installing Angular CLI
Creating Hello World app
Anatomy of an Angular workspace
Source files
Configuration files
Node modules
End-to-end tests
Deep dive into Angular CLI
Conclusion
Points to remember
Questions
References

4. Building Blocks of Angular App

Introduction
Structure

Objectives
Breaking down the Angular app
Modules
Components
Working with multiple components
Services
Module level versus component level services
Directives
Pipes
Decorators
Importing modules
Organizing code
Conclusion
Points to remember
Questions
References

5. Routing

Introduction
Structure
Objectives
Routing in traditional websites
Routing in single page application
Understanding routing in Angular
<base href> and routerLink
Adding routes to existing app
Passing URL parameters
Nested routes
Snapshot versus observables
Setting up redirects
Handling page not found

Showing active routes
Passing query parameter
Routing using TypeScript
Lazy loading and preloading
CanActivate, CanActivateChild, CanDeactivate
Fragments
Router data
Router state
Debugging routes
Conclusion
Points to remember
Questions
References

6. Styling and Theming

Introduction
Structure
Objectives
Styling using styleUrls metadata
Styling using styles metadata
Inline style in template
<link> tag in template
Using CSS frameworks
Importing library packages
View encapsulation
Global styles
CSS preprocessors
Using SCSS @mixin
Angular specific selectors
Authoring style changes

Theming
Setting style using TypeScript
Using Tailwind with Angular
Styling best practices
Conclusion
Points to remember
Questions
References

7. TypeScript Language Constructs

Introduction
Structure
Objectives
Introduction to TypeScript
Type annotation
Classes
Interfaces
Modules
Enums
TypeScript config file
Conclusion
Points to remember
Questions
References

8. Lifecycle Hooks

Introduction
Structure
Objectives
Lifecycle hooks
ngOnChanges

ngOnInit
ngDoCheck
ngAfterContentInit
ngAfterContentChecked
ngAfterViewInit and ngAfterViewChecked
ngOnDestroy
DestroyRef
Ordering of lifecycle hooks
Conclusion
Points to remember
Questions
References

9. Data Binding and Events Handling

Introduction
Structure
Objectives
Data binding
Interpolation
Property binding
Event binding
Two-way data binding
Conclusion
Points to remember
Questions
References

10. Deployment

Introduction
Structure
Objectives

Building your Angular app
Choosing a hosting service
Deployment using Angular CLI
Deploy to Azure Static Web App
Adding a custom domain
Conclusion
Points to remember
Questions
References

11. User Input Using Forms

Introduction
Structure
Objectives
User input events handling
Introduction to forms
Template driven forms
Form validation
Reactive forms
Nested form groups and form array
Form builder and validation
Listening for changes
Conclusion
Points to remember
Questions
References

12. Observables and Signals

Introduction
Structure
Objectives

Understanding observables and observers
Reactive extensions for JavaScript
Operators in RxJS
Creational operators
Pipeable operators
Combining multiple observables
Subject and multicast
Using observables in Angular
Event handling using observables
HTTP request handling using observables
Reactive forms using observables
Router events using observables
Signals
Conclusion
Points to remember
Questions
References

13. API Calls

Introduction
Structure
Objectives
Application programming interface
Handling REST API
GraphQL calls
Working with WebSockets
Conclusion
Points to remember
Questions
References

14. Authentication and Authorization

Introduction
Structure
Objectives
Understanding authentication and authorization
Sign up and sign in API endpoints
Setting up an authentication service
Implementing user registration
Adding a sign in page
Securing routes with Auth Guard
Managing user sessions
Role-based authorization
JSON Web Tokens in Angular
Handling authentication errors
Conclusion
Points to remember
Questions
References

15. Testing

Introduction
Structure
Objectives
Types of testing
Unit testing in Angular
Testing components
Testing services
Testing for async code using spy
Testing directives and pipes
End-to-end tests
Code coverage

Running tests in CI/CD
Conclusion
Points to remember
Questions
References

16. Debugging

Introduction
Structure
Objectives
Importance of debugging
Using breakpoints and logging
Angular DevTools
Performance profiling
Debugging templates and components
Debug code in IDE using breakpoints
NgRx DevTools
Conclusion
Points to remember
Questions
References

17. Exception Handling

Introduction
Structure
Objectives
Significance of error handling
Local error handling
Global error handling
Third-party log monitoring
Handling HTTP errors

Conclusion
Points to remember
Questions
References

18. Internationalization and Localization

Introduction
Structure
Objectives
Internationalization and localization
Performing internationalization
Locale
Localizing Angular app
Best practices for i18n
Displaying dates, currency, and numbers
Switching between locales in the UI
Handling bidirectional text
Conclusion
Points to remember
Questions
References

19. Accessibility

Introduction
Structure
Objectives
Understanding disability and impairment
Guidelines and standards for accessibility
Adding ARIA attributes
Using Angular Material Design
Using Angular Component Dev Kit

High contrast mode
Testing and evaluating accessibility
Conclusion
Points to remember
Questions
References

20. Animation

Introduction
Structure
Objectives
Simple animation using CSS
Animation module in Angular
Reusing animation
Note on accessibility
Conclusion
Points to remember
Questions
References

21. Micro Frontends

Introduction
Structure
Objectives
Introduction to micro frontend
Implementing a micro frontend in Angular
Working with different frameworks
Conclusion
Points to remember
Questions
References

22. Charts

Introduction

Structure

Objectives

Choosing a charting library

Bar chart using ng2-charts

Conclusion

Points to remember

Questions

References

23. Performance and User Experience

Introduction

Structure

Objectives

Achieving 60fps

The RAIL model

Measuring website performance

Lazy loading and preloading

Angular change detection optimization

Using observables with OnPush

Using pure pipes for heavy computation

Optimize animations

Using virtual scrolls

Optimize *ngFor in template

Optimize image lazy loading

Load testing

Conclusion

Points to remember

Questions

References

24. Handling Offline Mode with Progressive Web App

Introduction

Structure

Objectives

Understanding offline capabilities

Offline-first approach

Service workers

Progressive Web Apps

Background Sync and custom service worker

Conclusion

Points to remember

Questions

References

25. Advanced Deployment

Introduction

Structure

Objectives

Content delivery network

Load balancers

Jamstack using Scully

Deploy Angular app using Firebase

Conclusion

Points to remember

Questions

References

26. Responsiveness

Introduction

Structure

Objectives
Using media queries
Conditional rendering with breakpoints
Mobile-first approach and performance factors
Flex layout and grid controls
Responsiveness using Angular Material
Responsiveness on server side
Bootstrap
Conclusion
Points to remember
Questions
References

27. Design System and Tools

Introduction
Structure
Objectives
Design system
Angular Material
Figma design tool
Theming
Conclusion
Points to remember
Questions
References

28. Creating a Mobile App using Angular

Introduction
Structure
Objectives
Advantages of a native app

Creating a native app
Making an Angular app native
Adding native features
Conclusion
Points to remember
Questions
References

29. State Management

Introduction
Structure
Objectives
The need for a state management
Approaches to state management
Local component state
Service based state
NgRx
Debugging state changes
Conclusion
Points to remember
Questions
References

30. Telemetry and Instrumentation

Introduction
Structure
Objectives
Instrumentation of Angular app
Setup Application Insights
Conclusion
Points to remember

Questions
References

31. Server-side Rendering and Search Engine Optimization

Introduction
Structure
Objectives
Server-side rendering
Hydration and state transfer
Search engine optimization
Setting title and metadata
Schema.org: Structured data markup
Avoiding hash-based URLs
Canonical URLs
Avoid slow or blocked resources
Make it mobile-first
Testing SEO
Skipping a page from crawlers
Conclusion
Points to remember
Questions
References

32. AI Tools and Chatbots

Introduction
Structure
Objectives
GitHub Copilot
Copilot with Angular
Create a chatbot using OpenAI API
Conclusion

Points to remember
Questions
References

33. Security

Introduction
Structure
Objectives
Cross-site scripting prevention
Content security policy
Cross-origin resource sharing
Cross-site request forgery
Security best practices
Conclusion
Points to remember
Questions
References

Index

CHAPTER 1
Introduction to Web Apps and Angular

Introduction

In this chapter, you will learn about the basics of frontend development and the role of Angular in creating dynamic web pages. You will get a brief overview of how browsers render web pages, what are the main components of a web page, and how Angular simplifies UI development with its powerful features. You will also learn the prerequisites and essentials for getting started with Angular. This chapter is aimed at absolute beginners who have no prior experience in web development.

Structure

The chapter covers the following topics:

- Brief history of frontend development
- Evolution of Angular
- Understanding how browsers work
- Building blocks of a web page
- Website versus web app
- Single page applications
- TypeScript
- Understanding how Angular works
- Client-side versus server-side rendering

Objectives

By the end of this chapter, you will be able to understand how browsers render a web page, what differentiates a web app from a website, what are **single page applications** (**SPAs**) and a brief about TypeScript. You will learn how Angular works at a high level and the different types of rendering techniques, namely—**client-side rendering** (**CSR**) and **server-side rendering** (**SSR**).

Brief history of frontend development

Frontend development has come a long way since the early days of the internet. In the 1990s, websites were simple and mostly consisted of static pages with basic **Hypertext Markup Language** (**HTML**) and CSS. As the internet grew more sophisticated, so did frontend development. The introduction of JavaScript in 1995 allowed for more dynamic and interactive web pages. In the early 2000s, **Asynchronous JavaScript and XML** (**AJAX**) became popular, allowing for even more seamless user experiences on the web.

As the web continued to evolve, frontend development became increasingly complex. The rise of mobile devices in the late 2000s led to responsive design and mobile-first development, which allowed websites to adapt to different screen sizes.

Today, frontend developers must not only be proficient in HTML, CSS, and JavaScript, but also in frameworks and libraries such as Angular, React, and Vue.js. These tools allow developers to create complex web applications that run seamlessly on a variety of devices.

Evolution of Angular

Angular is a popular JavaScript framework used for building complex web applications. It was first introduced by *Google* in 2010 as AngularJS, a framework for building dynamic, single page web applications. AngularJS was based on the **model-view-controller** (**MVC**) architecture pattern and provided a number of powerful features, such as two-way data binding, dependency injection, and directives.

In 2016, Angular 2 was released, which was a complete rewrite of AngularJS. Angular 2 introduced a number of new features and improvements, including a more modular architecture, improved performance, and support for mobile development.

Since then, Angular has continued to evolve and improve, with the release of Angular 16 in 2023 during the writing of this book. Today, Angular is one of the most popular frontend development frameworks, with a large and active community of developers and contributors.

Understanding how browsers work

When you type a website address, also known as a **Uniform Resource Locator** (**URL**), on the browser, there are a series of steps the browser performs to render the page to you. To put it briefly, it carries out the following actions:

1. **Fetching the HTML document**: The browser sends a request to the server to get the HTML document of the website. The server responds with the HTML document, which is a text file. It also returns a bunch of meta data in the

response header to indicate the type of data in the response. One such header is the Content-Type. Content-Type header indicates the media type of the data in the response body. A media type also known as a **Multipurpose Internet Mail Extensions** (**MIME**) type indicates the nature and format of a document, file, or assortment of bytes. It is a string in the format type/subtype. A few frequently used MIME types are text/plain, text/html, text/css, text/javascript, image/jpeg, etc. In addition to the MIME types, the Content-Type header may also indicate character encoding, for instance, text/html; **charset=utf-8**. The browser converts the raw bytes of data into characters based on the character encoding. The following diagram shows a simplified network call between the browser and the web server. In a real-world scenario, there might be several hoping points in between these endpoints, as shown in the following figure:

Figure 1.1: *Browser requesting a web page from a web server*

2. **Parsing the HTML and building the DOM**: The browser parses the HTML document from top to bottom, creating a tree-like structure of nodes that represent the elements and attributes in the HTML. This tree-like structure is called the **Document Object Model** (**DOM**), and it represents the content and structure of the web page. Note that the browser first parses and renders the head tag and then the body. The browser also fetches any external resources referenced in the HTML, such as images, stylesheets, scripts, fonts, etc. The process is shown in the following figure:

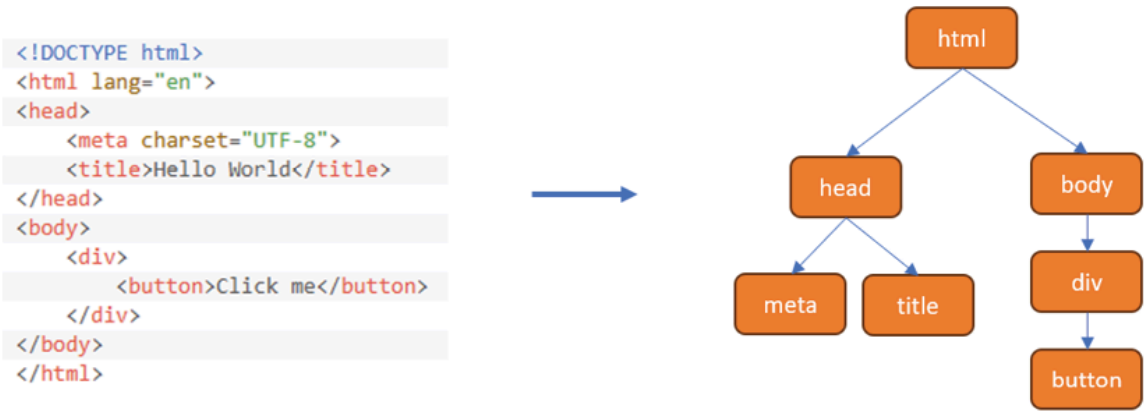


Figure 1.2: *Parsing of HTML file into a DOM*

3. **Parsing the CSS and building the CSSOM**: The browser parses any CSS files or inline styles that are linked or embedded in the HTML document, creating another tree-like structure of nodes that represent the selectors and

rules in the CSS. This tree-like structure is called the **CSS Object Model** (**CSSOM**), and it represents the style of the web page.

4. **Executing the JavaScript**: The browser executes any JavaScript files or inline scripts that are linked or embedded in the HTML document, either synchronously or asynchronously depending on their attributes. The JavaScript code can manipulate both the DOM and the CSSOM, changing the content, style or layout of the web page dynamically.
5. **Merging the DOM and CSSOM to construct the render tree**: The browser combines the DOM and the CSSOM to form a new tree-like structure called the **render tree**, which contains only the nodes that are visible on the web page. The render tree filters out any nodes that are hidden by CSS properties such as display: none or visibility: hidden, or by JavaScript code.
6. **Calculating layout and paint**: The browser calculates the size and position of each node in the render tree, taking into account factors such as margins, paddings, borders, box model, floats, flexbox, grid, etc. This process is called **layout**, and it determines how much space each node occupies on the web page. The browser then paints each node in the render tree onto a pixel grid, using colors, gradients, images, shadows, etc. This process is called **paint** or **rasterization**, and it determines how each node looks on the web page.
7. **Displaying on screen**: The browser displays the final output of the layout and paints on screen, using hardware acceleration, if possible, to improve performance. The browser also handles any user interactions such as scrolling, clicking, typing, etc., and updates the web page accordingly by repeating some or all of these steps. This process is recalculating the position and geometry of certain parts of a webpage is called **reflow**.

The series of steps can be visualized as follows:

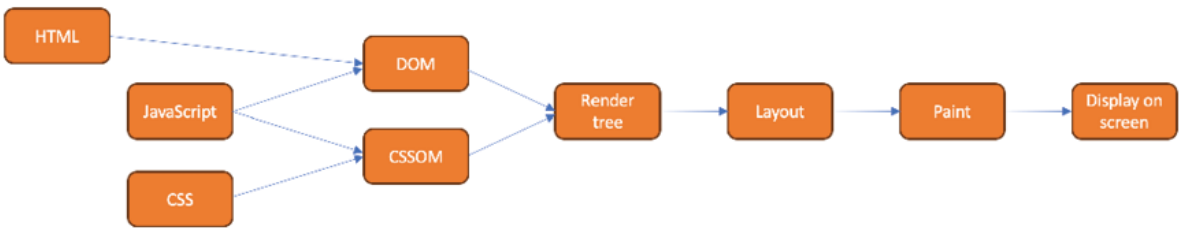


Figure 1.3: *Series of steps to render a page on display*

If you are interested to explore further, please refer to the article on web dev to understand how browsers work. The website link is available in the references section at the end of this chapter.

Building blocks of a web page

A web page is composed of different elements that define its content, structure, style, and functionality. One of the main building blocks of a web page is HTML

which uses tags also called **elements** to mark up the content of a web page, such as headings, paragraphs, images, links, etc. HTML tags are enclosed in angle brackets (< and >) and usually come in pairs, such as ** ** and ** **. HTML tags can have attributes that provide additional information or functionality, such as class, id, href, src, etc. HTML tags can be nested inside each other to create complex structures and layouts.

Here is an example code of a very simple HTML page that shows a button:

```
1. <!DOCTYPE html>
2. <html lang="en">
3. <head>
4.     <meta charset="UTF-8">
5.     <title>Hello World</title>
6. </head>
7. <body>
8.     <div>
9.         <button>Click me</button>
10.     </div>
11. </body>
12. </html>
```

If you open this page in a browser, it will look like the following figure:

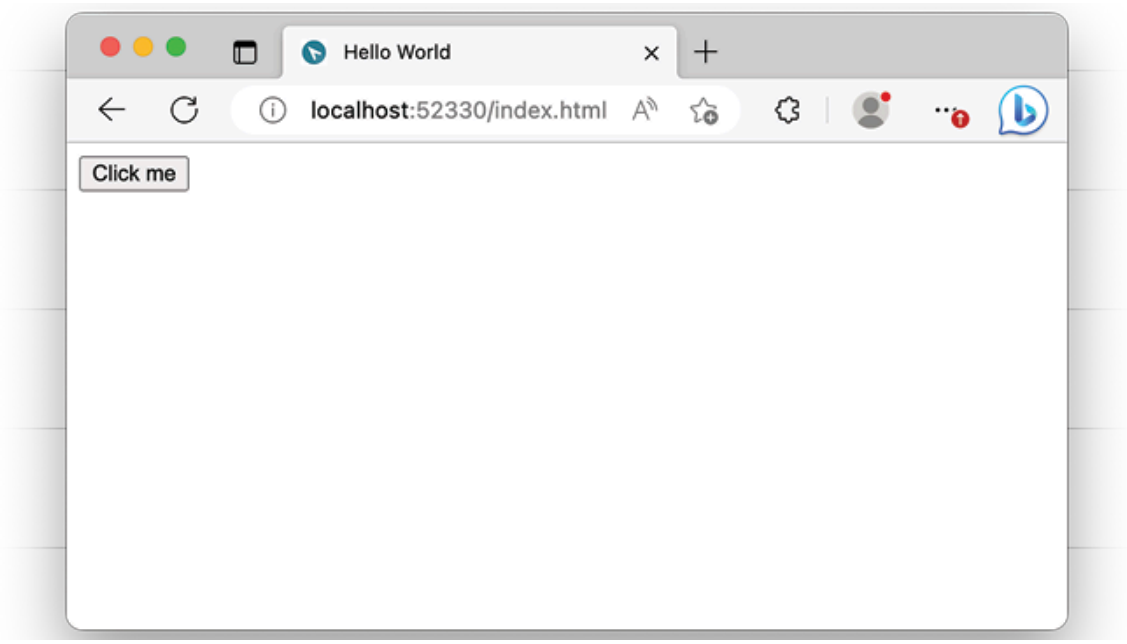


Figure 1.4: *A simple HTML page with a button*

Another building block of a web page is CSS. CSS is the technology that controls the appearance and design of a web page. CSS uses rules to apply styles to HTML elements, such as colors, fonts, backgrounds, borders, etc. CSS rules consist of

selectors and declarations. **Selectors** specify which elements to style, and declarations specify how to style them. CSS rules can be written in a separate file (external stylesheet), inside the HTML document (internal stylesheet), or directly on an HTML element (inline style). CSS rules can be inherited, overridden, or cascaded depending on their specificity and order.

Consider the previous code. You can add style statements for the parent **div** and **button** as follows:

```
<head>
  <style>
    #parent {
      padding: 50px;
    }

    button {
      color: dodgerblue;
      background-color: white;
       border: 1px solid dodgerblue;
      padding: 10px 20px;
      border-radius: 5px;
    }
  </style>
</head>
<body>
  <div id="parent">
    <button>Click me</button>
  </div>
</body>
</html>
```

It renders a button as shown below:

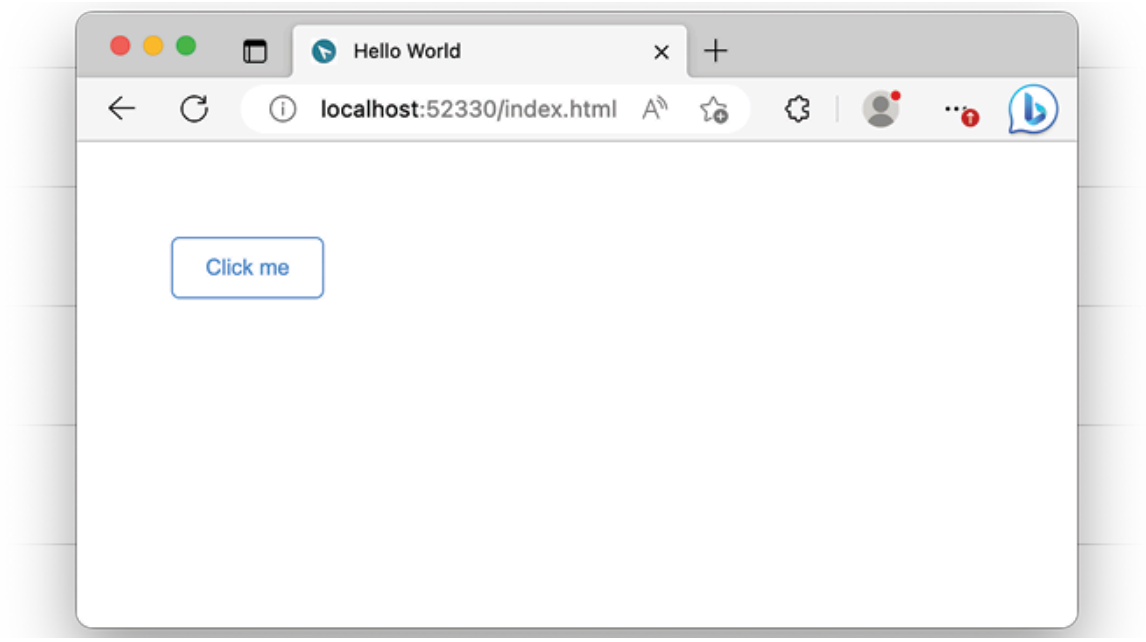


Figure 1.5: *Styled button using CSS*

The third building block of a web page is JavaScript, which is the programming language that adds interactivity and functionality to a web page. JavaScript can manipulate both the HTML and the CSS of a web page, changing the content, style, or layout dynamically. JavaScript can also respond to user events, such as clicks, scrolls, inputs, etc., and perform actions accordingly. JavaScript can be written in a separate file (external script), inside the HTML document (internal script), or directly on an HTML element (inline script). JavaScript can also use libraries or frameworks that provide additional features or simplify the coding process.

Let us add JavaScript code to show a message box on button click:

```
<body>
    <div id="parent">
        <button id="child">Click me</button>
    </div>
    <script>
        document.getElementById("child").addEventListener("click", showAlert);
        function showAlert() {
            alert('The button is clicked!');
        }
    </script>
</body>
```

Now when you click the button, it should show an alert message like the following:

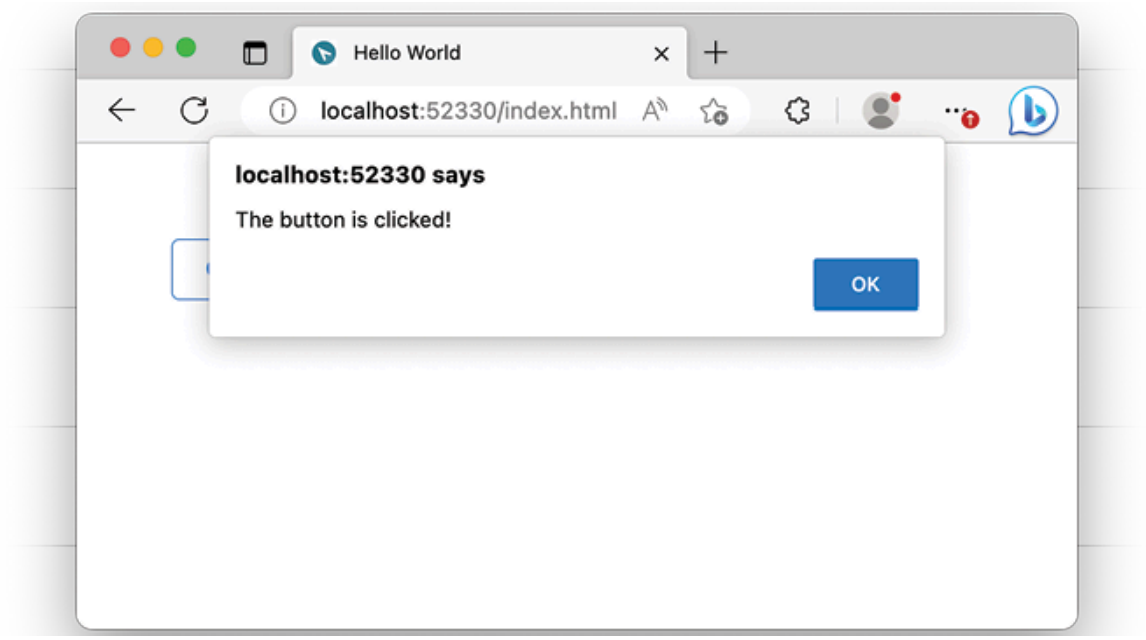


Figure 1.6: *Adding interaction logic to button*

Website versus web app

Before we try to understand how Angular works, it is helpful to know what a web application is and how that is different from a simple website. Traditionally, web pages were content based and provided information on certain topics. For instance, Wikipedia, a free online encyclopedia, is a website. It contains individual web pages for a wide variety of topics. These web pages are connected with hyperlinks so that the user can jump from one topic to another in respective web pages. As the internet grew, more complex scenarios needed to be addressed. For example, an email client like Gmail is not just a web page or a website. It requires the user to identify themselves to provide content specific to that user. The user gets personalized features like email, calendar, chat, etc. So, sites like Gmail are referred to as web apps rather than websites. And it is for such web apps that a framework like Angular is useful. Let us take a quick look at what differentiates a web app from a website. A web app and a website are both accessed through a web browser, but they have different purposes, functions, and levels of interactivity.

A website is a collection of web pages that provide information or content to the user. A website mainly displays content, such as text, images, videos, audio, etc., and may have some interactive elements, such as links, forms, buttons, etc. A website is static and informative, meaning that it does not change its content or appearance based on the user's input, preferences, or actions. It can only store data on the server-side and use it to update the content periodically. It usually does not require authentication, meaning that the user can access the website's information or content without logging in or signing up. Authentication is only needed for some websites that offer restricted or premium content.

A web app is a piece of software that provides specific functionality or service to

the user. A web app allows the user to interact with the software or program and perform tasks, such as sending emails, making payments, chatting online, etc. A web app is dynamic and responsive, meaning that it can change its content or appearance based on the user's input, preferences, or actions. A web app can also store data on the server or the client-side and use it to personalize the user experience. A web app usually requires authentication, meaning that the user must log in or sign up to access the web app's features or services. Authentication ensures that only authorized users can use the web app and access their personal data.

Single page applications

A web app may be a multi-page or a **single page application** (**SPA**). When the user clicks on a hyperlink on a page, if the browser needs to reload and fetch the entire content of the target page, that is referred to as multi-page. This causes a significant delay in page load and hampers user interaction with the page in a timely manner. A SPA fetches only the content needed to refresh a section of the page and renders inline without refreshing the entire page. This gives the feel of a native application and an improved user experience. Again, consider how we interact with Gmail—clicking on email subjects on Gmail does not refresh the entire page. It shows the relevant email inline almost instantaneously depending upon network bandwidth. This is an example of a SPA.

A **SPA** is a web application or website that fits on a single web page with the goal of providing a more fluid user experience similar to a desktop application. A SPA loads a single HTML file and dynamically rewrites that page as the user interacts with it. All subsequent interaction is done through JavaScript and AJAX calls to a server backend that handles data persistence and business logic.

SPAs have some advantages and disadvantages compared to traditional multi-page applications. One of the advantages is that SPAs are faster and more responsive, as they only need to update the parts of the page that change, rather than reloading the entire page. Another advantage is that SPAs are easier to develop and debug, as they use a single language (JavaScript) for both frontend and back-end and have a simpler structure and state management.

However, SPAs also have some drawbacks. One of the disadvantages is that SPAs are harder to optimize for search engines, as they rely on JavaScript for rendering, which may not be supported or executed by some crawlers. Another disadvantage is that SPAs are more vulnerable to security risks, as they expose more logic and data to the client-side, which may be manipulated by malicious users or scripts. A third disadvantage is that SPAs are less accessible, as they may not support some browser features or assistive technologies, such as back button, bookmarking, history, or screen readers. These are general points of concern; however, the SPA ecosystem has evolved in the last decade and most of the issues with performance, security,

and accessibility have been addressed by many frameworks, libraries, and browser APIs.

TypeScript

Angular is a platform and framework for building single page client applications using HTML, CSS and TypeScript. Angular is written in TypeScript. TypeScript is a programming language that builds on JavaScript, adding syntax for types. TypeScript is a superset of JavaScript, which means that any valid JavaScript code is also valid TypeScript code. TypeScript is developed and maintained by *Microsoft* and is widely used for developing large-scale web applications.

Some of the benefits and features of TypeScript are:

- TypeScript adds static typing to JavaScript, which helps to catch errors at compile time, rather than at runtime. Static typing also enables better tooling, such as code completion, refactoring, and documentation.
- TypeScript supports object-oriented programming features, such as classes, interfaces, inheritance, generics, and decorators. These features make the code more structured, reusable, and maintainable.
- TypeScript can transpile to any version of JavaScript, which means that it can run on any browser or platform that supports JavaScript. TypeScript also supports modern JavaScript features, such as modules, arrow functions, async/await, etc.
- TypeScript can work with any existing JavaScript libraries or frameworks, such as React, Angular, Vue, etc. TypeScript can also use type definitions from the DefinitelyTyped project, which provides types for thousands of popular JavaScript libraries.

Angular implements core and optional functionality as a set of TypeScript libraries that you import into your applications. The book will explore this in later chapters.

Understanding how Angular works

Angular adds structure and design patterns to frontend development, allowing developers to create more advanced web applications that are easier to maintain and update. We will take a deep dive into an Angular application in *Chapter 3, Create Your First Angular App,* and *Chapter 4, Building Blocks of Angular App*. In this section, we will try to provide a 36,000-foot view of how the Angular application loads into a browser.

Angular applications are written using TypeScript. Browsers do not understand TypeScript. So, every Angular application needs to be transpiled into JavaScript code. When an Angular app is built, it generates an **index.html** file and a few script files, amongst other files. When the browser renders an Angular application, first, it fetches the **index.html** file. For a hello world Angular application, the **index.html**

file might look as simple as this:

```
1. <!doctype html>
2. <html lang="en" data-critters-container>
3. <head>
4.   <meta charset="utf-8">
5.   <title>HelloWorld</title>
6.   <base href="/">
7.   <meta name="viewport" content="width=device-width, initial-scale=1">
8.   <link rel="icon" type="image/x-icon" href="favicon.ico">
9. <link rel="stylesheet" href="styles.ef46db3751d8e999.css"></head>
10. <body>
11.   <app-root></app-root>
12.   <script src="runtime.76640d59bb9529a3.js" type="module"></script>
13.   <script src="polyfills.cef9d16f4631fb75.js" type="module"></script>
14.   <script src="main.7603a62efea7a947.js" type="module"></script>
15. </body>
16. </html>
```

You will notice three script tags here. When the browser parses this HTML, it loads the Angular framework from the **runtime.js** script file(*line 12*). Once the Angular framework starts execution, it loads **the main.js script** file (*line 14*), the entry point to an Angular app. An Angular app is defined by many modules. We will look into the details of these files in *Chapter 3, Create Your First Angular App* and understand modules are in *Chapter 4, Building Blocks of Angular App.*

Angular uses a component-based architecture, which means that the application is broken down into smaller, reusable parts called components. They are classes that contain the logic, data, and presentation of a specific part of the web page. These components can then be combined to create more complex applications.

The **index.html** file contains an **<app-root>** element (*line 11*). This is where Angular inserts the root component of the application. Once the application DOM is loaded into the browser, Angular manages all user interaction on the page and refreshes sections within the page as needed.

Client-side versus server-side rendering

CSR and SSR are two ways of generating web pages on the browser. They have different advantages and disadvantages, depending on the needs and goals of the website.

CSR is a method where the browser requests a minimal HTML file from the server, and then uses JavaScript to generate the rest of the HTML content dynamically.

This makes the server faster and more efficient, as it only sends a small amount of data to the browser. It also allows more interactivity and functionality for the user, as the JavaScript code can manipulate the DOM and respond to user events. However, this also makes the browser slower and less reliable, as it has to download, parse, and execute a large amount of JavaScript code before rendering the page.

SSR is a method where the server generates a complete HTML file for each page request and sends it to the browser for display. This makes the browser faster and more reliable, as it receives a ready-to-render HTML file that does not depend on JavaScript. It also consumes less resources and bandwidth from the browser, which may improve performance and user experience. However, this also makes the server slower and more costly, as it has to process and render each page request individually. It also limits interactivity and functionality for the user, as the JavaScript code can only run after the page is loaded. It requires a full page reload for each navigation or change. On the other hand, SSR is more friendly for SEO, as search engines can easily crawl and index the HTML content. It also enhances accessibility, as browsers or assistive technologies can display the content without relying on JavaScript. If the page mostly contains static content and involves little to no user interaction, then **static site generation** (**SSG**) is a preferred technique. In SSG, the pages are prebuilt during build time and can be cached at various network edges for faster access. This can greatly improve the SEO rating as well as the user experience on the page.

Generally, the rule of thumb is to generate all public-facing pages on the server-side (SSR or SSG) for SEO and to use CSR for all the user-centric pages, which typically show after authentication and are not indexed by search engines. We will take a deep dive into these rendering techniques in *Chapter 31, Server Side Rendering and Search Engine Optimization*.

Before we get started

We hope that you get a fair overview of how websites render on a browser and how Angular works at a high level. Please familiarize yourself with TypeScript, if you are not, as that is a pre-requisite to learning Angular. We will look at some language constructs of TypeScript in *Chapter 7*, but it is a good idea to get acquainted with TypeScript before you continue with the next chapters.

Conclusion

Angular has been adopted by the developer community since 2010 as a framework of choice for frontend development, especially for enterprise apps. The framework was rewritten in 2016 to meet the complex requirements of a modern web app and user interactions. Rendering a web page on the browser is a complex process and involves a series of steps. It is useful to understand these steps to be able to

optimize the page load for an application. In this chapter, we looked at how Angular works at a high level and the pros and cons of CSR versus server-side.

In the next chapter, we will learn how to set up the development environment for Angular projects.

Points to remember

- Browser fetches and parses multiple files to render a page.
- HTML, JS, and CSS are the building blocks of a web page.
- Web app allows a page to be personalized for a user.
- Single page applications allow for a seamless user experience.
- Angular apps are written in TypeScript.
- Angular breaks the application into components.
- A web page can be rendered on the client-side browser, or on the server.

Questions

1. How does a browser render a web page?
2. Explain the role of HTML, JavaScript, and CSS with an example.
3. Why is TypeScript used instead of JavaScript for Angular development?
4. Explain the advantage of a single page application over traditional websites with an example.

References

- How browsers work:
  https://web.dev/howbrowserswork/

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 2
Setting Up the Development Environment

Introduction

To get started with Angular development, it is important to set up your development environment properly. This will ensure a smooth and efficient workflow, allowing you to create and test your Angular projects with ease. In this chapter, we will walk you through the steps required to set up your development environment, including installing the necessary software and configuring your system to work with Angular. Whether you are new to Angular or an experienced developer, this guide will help you get up and running quickly, so you can start building great web applications.

Structure

The chapter covers the following topics:

- Role of Node.js and npm
- Node version manager
- Integrated development environment
- WebStorm
- Visual Studio Code
- Installing VS Code and extensions
- Quick introduction to VS Code features

- AI assisted tools in VS Code
- Browser developer tools

Objectives

By the end of this chapter, you will be able to understand how to set up tools and editors required to code and build Angular apps. You will learn how to set up **Visual Studio Code** (**VS Code**) with extensions as the development environment for Angular.

Role of Node.js and npm

Node.js and npm are powerful tools that have revolutionized how developers build applications. Node.js is a server-side JavaScript runtime that enables developers to build fast and scalable network applications. It is built on top of the V8 JavaScript engine, the same engine that powers *Google Chrome* and *Microsoft Edge* browsers. Node.js allows developers to write server-side code using JavaScript, a language already familiar to many developers. This makes it easy for them to build robust and efficient applications without having to learn a new language or framework.

Node package manager (**npm**) is a powerful tool that comes bundled with Node.js and allows developers to easily manage and share their JavaScript modules. With npm, developers can easily install, update, and remove packages from their projects. npm has a vast library of packages that can be used to add functionality to any project. This includes everything from small utility libraries to large frameworks. Angular is one such framework that is installed through npm.

To install Node.js and npm, go to **https://nodejs.org/en/download** and download an installer appropriate for your operating system. Once downloaded, run the installer and follow steps in the installation wizard, which looks like the following figure:

***Figure 2.1**: Node.js installer for Mac OS*

The installer will automatically add Node.js binaries to **PATH** variable so that you can run it from the command line tool. It is recommended to install **long-time support** (**LTS**) version as it is well tested and stable. However, if you want to use the latest features of node, you may install the current version instead.

The above installer will install **npm** along with Node.js, but the **npm** packaged with Node.js installer may not be the latest one. To install the latest version of **npm**, run the following command on the terminal:

```
npm install -g npm@latest
```

Run the following command on the terminal to check the version of Node.js and **npm** installed:

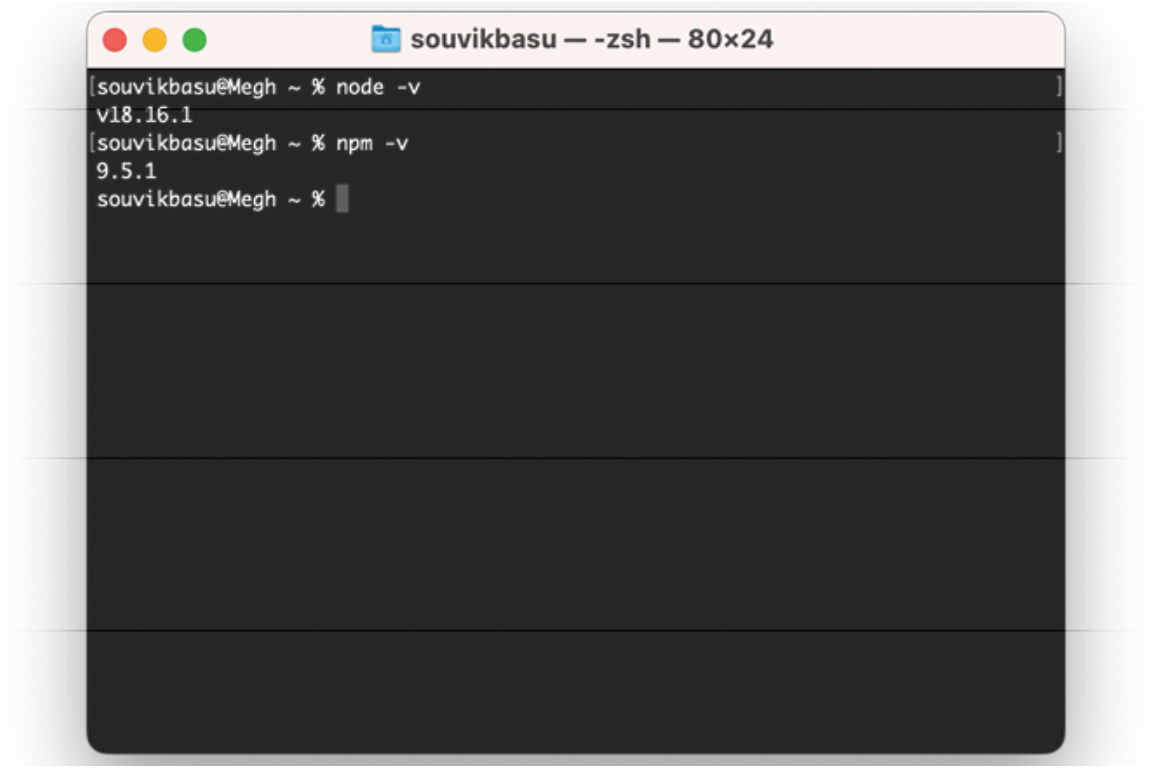


***Figure 2.2**: Node.js and npm versions*

The actual version number might differ on your command line based on the version of Node.js you installed.

Node version manager

If you work on multiple Node.js projects, sometimes you might need to work with different versions of node for each project. This is where a dedicated node environment for each project can be helpful. **Node version manager (NVM)** is a command line tool that helps you to install and manage different versions of Node.js and npm on your machine. It allows you to easily switch between these versions on a per-shell basis, which can be useful for testing your applications or packages with different node environments. NVM also lets you set a default node version for your system or for specific projects using a **.nvmrc** file. It is recommended that you install nvm even if you do not see an immediate need for switching between multiple node versions. It can prove to be handy when needed. For details on how to setup nvm on your machine, please check **https://github.com/nvm-sh/nvm**.

Integrated development environment

An **integrated development environment** (**IDE**) is a software application that provides a comprehensive set of tools for developers to create, test, debug, and deploy web applications. An IDE typically consists of a code editor, a compiler, a debugger, a testing framework, and a deployment tool. For Angular development, an IDE should support features such as code completion, syntax highlighting, refactoring, debugging, testing, and deployment. Let us take a look at them:

- **Code completion**: It helps developers write code faster and avoid errors by suggesting relevant keywords, variables, methods, and properties based on the context and the code syntax. For example, if the developer types *ng* in the code editor, the IDE may suggest **ngOnInit**, **ngOnDestroy**, **ngModel**, and other Angular-specific keywords. Code completion also helps developers to avoid spelling mistakes and typos that may cause bugs or compilation errors.
- **Syntax highlighting**: This makes the code more readable and easier to understand by using different colors and fonts for different elements, such as keywords, variables, strings, comments, operators, and brackets. It also helps developers identify each element's type and scope and spot syntax errors or missing characters. For example, if the developer forgets to close a quotation mark or a parenthesis in the code, the IDE may highlight the rest of the code in red or show an error message.
- **Refactoring**: This allows developers to change the structure and design of the code without affecting its functionality. Refactoring can improve the code's readability, maintainability, reusability, and performance by applying common patterns and principles such as naming conventions, modularization, abstraction, encapsulation, inheritance, and polymorphism. For example, if the developer wants to rename a variable or a method in the code, the IDE can automatically update all the references to that variable or method throughout the project. Refactoring can also help developers to avoid code duplication or complexity by extracting common logic into separate functions or classes.
- **Debugging**: It helps developers to find and fix errors in the code by setting breakpoints, inspecting variables, and stepping through the execution. Breakpoints are points in the code where the execution stops and allows the developer to examine the state of the application

at that moment. Variables are data values stored in memory and can be accessed or modified by the code. Stepping through the execution refers to running the code line by line or jumping from one breakpoint to another. This can help developers understand how the code works, what causes errors or unexpected behaviors, and how to correct them.

- **Testing**: This helps developers to ensure the quality and performance of the web application by running unit tests, integration tests, and end-to-end tests. Testing can help developers to verify that the web application meets the requirements and specifications, that it works correctly under different scenarios and conditions, and that it does not have any bugs or defects.
- **Deployment**: It helps developers to publish the web application to a server or a cloud platform where it can be accessed by users over the internet. Deployment can involve various steps such as building, bundling, minifying, compressing, transpiling, and uploading the web application files. **Building** means compiling the source code into executable files that can run on different browsers or devices. **Bundling** refers to combining multiple files into one file to reduce the number of requests and improve loading speed. **Minifying** is the process of removing unnecessary characters, such as spaces or comments, from the files to reduce their size. **Compressing** means applying algorithms such as gzip or brotli to further reduce the size of the files. **Transpiling** is converting one programming language into another programming language that is more compatible or efficient for a specific target environment. **Uploading** means transferring the web application files from the local machine to a remote server or cloud platform using protocols such as FTP or SSH.

VS Code, WebStorm, and Sublime Text are some of the most popular IDEs and code editors for Angular development. These IDEs offer extensions and plugins that integrate with Angular CLI, TypeScript, RxJS, and other Angular-related technologies.

WebStorm

WebStorm is a powerful and smart IDE for web development, especially for frontend development and server-side JavaScript. WebStorm is a full-

fledged IDE that provides a comprehensive set of features and tools for Angular development, as listed in the above section. WebStorm has more advanced and intelligent code analysis and refactoring tools than code editors like VS Code, which can help improve the code quality and maintainability of Angular projects. WebStorm can detect and fix errors, warnings, style issues, and best practices in the code, as well as suggest optimal ways to structure and design the code using common patterns and principles. WebStorm also has better integration with Angular CLI, TypeScript, RxJS, and other Angular-related technologies, which can enhance the functionality and expressiveness of the code.

Visual Studio Code

Visual Studio Code (**VS Code**) is a lightweight and extensible code editor that supports a wide range of languages and technologies, including Angular. VS Code offers many features and benefits for Angular development, such as customizability, productivity, performance, and open-source. However, VS Code is not a full-fledged IDE and may require installing some extensions or plugins to get the same level of functionality and support as WebStorm.

When it comes to Angular development, VS Code is the go to code editor for many developers. VS Code is powerful code editor that supports a wide range of languages, frameworks, and technologies, including Angular.

It offers many features and benefits that make it a preferred choice for Angular development, such as:

- **Extensibility**: VS Code has a rich marketplace of extensions that enhance its functionality and provide support for Angular and related technologies such as TypeScript, RxJs, Angular CLI, NgRx, Markdown, and more. These extensions offer features such as code completion, syntax highlighting, refactoring, debugging, testing, deployment, and more. We will explore a few of the extensions in the next section.
- **Customizability**: VS Code allows developers to customize its appearance and behavior to suit their preferences and needs. Developers can change the theme, font size, color scheme, keyboard shortcuts, editor settings, and more. Developers can also create their

own extensions or snippets using the VS Code API.

- **Productivity**: VS Code helps developers to write code faster and easier by providing features such as IntelliSense, Emmet, Live Share, Git integration, and code actions. We will explore these with examples in the later sections.
- **Performance**: VS Code is a fast and responsive code editor that runs smoothly on different platforms and devices. VS Code has a modular architecture that separates the core editor from the extensions and the language servers. This means that the editor can run independently from the extensions, and the language servers can run in separate processes or on remote machines. This reduces the memory usage and the CPU load of the editor and improves its performance and stability.
- **TypeScript support**: VS Code has excellent support for TypeScript, which is what Angular is built on.
- **Community**: It has a large and active community, which means plenty of resources are available if you need help or run into any issues.
- **Price**: VS Code is a free and open-source product that does not require any payment or license to use. This is a huge factor that contributes to the adoption of VS Code by the developer community. Some IDEs like WebStorm have better refactoring tools and allow faster code completion. But WebStorm is a proprietary and licensed product that requires a monthly or yearly subscription fee.

Installing VS Code and extensions

Since VS Code is the code editor of choice for developing Angular apps, we will explain the installation and usage details of VS Code. However, you may choose to go ahead with other IDEs or code editors as per your liking and comfort with the development environment. To install VS Code, go to **https://code.visualstudio.com/download** and download the appropriate installer for your operating system.

Since VS Code is a code editor, it heavily depends on extensions to provide meaningful features and productivity tools. To install extensions, run VS Code and go to the **Extensions** menu on the Action bar. Then search for extensions as shown:

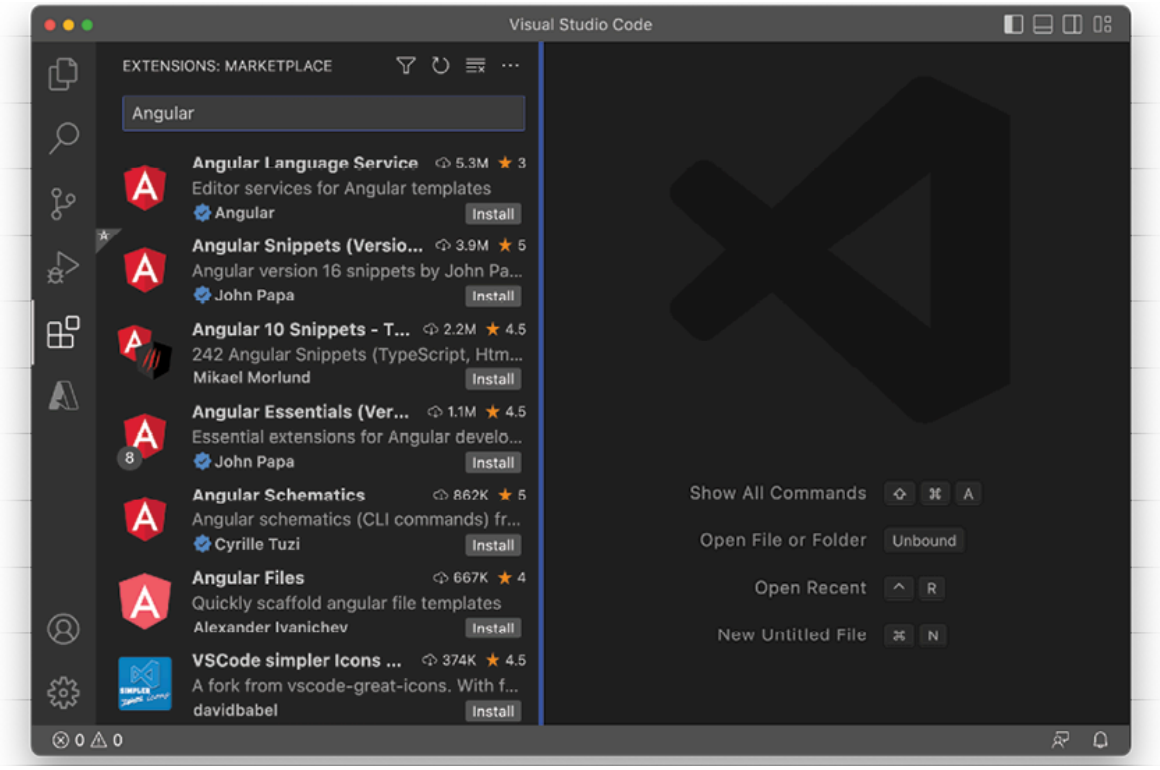


Figure 2.3: *Extensions in VS Code*

Following are the essential VS Code extensions you may need in Angular projects:

- **Angular Language Service**: It provides code completion, syntax highlighting, error checking, and navigation for Angular templates.
- **Angular Snippets**: It provides code snippets for common Angular elements such as components, directives, pipes, services, modules, etc.
- **Debugger for Chrome**: This allows debugging Angular applications running in the Chrome browser directly from VS Code. It allows setting breakpoints, stepping through the code, inspecting variables, and more.
- **Jasmine Test Explorer**: It allows running and debugging Jasmine tests for your Angular project. It also provides code coverage reports and test results in a graphical interface.
- **Prettier**: It is a code formatter that enforces a consistent style across your code base. It supports HTML, CSS, JavaScript, TypeScript, Angular templates, and more. It can also integrate with ESLint to fix style issues automatically.
- **ESLint**: It is a code linter that checks your TypeScript code for readability, maintainability, and functionality errors.

Apart from the above list, there are some helpful extensions to aid with productivity:

- **Angular Console**: It provides a graphical interface for running Angular CLI commands and generating Angular elements.
- **Path Intellisense**: It is used to auto-complete file paths when you are trying to import other files or assets into your code. It can save you time and avoid typos.
- **JSON to TS**: This is used to convert JSON objects to TypeScript interfaces, useful when working with RESTful APIs or mock data.
- **ngrok for VS Code**: It allows exposing your local web server to the internet using ngrok. This can be useful for testing your Angular application on different devices or sharing it with others.
- **GitLens**: This supercharges Git within VS Code and unlocks untapped knowledge within your repository. It helps you to visualize code authorship, navigate and explore Git history, gain valuable insights, and collaborate with your team.
- **TODO Highlight**: It highlights TODOs, FIXMEs, and any other annotations you wish within your code comments.
- **Peacock**: It subtly changes the color of your VS Code workspace to help you identify your editor when you have multiple instances, use VS Live Share, or use VS Code's Remote features.
- **Code Spell Checker**: This is a basic spell checker that helps you to catch common spelling errors.
- **Change Case**: It helps in automating the process of case conversion for the selected text. It supports various types of cases such as camelCase, snake_case, kebab-case, PascalCase, etc.

Quick introduction to VS Code features

It is helpful to understand your code editor to be productive. Here are few productivity features of VS Code:

- **IntelliSense** is a code completion feature that suggests relevant keywords, variables, methods, and properties based on the context and the syntax of the code. It works as shown:

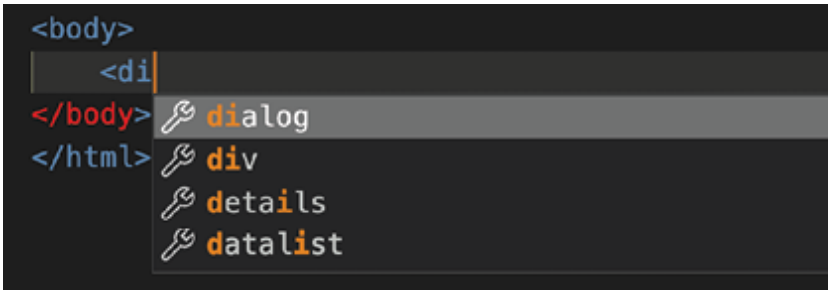


Figure 2.4: *IntelliSense in VS Code*

- **Code actions** are quick fixes or refactoring that can be applied to the code by clicking on a light bulb icon or using a keyboard shortcut. Take a look at the following figure for your reference:

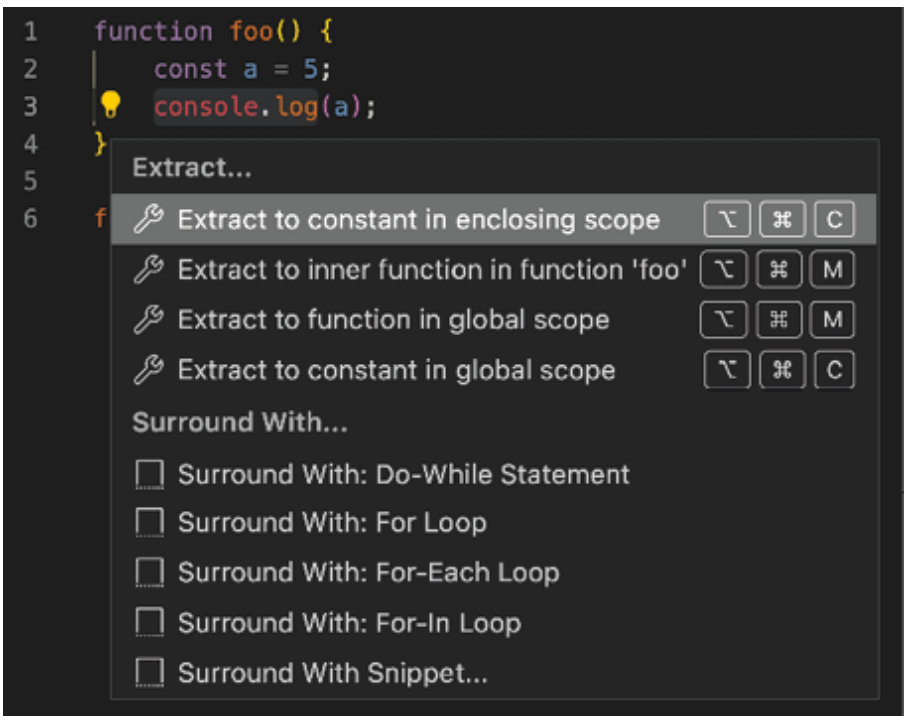


Figure 2.5: *Code actions in VS Code*

- **Emmet** is a feature that allows developers to write HTML and CSS code using abbreviations that expand into full code snippets. For instance, create a new html file and just type the exclamation mark —**!**. The following figure explains what happens:

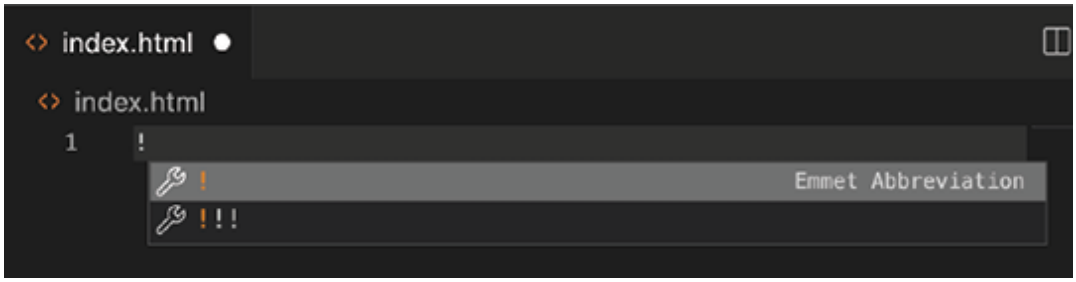


Figure 2.6: *Emmet in VS Code*

Then select the Emmet code action from the contextual menu. This will create a basic HTML structure as shown:

Figure 2.7: *Code generation using Emmet in VS Code*

- **Live Share** is a feature that allows developers to collaborate with other developers in real-time by sharing their code editor, terminal, debugger, and server.
- **Git integration** is a feature that enables developers to manage their code versioning and collaboration using Git commands and tools.

AI assisted tools in VS Code

We are in the AI era, and code assistance using AI is becoming a norm. VS Code comes with power integration of *GitHub Copilot* and its upgraded sibling, *Copilot X* (aka *Copilot Chat*), which can greatly enhance code quality. *GitHub Copilot X* is a remarkable tool that has revolutionized the coding experience for many developers. With its AI-powered assistance, it provides accurate and personalized suggestions that help improve productivity and the quality of code, allowing you to write

code faster and with less work. It draws context from comments and code to suggest individual lines and whole functions instantly.

What is more impressive is how seamlessly it integrates with VS Code, allowing developers to enjoy a familiar coding environment while using the tool. The integration is so smooth that it feels like an integral part of the VS Code experience.

GitHub Copilot X is powered by *OpenAI Codex*, a generative pretrained language model created by *OpenAI*. *GitHub Copilot X* can also help you with chat, terminal, pull requests, and documentation using its chat and terminal interfaces, support for pull requests, and early adoption of *OpenAI GPT-4*. *Copilot X* understands and responds to natural language commands, both in chat and voice. It helps developers review and improve pull requests. This saves developers time and helps them to produce higher-quality code. It answers questions about code documentation.

You can find more up to date details about *GitHub Copilot X* at the *GitHub* page mentioned in the references section.

To use Copilot Chat, go to Extensions menu and install **GitHub Copilot Chat** extension. Open Command Palette (*Cmd+Shift+P* on Mac/*Ctrl+Shift+P* on Windows/Linux) and type/select **Chat**: **Open Editor** (**GitHub Copilot**). The copilot window should appear. Keep your code open on one side and ask code tasks like **Generate unit tests for this code** in the Copilot panel as shown:

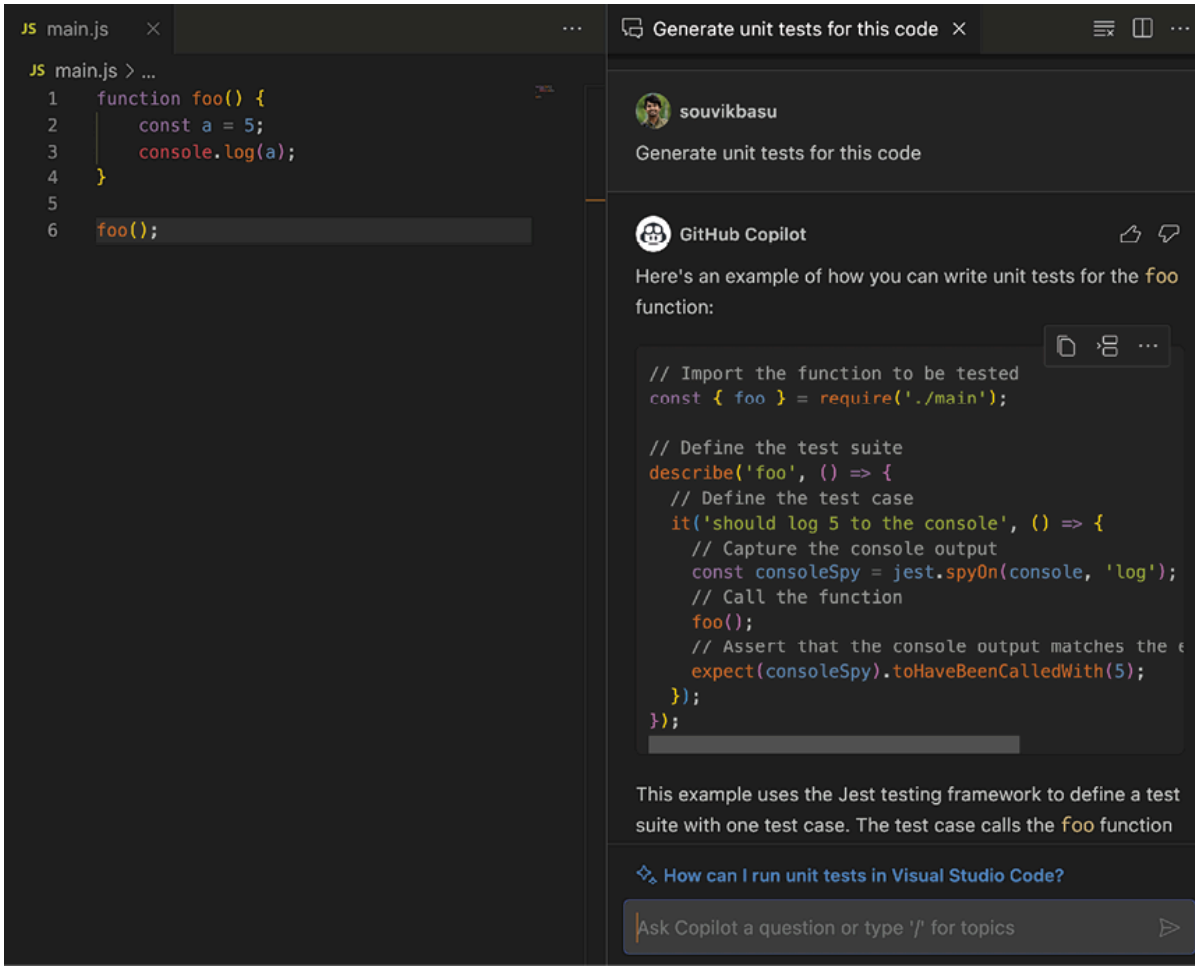


Figure 2.8: *GitHub Copilot Chat panel in VS Code*

Browser developer tools

Using the developer tools also known as **DevTools** on a browser to debug code is a standard practice in frontend development. Chrome and Edge browsers use the same underlying open-source code, which is Chromium. Hence the DevTools is the same for both. Similarly, Firefox, Safari, and other browsers also have their respective dev tools. If you use DevTools, install the corresponding browser extension for the Angular DevTools to navigate and inspect Angular pages effortlessly. The Angular DevTools URL to install the browser extension is added in the reference section at the end of the chapter. We will discuss these more in detail in *Chapter 16, Debugging*, where we explore various debugging techniques.

Conclusion

Choosing the right development environment is critical to any project.

With its rich set of extensions, VS Code is the most popular IDE for developing Angular projects. In this chapter, we looked at the features of an IDE and the steps to install VS Code. It is important to keep node, npm, and IDE upgraded from time to time to avail of the latest features and security patches.

In the next chapter, we will learn how to create our first Angular app using Angular CLI. We will also understand the structure of an Angular workspace and how the source code is organized into an Angular project.

Points to remember

- Install Node.js and npm to get started.
- Install WebStorm or VS Code as an IDE for Angular projects development.
- Install IDE extensions as needed to improve productivity.
- Take advantage of AI assisted tools for faster and error free code generation.

Questions

1. What is node and npm?
2. What is a Node version manager? Explain with a use case.
3. Name a few IDEs used for Angular development.
4. Explain a few features of VS Code to develop for Angular.
5. Name a few useful VS Code extensions.

References

- Copilot X:
  https://github.com/features/preview/copilot-x
- Angular DevTools:
  https://angular.dev/tools/devtools

CHAPTER 3
Create Your First Angular App

Introduction

If you are new to Angular, the Angular **command-line interface** (**CLI**) can be a lifesaver. It is a CLI tool that helps you generate and manage Angular projects with ease. With Angular CLI, creating your first Angular app is a breeze. You do not have to worry about setting up your project structure or configuring your build tools, as Angular CLI takes care of all that for you. It is a powerful tool that simplifies the development process and automates common tasks. In this chapter, we will be taking the first steps toward building your very own Angular app using Angular CLI. You will also explore the project folder structure and understand the purpose of each file and folder. Finally, you will dive deeper into the CLI options and learn how to customize and configure your Angular app.

Structure

The chapter covers the following topics:

- Installing Angular CLI
- Creating Hello World app
- Anatomy of an Angular workspace
- Deep dive into Angular CLI

Objectives

By the end of this chapter, you will have a solid foundation for

developing Angular apps with CLI. You will also understand how the various files are structured in an Angular project.

Installing Angular CLI

Angular CLI is a command-line interface tool that helps you create and manage Angular projects. You can install it globally using **npm**:

```
npm install -g @angular/cli
```

Creating Hello World app

Open the terminal window of your choice and go to the folder where you want to create your Angular app. Keep the path to this folder as short as possible to avoid issues with longer path names. Certain operating systems, like Windows, restrict a folder to having a maximum path string of 255 characters. Yet, the **node_modules** folder can go very deep, breaching this limit.

Run the following command to create a new Angular workspace:

```
ng new angular-hello-world
```

The tool will prompt for if you want to include routing. For now, select the default option not to include routing, as shown:

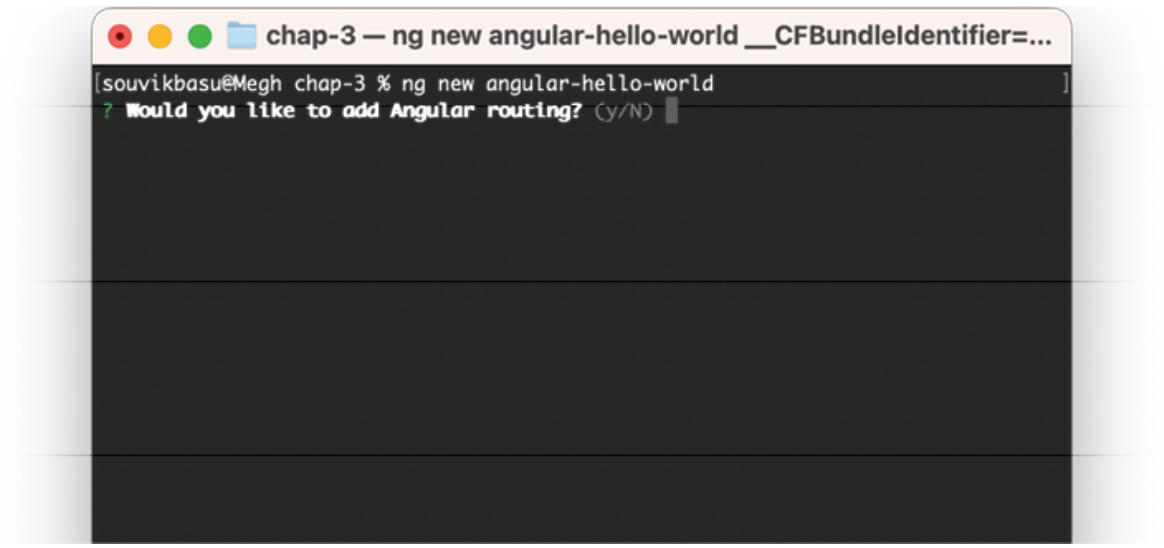


Figure 3.1: *Generate a new Angular project*

The next question is to select the styling library. There are a variety of styling options that Angular supports. We will take a deep dive into styling in *Chapter 6, Styling*. For now, select the default option—**CSS**, as

shown:

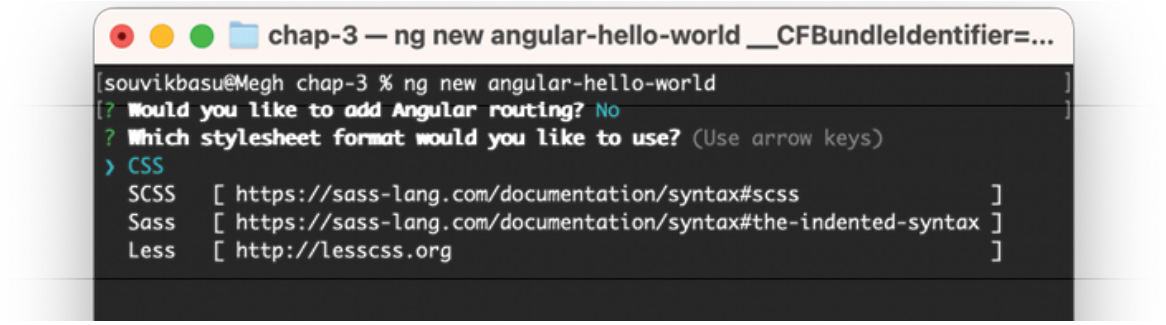


Figure 3.2: *Select stylesheet format*

The tool will start installing dependent packages to set up the project workspace. Within a minute or so, you should see a success message like this:

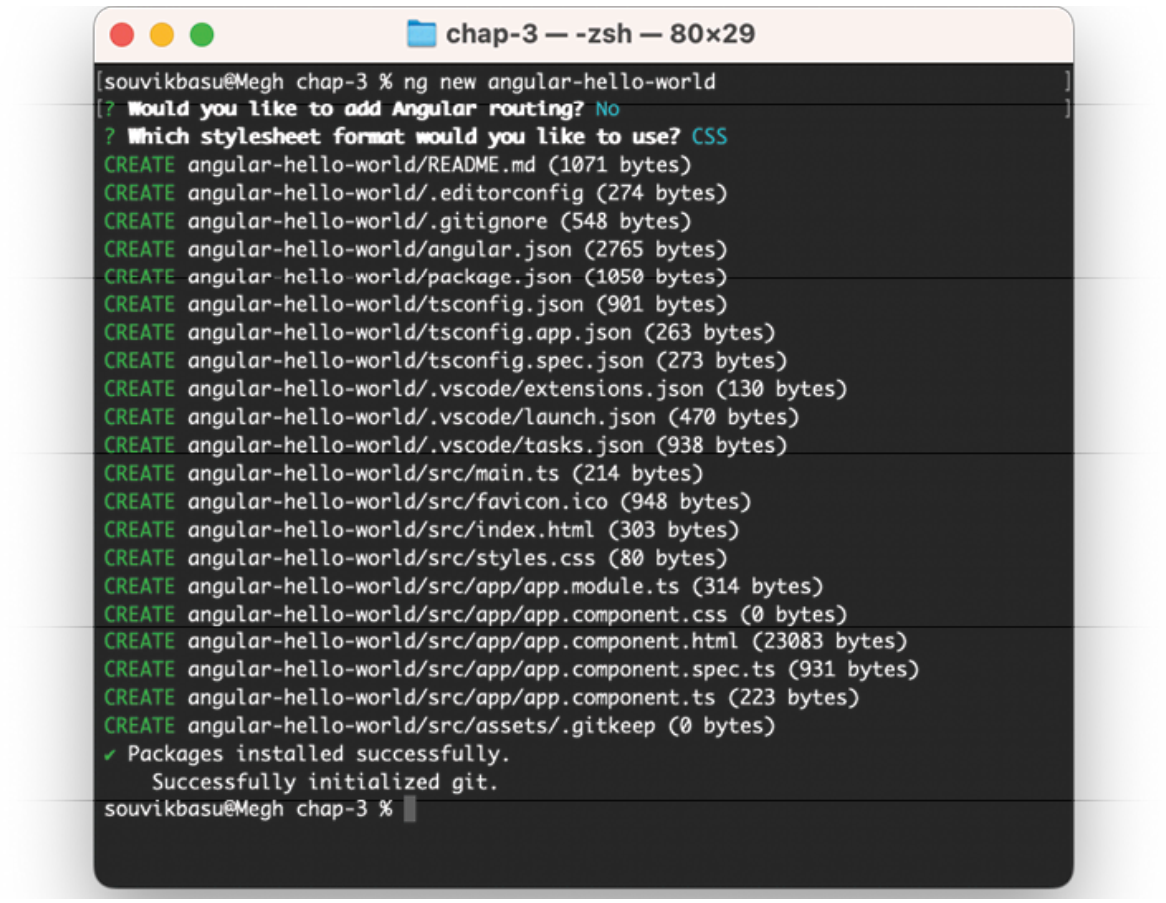


Figure 3.3: *Success message to generate a new project*

Jump into the newly created folder and run the development server to host the app:

cd angular-hello-world

ng serve --open

The **--open** option will automatically open your default browser and navigate to **http://localhost:4200/**.

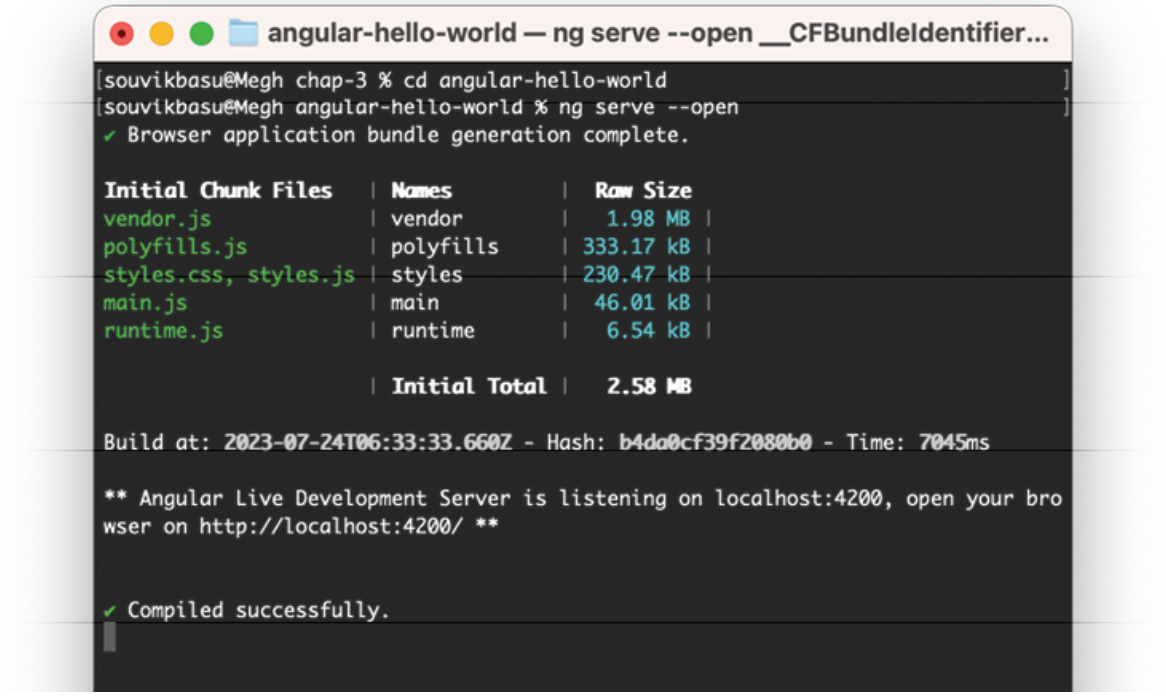


Figure 3.4: *Serve the app on dev server*

Congratulations! You just created your first Angular app! It is that simple. The following figure demonstrates a preview of the app:

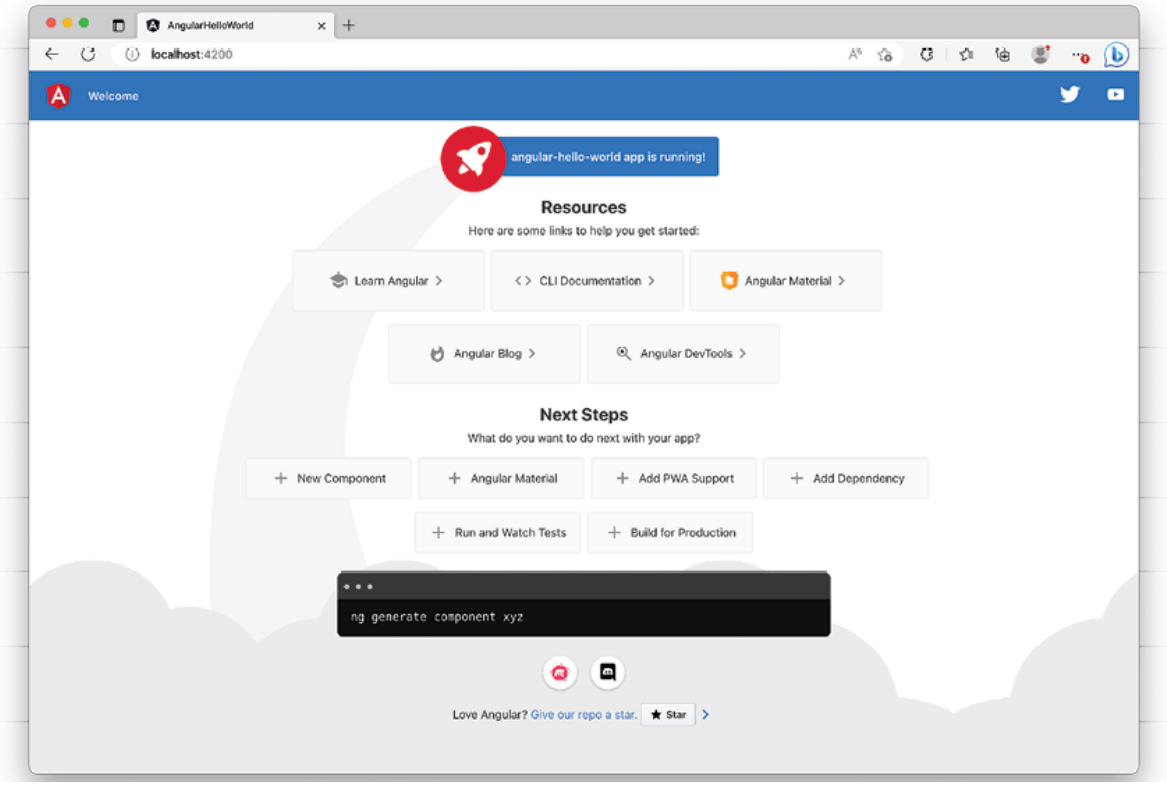


Figure 3.5: *App preview on the browser*

Anatomy of an Angular workspace

An Angular project can be either an application or a library that can be used by other applications. An Angular workspace is a way of organizing your Angular projects and configuring your build and development tools. It consists of configuration files, source files, node modules, and **end-to-end** (**e2e**) tests. You can create a workspace using the Angular CLI or manually. You can also create multiple projects in a workspace, either applications or libraries and share common dependencies and settings among them.

Source files

The workspace source files are the files that contain the actual code and assets for the projects in the workspace. Each project has its source folder containing subfolders and files for different purposes. For example, the project we created in the previous section has the following source folder structure:

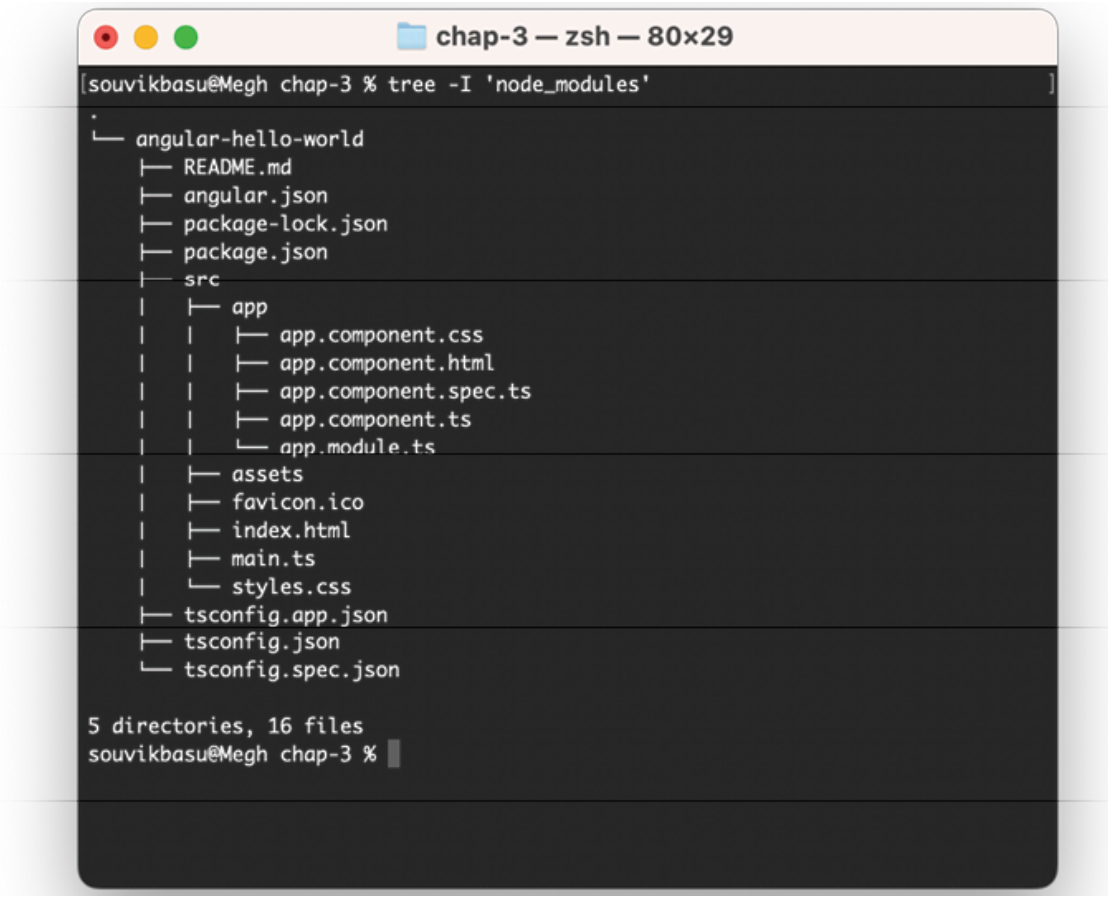


Figure 3.6: *Project folder hierarchy*

The main subfolders and files in the source folder are:

- app/: This folder contains the component files for the application. **app.component.module** file is the root module of the application. We will discuss more about app modules in the next chapter. The app.component.html file contains the HTML structure of the root application component. A best practice is to keep this file small in size to indicate just the application layout without inserting any further page structure or business logic into it. It is best to divide the page into separate components to handle specific regions within the page. We will discuss creating components in the next chapter. The business logic for the app component resides in **app.component.ts** file. The **app.component.css** file contains style statements. And finally, **app.component.spec.ts** file contains unit tests for the app component. We will dive deep into Testing in *Chapter 15, Testing*.
- assets/: This folder contains static assets such as images, icons, fonts, etc. These are kept in a separate folder so as to be able to move them to a **content delivery network** (**CDN**) servers which specializes in serving static files faster to the user by caching the files in a server

located nearer to the user. We will look at the techniques to deploy static content in CDN in *Chapter 10, Deployment*.

- index.html: This is the main HTML file that is served when someone visits your application. It contains a **<base>** element that specifies the base URL for all relative URLs in the application. It also contains an **<app-root>** element that is a placeholder for your root component template. We will look at a code example to understand these concepts in the next chapter.
- main.ts: This is the main entry point for your application. It bootstraps the root module of your application.
- styles.css: CSS style statements for your application. You can also use other style formats such as SCSS, Sass or Less, which we will discuss in *Chapter 6, Styling*.
- favicon.ico: It is the application icon to show in the browser tab. An application can specify a range of favicons to be used for different resolutions and platforms. For instance, we can keep icons for 16x16 px, 32x32 px, and other dimensions. The browser will pick up the best matching icon based on the platform and browser settings. This is also handy when creating a **Progressive Web App** (**PWA**) to show the icon on a mobile home screen. We will discuss the technique in *Chapter 24, Offline*. If nothing else is available, it will fall back to favicon.ico file to display in the browser tab.

A library project has a similar source folder structure, except it does not have an **index.html** or a **main.ts** file. Instead, it has a **public-api.ts** file that exports all public symbols from your library.

Configuration files

The workspace configuration files provide workspace-wide and project-specific settings for the build and development tools that the CLI uses, such as TypeScript, Karma, Protractor, etc. The main workspace configuration file is **angular.json**, which contains an object with the following properties:

- **version**: The version of the Angular CLI that created or updated the workspace.
- **newProjectRoot**: The path where new projects will be created, relative to the workspace root. By default, it is projects.

- **cli**: An object with CLI-specific configuration options, such as default schematics collection, analytics reporting, etc.
- **defaultProject**: The name of the default project for the workspace. By default, it is the same as the workspace name.
- **projects**: An object with keys corresponding to the names of the projects in the workspace, and values containing project-specific configuration options.

Each project in the projects object has its own properties, such as:

- **projectType**: The type of the project, either application or library.
- **root**: The root folder for the project files, relative to the workspace root.
- **sourceRoot**: The root folder for the project source files, relative to the workspace root. By default, it is **<project-root>/src**.
- **prefix**: The prefix to apply to generated selectors for the project components. By default, it is app for applications and lib for libraries.
- **architect**: An object with keys corresponding to the names of the build targets for the project, and values containing target-specific configuration options. For example, some common build targets are build, serve, test, lint, etc.

We will explore these options and configurations in detail in *Chapter 10, Deployment*.

Node modules

The workspace also contains a **node_modules** folder at the root level that provides npm packages to all projects in the workspace. These packages are specified in the **package.json** file and are installed by running **npm install** or **yarn install** commands. Some of these packages are Angular core libraries, such as **@angular/core**, **@angular/common**, **@angular/forms**, etc. Others are third-party libraries or tools that you can use in your projects, such as RxJS, bootstrap, angular-material, etc.

End-to-end tests

e2e tests are a way of testing your angular application from the user's perspective, using a real browser and simulating user actions and expectations. e2e tests in Angular can help you verify that your application works as intended, from start to finish, and catch any bugs or

errors that might occur in different scenarios. These tests are written using frameworks and tools such as Protractor, Jasmine, Cypress, Nightwatch, or WebdriverIO. You can run the tests using the ng e2e command, which will build and serve your application and then run the tests. Since Angular v12, the e2e folder has not been auto-generated. You can generate it using **ng e2e [project name]** command.

We will explore Testing in *Chapter 15*, *Testing,* where we will look at the different types of tests we can perform on an application and the tools suited for each type of testing.

Deep dive into Angular CLI

You can use the Angular CLI to perform various tasks, such as generating new projects, components, services, pipes, directives, and so on; building and serving your application for local development or production deployment; testing and linting your code; and adding external libraries or schematics to your project. Following are some of the most useful options that developers can use to streamline their development process:

- **ng new**: This option is used to create a new Angular project from scratch. We used this command to create the Angular Hello World app in the beginning of this chapter:

    ng new angular-hello-world

This will create a new workspace folder named **angular-hello-world** with all the files and dependencies needed for the project. You can also pass some options to customize your project, such as:

Option	Description
--dry-run	Prints what the command would do without actually doing it
--skip-install	Skips installing packages
--style	Specifies the style file format to use (such as CSS, SCSS, SASS, etc.)
--routing	Generates a routing module for the project

Table 3.1: *ng new options*

As an exercise, try generating the app again, this time with SCSS style files and a routing module:

ng new angular-hello-world --style=scss --routing

- **ng serve**: This will compile your application and start a development

web server at **http://localhost:4200/**. You can open this URL in your browser to see your application running. This provides a good preview of your application during development. The server will also automatically reload the page in the browser when you change any of the source files. Please note that this is a development server and not meant to be used in production for security reasons.

You can pass some options to customize your serve command, such as:

Option	Description
--open or -o	Opens the URL in your default browser after serving
--port or -p	Specifies the port number to use (default is 4200)
--prod or -prod	Builds the application for production (with optimizations)
--configuration	Specifies a configuration name

Table 3.2: *ng serve options*

For example, to serve your application on port **3000** and open it in your browser, you can use:

ng serve -o -p 3000

- **ng generate**: You can use the ng generate command to create various Angular components, such as components, services, directives, pipes, etc. For example:

ng generate component my-component

This will create a new folder named **my-component** under the **src/app** folder, with four files:

- **my-component.component.ts**
- **my-component.component.html**
- **my-component.component.css**
- **my-component.component.spec.ts**

It will also update the **app.module.ts** file to declare the new component.

You can also use aliases or shortcuts for the generate command, such as:

ng g c my-component

This does the same thing as the previous command. Some of the common aliases are:

Alias	Expands to

c	Component
s	Service
d	Directive
p	Pipe
m	Module
g	Guard

Table 3.3: *Shortcuts for ng generate schema*

We will explore each of these schema types in the next chapter.

You can pass some options to customize your generated files, such as:

Option	Description
--dry-run	Prints what the command would do without actually doing it
--flat	Creates the files without creating a folder
--inline-template or -t	Creates an inline template instead of a separate file
--inline-style or -s	Creates an inline style instead of a separate file
--skip-tests or -S	Skips creating a test file
--module	Declares the features in a specific module

Table 3.4: *ng generate options*

For example, to create a component with an inline template and style, and no test file, you can use:

```
ng g c my-component -t -s -S
```

- **ng build**: The ng **build** command compiles your application into an output directory that you can deploy to a web server.

In the above example, running **ng build** will create a new folder named **dist/angular-hello-world** with all the files needed to run your application. You can copy these files to any web server that can serve static files. We will explore ways to deploy an Angular app to production in *Chapter 10, Deployment.*

Following are the options to customize your build command:

Option	Description

--prod or -prod	Builds the application for production (with optimizations)
--output-path or -op	Specifies the output directory (default is dist/project-name)
--watch or -w	Watches for changes and rebuilds the application

Table 3.5: *ng build options*

Here is an example to build your application for production and output it to a folder named build:

```
ng build -prod -op build
```

- **ng test**: This command runs unit tests for your application using Karma and Jasmine. You can execute unit tests and integration tests to ensure that the application works as expected and meets all the requirements.

You can also specify various options to customize the testing process, such as:

Option	Description
--code-coverage	Generate a code coverage report
--watch or -w	Watches for changes and rerun the tests
--browsers	Choose browser to run the tests on
--configuration	Specify a named configuration

Table 3.6: *ng test options*

We will explore these options in *Chapter 15, Testing*.

- **ng e2e**: This command runs end-to-end tests for your application using Protractor. It also builds and serves your application before running the tests. You can use the shorthand **ng e** for this command.

Following are the options to customize the e2e testing process:

Option	Description
--dev-server-target	Overrides the dev server target used by Protractor
--webdriver-update	Updates webdriver binaries before running Protractor tests
--suite	Runs only specified suites from the Protractor configuration file

Table 3.7: *ng e2e options*

- **ng lint**: This option runs a static code analysis tool that checks the Angular application's code for code quality and conformity to coding

standards. This helps to detect and fix potential bugs, errors, or security vulnerabilities before deploying the application.

You can explore further options in the doc page for Angular CLI, which is provided in the References section at the end of the chapter.

Conclusion

The Angular CLI is a powerful tool that helps you create, develop, and manage Angular projects. It provides a set of commands that you can use to perform various tasks, such as generating new files, building your application, running tests, and deploying to a server. You can also pass different options to the commands to customize their behavior or output. Although you can create the project files manually, using the CLI will often speed up your development process.

This concludes the first section of the book to get started with Angular. The following section concentrates on creating a simple website using Angular that renders data on the page without involving user interaction. In the next chapter, we will look at modules, components, services, directives, decorators, and pipes, which are the building blocks of an Angular application.

Points to remember

- Angular CLI allows you to create a new workspace with a project.
- The CLI can generate schema, build for production, test, and run lint on the app.
- Angular workspace may contain one or more projects.

Questions

1. Explain how to generate a new project using Angular CLI.
2. How can you add a component to an existing project?
3. How can you run unit tests on a project?
4. What is the difference between dev and prod build in an Angular project?
5. What are the main subfolders and files in the source folder of an Angular workspace, and what are their purposes?

References

- Angular CLI:
  https://angular.io/cli

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 4
Building Blocks of Angular App

Introduction

In the previous chapter, we created our first Angular app. Now is a good time to dive deep into the building blocks of an Angular app and understand the role of each block. An Angular app is composed of a hierarchy of components. Components are the basic building blocks of any Angular app. Apart from components, there are concepts like modules, directives, services, pipes, etc., which help enhance the components capabilities and allow modularity and reusability of code.

Structure

The chapter covers the following concepts in Angular:

- Breaking down the Angular app
- Modules
- Components
- Services
- Directives
- Pipes
- Decorators
- Importing modules
- Organizing code

Objectives

By the end of this chapter, you will have a clear understanding of the core concepts of an Angular app, as listed above. These concepts will help you to develop a real-world Angular app using best coding practices like reusability and the Single Responsibility Principle.

Breaking down the Angular app

Let us create a new Angular workspace to understand each building block of the app. We will create an online store to sell fruits. If you skipped the previous chapter or need a refresher, please run the Angular CLI command below to create an Angular project:

ng new fruit-store

Select the default options in the prompt. This will generate the workspace folder **fruit-store**. To render the app on a local browser, run the following command:

ng serve --open

As we learned in the previous chapter, this will start a development server and open the app at **http://localhost:4200** using the default browser, as shown:

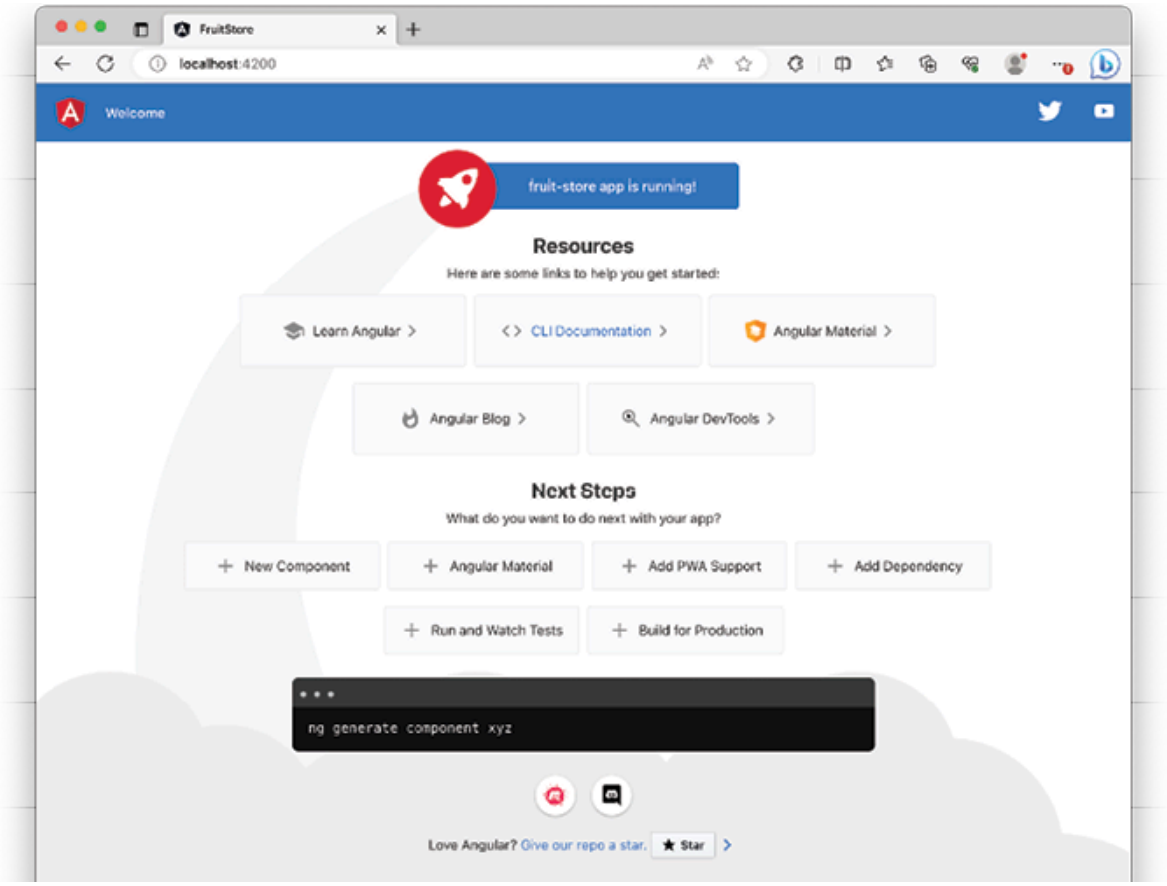


***Figure 4.1**: Generated Angular app*

When a browser tries to access any website, by convention, the web server returns **index.html** file. Once the html is fetched and parsed on the browser, other script files JavaScript, stylesheets CSS and image/icon assets are loaded as specified in the HTML. We will take a look at how this **index.html** file is generated.

Open the workspace folder **fruit-store** in VS Code or an IDE of your choice and scan through the files created, as shown in the following figure:

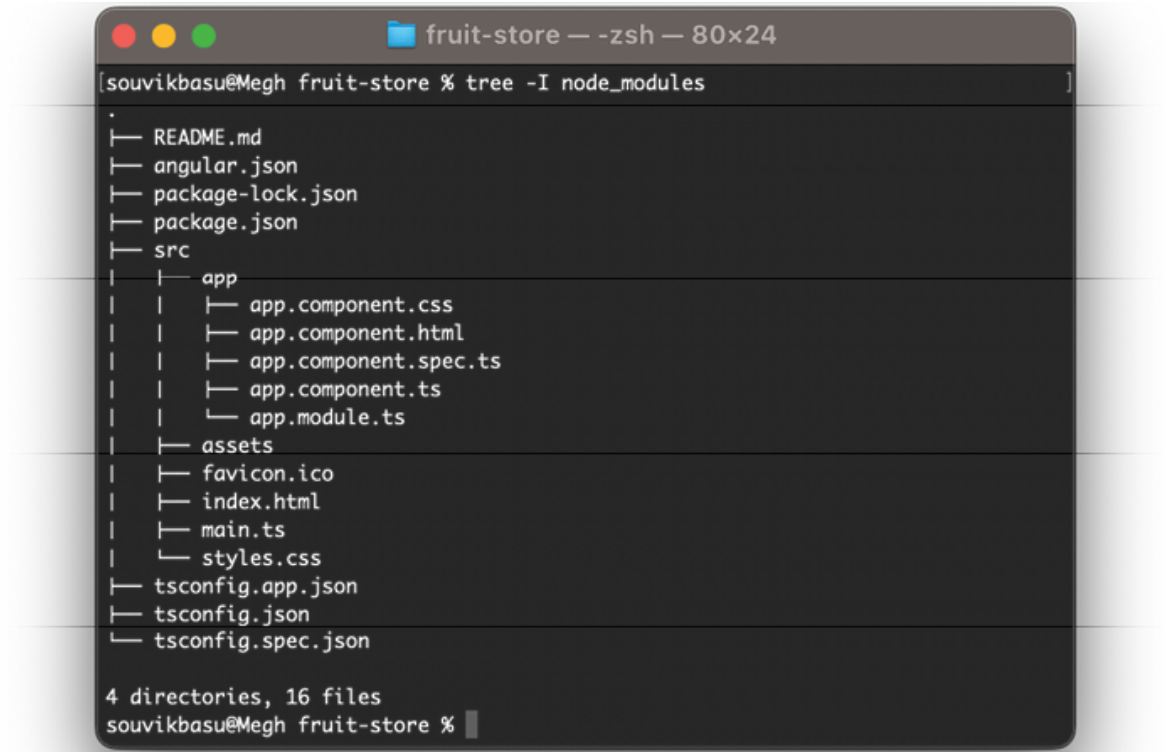


Figure 4.2: *App folder structure*

The files you see under the **src** folder are for development purposes and are not deployed on a web server as is. If you want to deploy this app to a production web server, you need to build the project first. Run the following command to build the project:

ng build

This will generate the **dist** folder under the **root** folder. The contents of this **dist** folder can be deployed to a web server. If you look at this folder, you will find the following files:

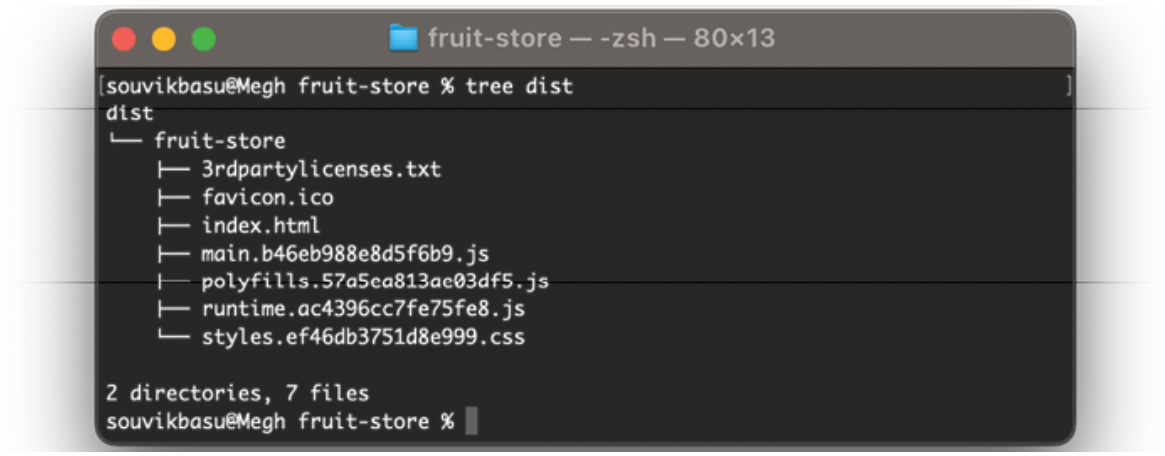


Figure 4.3: *dist folder structure*

The folder contains an **index.html** file which is the initial file served when a browser makes a call to the web server. Additionally, it includes a **favicon** file, a CSS file, and three JavaScript files:

- **runtime.*.js**: This is the first script that runs to load the Angular framework and other required libraries. A unique ID is added to the file name to ensure that when a new build is deployed, the web server does not return an old, cached file to the browser.
- **polyfills.*.js**: This script loads polyfills to support older browser versions. This ensures that our code can use the latest JavaScript features without worrying about browser compatibility.
- **main.*.js**: This is the entry point of our app.

Now let us look at the **src** folder from *Figure 4.2* to identify the corresponding source files that were used to build the **dist** folder:

- **src/index.html**: This corresponds to the **index.html** file created under **dist**. Try changing the **title** tag value to **Online Fruit Store** and refresh the browser window. The tab title will update to reflect the new value, as shown:

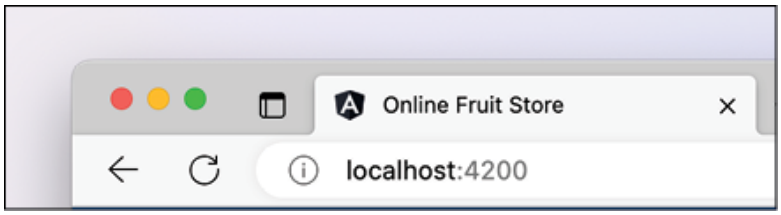


Figure 4.4: *App title on browser tab*

The **index.html** file in **src** folder looks very similar to the one in **dist** folder:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Online Fruit Store</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

You will, however, notice no **<script>** tags in this file. The script tags are injected into the file during the build process:

- src/styles.css: This file is used to generate the **styles.*.css** file in **dist** folder. The generated CSS file is not just **styles.css** but rather an accumulation of all the CSS files throughout the project clubbed into one.

- src/favicon.ico: This file is copied as is to the **dist** folder.
- src/main.ts: This is the entry point of the Angular app and is used to generate the **main.*.js** file in the **dist** folder. The build process parses **main.js** and looks at all other modules imported in the file. It then includes all referenced modules together. It repeats the steps for each file it parses. This way, a single script file is generated. Finally, the TypeScript file is transpiled into a JavaScript file **main.*.js**, as browsers only understand JavaScript.
- src/app: This contains code for the app component referenced in the **main.ts** file.

Let us take a detailed look at the **main.ts** file:

```
1. import { platformBrowserDynamic } from '@angular/platform-browser-
   dynamic';
2. import { AppModule } from './app/app.module';
3.
4.
5. platformBrowserDynamic().bootstrapModule(AppModule)
6.   .catch(err => console.error(err));
```

This is the simplest **main.ts** file possible. It uses an Angular runtime method **platformBrowserDynamic** that compiles and executes the app on different supported browsers. The **bootstrapModule** method takes **AppModule** as a param which is defined in the **src/app/app.module.ts** file. By convention, the **root** module of an Angular app is named **AppModule**. The next section gives an intro to Angular modules.

Modules

A module is a collection of components, services, directives, pipes, and other related code that defines a feature or functionality of an app. A module can import other modules to use their exported declarations and export their declarations for other modules to use. Let us look at our **AppModule** to understand this:

```
1. import { NgModule } from '@angular/core';
2. import { BrowserModule } from '@angular/platform-browser';
3.
4. import { AppComponent } from './app.component';
5.
6. @NgModule({
7.   declarations: [
8.     AppComponent
9.   ],
```

```
10.   imports: [
11.     BrowserModule
12.   ],
13.   providers: [],
14.   bootstrap: [AppComponent]
15. })
16. export class AppModule { }
```

Every Angular app has at least one module, called the **root** module or **AppModule**, which bootstraps the app and provides the main entry point. You will see a class defined above with the name **AppModule**. Angular uses class decorators to add attributes. *Line 6* defines a class decorator **@NgModule()** for **AppModule** that takes a metadata object to define the module. The following properties are listed in the above code:

- **declarations**: This is a list of components, pipes, and directives that belong to this module. Since this is a simple example, the only item you see is **AppComponent**, that is, the **root** component of the app. A real case may contain several items in a module declaration.
- **imports**: This is a list of other modules used in this module. In the above code, **BrowserModule** is imported that has methods to render the Angular app on a browser.
- **providers**: This is a list of services that this module provides. The list is empty in the above sample code. We will revisit this property later in the chapter with a more real-world case.
- **bootstrap**: This property is specific to the **root** module, as in the code above. It specifies the name of the **root** component to bootstrap the app. Using this property, the **AppModule** loads the **AppComponent** on the browser.

You can create additional modules to organize your code into logical units. We will take a look at this later in the chapter.

Components

Components are the visible blocks of an Angular app. They contain code to render the view on the browser and handle user interactions like clicking on a button. A tiny app might have a single component for the entire app page. A more real-world app will have a page composed of several components, which in turn might be composed of smaller children components. A component is a class that controls a view template, which is a section of the web page that displays data. Let us look at the app component created in our **fruit-store** app. The app folder consists of the following files:

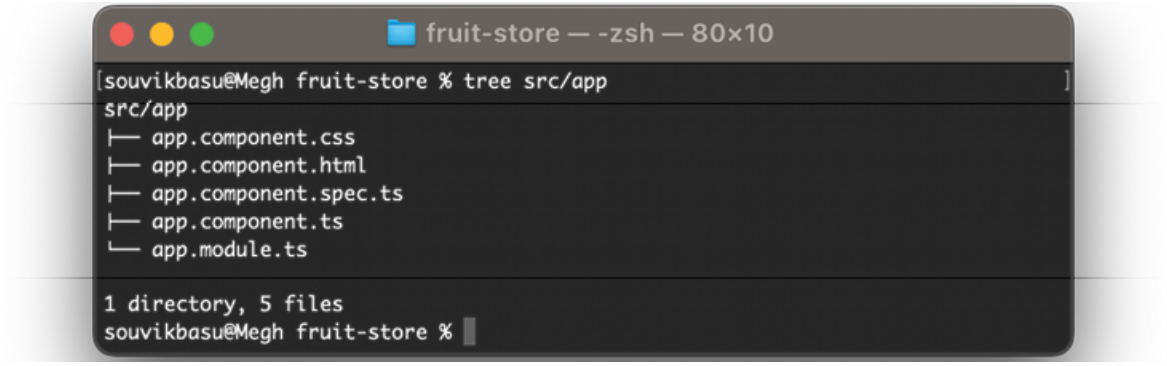


***Figure 4.5**: App component files*

This app component is created using four files:

- app.component.html: This is the view template of the component and defines how the component will be rendered. Any HTML content is valid. If you look at the content of this file, you will notice that the entire app HTML is added to this file.

  Run the following command on the terminal if not already running:

  ng serve --open

  The auto-generated view template by Angular CLI is too complex. To see a simpler example, delete the entire content of this file and replace it with the following HTML code:

  ```
  1. <div>The Fruit Store</div>
  ```

  The browser tab should auto-refresh to show the new text, as shown in the following screenshot:

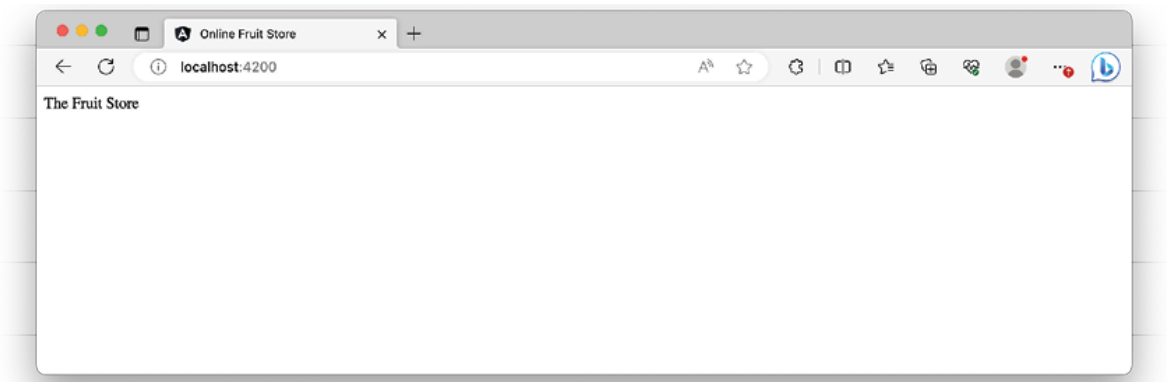


***Figure 4.6:** Simple view template*

- **app.component.ts**: This is the TypeScript file that defines the underlying class for the component. In our code, it looks like this:

  ```
  1. import { Component } from '@angular/core';
  2.
  3. @Component({
  4.  selector: 'app-root',
  ```

```
5.  templateUrl: './app.component.html',
6.  styleUrls: ['./app.component.css']
7. })
8. export class AppComponent {
9.  title = 'fruit-store';
10. }
```

You will notice that the class **AppComponent** is exported from this file, and it is the same name mentioned in the bootstrap property of the **AppModule**. This is how the **root** module figured out that it needed to load this component to bootstrap the app. Similar to a module, component attributes are defined using a class decorator—**@Component**. We import the **Component** decorator from the **@angular/core** package in this example. We then use the **@Component** decorator to define our **AppComponent**. Following are the properties we see above:

- **selector**: This represents the CSS selector that identifies the component that will render the component on the browser. If you recall, the **index.html** file has an HTML tag **<app-root></app-root>**. The same is mentioned in the **selector** property of the **AppComponent**. This is how the Angular runtime will know to render the **AppComponent** on the **<app-root>** HTML tag. In general, there are no restrictions on naming a **selector**. There is only one primary condition that is, it needs to be composed of at least two words separated by a hyphen.
- **templateUrl**: This identifies the HTML file that defines the view template. Usually, it will be the HTML file within the same component folder.
- **styleUrls**: This is an array of CSS/SCSS/SASS/Less files that defines styling for the component. During the build process, this property is used to bundle the CSS files associated with this component into a **styles.*.css** file.

Apart from the above list, many other properties of the **@Component()** decorator are used to define a component. To explore an exhaustive list of such properties, please refer to the Angular official documentation link in the *Reference* section.

The Angular runtime creates data binding between the component class and the template HTML. To demo this, let us add the store name as a class property in the **app.component.ts** file:

```
1. export class AppComponent {
2.  storeName = 'The Fruit Store';
3. }
```

Now, using string interpolation, use the property **storeName** in the view template file:

```
1. <div>Store name: {{ storeName }}</div>
```

This would show the name on the browser, as shown:

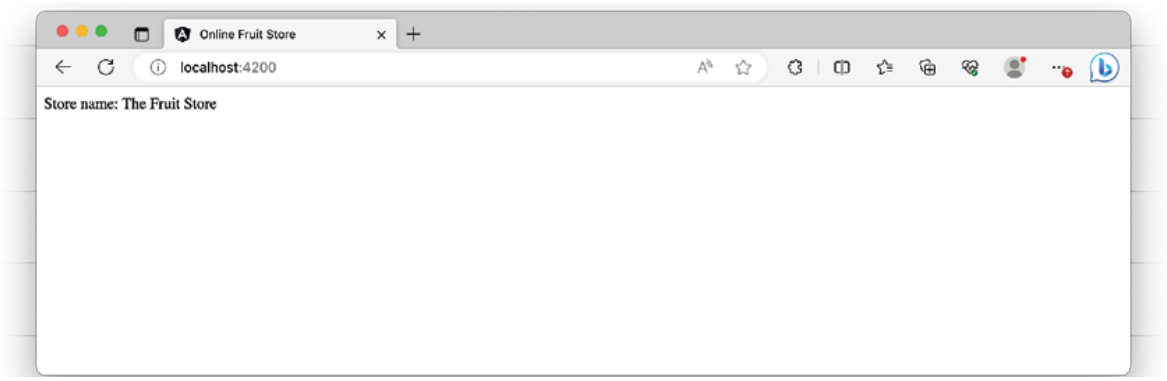


***Figure 4.7**: Data binding in view template*

Similarly, you may bind a method call to the template. The below component and template code changes would yield the same result as above:

```
1. export class AppComponent {
2.  storeName = 'The Fruit Store';
3.
4.  getStoreName() {
5.  return this.storeName;
6.  }
7. }
```

```
1. <div>Store name: {{ getStoreName() }}</div>
```

A component can have properties, methods, inputs, outputs, and lifecycle hooks. We will explore lifecycle hooks in *Chapter 8, Lifecycle Hooks* and inputs and outputs in *Chapter 9, Data Binding and Events Handling*.

- **app.component.css**: This file defines the style statements for the component. To see this in action, add **class** attributes to HTML tags in the view template file and add corresponding style statements in **app.component.css** file as follows:

```
1. <div class="title">Store name: <span class="store-name">{{
   getStoreName() }}</span></div>
```

```
1. .title {
2.  text-align: center;
3.  font-size: 24px;
4.  font-weight: 600;
```

```
}

.store-name {
 color: blue;
}
```

The browser page should auto-refresh to show the new font size, text alignment and font color, as shown in the following figure:

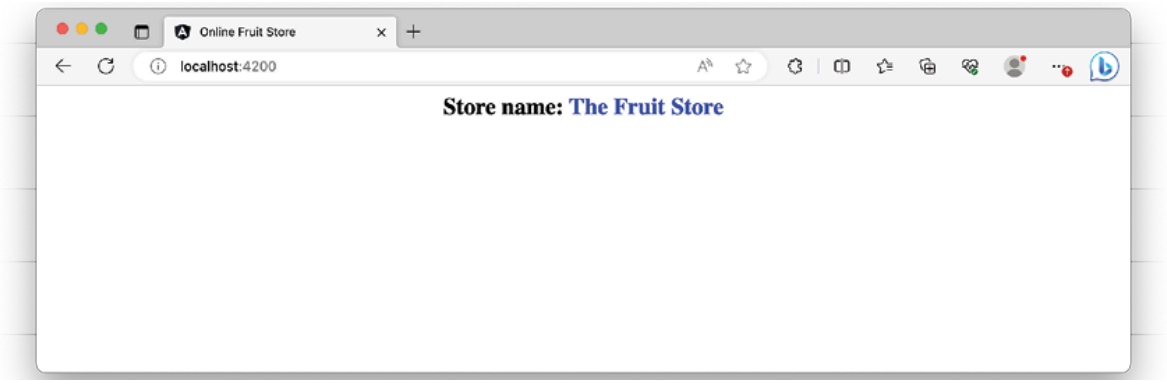


***Figure 4.8**: Styling using CSS*

- **app.component.spec.ts**: This file contains unit tests for the component. Writing tests for all code paths ensures that the code will behave as expected in production. Angular CLI downloads and installs all dependencies to test your app using the Jasmine testing framework and Karma test runner.

  Add two tests to the **app.component.spec.ts** file—one to check whether the component loads successfully and another one to test the store name we added:

```
import { TestBed } from '@angular/core/testing';
import { AppComponent } from './app.component';

describe('AppComponent', () => {
 beforeEach(() => TestBed.configureTestingModule({
 declarations: [AppComponent]
 }));

 it('should create the app', () => {
 const fixture = TestBed.createComponent(AppComponent);
 const app = fixture.componentInstance;
 expect(app).toBeTruthy();
```

```
13. });
14.
15. it(`should have as storeName 'The Fruit Store'`, () => {
16. const fixture = TestBed.createComponent(AppComponent);
17. const app = fixture.componentInstance;
18. expect(app.storeName).toEqual('The Fruit Store');
19. });
20. });
```

To run all the unit tests in your app, execute this command:

ng test

This would provide an excellent test report in the browser and auto-refresh as you modify your code, as shown:

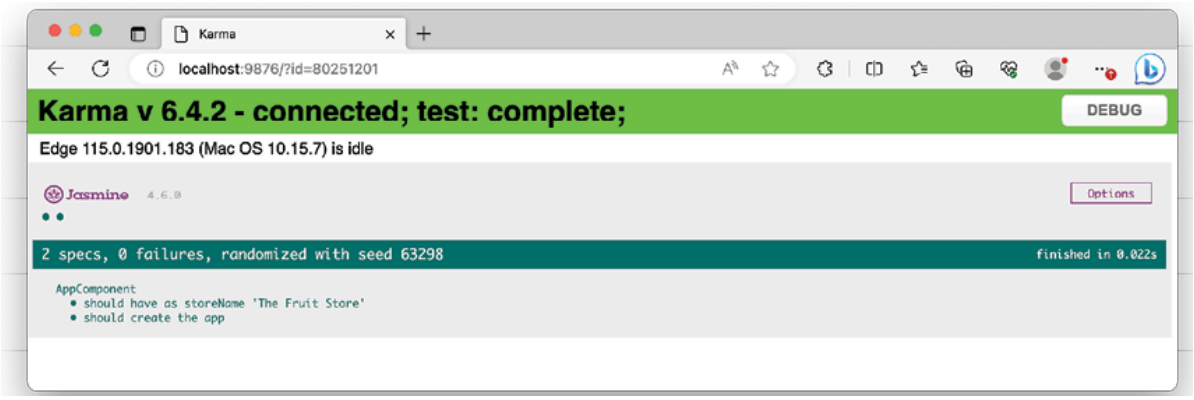


Figure 4.9: *Test report*

We will explore tests in depth in *Chapter 15, Testing*.

Working with multiple components

You can use Angular CLI to generate new components. Let us add a **home-page** component to display available products. Run the following command to generate a **home-page**:

ng generate component home-page

You will get the following output:

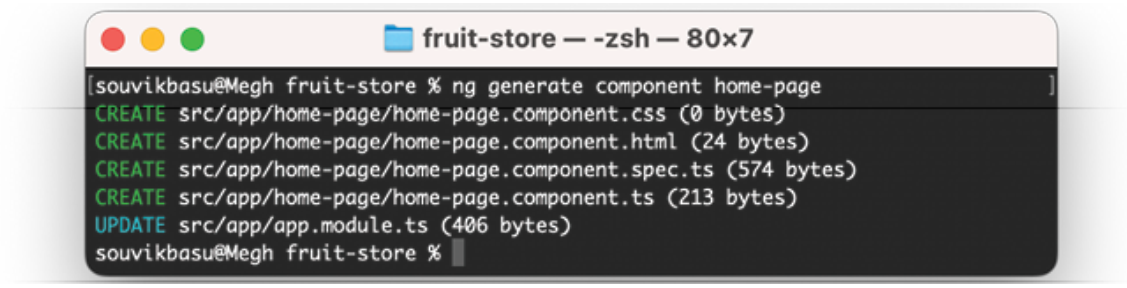


Figure 4.10: *Generating home-page component*

This adds a **home-page** component folder under the **app** folder with the same four types of files we discussed earlier:

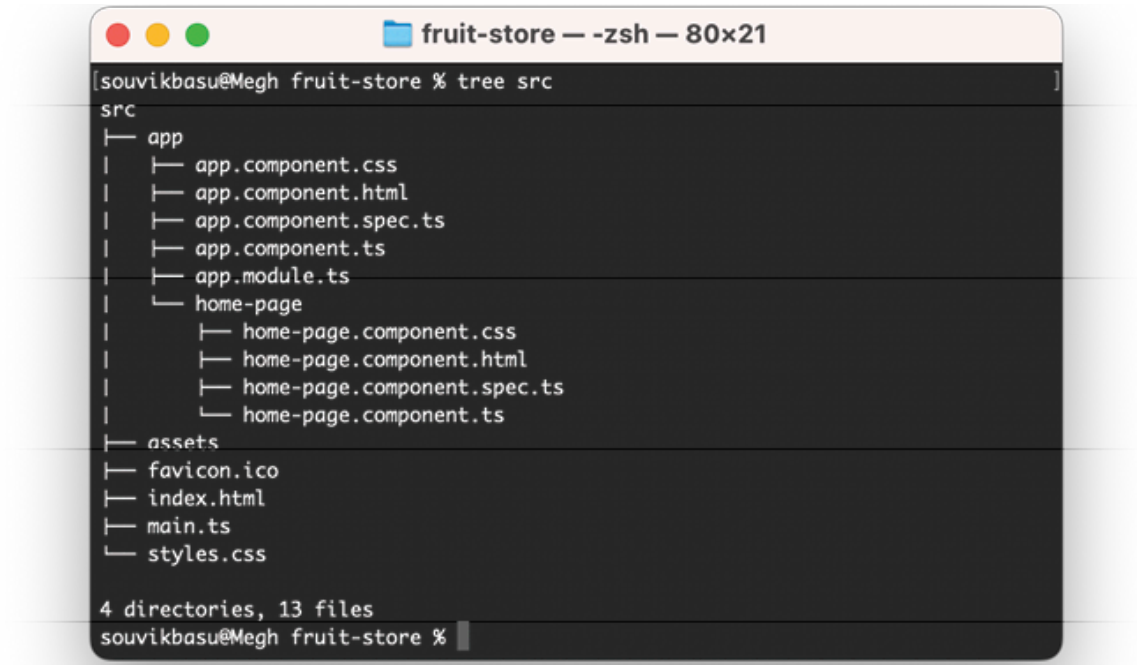


Figure 4.11: *Home-page component files*

You will also notice that the **app** module is modified to add the new component in the declaration property:

```
1  import { NgModule } from '@angular/core';
2  import { BrowserModule } from '@angular/platform-browser';
3
4  import { AppComponent } from './app.component';
5  import { HomePageComponent } from './home-page/home-page.component';
6
7  @NgModule({
8    declarations: [
9      AppComponent,
10     HomePageComponent
11   ],
12   imports: [
13     BrowserModule
14   ],
15   providers: [],
16   bootstrap: [AppComponent]
17 })
18 export class AppModule { }
```

Figure 4.12: *HomePageComponent added to AppModule declarations*

You may create each file manually if you so desire. Angular CLI makes it easier for us by generating the initial files and adding entries in the module declaration.

We will create a blank page with a label as a **home-page** and add a light pink

colored background to the page to be able to identify the corresponding DOM element on the browser:

```
<div id="home_page">Home-page</div>
```

```
#home_page {
    background-color: lightpink;
    width: 100%;
    height: 100vh;
}
```

Use this newly generated component inside the **AppComponent** to show an empty **home-page**:

```
<div class="title">Store name:
    <span class="store-name">{{ getStoreName() }}</span>
</div>
<app-home-page></app-home-page>
```

This should auto-refresh the page to show the newly created home-page, as shown in the following screenshot:

***Figure 4.13**: Showing home-page with a light pink background color*

This was a very simple demonstration of composing one component inside another. The same technique is used to create a tree of components under the app.

Services

Services are a way to share code across different parts of your app. They provide a common set of functionalities that multiple components, directives, or other services can use. A service can be injected into a component or another service using dependency injection, a technique that lets Angular provide instances of services when needed.

A component handles the user experience side of the app. Its responsibilities are to

manage app logic, render the view, and handle user interaction on the view. Tasks like making calls to the backend API, validating user input, or cross-cutting concerns like logging error messages are generally handled by services. This allows for a clear separation of concern. It also allows reusing common functionality across components using services.

To see services in action, let us add a **ProductService** to our app using Angular CLI:

ng generate service product

This creates two files: The **service** class file and a **spec** file to test the service:

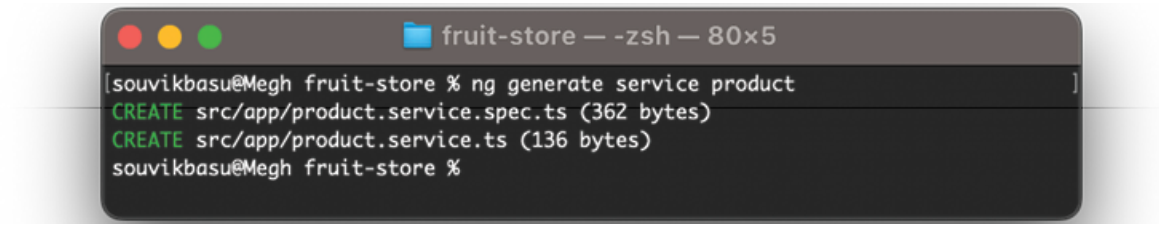


***Figure 4.14**: Generating a service*

As we saw for the modules and components, the service attributes are also defined using a class decorator:

```
1. import { Injectable } from '@angular/core';
2.
3. @Injectable({
4.   providedIn: 'root'
5. })
6. export class ProductService {
7.
8.   constructor() { }
9. }
```

To understand **@Injectable** decorator, let us go over the concept of dependency injection.

Dependency injection is a design pattern that allows components and services to depend on other objects or services without creating them directly. This means that instead of a component or service creating its own dependencies, it relies on the Angular dependency injection system to provide the necessary dependencies at runtime.

To inject a service in Angular, you must first register its provider with the dependency injection system. In the code above, the **@Injectable** decorator tells Angular dependency injection that this service class is available as a provider and can be injected into other components and services. The property **providedIn: 'root'** tells the injector that this service is available at the module level. When the

app bootstraps, Angular creates an injector that maintains a container to keep instances of such services. Once the service provider is registered, you can inject an instance of the service into any component or service that requires it by adding it as a parameter in the **constructor** function. When a component **constructor** is initialized, the injector looks at all parameters in the constructor to check if there is any parameter whose class is decorated with **@Injectable**. If there is, it looks for an instance in its container. If an instance exists, it injects that instance into the constructor parameter. If not, it creates an instance, keeps it in the container, and passes a reference to the instance in the constructor param. Once all dependencies are resolved, the component/service class is initialized.

Now let us add a property, called **products** to the service with fruit names as dummy values. A real-world case will have a more complex product object, but for now, we are exploring how to pass data from a service to components. Add a **getProducts()** method that would return this list of items:

```
1. import { Injectable } from '@angular/core';
2.
3. @Injectable({
4.   providedIn: 'root'
5. })
6. export class ProductService {
7.   products: string[] = ['Apple', 'Orange', 'Banana'];
8.
9.   getProducts() {
10.     return this.products;
11.   }
12.
13.   constructor() { }
14. }
```

Now, make a call from to this method from the **HomePageComponent** to fetch the items:

```
1. import { Component } from '@angular/core';
2. import { ProductService } from '../product.service';
3.
4. @Component({
5.   selector: 'app-home-page',
6.   templateUrl: './home-page.component.html',
7.   styleUrls: ['./home-page.component.css']
8. })
```

```
9.  export class HomePageComponent {
10.   products: string[];
11.
12.   constructor(productService: ProductService) {
13.     this.products = productService.getProducts();
14.   }
15. }
```

Change the view template of the **HomePageComponent** to render the items list:

```
1. <div id="home_page">
2.     <div>Product list
3.         <ul>
4.             <li *ngFor="let product of products">
5.                 {{ product }}
6.             </li>
7.         </ul>
8.     </div>
9. </div>
```

***ngFor** is a directive to loop through the **products**. We will look at it in the next section.

These items will now show on the cart component:

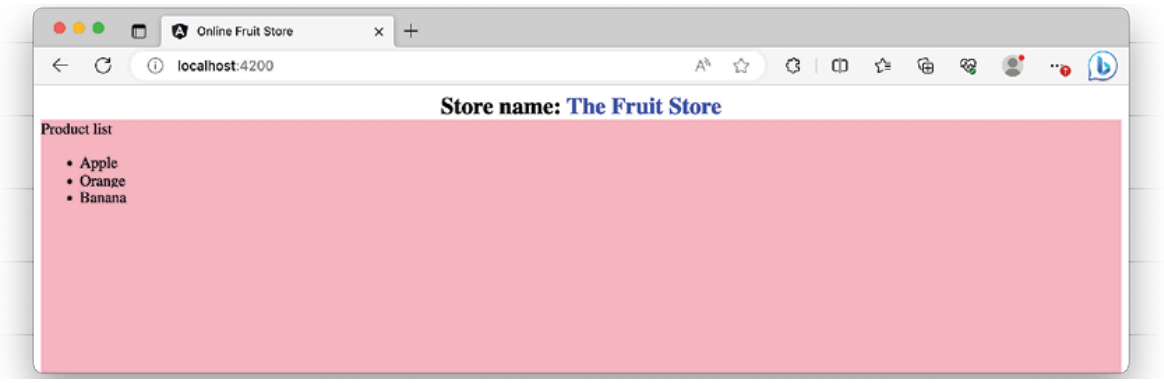


***Figure 4.15:** Show list of products*

The above code is a simple example of how a component can use a service to fetch data. The prime responsibility of a component is to render data on the page. For that purpose, it uses a service to get the actual data. Similarly, the **ProductService** only bothers itself with how to return data without considering how it will be used or rendered. Thus, the separation of concern is achieved.

Remember to add tests in **product.service.spec.ts** to verify that the **getProducts()** method returns the fruits array. This ensures that if the array is modified, our test will fail, and we will then need to take corrective actions or fix the test code to pass

the test:

```
1. import { TestBed } from '@angular/core/testing';
2.
3. import { ProductService } from './product.service';
4.
5. describe('ProductService', () => {
6.   let service: ProductService;
7.
8.   beforeEach(() => {
9.     TestBed.configureTestingModule({});
10.     service = TestBed.inject(ProductService);
11.   });
12.
13.   it('should be created', () => {
14.     expect(service).toBeTruthy();
15.   });
16.
17.   it('should return the list of products', () => {
18.  expect(service.getProducts()).toEqual(['Apple', 'Orange', 'Banana']);
19.   });
20. });
```

If your **ng test** is running, this test will show up in the test report:

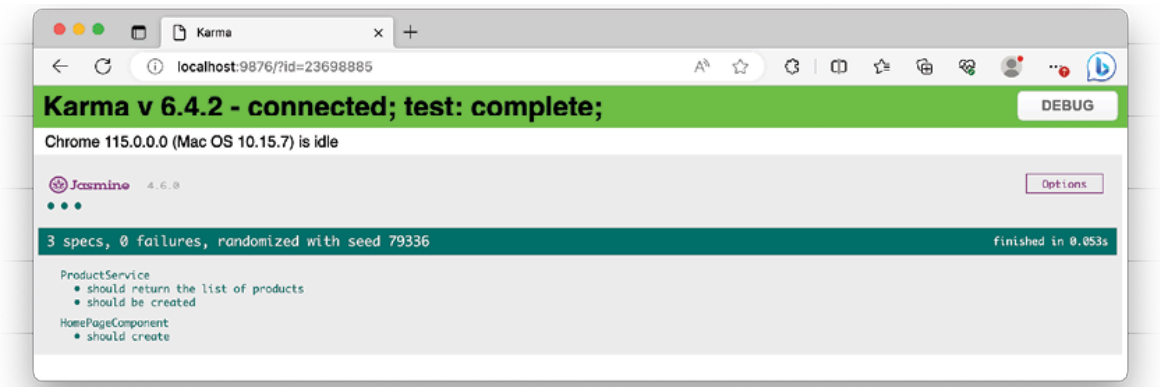


***Figure 4.16**: Test report showing service tests*

The **ProductService** may return an array of typed **Product** items instead of simple strings. To achieve this, we will first define the **Product** type. Let us create a **models** folder under the **src/app** folder and add a TypeScript class **Product**. Add product **name** and **price** properties:

```
1. export class Product {
```

```
    name: string;
    price: number;

    constructor(name: string, price: number) {
        this.name = name;
        this.price = price;
    }
}
```

Change the **ProductService** to use this class instead of string **product** names:

```
import { Injectable } from '@angular/core';
import { Product } from './models/Product';

@Injectable({
  providedIn: 'root'
})
export class ProductService {
  products: Product[] = [
    new Product('Apple', 6.99),
    new Product('Orange', 5.99),
    new Product('Banana', 3.99),
  ];

  getProducts() {
    return this.products;
  }

  constructor() { }
}
```

You might see a compile error now since the **home-page** is expecting an array of strings, as shown in the following figure:

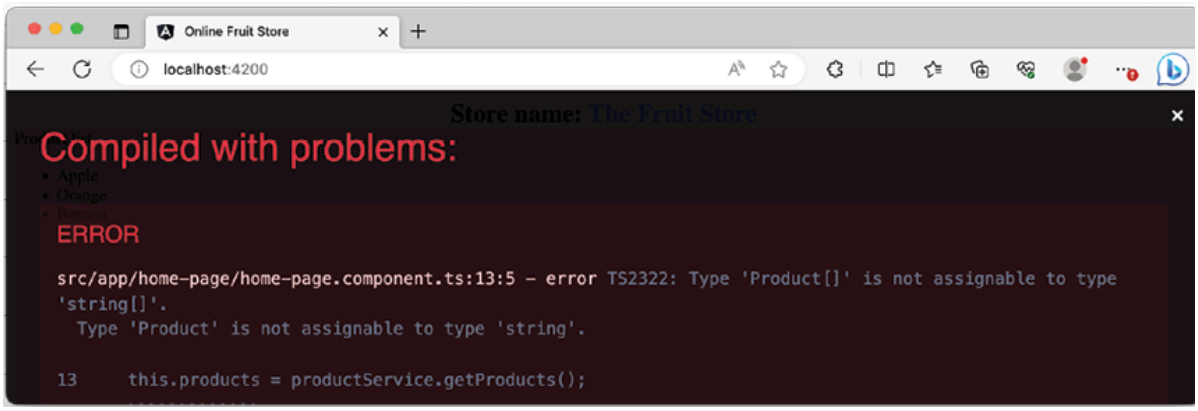


Figure 4.17: *Compile error in home-page*

Fix the **HomePageComponent** to consume the **Product** class and show the **product.name** and **product.price**:

```
export class HomePageComponent {
  products: Product[];

  constructor(productService: ProductService) {
    this.products = productService.getProducts();
  }
}
```

```
      <ul>
        <li *ngFor="let product of products">
          {{ product.name }} {{ product.price }}
        </li>
      </ul>
```

The product name and price would now show on the browser tab:

Figure 4.18: *Home-page showing product name and price*

If you rerun your tests, there will be compilation error in tests as the test is still checking for an array of strings, as displayed:

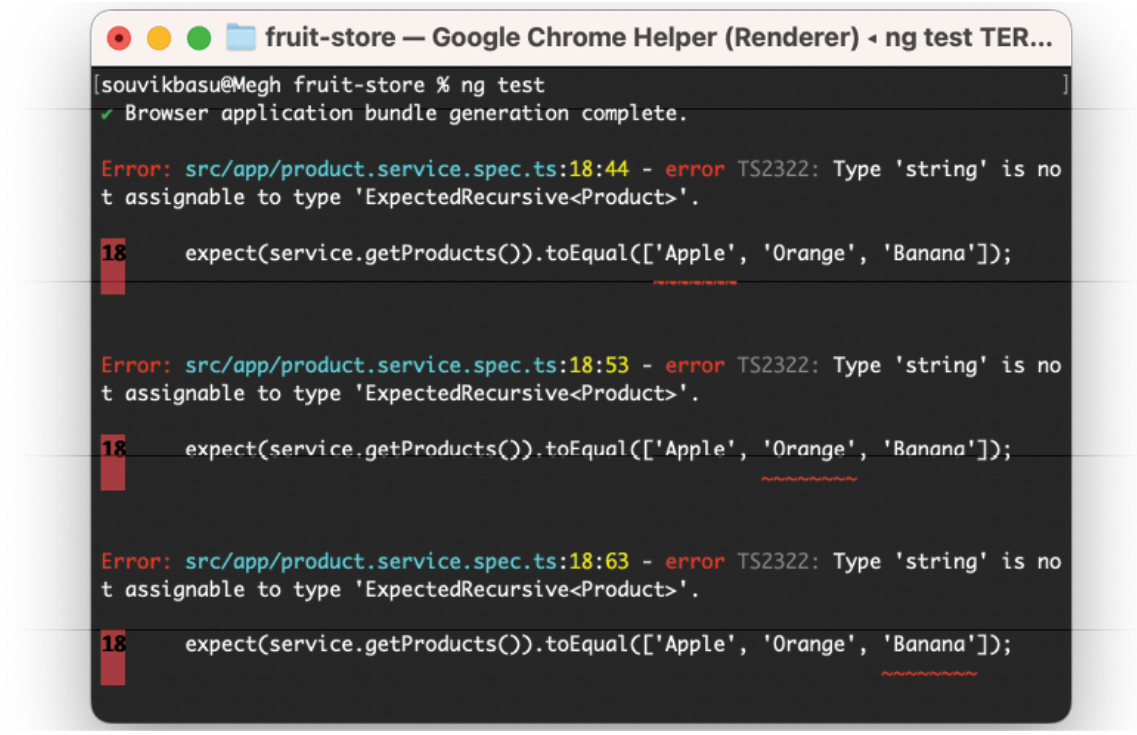


***Figure 4.19**: Error in tests*

Fix the service test by changing the type of expected array being compared:

```
1.  it('should return the list of products', () => {
2.    expect(service.getProducts()).toEqual([
3.      new Product('Apple', 6.99),
4.      new Product('Orange', 5.99),
5.      new Product('Banana', 3.99),
6.    ]);
7.  });
```

Rerun the tests. This time all tests should pass.

To demonstrate how a service works, we returned a hard-coded array of products from the service. A more realistic example of a service may be one that fetches this list from a backend API server. We will explore this in *Chapter 13, API calls*.

A service may consume other services to distribute responsibilities. For instance, if there are multiple services making backend calls, we can move the common functionality to make backend calls into a BackendService. A good example will be when a network call to the backend API server fails; the BackendService can handle the error by logging the error into the console. Having this common code in a BackendService helps avoid code duplication in all other services that want to fetch data from the API server and need to handle error case. Another example of a cross-cutting service is a LoggerService to log error messages to the console. This ensures that at a later point, if you want to send log messages to a log server rather than just logging in console, the new change will be limited to the LoggerService, and its consumers will not be impacted.

Module level versus component level services

In the previous section, we saw an example service that looked like this:

```
@Injectable({
  providedIn: 'root'
})
export class ProductService {
...
}
```

The **@Injectable** property **providedIn: 'root'** tells the injector to create a single instance of the service at the module level. This is the default behavior. The same could be achieved by removing the entry from the service file:

```
@Injectable()
export class ProductService { ... }
```

Adding the service provider name in the **providers** property in the module:

```
@NgModule({
  declarations: [
    AppComponent,
    HomePageComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [ProductService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

To test that a single instance of the service is created, add a method **addProduct()** to the **ProductService**:

```
  addProduct(product: Product) {
    this.products.push(product);
  }
```

Call this method from **HomePageComponent**:

```
export class HomePageComponent {
  products: Product[];

  constructor(productService: ProductService) {
    productService.addProduct(new Product('Watermelon', 2.99));
```

```
    this.products = productService.getProducts();
  }
}
```

Notice that the newly added fruit shows up on the page, as shown:

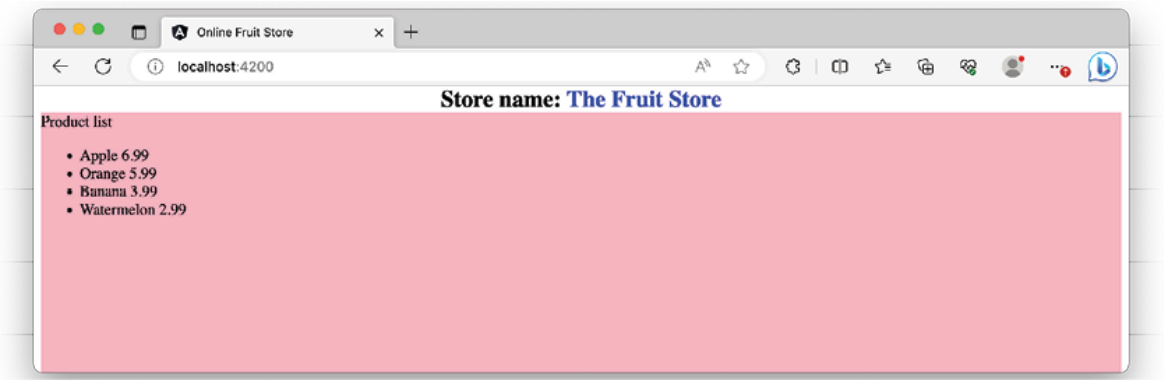


Figure 4.20: *Newly added fruit shows up in the list*

Now, add another component **ProductManagerComponent** that depends upon the same service and adds a new fruit **Peach** by calling the **addProduct** method:

```
export class ProductManagerComponent {
  constructor(productService: ProductService) {
    productService.addProduct(new Product('Peach', 6.49));
  }
}
```

This is not an ideal case to call the **addProduct()** method from a **constructor**, but we are calling it from the **constructor** for brevity.

Add this new component in the **AppComponent** so that its **constructor** is called:

```
<div class="title">Store name:
    <span class="store-name">{{ getStoreName() }}</span>
</div>
<app-home-page></app-home-page>
<app-product-manager></app-product-manager>
```

Now if you check the browser, you will notice that **Peach** is added to the list:

Figure 4.21: *Product list in HomePageComponent getting updated*

This means that there is a single instance of the service used by both components.

The other way to manage a service is by adding an entry in the **providers** property of **@Component** decorator. That creates an instance of the service for every instance of the component as well as a new instance for every component that adds this service in the **providers** property. To see this, remove **providers: [ProductService]** property from **app.module.ts** and add the same line in both the **HomePageComponent** and **ProductManagerComponent**. This time you will notice that Peach is not added to the list:

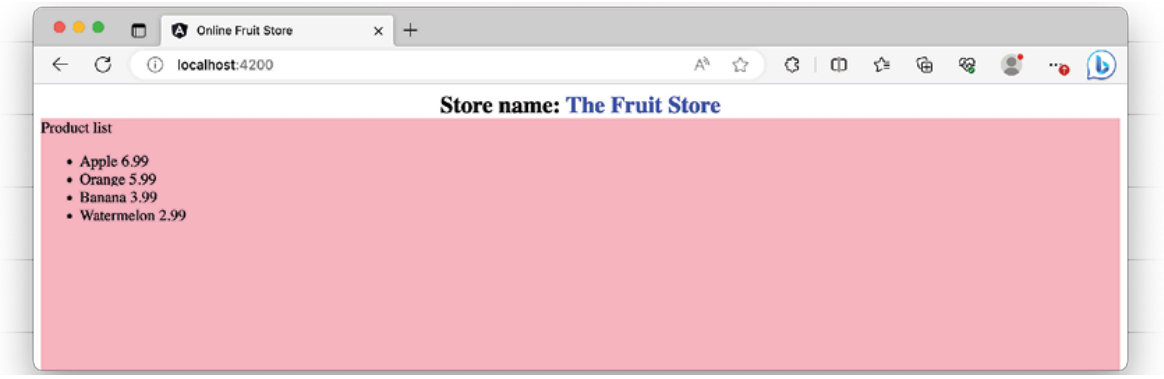


Figure 4.22: *Product list in HomePageComponent does not show Peach*

As an exercise, you can display the product list in **ProductManagerComponent** and verify what you see there.

Directives

A **directive** is a class that can modify the appearance or behavior of an element in the template. Directives are used to extend the functionality of HTML elements. They allow you to add custom behavior to an element or modify its appearance. Directives can be used to create reusable components, add event listeners, or manipulate the DOM.

There are two types of directives: Structural directives and attribute directives. **Structural directives** change the structure of the DOM by adding, removing, or replacing elements. **Attribute directives** change the attributes or styles of an

element.

Angular provides some built-in directives, such as ***ngFor**, ***ngIf**, and **[ngClass]**. The ***ngFor** and ***ngIf** directives are two of the most commonly used structural directives in Angular. The ***ngFor** directive allows you to iterate over an array or an iterable object and create a template for each item. We used this in our **HomePageComponent** to display the list of products:

```
<ul>
    <li *ngFor="let product of products">
        {{ product.name }}  {{ product.price }}
    </li>
</ul>
```

The ***ngFor** directive takes an expression of the form let item of items, where item is a local variable that holds the current item in the iteration, and item is the array or iterable object to iterate over. You can use the item variable to access the properties of each item in the template.

The ***ngIf** directive allows you to conditionally render a template based on a Boolean expression. If we want to handle the case when the products list is empty, we can use ***ngIf** directive as follows:

```
<div *ngIf="products.length === 0">
  No products available.
</div>
```

Both directives are prefixed with an asterisk to indicate that they are structural directives.

The **[ngClass]** directive in Angular is a way to dynamically add or remove CSS classes to an HTML element based on some conditions. You can use this directive to change the appearance or behavior of an element depending on the state of your component or app. You can pass an object that has CSS classes as its keys and Boolean expressions as its values. The directive will add the classes whose values are true and remove the classes whose values are false. In our component, you can add a class **expensive** to product rows where the price is more than **5**:

```
    <ul>
        <li *ngFor="let product of products">
  <div [ngClass]="{'expensive': product.price > 5}" >{{ product.name }} {{ product.price }}</div>
        </li>
    </ul>
```

Then add a CSS statement to highlight elements with **expensive** class names:

```
.expensive {
    color: red;
```

```
    font-weight: bold;
}
```

This should highlight the costly items:

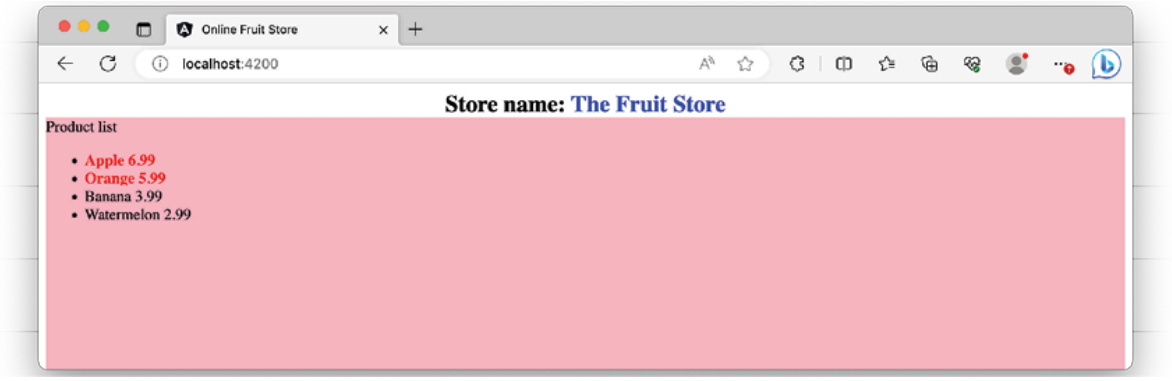


Figure 4.23: *Highlight costly items using [ngClass]*

You can also create your own custom directives. To define a directive, you use the **@Directive** decorator on a class.

Let us add a directive to highlight the price of a product if it is more than 5. Run the following command to create a directive:

ng generate directive highlight-price

This will create a **directive** class file and a test file and modify the **app** modules file, as shown:

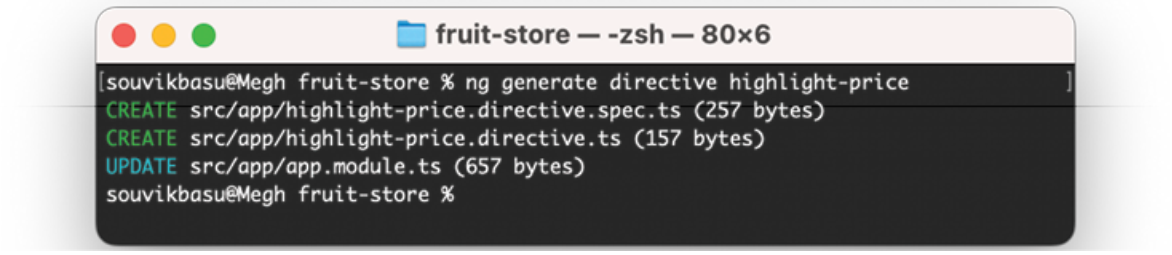


Figure 4.24: *Generating directive*

The **directive** class code generated is decorated with **@Directive** and specifies a **selector** property:

```
import { Directive } from '@angular/core';

@Directive({
  selector: '[appHighlightPrice]'
})
export class HighlightPriceDirective {
  constructor() { }
}
```

The directive name is added in the **declarations** property of the **AppModule**:

```
declarations: [
  AppComponent,
  HomePageComponent,
  ProductManagerComponent,
  HighlightPriceDirective
],
```

Let us change the directive code to highlight a price greater than **5**:

```
import { Directive, ElementRef, Input, OnInit } from '@angular/core';

@Directive({
  selector: '[appHighlightPrice]'
})
export class HighlightPriceDirective {
  @Input() appHighlightPrice = 0;

  constructor(private element: ElementRef) {}

  ngOnInit() {
    if (this.appHighlightPrice > 5) {
      this.element.nativeElement.style.backgroundColor = 'yellow';
    }
  }

}
```

Observe the following code change:

- **@Input** decorator is used on a class property when we want to receive a value for the property from the parent element using this directive.
- The element on which this directive is applied is sent as a param in the constructor so that the directive can make modifications to the element as needed.
- **ngOnInit** is a lifecycle hook called when Angular initiates the component. We saw earlier that the Angular dependency injection tries to inject all constructor parameters into a class. Once all the constructor parameters are injected, Angular initiates the constructor. Once the object is initiated, the **ngOnInit** lifecycle hook is called. This is a safe place to make any DOM manipulation. We will look at lifecycle hooks in depth in *Chapter 8, Lifecycle Hooks*. In the above code, the directive checks if the value of the **appHighlightPrice** property is more than **5**. If so, it styles the element with yellow background

color to highlight it.

Use this directive in the **home-page.component.ts** file as follows:

```
1.        <ul>
2.            <li *ngFor="let product of products">
3.  {{ product.name }} <span [appHighlightPrice]=product.price >{{
    product.price }}</span>
4.            </li>
5.        </ul>
```

Since the directive accepts a number, it is placed inside square brackets. This is called **property binding**. We will look at different types of data binding in *Chapter 9, Data Binding and Events Handling.*

This will highlight the pricey items on the browser:

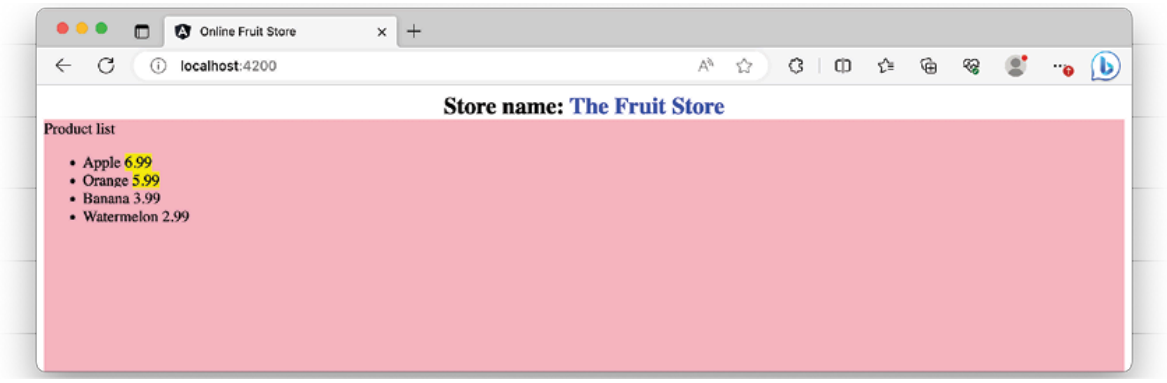


***Figure 4.25**: Highlight pricey items using directive*

If you run **ng test**, you will observe that the directive test fail now as the directive constructor now accepts one argument. Fix the test to pass an element as an argument:

```
1. describe('HighlightPriceDirective', () => {
2.   it('should create an instance', () => {
3.  const directive = new HighlightPriceDirective(new ElementRef(null));
4.     expect(directive).toBeTruthy();
5.   });
6. });
```

In *Chapter 15, Testing,* we will explore how to add tests for directives that can test its functionality.

Directives can also be used to add event handlers. In the above example, if you want to highlight a price only on mouse hover, you can add the following code to the directive:

```
1. export class HighlightPriceDirective {
2.   constructor(private element: ElementRef) {}
```

```
3.
4.   @HostListener('mouseenter') onMouseEnter() {
5.     this.highlight('yellow');
6.   }
7.
8.   @HostListener('mouseleave') onMouseLeave() {
9.     this.highlight(null);
10.   }
11.
12.   private highlight(color: string | null) {
13.     this.element.nativeElement.style.backgroundColor = color;
14.   }
15. }
```

Import the **Directive**, **ElementRef**, and **HostListener** decorators from the **@angular/core** package. Now that we are not comparing against any price, we removed the **@Input** decorator from the code. Similarly, we do not need to assign any value to the directive in its usage:

```
1.       <ul>
2.           <li *ngFor="let product of products">
3. {{ product.name }} <span appHighlightPrice >{{ product.price }}</span>
4.           </li>
5.       </ul>
```

This will show a price highlighted on mouse hover:

Figure 4.26: *Highlight price on mouse hover using directive*

Pipes

A **pipe** is a class that transforms data for display in the template. A pipe can take one or more parameters and return a new value based on some logic or formatting rules. Angular provides some built-in pipes, such as **DatePipe**, **UpperCasePipe**,

LowerCasePipe, **CurrencyPipe**, etc. Let us use the built-in **CurrencyPipe** to show the currency symbol beside the product prices. As the name suggests, you need to add the pipe character (|) inside the expression that displays the price property followed by the pipe's name, currency in this case:

```
<div id="home_page">
    <div>Product list
        <ul>
            <li *ngFor="let product of products">
 {{ product.name }} <span>{{ product.price | currency }}</span>
            </li>
        </ul>
    </div>
</div>
```

Notice the **$** symbol added before the price on the browser window:

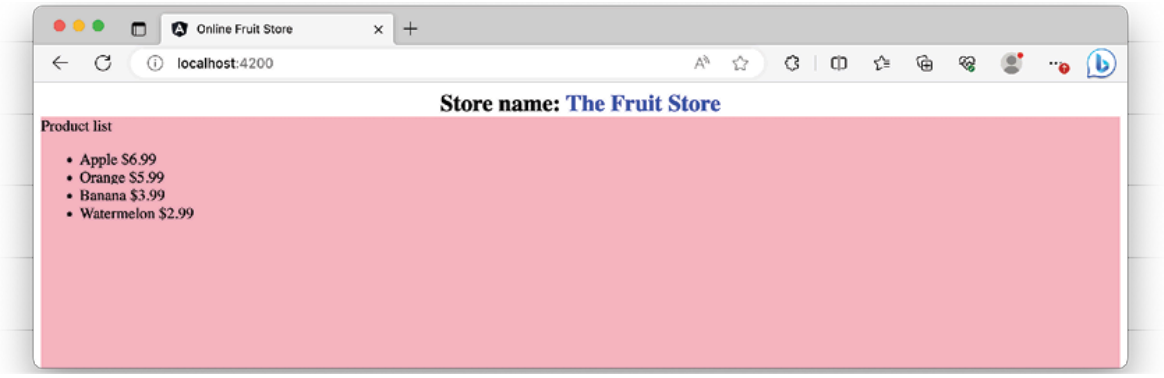


***Figure 4.27**: Show currency symbol in price*

A pipe can accept parameters to tweak it. For instance, the currency pipe we used above accepts three parameters:

- **currencyCode**: This is the code of the currency identified in *ISO 4217* (See the *Reference* section for the link). It allows us to identify the currency.
- **symbolDisplay**: This is a string field with **code**, **symbol** or **symbol-narrow** as possible values to show the currency. For the **Indian Rupee** (**INR**), the **code** will be INR, and the **symbol** will be ₹. Few currencies, like the Canadian Dollar, have more than one character in the symbol—CA$. For such currencies, **symbol-narrow** will return a single character like $.
- **digitInfo**: This determines how many digits to show, as explained below.

These parameters are positional, that is, we do not specify the name of the parameter but provide the values in order. If you want to show prices in INR, want to show the Rupee symbol instead of the code INR, and ignore the decimals, the above code can be written as follows:

```
<span>{{ product.price | currency:'INR':'symbol':'1.0-0' }}</span>
```

The param **digitInfo** is specified using three parts. **'1.0-0'** denotes:

- Shows minimum **1** integer digit.
- Shows a minimum of **0** digits after the decimal point.
- Shows a maximum of **0** digits after the decimal point.

The page will now look like this:

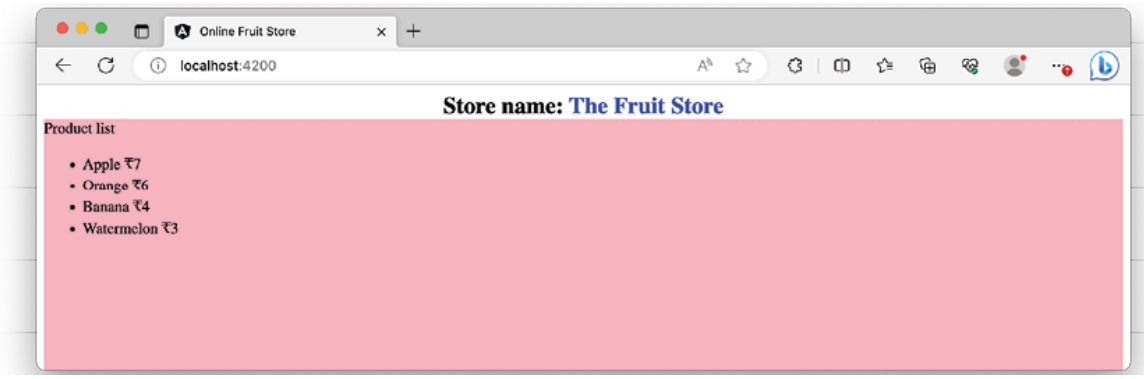


***Figure 4.28**: Show price in INR symbol*

Observe how 6.99 is now rounded to 7 with a Rupee symbol.

You can also create your own custom pipes. To define a pipe, you use the **@Pipe** decorator on a class and implement the **PipeTransform** interface. Say, in the above example, you want to convert USD 6.99 to INR. We will use a hard-coded conversion factor for brevity. In a real-world app, this conversion factor is dynamic and can be retrieved from financial institution APIs.

Run the following Angular CLI command to create a **convert-to-inr** pipe, and refer to *Figure 4.29* for the result:

ng generate pipe convert-to-inr

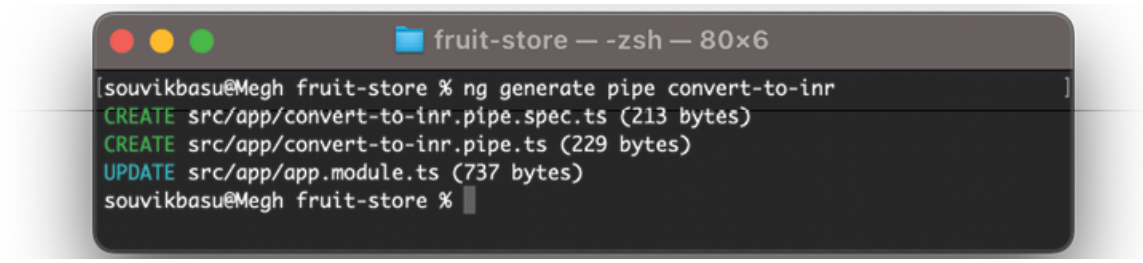


***Figure 4.29:** Generating pipe*

You will find a **pipe** class and test file created like what we saw before. Also, the **app** module is modified to add the pipe in the **declarations** property. Open the **pipe** class and add the following code to convert the price from USD to INR (this is an approximate conversion for demo purposes):

```
1. import { Pipe, PipeTransform } from '@angular/core';
2.
3. @Pipe({
```

```
4.   name: 'convertToInr'
5. })
6. export class ConvertToInrPipe implements PipeTransform {
7.
8.   conversionFactor = 83.01;
9.   transform(value: number): number {
10.     return value * this.conversionFactor;
11.   }
12.
13. }
```

We import the **Pipe** and **PipeTransform** decorators from the **@angular/core** package in this example. We then use the **@Pipe** decorator to define our **ConvertToInr**. We specify the name of our pipe using the **name** property. We also define a class for our pipe, which implements the **PipeTransform** interface. The **PipeTransform** interface requires us to implement a transform method, which is called when our pipe is used. In this example, our transform method takes a number as input (price in USD) and returns the converted value in INR.

Use this pipe in **HomePageComponent** as follows:

```
1. <span>{{ product.price | convertToInr }}</span>
```

The page should now show the converted price in INR:

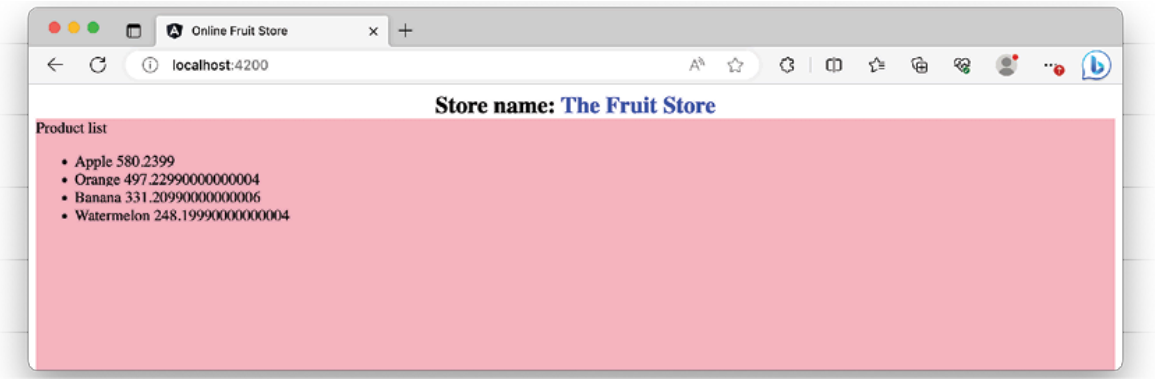


Figure 4.30: *Use custom pipe*

Using a digits param, we can restrict the number of decimal digits to show. This is to demo how a pipe can accept arguments:

```
1.   transform(value: number, digits: number): number {
2.     const convertedValue = value * this.conversionFactor;
3.     return parseFloat(convertedValue.toFixed(digits));
4.   }
```

Add the parameter value in the view template:

```
1. <span>{{ product.price | convertToInr:2 }}</span>
```

This will show two decimal digits on the browser:

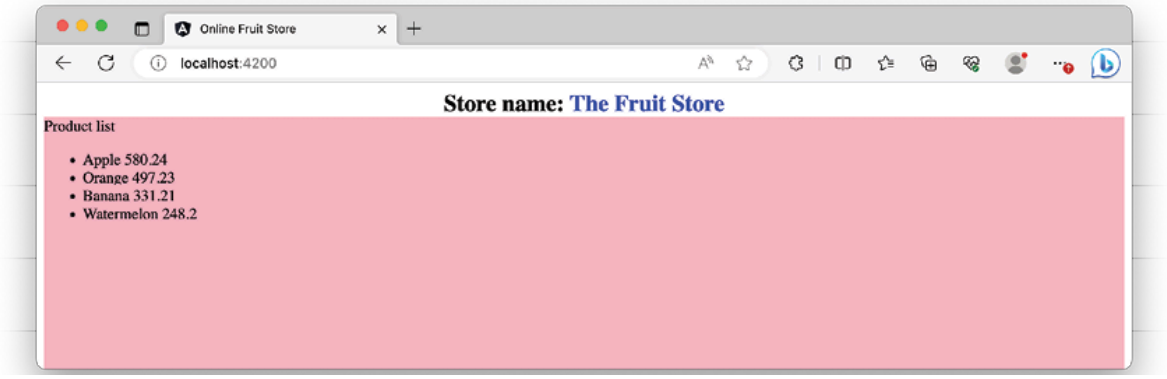


Figure 4.31: *Show 2 digits using pipe parameter*

You may also chain multiple pipes one after the other. Say, you want to show the Rupee symbol, you may add the built-in **CurrencyPipe** to the view template:

```
1. <span>{{ product.price | convertToInr:2 | currency:'INR': 'symbol' }}</span>
```

This should now show the Rupee symbol:

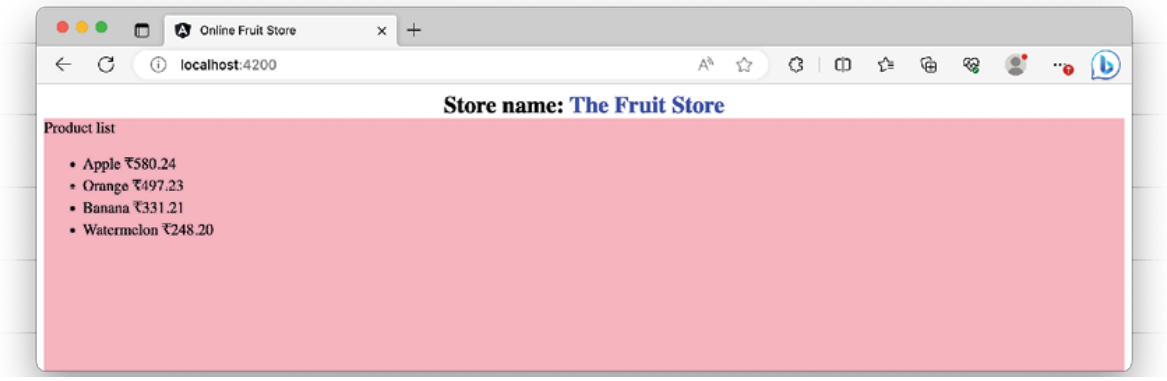


Figure 4.32*: Chain multiple pipes*

Decorators

A decorator is a function that adds metadata to a class, property, method, parameter, or argument list. Decorators are prefixed with an @ sign and are placed immediately before the code they apply to. We already used decorators in this chapter to define a module, component, service, directive, and pipe. Apart from these, Angular uses decorators to provide additional information, such as selectors, templates, providers, etc. Some examples of decorators are **@NgModule, @Component, @Injectable, @Directive, @Pipe, @Input, @Output**, etc. We will explore **@Input** and **@Output** decorators in *Chapter 9, Data Binding and Events Handling*.

You may also create custom decorators. You use the **@decoratorName** syntax to define a decorator. Create a folder **decorators** under **src/app** folder and add a file **log.decorator.ts**:

```
export function Log(target: any, name: string, descriptor: PropertyDescriptor) {
    const original = descriptor.value;

    descriptor.value = function (...args: any[]) {
        console.log(`Method ${name} called with args `, args);
        const result = original.apply(this, args);
        console.log(`Method ${name} returns `, result);
        return result;
    }

    return descriptor;
}
```

The **target** param represents the class containing the method we are decorating. Parameter **name** is the method name on which the decorator is applied, and **descriptor** is the actual method. The decorator adds console log messages before and after the method call for better debugging. The above decorator can be used on any method as follows:

```
@Log
getProducts() {
  return this.products;
}
```

Open dev tool on your browser to see the **Console** log messages:

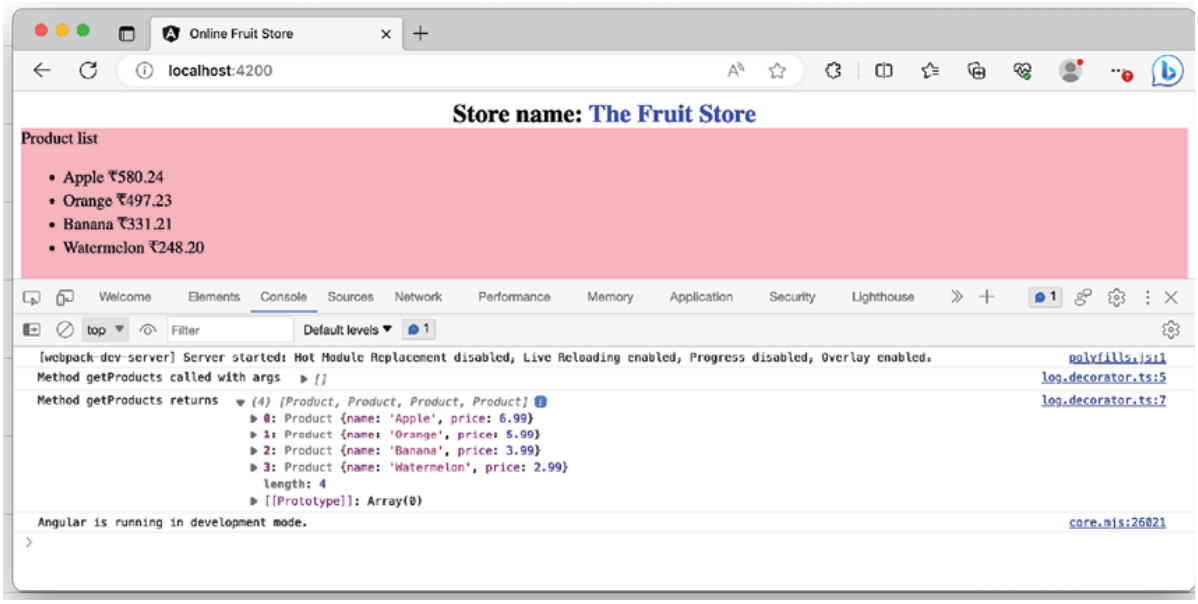


***Figure 4.33**: Custom decorator to show console log messages*

Importing modules

In the first section of this chapter, we introduced the concept of a module and explained **AppModule**. After adding the **HomePageComponent**, if you ran the **ng test**, you would have observed that the test failed with an error message as follows:

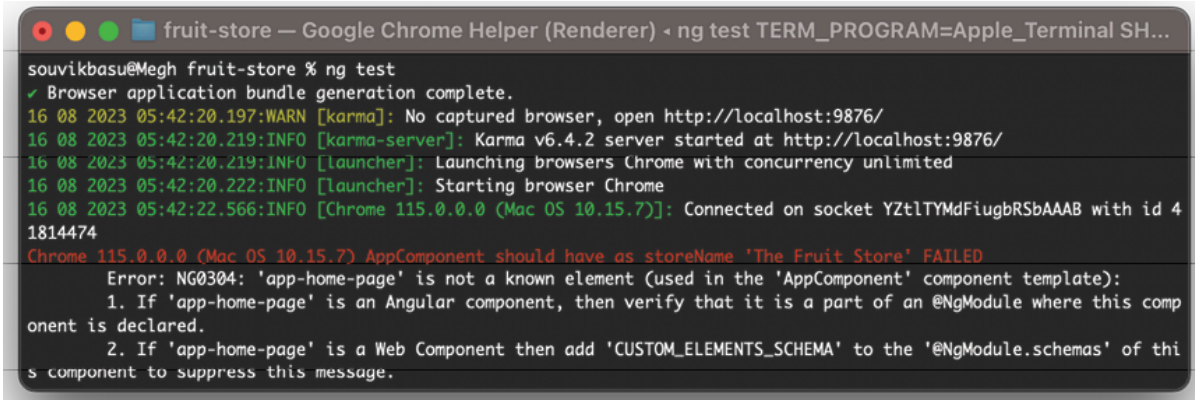


***Figure 4.34**: Test error*

This error occurs because Angular test expects the component to be exported from a module that it can import. Since the component is declared in the **root** module, it is difficult for the test runner to import it. A typical large-scale Angular app will have multiple modules. In our code, we can create a **ProductModule** that would contain all components, services, models, etc., related to showing the list of products and product details on the app. To create a module, run the following Angular CLI command:

ng generate module product

This will create a module file **products.module.ts**. as shown:

***Figure 4.35**: Generating module*

You can move **HomePageComponent**, **ProductManagerComponent**, **ProductService**, and **Product** model inside this module and export these as members. A general practice is to move all feature module members inside the module folder. In this case, it is the **src/app/product** folder. Move the above items inside this folder and update references to these members in import statements elsewhere in the app. Similarly, create a **shared** module to keep directives and pipes that may be shared across the modules.

The new folder structure looks like this:

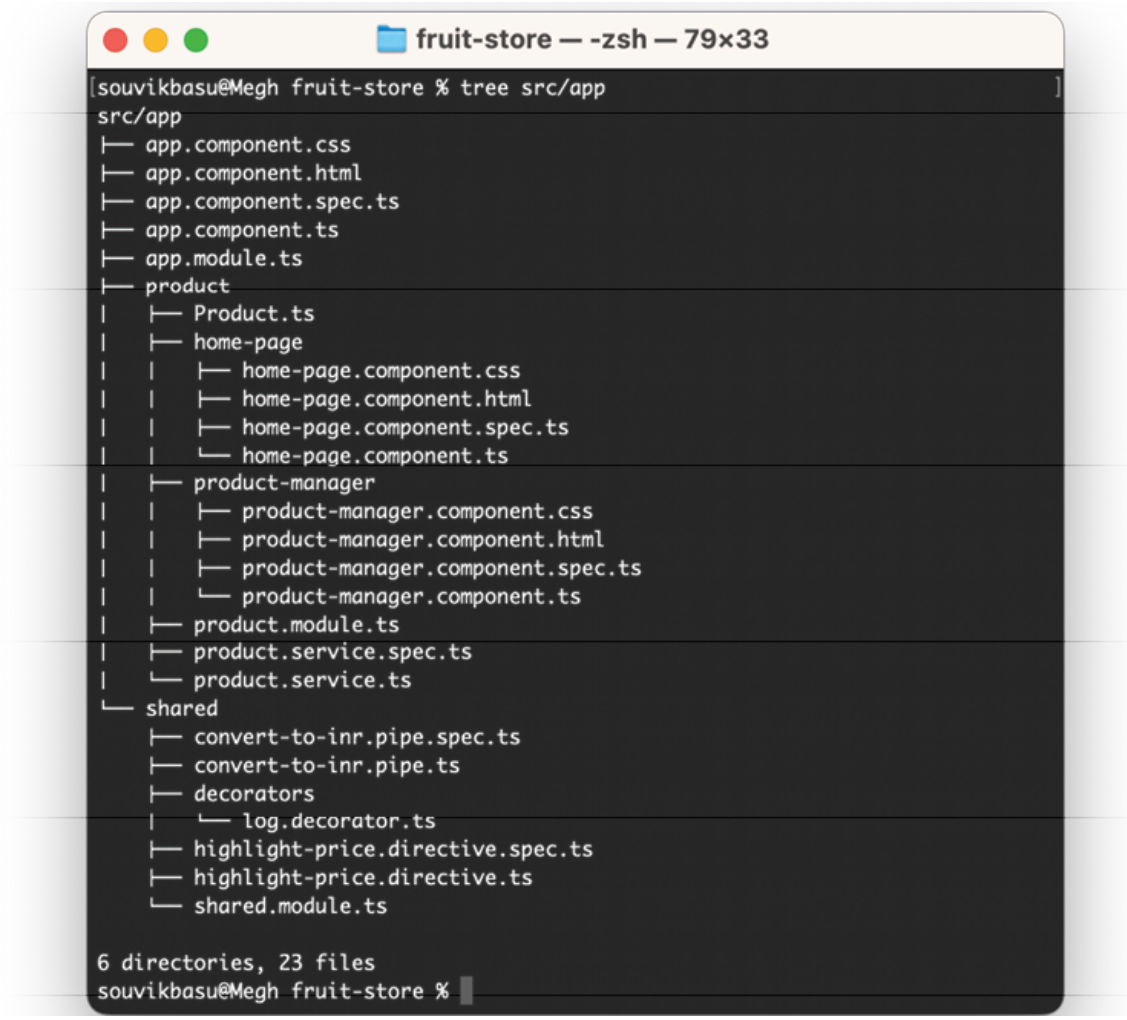


Figure 4.36: *New folder structure with feature modules*

shared.module.ts file exports its members in the module file:

```
1. @NgModule({
2.   declarations: [
3.     HighlightPriceDirective,
4.     ConvertToInrPipe
5.   ],
6.   imports: [
7.     CommonModule
8.   ],
9.   exports: [
10.     HighlightPriceDirective,
11.     ConvertToInrPipe
12.   ]
13. })
14. export class SharedModule { }
```

These shared items are used in the feature module file **product.module.ts** which, in turn **exports** its own members:

```
@NgModule({
  declarations: [
    HomePageComponent,
    ProductManagerComponent,
  ],
  imports: [
    CommonModule,
    SharedModule
  ],
  exports: [
    HomePageComponent,
    ProductManagerComponent
  ]
})
export class ProductModule { }
```

The **root** module file **app.module.ts** imports other feature modules:

```
@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    ProductModule
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

You will notice that the feature modules import **CommonModule** while the **root** module imports **BrowserModule**. Directives like ***ngFor**, ***ngIf** are available in the **CommonModule**. **BrowserModule** can only be imported into the **root** module to render the app on the browser.

Organizing code

Dividing an app into multiple modules helps to organize the app source code. But within a module as well we need to organize the code well so that it is easier to

manage and maintain the code. A detailed discussion on code organization is out of the scope of this book, but we will provide some hints around how you can go about it. There are many ways to organize your code which vastly depends upon your app. In general, the popular approach is to divide the app into folders per module and create folder for **components**, **services**, **directives** and **pipes**. To visualize this, our app code can look like this:

***Figure 4.37**: Organizing code into meaningful folders*

This is just an indication for a cleaner code structure. You can explore better ways that suits your app needs.

Conclusion

Understanding the concepts of modules, components, services, dependency injection, directives, pipes, and decorators is essential for building robust and

scalable apps in Angular. These concepts help to modularize the code and encourage reusability. Angular does not enforce how a code should be structured but provides these building blocks to promote best coding practices. By mastering these concepts, developers can create powerful and efficient Angular apps that meet the needs of their users.

In the next chapter, we will look at how to navigate between multiple pages in an Angular page using routing mechanisms.

Points to remember

- Modules help organize the app into smaller, reusable, and testable parts.
- Components define the UI and behavior of the app.
- Services provide support for backend API calls, user validation, and cross-cutting concerns.
- Dependency injection helps in providing the required dependencies to the components and services.
- Directives allow developers to add custom behaviors to the DOM.
- Pipes help transform data before displaying it.
- Decorators provide metadata that aids in configuring classes and functions.

Questions

1. Explain the difference between a component and a service.
2. Explain how the Angular dependency injection system works.
3. What are the different types of directives?
4. How can you create a custom pipe in Angular?

References

- Components:
  https://angular.io/api/core/Component
- ISO 4217:
  https://en.wikipedia.org/wiki/ISO_4217

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 5
Routing

Introduction

Routing is an essential feature in any web application framework, and Angular is no exception. It enables users to navigate through the application and view different pages or components. Routing in Angular is a powerful tool that allows for the creation of **single page applications** (**SPAs**) with multiple views. It provides a seamless user experience by dynamically loading content based on the user's actions. With Angular routing capabilities, developers can create complex applications with ease while ensuring a smooth user experience. In this chapter, we will explore the fundamentals of routing in Angular and how it can be used to build dynamic web applications.

Structure

The chapter covers the following topics:

- Routing in traditional websites
- Routing in single page application
- Understanding routing in Angular
- <base href> and routerLink
- Adding routes to existing app
- Passing URL parameters
- Nested routes
- Snapshot versus observables
- Setting up redirects
- Handling page not found
- Showing active routes
- Passing query parameter
- Routing using TypeScript
- Lazy loading and Preloading
- CanActivate, CanActivateChild, CanDeactivate
- Fragments
- Router data
- Router state
- Debugging routes

Objectives

By the end of this chapter, you will understand how routing works in Angular and the various routing types and directives available to provide a seamless user experience.

Routing in traditional websites

Before we look at routing in Angular, let us take a quick look at how routing works in a traditional website. In

a traditional website, routing is usually handled by the server-side framework or the web server itself. The server receives a request from the browser with a URL and then decides the content or function to return based on the URL. Following is an example of a Node.js web server code written using Express.js:

```
const express = require('express'); // Import the express module
const app = express(); // Create an app object

// register the location of the static assets
app.use( express.static( 'public' ));

// Define routes for different URL paths
app.get( "/", ( req, res ) => {
  // Send the HTML file for home page
  res.sendFile( path.join( __dirname + "/public/index.html" ));
});

app.get('/product', (req, res) => {
 res.send('This is the product page'); // Example of a dynamically generated page content
});


app.get('*', (req, res) => {
  res.send('This is the not found page'); // when route is not found
});

// Start the server and listen on port 8000
app.listen(8000, () => {
  console.log('Server is running on port 8000');
});
```

It uses the app object to define different routes with different callback functions based on the path. The callback functions use the **res.sendFile()** and **res.send()** methods to send different content as the response.

Routing in single page application

Routing works differently in a SPA like Angular. In Angular, when the user clicks on a link on a page to visit a different page, the runtime handles the request and renders a new page on the browser itself, without making a round-trip call to the web server. This saves a lot of time and provides a seamless user experience. An example of such an experience is Gmail, wherein once the user clicks on an email subject, the email body is instantly opened on the same page. You will notice that the URL changes when an email body is displayed, but the application does not call the web server to render the page. Hence the name SPA, as the user feels as if they are on the same page even while navigating to different links. A very simplistic way to explain the steps would be as follows:

1. When the browser loads an application page, all the view templates are loaded on the browser. Based on the URL, a particular view is shown to the user.
2. When the user clicks on any navigation link, the runtime figures out which view template to render in which region of the page based on the URL.
3. It then makes a call to the web server (or an API server) to only fetch data to show in the new view to be rendered.
4. Once data is available, it fills the view template with this data and renders it on the page. So once the view templates are fetched and rendered, only data is fetched from a web server in consecutive requests. The initial load of the page takes time to render the view template with data but all further user interactions render data on the page much faster than the time it would take to refresh an entire page on

every user action.

As mentioned, the above description is too simplistic. In a real-world application, more advanced techniques, such as lazy loading and preloading, are used, which further reduces the time to fetch and render the view. We will look at these techniques later in this chapter.

Understanding routing in Angular

Let us now understand how routing works in Angular. We will create an online fruit selling app like in the previous chapter, and add routing to it. Run the following command to create a new **fruit-store** app:

```
ng new fruit-store
```

Press *y* for the option to add routing. Select other default options, as shown:

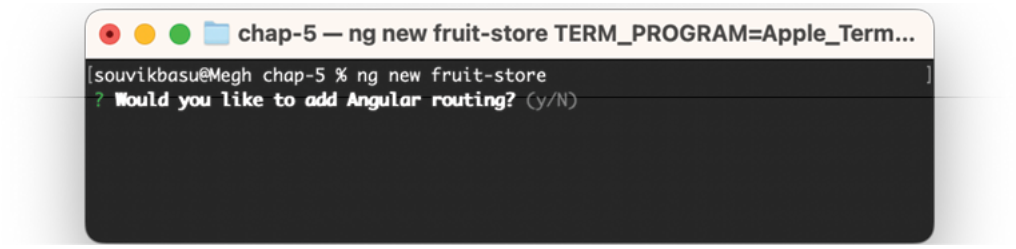


Figure 5.1: *Create a new project with routing*

Open the code in VS Code or in the IDE of your choice and take a look at **app.module.ts** file:

```
1.  ...
2.  import { AppRoutingModule } from './app-routing.module';
3.  ...
4.
5.  @NgModule({
6.    imports: [
7.      ...,
8.      AppRoutingModule
9.    ],
10.    ...
11.  })
12.  export class AppModule { }
```

Notice a new module being imported, called **AppRoutingModule**. By convention, the routing details are added into a file with the name **app-routing.module.ts** and kept in the **root** folder. Let us open the file:

```
1.  import { NgModule } from '@angular/core';
2.  import { RouterModule, Routes } from '@angular/router';
3.
4.  const routes: Routes = [];
5.
6.  @NgModule({
7.    imports: [RouterModule.forRoot(routes)],
8.    exports: [RouterModule]
9.  })
10.  export class AppRoutingModule { }
```

Line 4 defines an array of type **Routes**, defined in the npm package **@angular/router**. *Line 7* uses the method **forRoot()** from **RouterModule** to define the routes at the root level. As of now, the **routes** array is empty. We will add three page-level components: **home**, **about**, and **contact** and use routing to navigate between the pages.

From now on, we will use the shorthand expressions in Angular CLI to generate components and other schema:

ng g c home

ng g c about

ng g c contact

Replace the content of **home.component.html** with the following:

```
<div>
    <a href="/about">About</a>
</div>
<div>
    <a href="/contact">Contact</a>
</div>
```

These are simple HTML link tags to navigate to **/about** or **/contact**. Replace the content of **app.component.html** to render just the **home** component. We will keep it as simple as possible for now:

```
<app-home></app-home>
```

If your dev server is not running, run the following command:

ng serve --open

This should open the app in your default browser as follows:

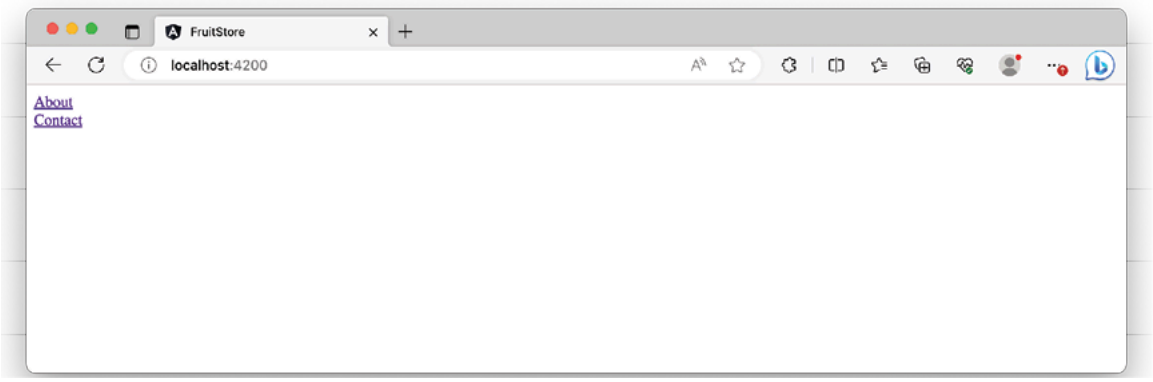


Figure 5.2: *App rendered on the browser*

The routes are currently not implemented. Try clicking on the **About** link and observe that the browser address bar momentarily changes to **http://localhost:4200/about** and returns to the home page.

Now add the following routes in **app-routing.module.ts**:

```
const routes: Routes = [
  { path: '', component: HomeComponent }, // return the home component
  { path: 'about', component: AboutComponent }, // return the about component
  { path: 'contact', component: ContactComponent }, // return the contact component
];
```

Try clicking the About link again and now observe that the browser address bar stays at **http://localhost:4200/about** and does not return to the home page. The page, however, does not render the **AboutComponent** yet. This is because we have not told Angular Router where to render the **AboutComponent**. Replace the content of **app.component.html** file with the following:

```
<router-outlet></router-outlet>
<!-- Component will be rendered here -->
```

The **<router-outlet>** is a placeholder in the view template to render the component associated with a route. The component associated with the route is rendered below the **<router-outlet>** tag.

Now, a click of the **About** link should show the **AboutComponent** on the browser. Similarly, clicking on the **Contact** link should show the **ContactComponent**.

<base href> and routerLink

As we saw in previous sections, in a traditional application, routing is handled by the server, and in an SPA, it

is handled on the client-side inside the browser. In the above example code, when the user types **http://localhost:4200/about** in the browser, the browser does not send a server request but instead allows Angular to resolve the URL on the client-side. This is made possible by **history.pushState()** in modern HTML5 compliant browsers. To resolve the URL on the client-side, the application needs to know the base URL. The **base href** is an element that is added to the **<head>** section of the HTML file and is used to specify the base URL for all relative URLs in the application. This is particularly useful when deploying an Angular application to a subdirectory on a server, as it allows the application to work properly with relative URLs.

The **base href** is set in the **index.html** file of the application, and its value is the base URL for all relative URLs in the application. Open **src/index.html** file and notice the **base href** defined inside the head tag:

```
1. <head>
2.   <meta charset="utf-8">
3.   <title>Online Fruit Store</title>
4.   <base href="/">
5.   <meta name="viewport" content="width=device-width, initial-scale=1">
6.   <link rel="icon" type="image/x-icon" href="favicon.ico">
7. </head>
```

In *line 4* above, if you set **<base href="/my-app/">**, it would mean that all relative URLs in the application will be relative to the **/my-app/** URL. For example, if we have a route defined as **product**, the URL for that route would be **/my-app/product**. It is important to note that the **base href** must match the base URL of the application. Otherwise, the application will not work properly. For example, if the application is deployed to the **root** directory of a server, the **base href** should be set to **"/"**. In addition, when using the **base href**, we need to make sure that all of the links in the application are relative to the **base href**.

You can use the built-in **routerLink** directive to create a relative path with respect to **base href**. The previous code in home.component.html can be re-written using **routerLink** as:

```
1. <a routerLink="/about">
2.   ...
3. </a>
```

This directive is defined in **RouterModule**. So, you need to import **RouterModule** in the module where this view template is defined.

The same can also be written using data binding expression as follows:

```
1. <a [routerLink]="'/about'">
2.   ...
3. </a>
```

What the above code means is that **routerLink** is now data bound to the value on the right-hand side. We can put any variable or method name on the right-hand side. In this case, we want to bind to a string value **'/about'**. We will explore data binding in detail in *Chapter 9, Data Binding and Events Handling.*

Adding routes to existing app

In the previous section, we created an Angular app from scratch and opted for the routing option during project creation. You can also add routing into an existing app. To see this in action, let us create a copy of the **fruit-store** app we created in the previous chapter. The following steps will work on any existing app.

Make sure the app is running. Now run the following command to create the **app-routing.module.ts** file in the **root** folder:

```
ng g m app-routing –flat –module=app
```

-flat tells Angular CLI to generate the new module at the **root** folder, and **-module=app** is to import this module into **AppModule**. Notice that the **app-routing.module.ts** file is created, and **app.module.ts** file is updated, as shown:

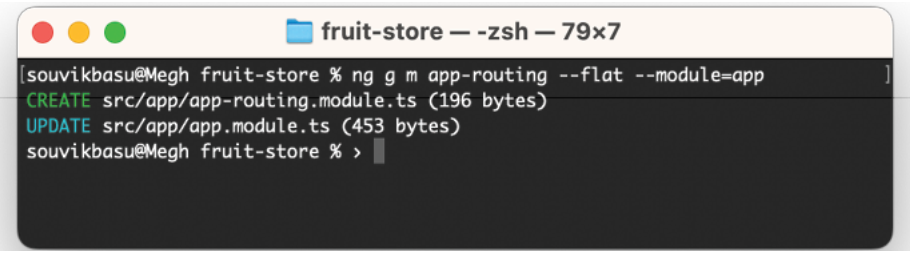


Figure 5.3: *Adding app routing module*

The change in **app.module.ts** is that **AppRouterModule** is imported into **AppModule**.

Add a **ProductDetailsComponent** to show details about each product available in the store. Recall that we had created a **product** module in the **fruit-store** app. You can run the following command to add the component inside the **product** module:

Ng g c product/product-details

The output will be as follows:

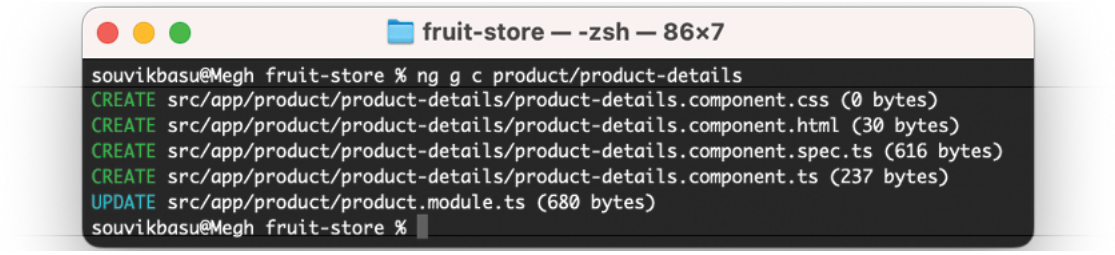


Figure 5.4: *Adding product details component*

Notice that the **ProductDetailsComponent** gets added to the declarations of **product.module.ts** file.

Now add routes to **app-routing.module.ts** in line with how we added in the previous section:

```
1. const routes: Routes = [
2.   { path: '', component: HomePageComponent },
3.   { path: 'details', component: ProductDetailsComponent },
4. ];
```

Replace the content of **app.component.html** to show the **<router-outlet>** instead of the individual components, as shown:

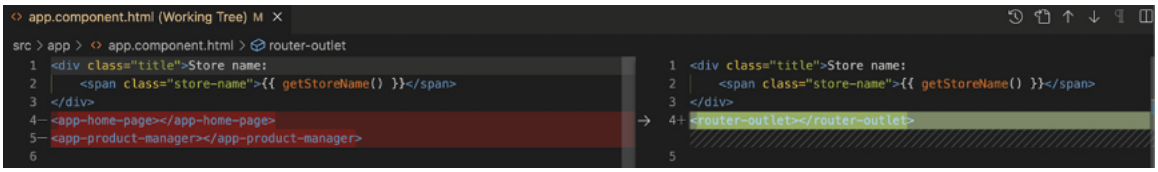


Figure 5.5: *Using router outlet to display components*

We want to make each item in the list of **products** clickable, which should lead to the product details page. To achieve this, wrap each item in the list of products with an **<a >** tag:

```
1. <li *ngFor="let product of products">
2.   <a routerLink="/details">
3.  <div [ngClass]="{'expensive': product.price > 5}">{{ product.name }} {{ product.price }}</div>
4.   </a>
5. </li>
```

Let us save and open the page in the browser. Notice that each item in the list of fruits now has an underline indicating that it is a link:

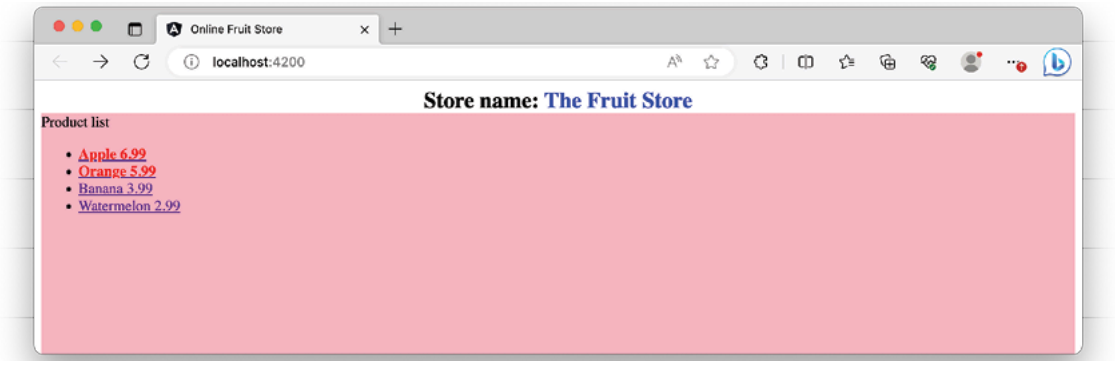


Figure 5.6: *Clickable product links*

Click on any item and watch the browser address bar change to **http://localhost:4200/details,** and the page now shows the **ProductDetailComponent**:

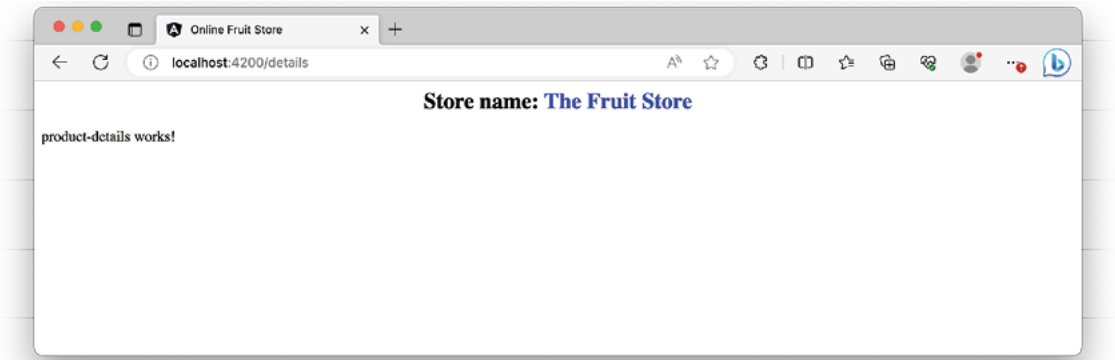


Figure 5.7: *Navigate to /details*

This demonstrates how you can add routing to an existing application.

Passing URL parameters

In the previous section, we navigated to the product details page on click of a fruit item from the list. But the product details page does not have any details about the item clicked.

To show details about the product, let us pass a fruit name, say Apple, in the link as follows:

```
<a routerLink="/details/apple">
  ...
</a>
```

Since we are now passing the product name to the **/details** route, we need to add that to the routes definition in the **app-routing.module.ts** file:

```
const routes: Routes = [
  { path: '', component: HomePageComponent },
  { path: 'details/:name', component: ProductDetailsComponent },
];
```

:name denotes that any value passed will be stored in a route param **name**. Modify **product-details.component.ts** file to receive the product name from the route. We usually retrieve the value of the parameter in the **ngOnInit** life cycle hook when the component is initialized. To load router parameters, Angular provides **ActivatedRoute** service. This service keeps track of the currently activated route in the loaded component. **ActivatedRoute** contains a **snapshot** property, which holds the parameters for the current route. Change **product-details.component.ts** as follows to extract the route parameter—**name**:

```
...
import { ActivatedRoute } from '@angular/router';

...
```

```
export class ProductDetailsComponent {
  productName?: string;

  constructor(private activatedRoute: ActivatedRoute) { }

  ngOnInit() {
    this.productName = this.activatedRoute.snapshot.params['name'];
  }
}
```

Display the extracted property **productName** in the view template file **product-details.component.html**:

```
<div>
    <div>Product Name: {{ productName }}</div>
</div>
```

Now, when you click on any fruit name on the home page, it goes to **/details/apple** and shows the following:

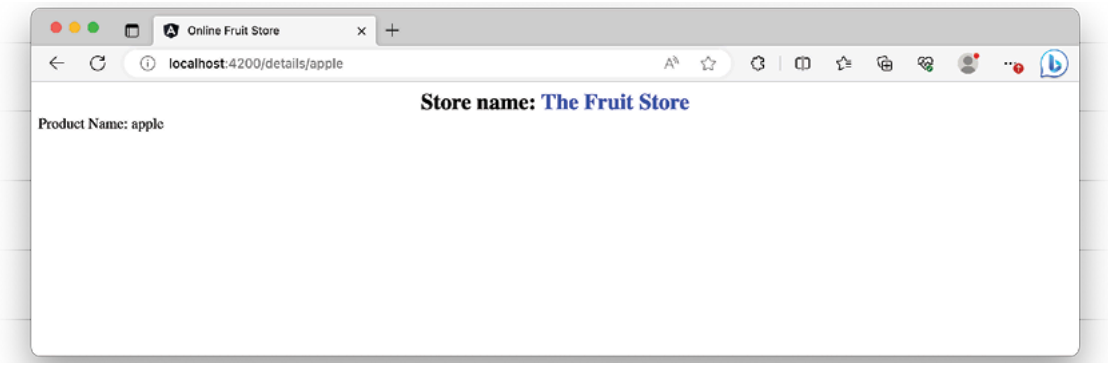


Figure 5.8: *Adding route parameter*

For now, we added a hard-coded link **/details/apple** to all the fruit links. We want to pass the name of the fruit being clicked. This is where **routerLink** will be useful. **routerLink** can bind to an array in which each item defines a part of a path. For instance, if we want to navigate to **/details/apple**, this path can be broken down in an array as **['/details','apple']**.

Change the view template code to use an array format:

```
            <a [routerLink]="['/details', 'apple']">
    ...
</a>
```

Observe that the browser page behaves the same as before. Now change the second item in the array to **product.name** as follows:

```
<li *ngFor="let product of products">
  <a [routerLink]="['/details', product.name]">
  <div [ngClass]="{'expensive': product.price > 5}">{{ product.name }} {{ product.price }}</div>
  </a>
</li>
```

Now click on **Orange** on the home page and see that the product details page shows the value as **Orange**:

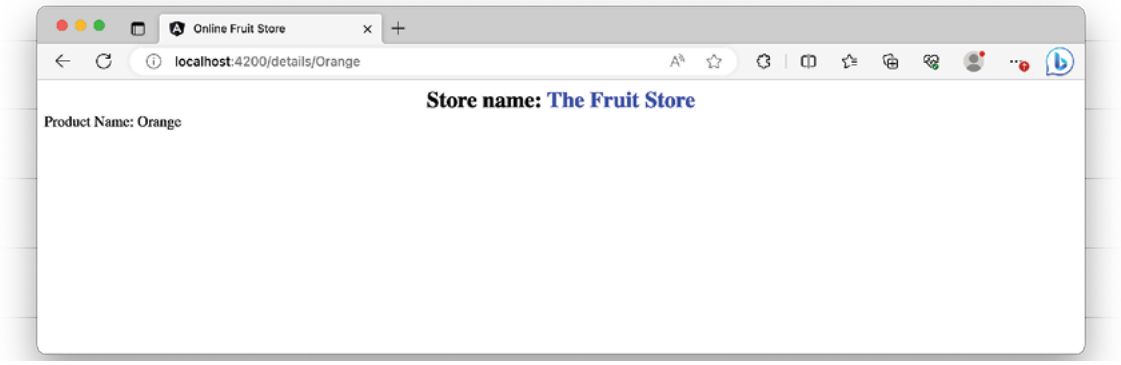


Figure 5.9: *Navigate to Orange details*

If you want to show other details on the product page, you can now fetch product details based on the product name. In a real-world app, you would want to send an ID or a slug instead of a product name, as the product name may not be unique across products. We are using the name so as to keep the code as simple as possible to demo routing features. Let us add a method in **product.service.ts** file to return a product given its name:

```
getProduct(name: string) {
  return this.products.find(product => product.name === name);
}
```

Now, call this method from **product-details.component.ts** file:

```
export class ProductDetailsComponent {
  productName?: string;
  product?: Product;

 constructor(private activatedRoute: ActivatedRoute, private productService: ProductService) { }

  ngOnInit() {
    this.productName = this.activatedRoute.snapshot.params['name'];
    if (this.productName) {
      this.product = this.productService.getProduct(this.productName);
    }
  }
}
```

Now you can see the price details fetched as well in the following figure:

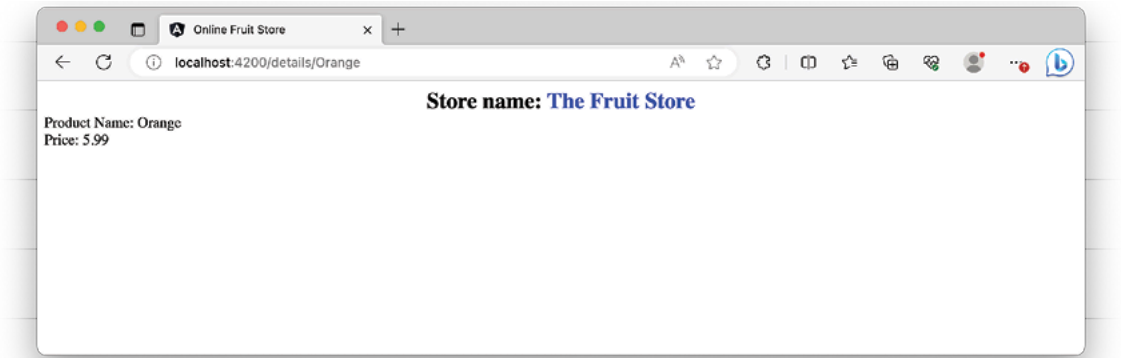


Figure 5.10: *Product details fetched*

Nested routes

Nested routes can help to organize a page layout into a reasonable hierarchy. Let us rearrange our application to separate out a product page from the home page, so that we can include more features on the home page like promotions, deals, and contact details. Based upon the techniques we learned till now, create separate

components for each of these features, create respective routes and add links in the home page. For brevity, we will only show the routes and the modified home page view template below:

```
const routes: Routes = [
  { path: '', component: HomePageComponent },
  { path: 'products', component: ProductComponent },
  { path: 'deals', component: DealsComponent },
  { path: 'details/:name', component: ProductDetailsComponent },
];
```

Create a list of top-level menu items and a router outlet in **app.component.ts**:

```
<div class="title">
    <span class="store-name">{{ getStoreName() }}</span>
</div>
<ul class="menu">
    <li class="menu-item">
        <a routerLink="/">
            Home
        </a>
    </li>
    <li class="menu-item">
        <a routerLink="/products">
            Products
        </a>
    </li>
    <li class="menu-item">
        <a routerLink="/deals">
            Deals
        </a>
    </li>
</ul>
<div class="main">
    <router-outlet></router-outlet>
</div>
```

To clean up the **HomeComponent** move the code in **home.components.html** that shows the list of products to **product.component.html**. Add dummy text in **home.components.html** and **deals.component.html** files to be able to differentiate when these menu items are clicked. Notice that the list of products shows when you click on the **Products** menu at the top:

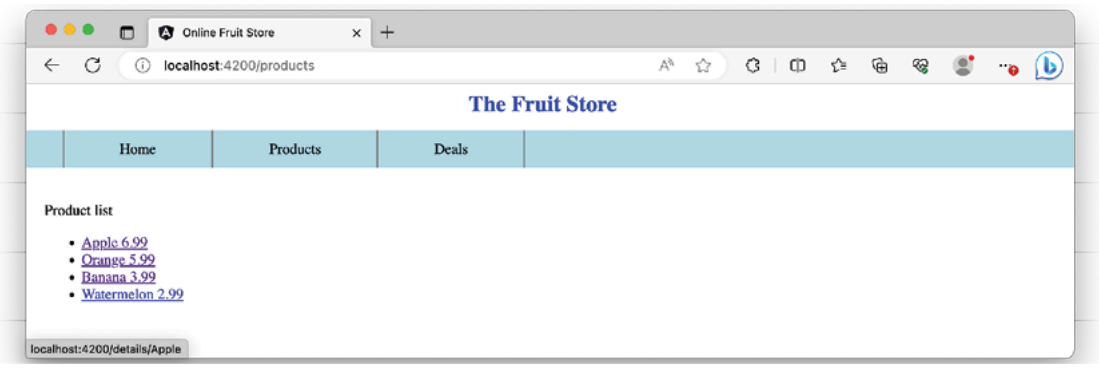


Figure 5.11: *Adding top-level menu*

As of now, the code has a single level of routing that is very similar to the previous app. Clicking on **Home**,

Products, and **Deals** menu items renders the respective component in the region below it. If you click on any product link like **Apple** or **Orange**, you will notice that the details component renders in the same region, replacing the list of products:

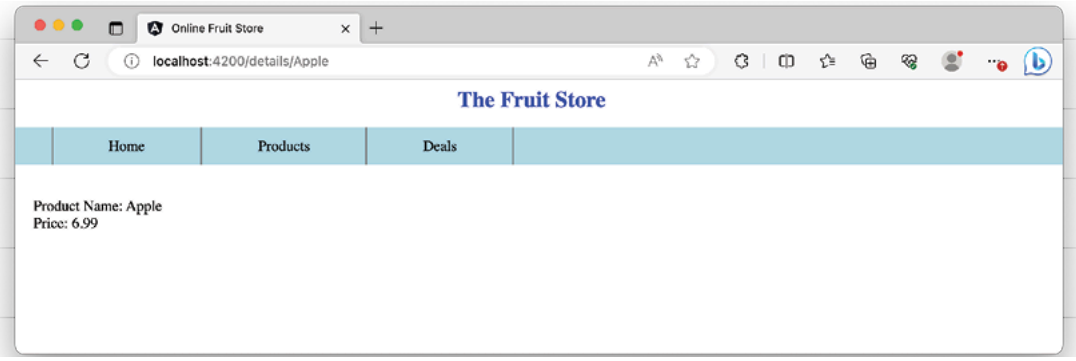


Figure 5.12: *Routing in the same level*

This is because **/details/:name** route is added at the same level as other routes and shows up below the same **<router-outlet>** declared in **app.component.html**.

Now if you want to display the product details beside the list of products, you will need to define another level of route. Add the following code in **product.component.html**:

```
<div id="product_page">
  <div class="product-list">
    <h4>Product List</h4>
    <ul>
      <li *ngFor="let product of products">
        <a [routerLink]="['details', product.name]">
<div [ngClass]="{'expensive': product.price > 5}">{{ product.name }} {{ product.price }}</div>
        </a>
      </li>
    </ul>
  </div>
  <div class="product-details">
    <router-outlet></router-outlet>
  </div>
</div>
```

Notice that now the **routerLink** points to **'details'** and not **'/details'**. This allows for adding a relative path with respect to the existing path. Also, notice the additional **<router-outlet>** added to this view template. It indicates that any route relative to this component will be displayed under this router outlet.

Using CSS, display **.product-list** and **.product-details** side by side:

```
#product_page {
  display:flex;
}

.product-list {
  width: 30%;
}
```

If you click on any fruit now, you will see the details shown in the panel to the right:

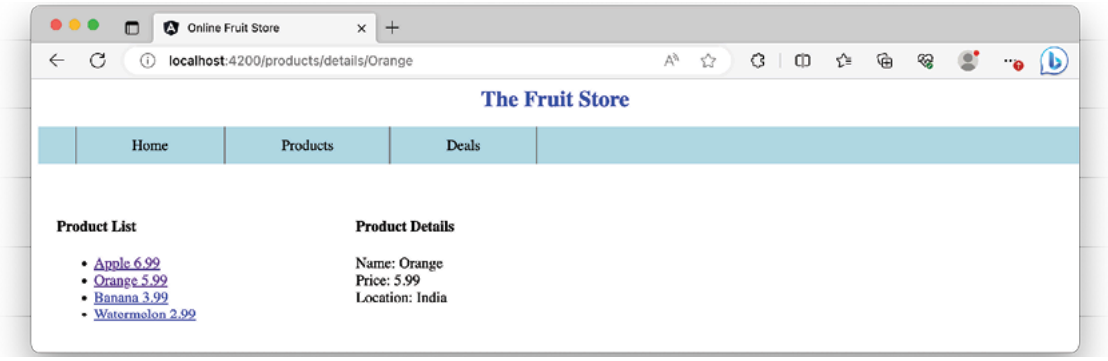


Figure 5.13: *Show details on a side panel using nested routes*

This nesting logic can be extended to any number of levels. For instance, you might want to show reviews for a product on click of a review link. This will be left as an exercise for you.

Snapshot versus observables

A keen reader would have noticed that in the above example, the routing to show product details worked only for the first link clicked. For instance, if you click **Apple**, the product details panel will show the details about Apple. Now if you click on **Orange**, it will not show details for Orange, although the address bar in the browser changed to **http://localhost:4200/products/details/Orange**. This happens because we are retrieving the route in the **ngOnInit()** lifecycle method. We will go into the details of lifecycle methods in *Chapter 8, Lifecycle Hooks*. But for the purpose of discussion here, we will explain it in short. The method **ngOnInit()** is called only during the initialization of a component. Since the product details component is already initialized and visible on the page, when you click on the second link for Orange, **ngOnInit()** is not called again to fetch the changed route. The **snapshot** property in **ActivatedRoute** provides the current route values. It does not provide any signal when the route changes. Since we are interested in listening to changes in the route, we need to subscribe to **paramMap** or **params** observable property of **ActivatedRoute**. We will understand observables in the next chapter, but to give a short intro, **observables** are like a stream of events being emitted by a source. If you subscribe to the source, your subscribing method will be invoked every time there is a new event emitted. Such a source is called an **observable** since these can be observed for events. The properties **paramMap** and **params** are observables in **ActivatedRoute**. In **product-details.component.ts** file, we can use either of them to read the name route param. The **paramMap** observable emits the **paramMap** object, while the **params** observable emits the **params** array:

```
ngOnInit() {
  this.activatedRoute.paramMap.subscribe(paramMap => {
    this.productName = paramMap.get('name');
    if (this.productName) {
this.product = this.productService.getProduct(this.productName);
    }
  });
}
```

Now clicking on **Apple** or **Orange** will show the respective product details.

The same code can be written using **params** observable as follows:

```
ngOnInit() {
  this.activatedRoute.params.subscribe(param => {
    this.productName = param['name'];
    if (this.productName) {
this.product = this.productService.getProduct(this.productName);
    }
  });
}
```

It is a good practice to unsubscribe in the **ngOnDestroy()** lifecycle method to avoid memory leaks. Otherwise,

the subscription will continue even after you move away to other routes and the component is removed from the page and garbage collected:

```
routeSub$: Subscription | null = null;


ngOnInit() {
  this.routeSub$ = this.activatedRoute.params.subscribe(param => {
    this.productName = param['name'];
    if (this.productName) {
this.product = this.productService.getProduct(this.productName);
    }
    });
  }

  ngOnDestroy() {
    this.routeSub$?.unsubscribe();
  }
```

paramMap and **params** are very similar in their intent. There was a discussion within the Angular community to deprecate **param** in favour of **paramMap**, but that is not yet decided. So, for the maintainability of code, prefer to use **paramMap** wherever possible.

Setting up redirects

You can use **redirectTo** prop in the route path to redirect to another existing path. It is a general practice to redirect the default path to a predefined path. For example, in our routes, we can define a redirect to **HomeComponent** as follows:

```
const routes: Routes = [
  { path: 'home', component: HomePageComponent },
  {
    path: 'products', component: ProductComponent,
children: [{ path: 'details/:name', component: ProductDetailsComponent }]
  },
  { path: 'deals', component: DealsComponent },
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: '**', component: NotFoundComponent},
];
```

Especially when you are trying to refactor routes, **redirectTo** can be a very helpful property.

Handling page not found

How do we handle a route which is not on the **routes** list? Generally, websites show a page showing the message **Page not found**. To try this out, add a component **NotFoundComponent** to show such a message:

ng g c not-found

Replace the view template with a message like this:

```
<p>Page not found</p>
```

This can be beautified with branding details and a link to the home page or other promotions as needed. For now, we will keep it simple as above. Add the following route to **app-routing.module.ts**:

```
{ path: '**', component: NotFoundComponent } // fall back route
```

Make sure to add this route definition at the end, as the first matching router wins. Hence, wildcard routes like the above are placed at the end.

Now, type any URL in the browser that is not handled in routes. It should show the **NotFoundComponent** as

follows:

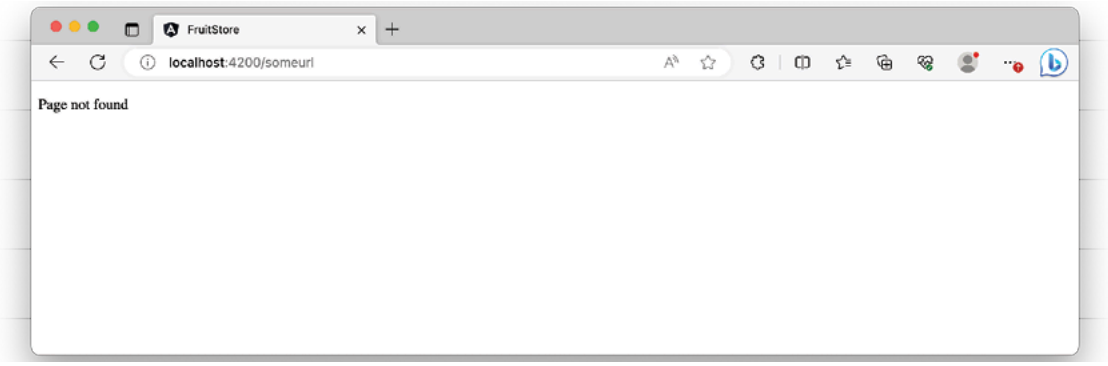


Figure 5.14: *Page not found*

Showing active routes

You can use **routerLinkActive** to add CSS to the active link. To see it in action, modify the **product** component to add the class **active** as follows:

```
<a [routerLink]="['details', product.name]" class="product-link" routerLinkActive="active">
  …
</a>
```

Now add CSS to add **background** and **font-weight** based on the class **active**:

```
.product-link {
    display: block;
    padding: 5px;
    margin: 10px;;
}

.product-link.active {
    background-color: #cfd8dc;
    font-weight: bold;
}
```

The page will highlight the selected product now, as shown:

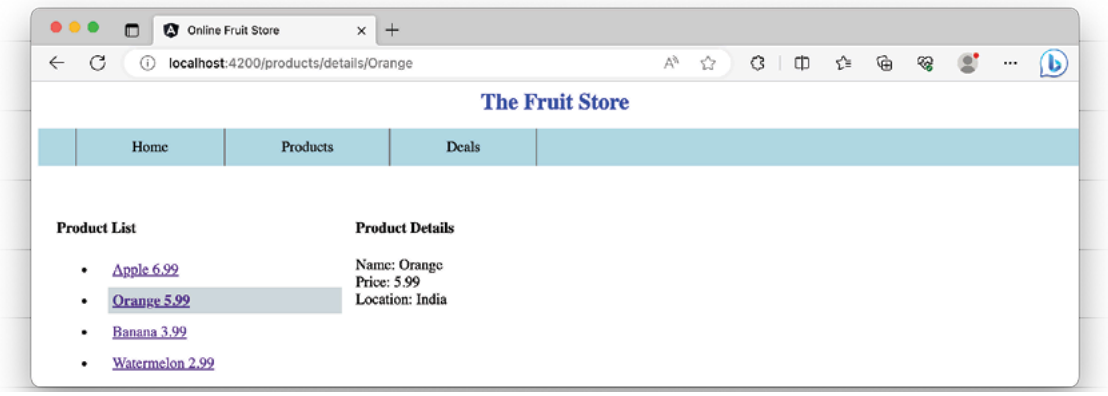


Figure 5.15: *Highlight active route*

You may also use an expression or a method call to fetch the name of the class name dynamically:

```
<a [routerLink]="['details', product.name]" class="product-link" [routerLinkActive]="getActiveClassNam
```

Create the method in **product.component.ts** file as follows:

```
  getActiveClassName(product: Product) {
```

```
    return product.name.toLowerCase();
  }
```

Now change **product.component.css** file to return a color specific to the selected product:

```
.orange {
  background-color: orange;
}

.apple {
  background-color: red;
}
```

This example does not have much significance in a real-world app but is just to show you how to set the class name dynamically.

Let us explore another case of using **routerLinkActive**. Open **app.component.html** and add this directive to all the top-level menu items as follows:

```
<ul class="menu">
  <li class="menu-item">
    <a routerLink="/" routerLinkActive="active">
      Home
    </a>
  </li>
  <li class="menu-item">
    <a routerLink="/products" routerLinkActive="active">
      Products
    </a>
  </li>
  <li class="menu-item">
    <a routerLink="/deals" routerLinkActive="active">
      Deals
    </a>
  </li>
</ul>
```

Add a CSS that sets the parent's background to a different shade of blue to differentiate the **active** menu:

```
.menu-item:has(> .active) {
  color: white;
  background-color: lightskyblue;
}
```

Now, click on the **Product** menu, and you will observe that both **Home** and **Product** are shown as active:

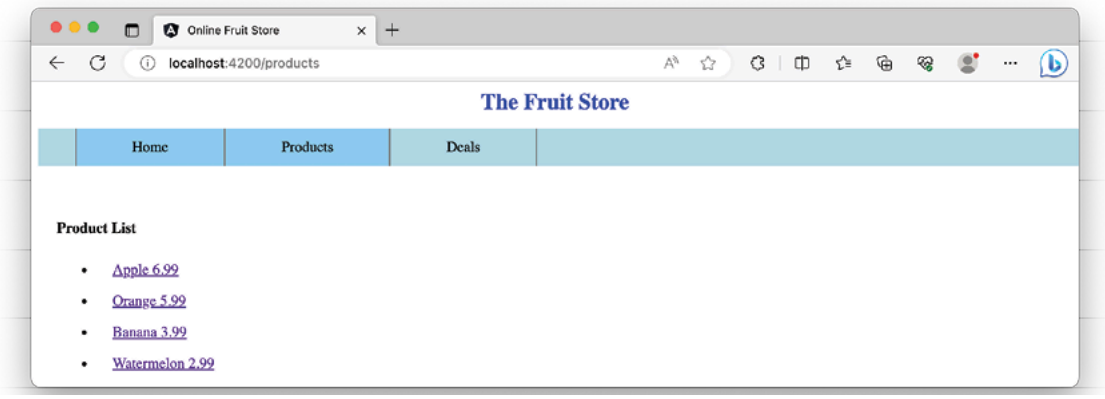


Figure 5.16: *Both Home and Product are shown as active*

This is happening because when you set the **routerLinkActive** directive on any link, it sets the class name for all **routerLink** which matches the URL. Since we added this directive for **routerLink** values **''** and **'product'**, the URL **localhost:4200/product** also matches for the path **''**. Hence the menu item for **HomeComponent** also shows as active. To avoid this, add **[routerLinkActiveOptions]="{exact: true}"** to home page link as follows:

```
<a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{exact:true}">
    Home
</a>
```

Now only the **Product** menu item will show as active as **/product** does not match the **''** path anymore.

Passing query parameter

The route for product details is set as follows:

```
[{ path: 'details/:name', component: ProductDetailsComponent }
```

The same can also be written by extracting the **:name** slug in the path into a URL query parameter. Change the path as follows:

```
{ path: 'details', component: ProductDetailsComponent }
```

Change the **product.component.html** file to remove **/:name** from **routerLink** and add the **queryParams** directive as follows:

```
<li *ngFor="let product of products">
 <a routerLink="details" [queryParams]="{name: product.name}" class="product-link" >
   ...
  </a>
</li>
```

The **queryParam** can be accessed in the component in a very similar way to how we accessed the route param. Just subscribe to **queryParams** observable in **ActivateRoute**:

```
this.routeSub$ = this.activatedRoute.queryParams.subscribe(param => {
    this.productName = param['name'];
    if (this.productName) {
this.product = this.productService.getProduct(this.productName);
    }
  });
```

Now clicking on a product link will show the product name as a query parameter in the browser address bar, like **http://localhost:4200/products/details?name=Orange**.

The advantage of using a query param is that it allows optional params to be added to the path. Also, it is easy to add and remove multiple query parameters. We can add another query param as follows:

```
<a routerLink="details" [queryParams]="{name: product.name, variety: product.variety}" class="product-
```

The same could be done in the route path as well like this:

```
{ path: 'details/:name/:variety', component: ProductDetailsComponent }
```

However, it is easier to add or remove **queryParams** as the route definitions do not need to be modified. This enables loose coupling between the route definitions, the component invoking the route and the component being rendered. If we are certain about the parameters, we can add those to the route. Otherwise, **queryParams** allow more flexibility.

Routing using TypeScript

There might be scenario where you may need to redirect to a route based on business logic. You can perform such dynamic navigation using the router object as follows:

```
this.router.navigate( ['details', {name: product.name}]);
```

Lazy loading and preloading

Lazy loading is a technique used in Angular to load only the modules that are required by the user at a given time. This approach helps to reduce the initial load time and improve the overall performance of the application. With lazy loading, modules are loaded on-demand, which means that the user only downloads the necessary resources when they navigate to a specific path. This way, the application can start faster and provide a more seamless experience to the user.

In our app, let us add a menu item in **app.component.ts** to take the user to the **Contact Us** page:

```
<li class="menu-item">
  <a routerLink="/contact" routerLinkActive="active" >
    Contact Us
  </a>
</li>
```

The probability of a user clicking on the **Product** menu is high, while that of clicking the **Contact Us** menu is relatively low. If we add a **ContactComponent** as part of the app, the component will be added as part of the initial page payload. Thus, **ContactComponent** will be loaded into the browser even though the user might not click the **Contact Us** menu. When more such components are added, the app payload bloats up. This causes the app to take significant time to load on the browser. To provide a better user experience, we can create a separate module for the Contact Us page and load only when the user clicks on the **Contact Us** menu. Angular CLI allows to creation of such feature modules and generates code to lazy load. Run the following command:

ng generate module contact --route contact --module app.module

This will generate a **contact.module.ts** file, add a **contact** component folder inside the module, and add an entry in **app-routing.module.ts** file to lazy load the **contact** module in route **'contact'**:

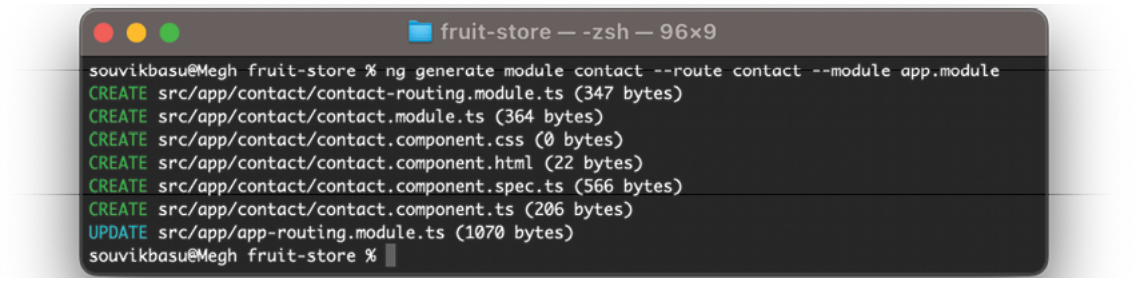


***Figure 5.17:** Add contact module with component and routes*

Open the **app-routing.module.ts** file to see the generated route:

```
{ path: 'contact', loadChildren: () => import('./contact/contact.module').then(m => m.ContactModule) },
```

The above command also adds **contact-routing.module.ts** file to generate routes within **contact**:

```
const routes: Routes = [{ path: '', component: ContactComponent }];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class ContactRoutingModule { }
```

This routes array is declared as a child route using **RouterModule.forChild(routes)** method as against a root route in **app-routing.module.ts** file that uses **forRoot()**. There can be only one root route created for an app but multiple child routes are allowed. Also, notice the default route created to navigate to **ContactComponent**. Now if you click on the **Contact Us** menu, you will notice that the module payload **src_app_contact_contact_module.ts.js** is fetched dynamically, and the **ContactComponent** is rendered on the page:

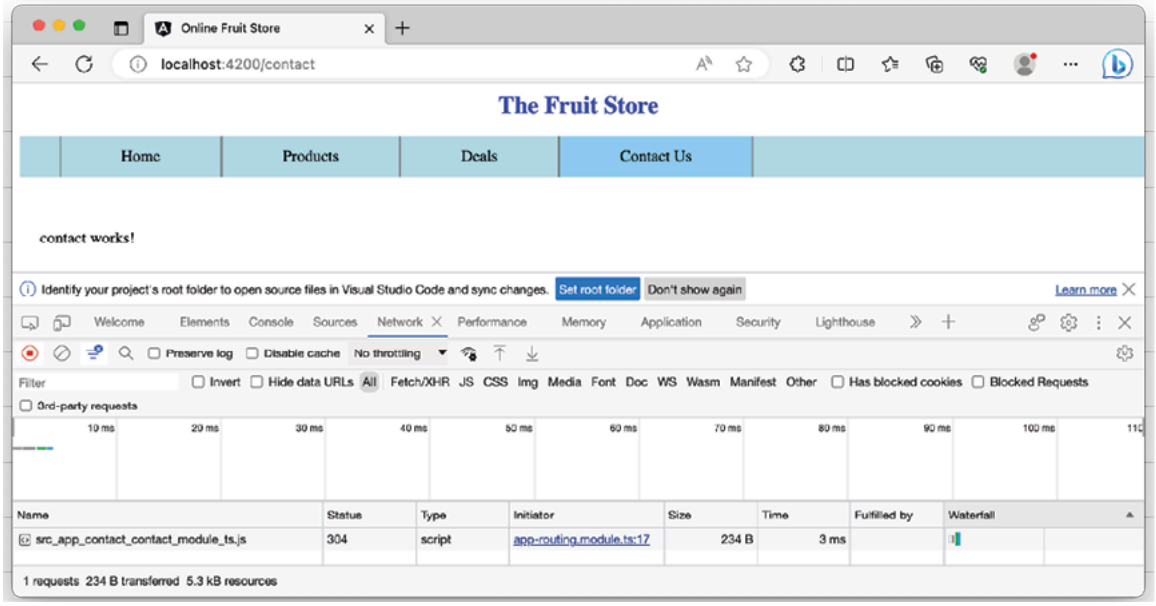


Figure 5.18: *Lazy loading module*

On the other hand, preloading is a technique that allows the application to load some modules in the background while the user is interacting with the application. This way, when the user navigates to a new section of the application, the necessary resources are already loaded, and the user does not have to wait for them to be downloaded. Preloading is particularly useful for applications with complex routing structures, where it is difficult to predict which modules will be required next. By preloading modules, the application can provide a faster and more responsive experience to the user.

To preload all modules and standalone components that were previously set as lazy loaded, you can import the **PreloadAllModules** token from the Angular Router:

```
1. import { PreloadAllModules, RouterModule, Routes } from '@angular/router';
2. ...
3. @NgModule({
4.   imports: [RouterModule.forRoot(routes, {
5.     preloadingStrategy: PreloadAllModules
6.   })],
7.   exports: [RouterModule]
8. })
9. export class AppRoutingModule { }
```

In the above example, the **PreloadAllModules** strategy is used to preload all the modules in the background while the user is interacting with the application. This way, when the user navigates to a new section of the application, the necessary resources are already loaded, and the user does not have to wait for them to be downloaded.

CanActivate, CanActivateChild, CanDeactivate

CanActivate is a route guard that allows you to control access to a route. It checks whether a user is authorized to access a particular route and either grants or denies access accordingly. To demonstrate this, add a menu item to go to **/orders** route, create an **OrdersComponent** and create a route to render **OrdersComponent** for **/orders** path. Since you need to identify the user before we can show the orders, we can add a **CanActivate** directive to the route as follows:

```
1. import { Injectable } from '@angular/core';
2. import { CanActivate, ActivatedRouteSnapshot, RouterStateSnapshot, UrlTree, Router } from '@angular/r
3. import { Observable } from 'rxjs';
4.
5. @Injectable({
```

```
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {}

  canActivate(
    next: ActivatedRouteSnapshot,
  state: RouterStateSnapshot): Observable<boolean | UrlTree> | Promise<boolean | UrlTree> | boolean |
  UrlTree {
      const currentUser = localStorage.getItem('currentUser');
      if (currentUser) {
          // authorized, return true
          return true;
      }

      // not logged in, redirect to login page
      this.router.navigate(['/login']);
      return false;
  }
}
```

Now, clicking on the **Orders** menu will take the user to the login route if the **currentUser** is not saved in **localstorage**. We will look at authentication techniques in *Chapter 14, Authentication and Authorization*. For now, assume that a separate login flow saves the logged-in user in the local storage.

Other related concepts to **CanActivate** are **CanActivateChild**, **CanDeactivate**, **Resolve**, and **CanLoad**. **CanActivateChild** is similar to **CanActivate** but applies to child routes. The key difference is that it runs before any child route is activated. **CanDeactivate** allows you to prevent a user from leaving a route with unsaved changes. You might have seen such a feature in many websites, wherein once you type some text and then try to navigate away from the page, there will be an alert box confirming if you want to save changes or cancel routing. **Resolve** allows you to fetch data before a route is activated. **CanLoad** allows you to prevent a module from being loaded lazily.

Fragments

In Angular routing, fragments are used to navigate to a specific part of a page. They are identified by a hashtag (#) followed by the fragment identifier. In the code example provided earlier, we can add a fragment to the end of the URL to navigate to a specific part of the page. For example, if we want to navigate to a component with an ID of **#chat** on the **contact** route, we can do so by appending **#chat** to the URL like this:

```
<li class="menu-item">
 <a routerLink="/contact" fragment="chat" routerLinkActive="active">
     Contact Us
  </a>
</li>
```

In the contact component's HTML file, we can add an element with **id="chat"**:

```
<p>Contact Us</p>
<div id="chat">
  This is the chat section in the contact page.
</div>

```

So, when the user navigates to **http://localhost:4200/contact#chat**, the browser will scroll to the element with the id of **chat** on the **contact** page.

Router data

Router data is used to pass information to a component when a route is activated. This data can be used to set properties, initialize services, or perform any other necessary tasks. To add data to a route, we can use the **data** property in the route configuration:

```
{ path: 'deals', component: DealsComponent, data: { title: 'Latest Deals and Discounts' } }
```

To access the data in a component, we can use the **ActivatedRoute** service as follows:

```
export class DealsComponent {
  title?: string;
  constructor(private activatedRoute: ActivatedRoute) { }

  ngOnInit() {
    this.title = this.activatedRoute.snapshot.data['title'];
    // set page title here
  }
}
```

Router state

A router state is used to store state information about a route. This can be useful when we want to preserve data between route changes or when we want to pass data to child routes. To add a state to a route, we can use the **state** property in the navigation extras. For example, we could create a new session id in AuthGuard while navigating to the **/login** route:

```
    const sessionId = 12345;
    this.router.navigate(['/login'], { state: { sessionId } });
```

This state can be accessed at the **LoginComponent** as follows:

```
  sessionId?: string;
 constructor(private activatedRoute: ActivatedRoute, private router: Router) { }

  ngOnInit() {
 this.sessionId = this.router.getCurrentNavigation()?.extras.state?.sessionId;
 }
```

Note that the **state** object can also be retrieved asynchronously using observables. This is useful when the state data needs to be fetched from an API or a server.

Debugging routes

You can add **{ enableTracing: true }** as a configuration while importing routing module as follows:

```
@NgModule({
  imports: [RouterModule.forRoot(routes, { enableTracing: true })],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

This will add verbose logs into the browser dev tool **Console** when you navigate to any route:

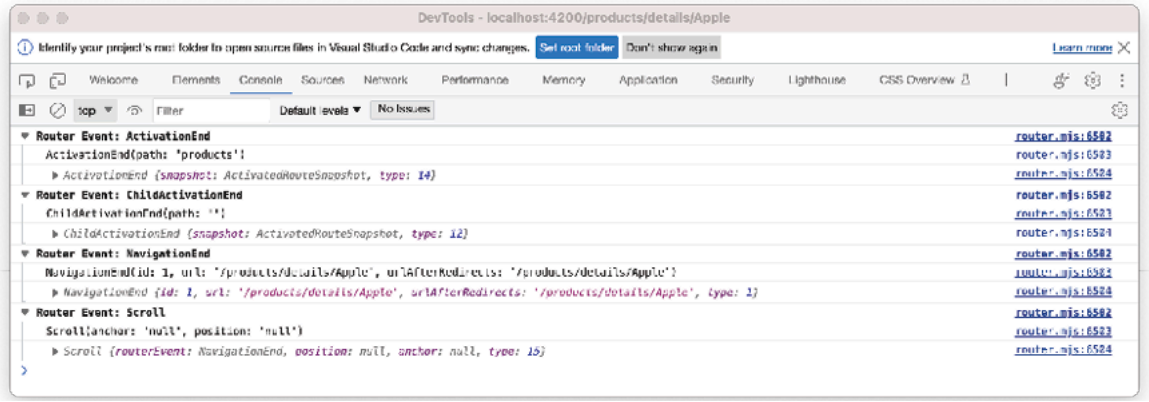


Figure 5.19: *Debugging route navigation on dev tool console*

Check the *References* section for details about these events.

Conclusion

Angular routing is a powerful feature that allows us to build SPAs with multiple views. With Angular routing, we can define and navigate multiple routes, pass data between components, protect routes with guards, and more. By understanding the concepts of routing in Angular, we can build complex and robust applications that provide a seamless user experience. Whether it is navigating between pages, sharing URLs, or passing data between components, Angular routing is an essential aspect of building modern web applications.

In the next chapter, we will learn the various ways to style Angular components.

Points to remember

- Client-side routing creates a seamless user experience.
- Base href allows to create a relative path for the app routes.
- Angular CLI makes it easy to add feature modules with its own route definitions.
- routerLink can take string routes or bind to expressions or method calls.
- Nested routes make it easy to divide the app into meaningful layouts and regions.
- It is easy to show the active routes using routerLinkActive.
- Lazy loading and preloading are essential tools for a better user experience.
- CanActivate allows to validate pre-conditions before initiating a route navigation.

Questions

1. Explain the role of history.pushState() browser API in client-side routing.
2. Why should the catch-all route be added to the end of all routes?
3. Explain the difference between forRoot() and forChild() methods.
4. Explain the advantages of using query parameters.
5. Explain the difference between Snapshot and observables in routing.

References

- routerLink:
  https://angular.io/api/router/RouterLink
- ActivatedRoute:
  https://angular.io/guide/router-reference#activated-route
- Router events:
  https://angular.io/guide/router-reference#router-events

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 6
Styling and Theming

Introduction

Styling in Angular is the process of applying CSS styles to the components and elements of an Angular application. Angular provides various ways to style components, depending on the scope and source of the styles. You can define component-specific styles or even global styles. You can also use CSS preprocessors such as Sass or Less to enhance your styling capabilities. Angular also supports view encapsulation, which controls how the styles are applied and isolated for each component. Additionally, Angular provides some special selectors that you can use in your component styles, such as :host, :host-context, and ::ng-deep, to target specific elements or scenarios. In this chapter, we will explore the different ways of styling Angular components and see some examples of using them.

Structure

The chapter covers the following topics:

- Styling using styleUrls metadata
- Styling using styles metadata
- Inline style in template
- <link> tag in template
- Using CSS frameworks
- Importing library packages
- View encapsulation
- Global styles
- CSS preprocessors (SCSS, Less)
- Using SCSS @mixin
- Angular specific selectors
- Authoring style changes
- Theming
- Setting style using TypeScript
- Using Tailwind with Angular
- Styling best practices

Objectives

By the end of this chapter, you will understand the various ways to style Angular components. You will also know some advanced style techniques specific to Angular.

Styling using styleUrls metadata

This is the most commonly used method to style Angular components. We already used this method in previous chapters to style our components. To see an example, open the fruit-store app code created in *Chapter 5, Routing,* and open the **app.component.ts** file in VS Code. You will see the component metadata set as

follows:

```
1. @Component({
2.   selector: 'app-root',
3.   templateUrl: './app.component.html',
4.   styleUrls: ['./app.component.css']
5. })
```

This is the default styling method used by the Angular CLI. You can regenerate this code easily by running the command to create a new project like:

ng new fruit-store

styleUrls is an array of relative paths to stylesheet files, in this case, **./app.component.css** file. You can specify more than one file. The general convention is to specify one CSS file for the component and other common CSS files if applicable. Open the CSS file and change the store name color from blue to **red** as follows:

```
1. .store-name {
2.     color: red;
3. }
```

Observe the store name color in the browser:

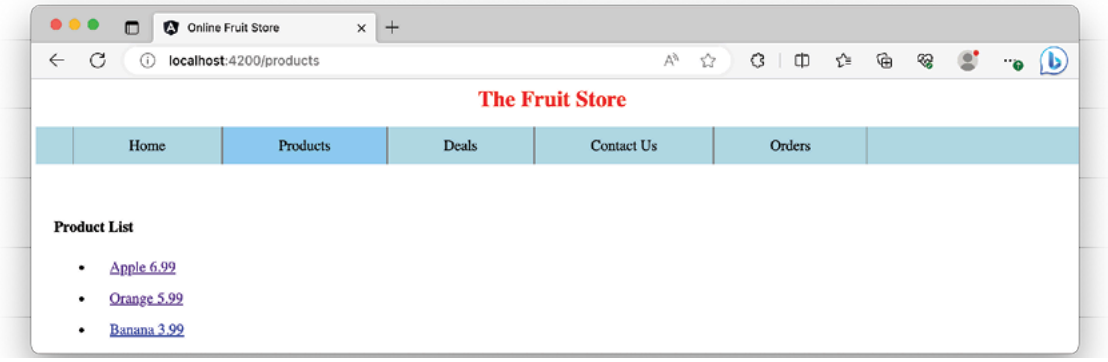


Figure 6.1: *Styling using styleUrls*

Similarly, you may set other CSS statements in the file as you like. All CSS statements are valid to use in this file.

To see multiple **styleUrls** in action, add another file, **app.component.additional.css** in the same folder and update the metadata as follows:

```
1. styleUrls: ['./app.component.css', './app.component.additional.css']
```

Add the following CSS statement in **app.component.additional.css**:

```
1. .store-name {
2.     color: green;
3. }
```

Now observe that the name changes to **green**:

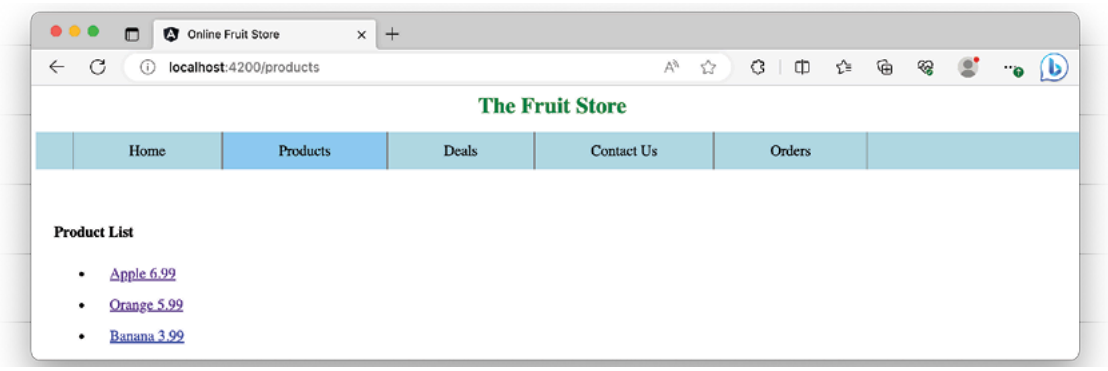


Figure 6.2: Multiple style files in styleUrls

The order in which the file names are added to the metadata determines which one takes precedence over the other. Try changing the order to the following and see that the store name color changes back to red:

```
styleUrls: ['./app.component.additional.css', './app.component.css']
```

Styling using styles metadata

This is the simplest method to style Angular components. Simply add the style statements in the styles metadata of a component. To see it in action, let us remove the previously added **styleUrls** metadata on the **app.component.ts** file and add a **styles** metadata as follows:

```
styles: ['.store-name { color: orange; }'],
```

Notice in the browser that the color of the store name changes to **orange**. You can style the other elements on the page by adding more items to the styles array as follows:

```
styles: ['.title { text-align: center }', '.store-name { color: orange }']
```

This method is usually used for single-line styling. You can add multi-line styles using a backtick (`) instead of single quote (') as follows:

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styles: [
    `.title {
      text-align: center;
      font-size: 24px;
      font-weight: 600;
  }`,
    `.store-name {
       color: red;
    }
    `],
})
export class AppComponent {
/* ... */
}
```

Although simplistic in nature, this method becomes cumbersome to maintain style code once the style statements start getting bigger. Hence, for all practical purposes, using the **stylesUrls** metadata is a preferred method to style a complex component. If you want, you can use both **styles** and **styleUrls** metadata, in which case styles will take precedence.

Inline style in template

Another simplistic approach to styling a component is to set inline styling within the template. For instance, to set the **store-name** in **red**, we could add the **style** attribute to the div in the template as follows:

```
<div class="title">
  <span class="store-name" style="color: red;">{{ getStoreName() }}</span>
</div>
```

You may also bind a style statement using an expression as follows where the **getTextColor()** method returns a color name based on business logic:

```
<div class="title">
  <span class="store-name" [style.color]="getTextColor()">{{ getStoreName() }}</span>
</div>
```

Again, this approach is suitable when you want to keep the styles closer to the template and works well when

you want to style an element or two. It does not allow separation of concern of structure and presentation of the view, and hence, it is not recommended in general. For more complex styling **styleUrls** helps to create more readable and maintainable code.

<link> tag in template

You may also add a **link** tag in the template like you would in an HTML document, as shown:

```
@Component({
  selector: 'app-demo',
  template: `
    <link rel="stylesheet" href="../../assets/demo.component.css">
    <p>
      demo works!
    </p>
  `,
})
export class DemoComponent {
  /* ... */
}
```

You need to make sure that you add a relative path to the **link href** so that the AOT compiler can find the referred file. This can be a useful method if you want multiple components to use a particular CSS file and yet not make it a global style. If you want to include a **link** tag for the entire application, for instance, a **link** to the Bootstrap library, then it is better to add such links in **index.html** file rather than in individual component templates. We will see an example of this in the next section.

Using CSS frameworks

CSS frameworks such as Bootstrap, Foundation, and Materialize provide a set of pre-built CSS styles and components that you can use in your Angular application.

Here is an example of how you can use Bootstrap to add a navigation bar to your Angular application. Add a link tag to the **index.html** file as follows:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet" integ
```

Now use the CSS class names from Bootstrap to style the navigation bar in **app.component.html**:

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <a class="navbar-brand" href="#">The Fruit  Store</a>
 <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav" aria-
  controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav">
      <li class="nav-item active">
        <a class="nav-link" href="#">Home</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="#">Products</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="#">Deals</a>
      </li>
    </ul>
```

```
18.   </div>
19. </nav>
```

Importing library packages

It is a common practice in Angular projects to use libraries like **Angular Material**. This helps a lot in bootstrapping your app with professional-looking atomic controls like buttons, text fields, etc., which are industry-tested and come with built-in accessibility, internationalization, transitions, and theme support. To use such a library package, add the package dependency as follows:

```
ng add @angular/material
```

Import the components from the library that you want to use into **AppModule**:

```
1. import { MatButtonModule } from '@angular/material/button';
2.
3. @NgModule({
4.   imports: [MatButtonModule],
5.   exports: [MatButtonModule]
6. })
7. export class AppModule {
8.   /* ... */
9. }
```

Then use the **mat-button** directive on any button as follows:

```
1. <button mat-button>Click me!</button>
```

For more details on using **Angular Material**, please see the link in the *References* section.

View encapsulation

As its name suggests, CSS cascades down the DOM tree. So, a style statement applied to a parent element automatically applies to its children unless overridden by child elements. But in the case of Angular, it is important to understand that styles defined for a component are scoped for that component and are not available for any other component. For an analogy, imagine these as private variables of the given component. Other components, whether outside, sibling, or even nested components, do not have access to the styles defined for a particular component. This technique of scoping styles to each component is called view encapsulation. To demo this, create two simple components, **comp-a** and **comp-b**. You can use the following command to inline the style and template in the metadata and skip the test file. This is handy for creating components for demo purposes:

```
ng g c comp-a --inline-style --inline-template –skip-tests
```

Notice that now the **.html**, **.css** and **.spec.ts** files are not created:

Figure 6.3: *Creating a component with inline styles and template with no tests*

Similarly, create **comp-b**. Set the inline style to increase **font-size** in only **CompAComponent** to a large size as follows:

```
1. @Component({
2.   selector: 'app-comp-a',
3.   template: `
4.     <p>
5.       comp-a works!
6.     </p>
```

```
  `,
  styles: [ `p { font-size: 30px; }`
  ]
})
export class CompAComponent {
}
```

Keep **comp-b** as it is and show both the components as siblings within **app.component.html**:

```
<app-comp-a></app-comp-a>
<app-comp-b></app-comp-b>
```

Notice that although you applied style to **<p>** tag in **CompAComponent**, it did not apply to **<p>** tag of **CompBComponent**:

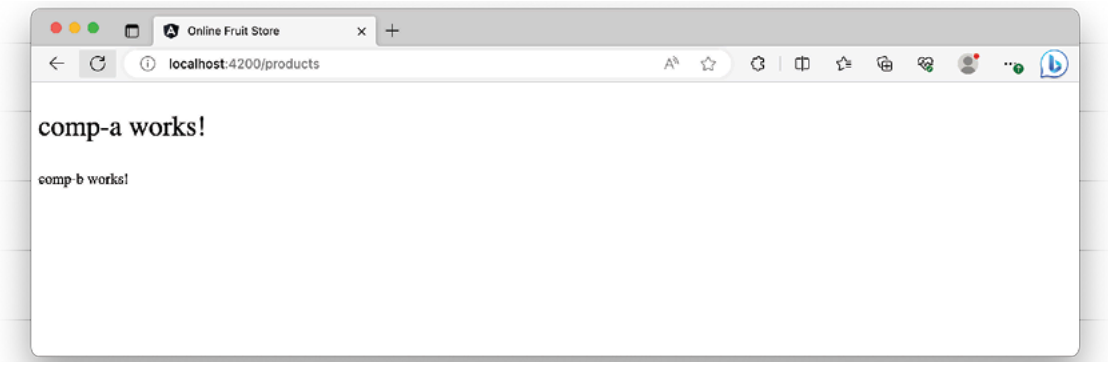


Figure 6.4: *Demo of View encapsulation*

If you inspect CompA text in the dev tool, you will notice that the style is scoped with a unique ID for CompA, which is added as an attribute to its DOM. This prevents the style from applying to **<p>** tag of CompB, as shown:

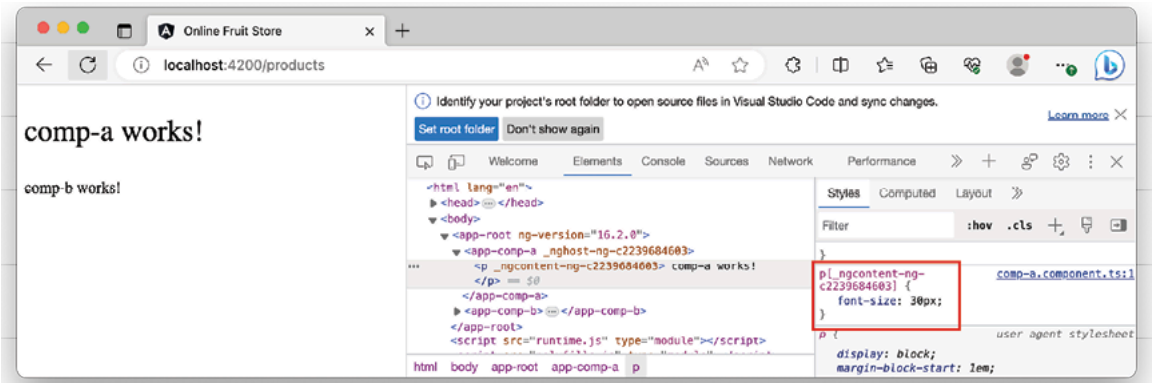


Figure 6.5: *Scoping styles with component ID*

CSS was created with reusability and performance in mind. In an HTML page, when certain elements have the same look and feel, CSS class names can be reused across such elements without having to rewrite styles individually for each element. Cascading of stylesheets is a feature that helps keep style statements small, lean, and performant. However, in complex applications, an element may mistakenly use a class name already defined in the style files that is used somewhere else in the application. This causes styles created for one element to be applied to the other. So, the wrong styles are applied to an element. Such issues are difficult to debug and to determine the source of styles applied to elements. Hence, view encapsulation in Angular is a very useful feature to avoid such a scenario.

Remove CompB from **app.component.html** and add it inside CompA as a child as follows:

```
@Component({
  selector: 'app-comp-a',
  template: `
```

```
    <p>
      comp-a works!
    </p>
    <app-comp-b></app-comp-b>
  `,
  styles: [ `p { font-size: 30px; }`
  ]
})
export class CompAComponent {
}
```

You will see that even in this case, the style applied to **<p>** tag of CompA does not apply to that of CompB.

In *Figure 6.5*, you will notice that there are two kinds of attributes added to CompA:

- **_nghost-ng-<comp id>**: This is applied to the element encapsulating the component view. In terms of shadow DOM, this is the shadow root.
- **_ngcontent-ng-<comp id>**: This is applied to the child elements within the component that are styled.

You can change the view encapsulation mode for a component by using the encapsulation property in the component metadata. Angular supports three modes of view encapsulation:

- **ViewEncapsulation.Emulated**: This is the default mode, as described above. It emulates the behavior of shadow DOM by adding attributes to the elements and modifying the CSS selectors. The styles are added to the **<head>** tag but are scoped with the component ID so that they apply to the specific components.
- **ViewEncapsulation.None**: This mode disables view encapsulation for the component. It applies the styles globally without any scoping or modification. It can be useful for defining common styles or themes for your application or when you deliberately want parent component style statements to be applied to child components. Even in this mode, the style statements are applied to the **<head>** tag, but the style statements are not scoped with the component ID and hence apply globally.
- **ViewEncapsulation.ShadowDom**: This mode uses the native shadow DOM implementation of the browser, if available. It creates a shadow root for the component and attaches a shadow DOM to it. It applies the styles only to the elements inside the shadow DOM and isolates them from the rest of the page.

If you are not familiar with shadow DOM, please refer to the link provided in the *References* section. In short, a shadow DOM creates a boundary around itself such that although it is rendered in the browser as part of the DOM tree, the styles defined inside it are not exposed to other elements in the DOM tree. This creates an encapsulation of the view within a shadow DOM.

To see the above view encapsulation in action, add the following metadata to CompA:

```
@Component({
  selector: 'app-comp-a',
  template: `
    <p>
      comp-a works!
    </p>
    <app-comp-b></app-comp-b>
  `,
  styles: [ `p { font-size: 30px; }`
  ],
  encapsulation: ViewEncapsulation.None,
})
export class CompAComponent {
}
```

Now the style is applied to both the components, and the **_nghost** and **_ngcontent** attributes are not added:

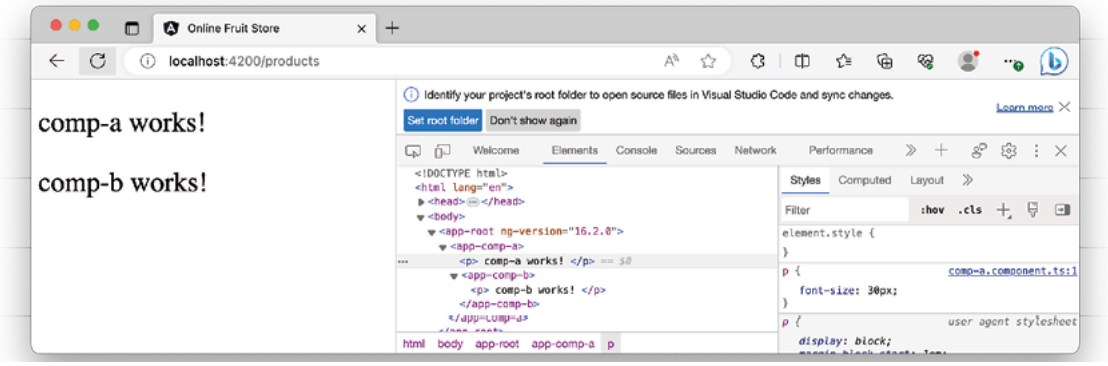


Figure 6.6: *Global styles with ViewEncapulation.None*

If you set **encapsulation: ViewEncapsulation.ShadowDom**, the styles are set specifically on the shadow DOM host. In our code, if we set it on CompA, it creates a shadow DOM for CompA and since CompB is used within CompA, all styles applied to CompA are now applied to CompB as well, as shown in the following figure:

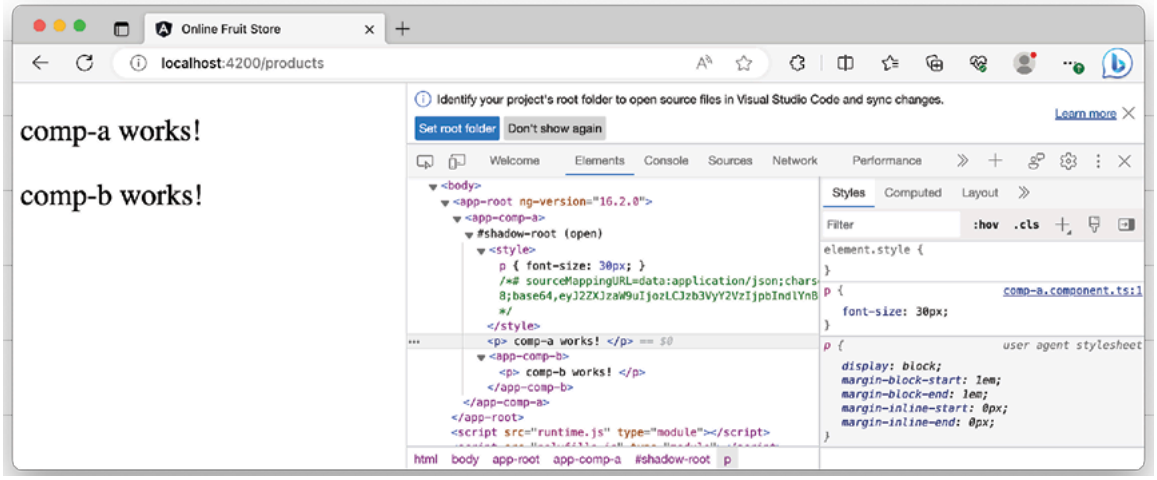


Figure 6.7: *Shadow Root with ViewEncapsulation.ShadowDom*

If CompB is moved out to be a sibling of CompA, then the styles of CompA will not apply to CompB.

Global styles

You can define global styles in **src/styles.css** file. Styles applied in this file will apply to all components. Set the following style in the **src/styles.css** file:

```
p {
    font-size: 50px;
}
```

This will apply to all elements on the page. Any style applied to the component-specifically will override global styles. For instance, since we applied **font-size** as **30px** for **<p>** tag in CompA, that will take precedence:

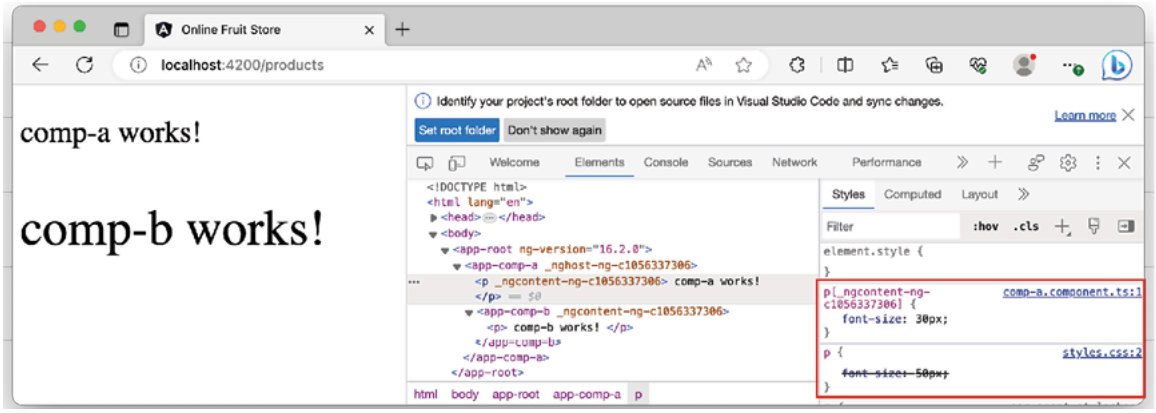


Figure 6.8: *Component style overriding global style*

CSS preprocessors

CSS preprocessors are popular tools that help developers write CSS code more efficiently. In an Angular application, you can use CSS preprocessors such as Sass, SCSS, or Less to enhance your styling capabilities.

Sass and SCSS are the most popular CSS preprocessors and have been widely adopted by the Angular community. Sass/SCSS provides a set of features that are not available in plain CSS, such as variables, nested rules, mixins, and functions. These features allow you to write more modular and reusable CSS code, which can save you time and effort in the long run. A detailed description of Sass or SCSS is out of the scope of this book. You can refer to the link in the *References* section to understand the language constructs of Sass/SCSS. To create a project with SCSS, you can use the **-style** switch as follows:

ng new fruit-store –style=scss

This will add the dependent **npm** packages to support SCSS and create **.scss** files per component instead of **.css**.

Less is another popular CSS preprocessor that is similar to Sass in many ways. Less also provides variables, mixins, and functions, but it has a simpler syntax compared to Sass. Less is a good choice if you are new to CSS preprocessors or if you prefer a simpler syntax. Please see the *References* section for details on Less syntax.

You can use the following command to create a project with Less:

ng new fruit-store –style=less

Similarly, now **.less** file will be created per component instead of **.css**.

Angular used to support Stylus as well but has been deprecated now due to relatively low usage (about 0.3% of users).

Using SCSS @mixin

SCSS mixins are a way of reusing a block of style declarations in different places. They can help you avoid repeating code and make your styles more modular and maintainable. To use SCSS mixins in Angular, you need to follow these steps:

1. Define a mixin using the **@mixin** directive, followed by a name and optionally some parameters. For example, you can define a mixin for setting the **font-size** and **color** of a text element. Create a file **src/mixins.scss** and add the following code:

```
@mixin text-style($size, $color) {
  font-size: $size;
  color: $color;
}
```

2. Import the mixin file in your component's SCSS file using the **@import** directive:

```
@import 'src/mixins';
```

3. Use the mixin in your component's style rules by calling it with the **@include** directive, followed by the name and optionally some arguments. For example, you can use the **text-style** mixin to style a paragraph

element:

```
p {
  @include text-style(16px, blue);
}
```

4. This will compile to the following CSS:

```
p {
  font-size: 16px;
  color: blue;
}
```

You can also use SCSS mixins from external libraries, such as Bootstrap, by importing them from the **node_modules** folder.

Angular specific selectors

Angular also provides some special selectors that you can use in your component styles to target specific elements or scenarios. These selectors are based on the shadow DOM style scoping. Some of these selectors are as follows:

- **:host**: This selector targets the element that hosts the component. You can add CSS selectors for all the inner elements of a component to apply styles. However, there are scenarios when you want to apply a style on the component host element, for instance, when you want to set the border color of a component. For such cases, you can set styles on **:host** selector. You can try this on **app-comp-a.component.css** as follows:

```
:host {
  display: block;
  border: 1px solid black;
}
```

This selector will apply the styles to the **<app-comp-a>** element that hosts the **CompAComponent**. As we saw earlier, this will add a scoped style to the element with **_nghost-ng-<comp id>** attribute as follows:

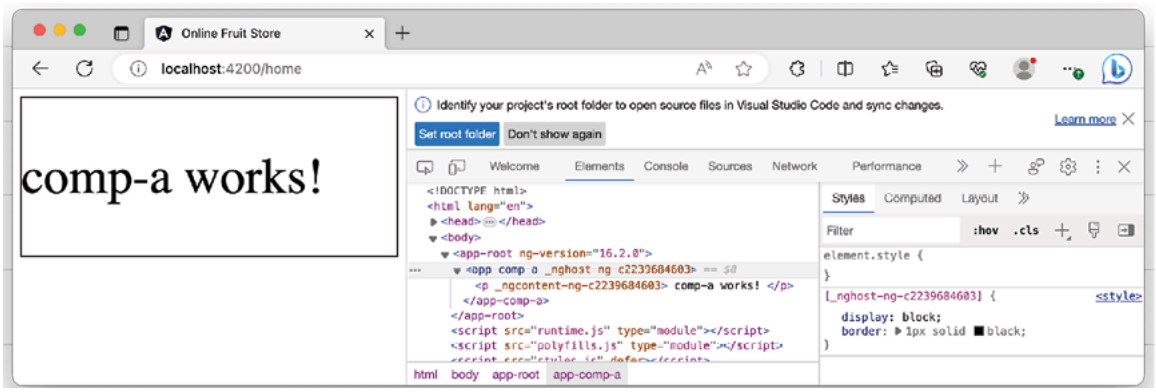


Figure 6.9: *Applying styles to :host*

- **:host-context**: This selector targets the element that hosts the component based on some external condition. For example:

```
:host-context(.dark-theme) {
  background-color: black;
  color: white;
}
```

This selector will apply the styles to the **<app-comp-a>** element that hosts the **CompAComponent** only if it is inside an element with the class **.dark-theme**.

- **::ng-deep**: This selector allows you to target elements inside a component's template, even if they are inside another component's template. For example:

```
::ng-deep p {
  font-size: 30px;
}
```

This selector will apply the styles to all **<p>** elements inside an **<app-comp-a>** element in the DOM tree, even if they are part of another component's template.

Authoring style changes

CSS custom properties, also known as **CSS variables**, allow you to define reusable values in CSS that can be used throughout your application. In Angular, you can use CSS custom properties to define and reuse values across your components, making it easier to maintain your application's styles.

In our CompA, we can define a CSS custom property to set color using the **--** prefix as follows:

```
:host {
  --primary-color: blue;
}
```

Once the variable is defined, it can be used in any CSS statement as follows:

```
p {
  color: var(--primary-color);
}
```

You may also use your CSS custom properties in your template. For instance, the above code could be re-written as follows:

```
<p [style.color]="'var(--primary-color)'">
  comp-a works!
</p>
```

A cleaner practice is to set CSS custom properties globally so that all components can leverage them. If you want to use your CSS custom properties globally across your application, you can define them in **src/styles.css** file as follows:

```
:root {
  --primary-color: #007bff;
}
```

Here, we have defined the **--primary-color** custom property in the **:root** selector. We can then use the custom property in any component's CSS file.

Theming

Theming is a way of customizing the appearance of your application by using CSS variables or SCSS mixins. Instead of using hard-coded color and font styles, if these are stored in CSS variables, it makes it easy to change between different themes like light mode, dark mode, or any specific color palette. You can create different themes for your application and switch between them dynamically.

In the previous section, we saw how to create CSS variables and use them in our component styles. We can leverage that technique to create theme support. There are several ways to implement themes in Angular. One easy way is to define the colors inside the **AppComponent** and provide a method to switch between various themes based on user preference as follows:

```
import { Component, Renderer2 } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.scss']
```

```
})
export class AppComponent {
  constructor(private renderer: Renderer2) {}

  // A method to switch between light and dark themes
  switchTheme(isDark: boolean) {
    // Define some colors for light and dark themes
    const lightTheme = {
      '--primary-color': '#1976d2',
      '--accent-color': '#e91e63',
      '--warn-color': '#f44336',
    };

    const darkTheme = {
      '--primary-color': '#90caf9',
      '--accent-color': '#ff80ab',
      '--warn-color': '#ff8a80',
    };

    // Get the root element of the document
    const root = document.documentElement;

    // Loop through the theme colors and set them as CSS variables
    const theme = isDark ? darkTheme : lightTheme;
    for (const [key, value] of Object.entries(theme)) {
      this.renderer.setStyle(root, key, value);
    }
  }
}
```

If the number of variables is huge, you can refactor the above code to place theme variables in separate **light-theme.scss** and **dark-theme.scss** files and import dynamically as needed.

Setting style using TypeScript

If you want to apply style on a specific element in the view template based on business conditional paths, you can do it in two ways. One way would be to use **Renderer2** as we saw in the section above. The other way would be to use **ElementRef**. The code would look like this:

```
@Component({
  selector: 'app-comp-a',
  template: `
    <p #para1>
      comp-a works!
    </p>
    <button (click)="changeColor('blue')">Change color</button>
  `})
export class CompAComponent {

  @ViewChild('para1') para: ElementRef | undefined;

```

```
  // A method to change the color of the para element
  changeColor(color: string) {
    // Get the native DOM element
    const paraElement = this.para?.nativeElement as HTMLElement;

    // Set the style property of the element
    paraElement.style.color = color;
  }
}
```

Yet, this is not a recommended practice as it can cause security issues and break server-side rendering.

Using Tailwind with Angular

Tailwind CSS is a utility-first CSS framework that allows you to create custom designs using predefined classes. It has gained increasing popularity in recent times. It is compatible with Angular. To use Tailwind CSS with Angular, you can add the Tailwind CSS package as follows:

npm install -D tailwindcss

Generate a Tailwind configuration file using the **tailwindcss** command. This file will let you customize your theme, plugins, and other options. Run this command in your terminal:

npx tailwindcss init

Now, edit the **tailwind.config.js** file and add the paths to your Angular template files in the content property. This will enable Tailwind to scan your HTML files and remove unused classes. For example, you can add something like this:

```
module.exports = {
  content: [
    './src/**/*.html',
    './src/**/*.ts',
  ],
  // ...
}
```

Import the Tailwind **base**, **components**, and **utilities** styles in your global style file, **src/styles.scss**. You can use the **@tailwind** directive to do this. For example, you can add something like this:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Now you can start using Tailwind classes in your Angular components to style your elements. For example, you can try the following code in **app.component.html** file:

```
<h1 class="text-4xl font-bold text-center text-blue-600">Text styled by Tailwind</h1>
```

This will apply the following styles to your heading element:

```
h1 {
  font-size: 2.25rem;
  font-weight: 700;
  text-align: center;
  color: #60a5fa;
}
```

You can also use Tailwind directives and functions in your component styles to extend or customize the classes. For example, you can add something like this in your **app.component.scss** file:

```
@import 'tailwindcss/base';
@import 'tailwindcss/components';
@import 'tailwindcss/utilities';
```

```
4.
5. h1 {
6.   @apply text-4xl font-bold text-center text-blue-600;
7.   @screen md {
8.     @apply text-6xl;
9.   }
10. }
```

This will apply the same styles as before but also increase the font size on medium screens. You can find more about Tailwind CSS in the link provided in the *References* section.

Styling best practices

When it comes to styling in Angular, there are some best practices you can follow to ensure your styles are consistent, maintainable, and scalable. Some of these best practices include using CSS and SCSS to organize your styles, avoiding inline styles, and using component-specific stylesheets to keep your code modular and reusable.

It is recommended to use view encapsulation to prevent styles from leaking into other components that might create intended style changes. It is also important to avoid using complex selectors and instead use class names and attribute selectors to target specific elements.

It is best to avoid style changes using TypeScript API as those are performance intensive operations.

Conclusion

Styling is a fundamental aspect of building a successful Angular application. A well-thought-out design and styled user interface can create a pleasant user experience and improve user engagement on the page. By using CSS and SCSS, along with Angular powerful styling features, you can create a consistent and visually appealing user interface that enhances the user experience. By using CSS custom properties, SCSS variables, and mixins, you can create reusable styles that can be applied across your application. This approach makes it easier to maintain your application's styles and allows you to switch between themes dynamically.

In the next chapter, we will go over the language constructs of TypeScript in detail.

Points to remember

- There are many ways to style components in Angular.
- Using **styleUrls** metadata is the most recommended way, as it creates a more maintainable code.
- One can leverage many libraries and packages for pre-built styles.
- SCSS offers many advantages over pure CSS for writing efficient code.
- It is best to avoid style changes in TypeScript API.
- Tailwind CSS is a popular utility library of predefined CSS classes.

Questions

1. Which metadata should be preferred for styling? styles or styleUrls? Why?
2. What is the difference between adding a <link> tag in a component view template versus in the index.html file?
3. Name a few CSS frameworks and libraries that can be used in Angular projects.
4. Explain how to switch between light and dark mode in an Angular app.
5. As an exercise, style the entire fruit store application components to provide a user-appealing look and feel.

References

- Shadow DOM:
  https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM
- Sass/SCSS:

https://sass-lang.com/

- Less:

  https://lesscss.org/

- Angular Material:

  https://material.angular.io/guide/getting-started

- Tailwind CSS:

  https://tailwindcss.com/

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 7
TypeScript Language Constructs

Introduction

TypeScript is a powerful language that offers several language constructs to help make your code in Angular more robust and maintainable. These language constructs include type annotations, interfaces, enums, modules, classes, and more. By using these constructs in your Angular application, you can catch errors early, improve the clarity and safety of your code, and make your code more maintainable. In this chapter, we will explore some of the key TypeScript language constructs used in Angular.

Structure

The chapter covers the following topics:

- Introduction to TypeScript
- Type annotation
- Classes
- Interfaces
- Modules
- Enums
- TypeScript config file

Objectives

By the end of this chapter, you will understand some of the language constructs provided by TypeScript on top of regular JavaScript that helps to organize the code better in Angular.

Introduction to TypeScript

TypeScript is a superset of JavaScript first introduced by *Microsoft* in 2012. It is an

open-source language that offers several features to help make your code more robust and maintainable. In the early 2010s, JavaScript as a language was still evolving and was not suited for a complex project. Developers faced issues with type incompatibility at runtime. Runtime errors in the browser were difficult to debug. JavaScript is the only language that runs on all browsers. As frontend technologies evolved, there was a need to add type checking in JavaScript code. The JavaScript standard was lagging behind the industry demand, and having a type check implementation in all browsers was not feasible at the given time. JavaScript running on the server-side Node.js faced similar challenges. These created a need for a community-based implementation of a language that could provide type annotation on top of regular JavaScript code. That is how TypeScript evolved.

TypeScript introduced static type, which could catch type errors at compile time. TypeScript code can be compiled into plain JavaScript using the TypeScript compiler, which also performs type checking and other validations. TypeScript is designed to make JavaScript development more reliable, scalable, and productive. TypeScript can run on any browser, any host, and any operating system that supports JavaScript.

Some of the main benefits of using TypeScript are:

- TypeScript can help you catch errors at compile time rather than at runtime. This can save you time and effort in debugging and testing your code. For example, TypeScript can detect if you pass a wrong type of argument to a function or if you access a property that does not exist on an object.
- TypeScript can help you document your code by specifying the types of variables, parameters, functions, and properties. This can make your code more readable and maintainable, as well as enable better tooling support, such as code completion, refactoring, and navigation.
- TypeScript can help you use modern JavaScript features that may not be supported by older browsers or environments. TypeScript supports features such as classes, modules, decorators, generics, etc., which can make your code more expressive and concise. TypeScript can compile these features to compatible JavaScript code that can run on any target platform.

TypeScript is a superset of JavaScript, which means that any valid JavaScript code is also valid TypeScript code. You can gradually adopt TypeScript in your existing JavaScript projects by adding type annotations and enabling strict mode. Angular adopted TypeScript from version 2+. Most other popular frameworks and libraries like React, Vue, and Node.js use TypeScript today because of the benefits it provides over plain JavaScript.

An exhaustive description of TypeScript features will be out of the scope of this book, but we will provide a short description of the language features used in an Angular project so that you feel comfortable using them while working on the code. Please refer to the link to official TypeScript documentation in the *References* section.

TypeScript files generally use the extension **.ts**. To compile any TypeScript file into

JavaScript, you can use the TypeScript compiler—**tsc**. This compiler can be downloaded from **npm** by running the following command:

```
npm install typescript --save-dev
```

Once installed, run the following command to compile a TypeScript file, say, **input.ts** as follows:

```
tsc input.ts
```

This will generate a JavaScript file **input.js**.

You can also install TypeScript globally with the following command:

```
npm install -g typescript
```

If you prefer not to download TypeScript, you can also run the compiler using **npx** as follows:

```
npx tsc input.ts
```

For more options to download the TypeScript compiler, check the *References* section.

If you wish to test TypeScript online, use the TypeScript playground link from the *References* section.

Type annotation

TypeScript allows you to annotate variables, parameters, functions, and properties with types, such as string, number, Boolean, array, object, function, etc. You can also define your own custom types using interface, type, or class keywords. TypeScript supports both basic types (such as string or number) and advanced types (such as union, intersection, literal, generic, etc.). Types help you catch errors at compile time, document your code, and enable better tooling support. For example, you can use types to define the shape of your data models, the inputs and outputs of your components, the parameters and return values of your services, etc.

Let us look at a few simple examples of type annotation. In the below code, defining a variable as a string type will throw an error at compile time if you try to assign a non-string value to it:

```
1. // Annotating a variable with a type
2. let name: string = "Sam";
```

Similarly, the parameters and return type of a function can be typed to catch any compile time error:

```
1. // Annotating a function parameter and return type
2. function greet(name: string): string {
3.   return “Hello, “ + name;
4. }
```

You may define an interface and then use that as a type for a JSON object as follows:

```
1. // Defining a custom type using an interface
2. interface Person {
```

```
    name: string;
    age: number;
}
```

This ensures that a variable of type **Person** must adhere to the interface definition:

```
// Using a custom type as an annotation
let person: Person = {
    name: "Veda",
    age: 25,
};
```

To test type safety, try to create an object without the **name** field as follows:

```
let person: Person = {
    age: 25,
};
```

During compilation, you will see an error as follows:

```
Property 'name' is missing in type '{ age: number; }' but required in type 'Person'.
```

Similarly, try setting the **name** field as a number instead of a string, as follows:

```
let person: Person = {
    name: 123,
    age: 25,
};
```

This will raise a compile time error as follows:

```
Type 'number' is not assignable to type 'string'.
```

JavaScript does not have any type annotations or declarations and relies on dynamic typing at runtime. The above examples can be written in JavaScript as follows:

```
// No type annotations in JavaScript
let name = "Sam";

// No type checking in JavaScript
function greet(name) {
    return "Hello, " + name;
}

// No custom types in JavaScript
let person = {
    name: "Veda",
    age: 25,
};
```

Classes

TypeScript supports the class-based object-oriented programming paradigm, which is also available in modern JavaScript. Classes are templates that define the structure and behavior of objects. They can have properties, methods, constructors, access modifiers (public, private, protected), static members, abstract members, etc. Classes can also inherit from other classes using the **extends** keyword or implement interfaces using the **implements** keyword. In Angular, we use classes to create components, services, directives, pipes, etc. In the previous chapters, we also looked at how to use decorators to add metadata to these classes, such as **@Component**, **@Injectable**, **@Directive**, etc. The following sample code shows definitions of a class in TypeScript:

```
1.  // Defining a class
2.  class Animal {
3.    // A property with a default value
4.    public name: string = "Unknown";
5.
6.    // A constructor that initializes the property
7.    constructor(name: string) {
8.      this.name = name;
9.    }
10.
11.   // A method that prints the name
12.   public speak() {
13.     console.log("My name is " + this.name);
14.   }
15. }
16.
17. // Creating an object from the class
18. let animal = new Animal("Leo");
19.
20. // Calling the method
21. animal.speak(); // My name is Leo
```

In the above code, notice how the field **name** has been declared with a **public** access modifier and is defined to be of type **string**:

```
1.  public name: string = "Unknown";
```

In JavaScript, we would define it as simply:

```
1.  name = "Unknown";
```

If the field may not be initialized during declaration or in the class constructor, then

we can define the field as nullable by setting the type as **string** or **undefined** as follows:

```
public name: string | undefined;
```

The same sentence can also be written as:

```
public name?: string;
```

If you want to allow setting **null** as a valid value, you can define the type as follows:

```
public name: string | null;
```

A class may be derived from another class as follows:

```
class Dog extends Animal {
  // A property specific to the subclass
  public breed: string;

  constructor(name: string, breed: string) {
    super(name); // Calling the superclass constructor
    this.breed = breed;
  }

  // This method  overrides the superclass method
  public speak() {
    console.log("Woof! I am " + this.name + ", a " + this.breed);
  }
}

let dog = new Dog("Max", "Labrador");
dog.speak(); // Woof! I am Max, a Labrador
```

A class may also implement an interface. We will see an example in the next section.

JavaScript also supports classes, but they are syntactic sugar over the prototype-based inheritance model. JavaScript classes do not have access modifiers, abstract members, or interface implementation. The **Animal** and **Dog** classes defined above can be written using JavaScript as follows:

```
class Animal {
  // A property with a default value (no access modifier)
  name = "Unknown";

  constructor(name) {
    this.name = name;
  }

```

```
9.  // no access modifier for the method
10.  speak() {
11.    console.log("My name is " + this.name);
12.  }
13. }
14.
15. let animal = new Animal("Leo");
16. animal.speak(); // My name is Leo
17.
18. // Defining a subclass that inherits from the superclass
19. class Dog extends Animal {
20.  // A property specific to the subclass (no access modifier)
21.  breed;
22.
23.  constructor(name, breed) {
24.    super(name); // Calling the superclass constructor
25.    this.breed = breed;
26.  }
27.
28.  // This method overrides the superclass method (no access modifier)
29.  speak() {
30.    console.log("Woof! I am " + this.name + ", a " + this.breed);
31.  }
32. }
33.
34. let dog = new Dog("Max", "Labrador");
35. dog.speak(); // Woof! I am Max, a Labrador
```

Interfaces

An interface in TypeScript is a way of defining a contract or a shape for an object, class, or function. An interface specifies the names and types of the properties and methods that an object, class, or function must have. An interface can also extend or inherit from other interfaces using the **extends** keyword. An interface can be used to describe the structure of a data model, the input and output of a component, the behavior of a service, etc.

For example, you can define an interface called **Person** that has two properties: **name** and **age**, of types **string** and **number** respectively. Then, you can use this interface to annotate a variable, a parameter, or a return value that must be an object

with those properties. You can also implement this interface in a class that defines a **constructor** and a method using those properties. Here is an example of how to use an interface in TypeScript:

```
1.  // Defining an interface
2.  interface Person {
3.    name: string;
4.    age: number;
5.  }
6.
7.  // Using the interface as an annotation for a variable
8.  let person1: Person = {
9.    name: "Veda",
10.    age: 25,
11.  };
12.
13.  // Using the interface as an annotation for a function parameter
14.  function greet(person: Person) {
15.    console.log("Hello, " + person.name + ". You are " + person.age + " years
     old.");
16.  }
17.
18.  // Using the interface as an annotation for a function return value
19.  function createPerson(name: string, age: string): Person {
20.    return {
21.      name: name,
22.      age: age,
23.    };
24.  }
25.
26.  // Implementing the interface in a class
27.  class Student implements Person {
28.    name: string;
29.    age: number;
30.
31.    constructor(name: string, age: number) {
32.      this.name = name;
33.      this.age = age;
34.    }
```

```
35.
36.    introduce() {
37.    console.log("I am " + this.name + ", and I am " + this.age + " years old.");
38.    }
39.  }
40.
41.  let student1 = new Student("Adi", 20);
42.  greet(student1); // Hello, Adi. You are 20 years old.
43.  student1.introduce(); // I am Adi, and I am 20 years old.
```

JavaScript does not have the concept of an interface.

Modules

TypeScript supports the modular programming paradigm, which allows you to organize your code into separate files or units called modules. Modules can export and import variables, functions, classes, interfaces, types, etc., using the **export** and **import** keywords. Modules help you avoid naming conflicts, reuse code across different files or projects, and structure your code in a logical way. In Angular, you can use modules to create feature modules, shared modules, lazy-loaded modules, etc. You can also use decorators to declare modules and their metadata, such as **@NgModule**, **@Import**, **@Export**, etc, as shown:

```
1.  // Defining a module in a file called math.ts
2.
3.  // Exporting a variable
4.  export const PI = 3.14;
5.
6.  // Exporting a function
7.  export function add(x: number, y: number): number {
8.    return x + y;
9.  }
10.
11.  // Exporting a class
12.  export class Calculator {
13.    // A method that uses the exported variable and function
14.    public calculateArea(radius: number): number {
15.      return PI * add(radius, radius);
16.    }
17.  }
18.
```

```
// Importing the module in another file called app.ts

// Importing everything from the module using an alias
import * as math from "./math";

// Using the imported variable, function, and class
console.log(math.PI); // 3.14
console.log(math.add(1, 2)); // 3
let calculator = new math.Calculator();
console.log(calculator.calculateArea(5)); // 157.0
```

JavaScript also supports modules, but they are not widely supported by all browsers or environments. JavaScript modules use the same export and import syntax as TypeScript, but they require a special attribute in the **script** tag to indicate that they are modules:

```
<script type="module" src="app.js"></script>
```

Enums

Enums are a feature of TypeScript that allows you to define a set of named constants that can be either numeric or string based. Enums can make your code more readable and expressive, as well as help you avoid errors and bugs. For example, you can use enums to represent different directions, status codes, user responses, etc.

To define an enum, you use the **enum** keyword followed by the name of the enum and a block of code that contains the enum members. Each enum member can have an optional initializer that specifies its value. If the initializer is omitted, the value is either 0 for the first member or 1 more than the previous member for the subsequent members. For example:

```
enum Direction {
  Up, // 0
  Down, // 1
  Left, // 2
  Right // 3
}

// A numeric enum with initializers
enum StatusCode {
  NotFound = 404,
  Success = 200,
  Accepted = 202,
```

```
  BadRequest = 400
}

// A string enum with initializers
enum UserResponse {
  No = "NO",
  Yes = "YES"
}
```

To use an enum, you can access its members as properties of the enum itself or assign them to variables or parameters of the same type as the enum. You can also use reverse mapping to get the name of an enum member from its value. For example:

```
// Accessing an enum member
console.log(Direction.Up); // 0

// Assigning an enum member to a variable
let currentDirection: Direction = Direction.Down;

// Using an enum member as a function parameter
function move(direction: Direction) {
  // Do something based on direction
}

// Calling a function with an enum member as an argument
move(Direction.Left);

// Using reverse mapping to get the name of an enum member from its value
console.log(Direction[2]); // Left
```

TypeScript config file

You might have observed the **tsconfig.json** file at the root of the Angular workspace we created in the previous chapters. For reference, a typical file looks like this:

```
/* To learn more about this file see: https://angular.io/config/tsconfig. */
{
  "compileOnSave": false,
  "compilerOptions": {
    "baseUrl": "./",
    "outDir": "./dist/out-tsc",
```

```
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "noImplicitOverride": true,
    "noPropertyAccessFromIndexSignature": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "sourceMap": true,
    "declaration": false,
    "downlevelIteration": true,
    "experimentalDecorators": true,
    "moduleResolution": "node",
    "importHelpers": true,
    "target": "ES2022",
    "module": "ES2022",
    "useDefineForClassFields": false,
    "lib": [
      "ES2022",
      "dom"
    ]
  },
  "angularCompilerOptions": {
    "enableI18nLegacyMessageIdFormat": false,
    "strictInjectionParameters": true,
    "strictInputAccessModifiers": true,
    "strictTemplates": true
  }
}
```

The **tsconfig.json** file is a configuration file for the TypeScript compiler that specifies the **root** files and the compiler options for a TypeScript project. In Angular, a given workspace contains several TypeScript configuration files that are inherited from the root **tsconfig.json** file. Following are some of the keys present in the **tsconfig.json** file in Angular and their meanings:

- compilerOptions: This key contains a set of options that affect how the compiler generates JavaScript code from TypeScript source files. Some of the common options are:
  - target: The ECMAScript version that the compiler will use to emit JavaScript code. For example, **target: es5** means that the compiler will generate ES5 compatible code.

- module: The module system that the compiler will use to organize and reference modules. For example, **module: system** means that the compiler will use SystemJS as the module loader.
- moduleResolution: It specifies how the compiler will resolve module imports. For example, **moduleResolution: node** means that the compiler will use Node.js style resolution algorithm to find modules.
- sourceMap: This determines whether the compiler will generate source map files that allow debugging TypeScript files directly in the browser.
- emitDecoratorMetadata: This option enables emitting design-type metadata for decorated declarations in source. This is required for Angular to use dependency injection and other features that rely on decorators. Setting it to true means that the compiler will emit metadata for classes and parameters decorated with **@Injectable**, **@Component**, etc.
- experimentalDecorators: This option enables experimental support for ES7 decorators, which are used extensively in Angular. For example, **experimentalDecorators: true** means that the compiler will accept decorator syntax and emit decorator calls in the output code.
- removeComments: This determines whether the compiler will remove comments from the output code.
- noImplicitAny: It controls whether the compiler will allow variables or parameters to have implicit any type, which means that they can hold any value. If set to false, the compiler will not report an error when a variable or parameter has no explicit type annotation, and no type can be inferred from its usage.
- exclude: List of files or folders that should not be included in the compilation.
- include: List of files or folders that should be included in the compilation.
- files: List of individual files that should be included in the compilation.
- extends: This key allows inheriting configuration options from another **tsconfig.json** file. Notice the file **tsconfig.app.json** in the **root** folder of our project. This config inherits from **tsconfig.json** using the key **extends**.

Conclusion

TypeScript offers a wide range of language constructs that can help make your code in Angular more robust and maintainable. By using features like type annotations, interfaces, enums, classes, and more, you can catch errors early, improve the clarity and safety of your code, and make your code more modular and reusable. These language constructs are particularly useful in Angular, where complex applications need to be built quickly and efficiently. They help ensure that your code is organized

and easy to maintain, making it easier to build and maintain large-scale applications. In the next chapter, we will go over the lifecycle hooks in Angular.

Points to remember

- TypeScript is a superset of JavaScript.
- TypeScript allows type annotations for variables and methods, which helps to catch errors at compile time.
- JavaScript does not have interfaces and enums.
- Modules help to organize our code better, which is needed in a complex app like Angular.

Questions

1. Explain the difference between type and interface.
2. Explain how a module definition in TypeScript is different from that in JavaScript.
3. Explain how to implement an enum in JavaScript.
4. What are the various ways to compile a TypeScript code into JavaScript?

References

- TypeScript handbook:
  https://www.typescriptlang.org/docs/handbook/intro.html
- TypeScript compiler download:
  https://www.typescriptlang.org/download
- TypeScript playground:
  https://www.typescriptlang.org/play

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 8
Lifecycle Hooks

Introduction

In Angular, components have a lifecycle that includes several different phases, from initialization to destruction. During these phases, Angular provides developers with a set of lifecycle hooks that can be used to perform specific actions at specific points in the lifecycle. These hooks allow you to interact with your component at different stages of its lifecycle, giving you fine-grained control over how it behaves. By using lifecycle hooks, you can perform actions like initializing data, detecting changes, and cleaning up resources.

Structure

This chapter covers the following topics:

- Lifecycle hooks
- ngOnChanges
- ngOnInit
- ngDoCheck
- ngAfterContentInit
- ngAfterContentChecked
- ngAfterViewInit and ngAfterViewChecked
- ngOnDestroy
- DestroyRef
- Ordering of lifecycle hooks

Objectives

By the end of this chapter, you will understand the different lifecycle hooks available in Angular and how to use them to build more robust and maintainable applications. There are eight lifecycle hooks, as described in the following sections.

Lifecycle hooks

Lifecycle hooks are special methods that Angular calls at specific points in the life cycle of a component or directive. They allow you to perform custom actions when the component or directive is created, updated, or destroyed. There are eight life cycle hooks available in Angular, as detailed in the sections below. Out of these, **ngOnInit**, **ngOnChanges,** and **ngOnDestroy** are the most used. Each of the lifecycle hooks is defined in its own interface. For instance, the interface **OnInit** defines the hook **ngOnInit**, **OnDestroy** defines **ngOnDestroy**, and so on. You need not implement all the interfaces, though.

ngOnChanges

The **ngOnChanges** hook is called when any data-bound property of the component or directive changes. It receives an object that contains the current and previous values of the changed properties. You can use this hook to react to changes in the input properties and perform any logic that depends on them. For instance, create a **counter** component that shows a **counter** passed as an input, as follows:

```
import { Component, Input, OnChanges, SimpleChanges } from "@angular/core";

@Component({
  selector: 'app-counter',
  template: `
    <p>Counter: {{ counter }}</p>
  `
})
export class CounterComponent implements OnChanges {
  @Input() counter: number = 0;
  ngOnChanges(changes: SimpleChanges) {
  console.log('Counter changed:', changes["counter"].currentValue, changes["counter"].previousValue);
  }
}
```

In the code above, we defined a component **CounterComponent** with an input property **counter**. We then implemented the **OnChanges** interface and defined the **ngOnChanges** method. This method takes a parameter **changes** of type **SimpleChanges**, which contains information about the changes to the input properties. We can use this method to perform actions whenever an input property changes.

Now, use this component in another component, say **AppComponent**. Pass on the **counter** as a prop. Also, add methods to **increment** and **decrement** the counter, as follows:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <app-counter [counter]="counter"></app-counter>
    <button (click)=»increment()»>+</button>
    <button (click)="decrement()">-</button>
  `
})
export class AppComponent {
  counter = 0;

  increment() {
    this.counter++;
  }

  decrement() {
    this.counter--;
  }
}
```

Click on the + and – buttons and observe the **Console** in the dev tool. It should show you messages indicating the current and previous values of the counter, as shown in the following figure:

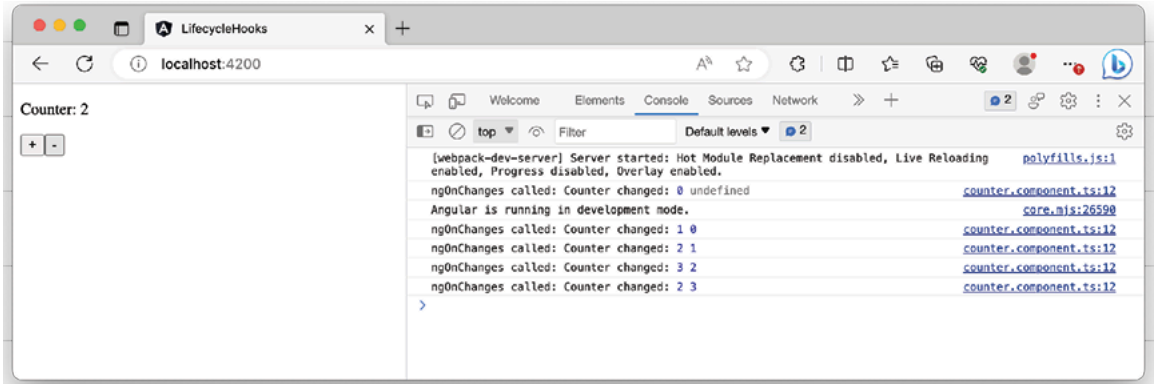


Figure 8.1: *ngOnChanges log messages*

This hook is useful when you need to perform actions based on changes to the component's input data. Note that this lifecycle hook is called when a value bound to an input property has changed, not when an internal field is modified. To verify this, make the counter an internal field in the **counter.component.ts** file as follows:

```
import { Component, DoCheck, Input, OnChanges, OnInit, SimpleChanges } from "@angular/core";

@Component({
  selector: 'app-counter',
  template: `
    <p>Counter: {{ counter }}</p>
    <button (click)=»increment()»>+</button>
    <button (click)="decrement()">-</button>
  `
})
export class CounterComponent implements OnChanges {
  @Input() counter: number = 0;

  ngOnChanges(changes: SimpleChanges) {
  console.log('ngOnChanges called: Counter changed:', changes["counter"].currentValue,
  changes["counter"].previousValue);
  }

  increment() {
    this.counter++;
  }

  decrement() {
    this.counter--;
  }
}
```

Render the component without the **counter** prop as follows:

```
@Component({
  selector: 'app-root',
  template: `
    <app-counter></app-counter>
  `
})
export class AppComponent {
```

```
8. }
```

Now, on clicking the + and – buttons, you will observe that the counter value changes, but there are no messages on the **Console** for calls made to **ngOnChanges**. Refer to the following figure:

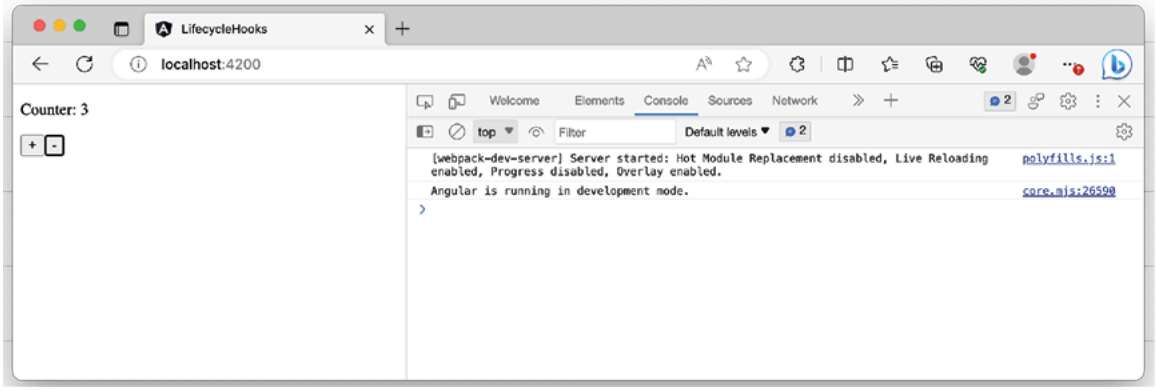


Figure 8.2: *ngOnChanges log messages not displayed on the internal field changes*

To handle changes to internal fields, we need to use the **ngDoCheck** lifecycle hook, which we will see later in this chapter.

ngOnInit

The **ngOnInit** hook is called once after the component or directive is initialized and before the component is displayed. It is called after the first **ngOnChanges** call. It is a good place to initialize any data or logic that is not dependent on the input properties. Add this hook in the above code example in the **counter.component.ts** file to observe the sequence of call to hooks:

```
1. ngOnInit() {
2.   console.log('Counter component initialized');
3. }
```

Refresh the browser page and observe the order of console log messages, as shown in the following figure:

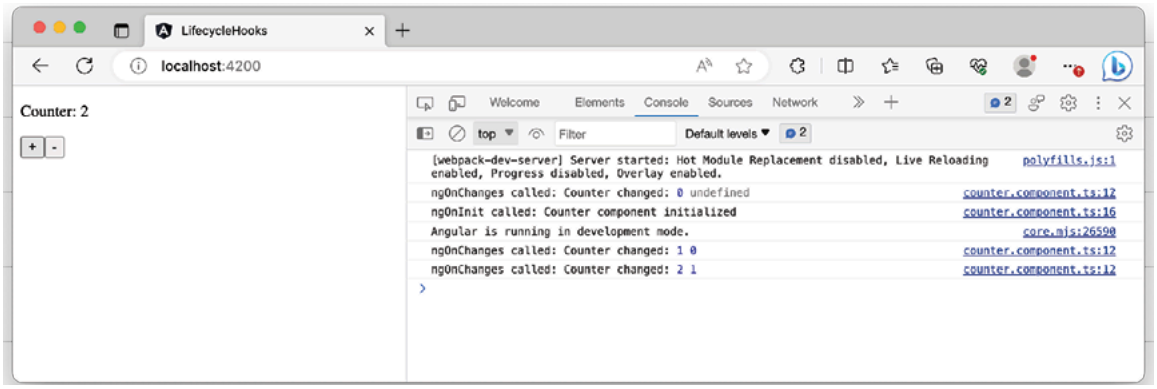


Figure 8.3: *ngOnInit log message*

ngDoCheck

The **ngDoCheck** hook is called every time Angular runs change detection on the component or directive. It allows you to implement your own custom change detection logic and detect changes that Angular might miss. In *Figure 8.2*, we saw that changes to the internal fields of a component are not handled by the **ngOnChanges** hook. To handle these changes, replace the **OnChanges** interface with the **DoCheck** interface and implement **ngDoCheck** method as follows:

```
1. export class CounterComponent implements DoCheck {
2.   ...
3.   ngDoCheck() {
```

```
    console.log(‘ngDoCheck called: Counter changed: ‘, this.counter);
  }
  ...
}
```

Notice now that the hook is triggered every time the **counter** changes:

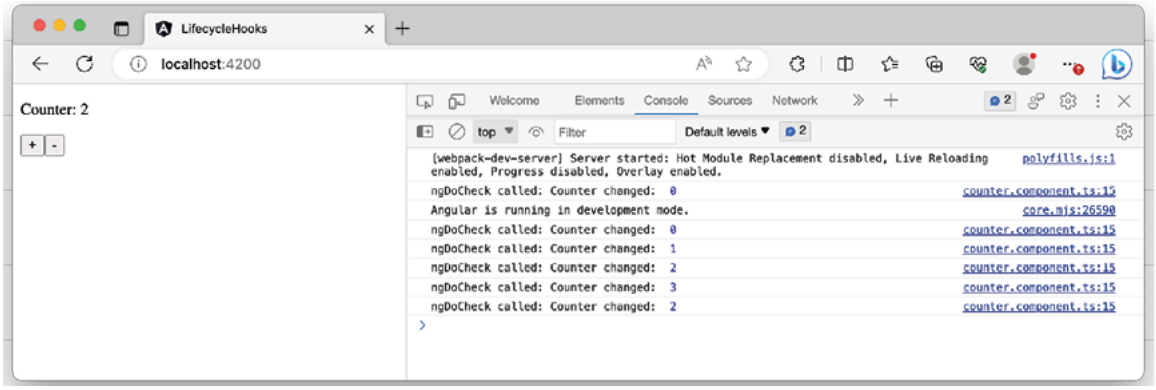


Figure 8.4: *ngDoCheck log messages*

Another case where this lifecycle hook is useful is when the reference of an input does not change, but the value changes. **ngOnChanges** only triggers when the actual instance of the input property changes, not when the content of the property changes. For example, if the input property is an array or an object, **ngOnChanges** will not fire when the elements or properties of the array or object change. **ngDoCheck**, on the other hand, can detect changes in the content of the input property, as well as changes in other properties of the component. Let us create a **ListComponent** that implements both the **OnChanges** interface as well as the **DoCheck** interface and accepts a list of strings as input:

```
import { Component, DoCheck, Input, OnChanges, SimpleChanges } from "@angular/core";

@Component({
  selector: 'app-list',
  template: `
    <ul>
      <li *ngFor="let item of items">{{ item }}</li>
    </ul>
  `
})
export class ListComponent implements OnChanges, DoCheck {
  @Input() items: string[] = [];
  previousLength: number = 0;

  ngOnChanges(changes: SimpleChanges): void {
    console.log('ngOnChanges called: ', changes);
  }

  ngDoCheck() {
    if (this.items.length !== this.previousLength) {
      console.log('ngDoCheck called: ', this.items);
      this.previousLength = this.items.length;
    }
  }
}
```

Render this component from **app.component.ts** and pass a list of fruit names as input. Also implement methods to add to the list and to clear the list, as follows:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <app-list [items]=»items»></app-list>
    <button (click)=»add()»>Add</button>
    <button (click)=»clear()»>Clear</button>
  `
})
export class AppComponent {
  items: string[];

  constructor() {
    this.items = ['Apple', 'Banana', 'Cherry'];
  }

  add() {
    // Add a random item to the array
  let fruits = ['Apricot', 'Blueberry', 'Coconut', 'Date', 'Elderberry'];
    let randomIndex = Math.floor(Math.random() * fruits.length);
    this.items.push(fruits[randomIndex]);
  }

  clear() {
    this.items = [];
  }
}
```

When the **Add** button is clicked, observe in the dev Console that **ngOnChanges** is not triggered, but **ngDoCheck** is:

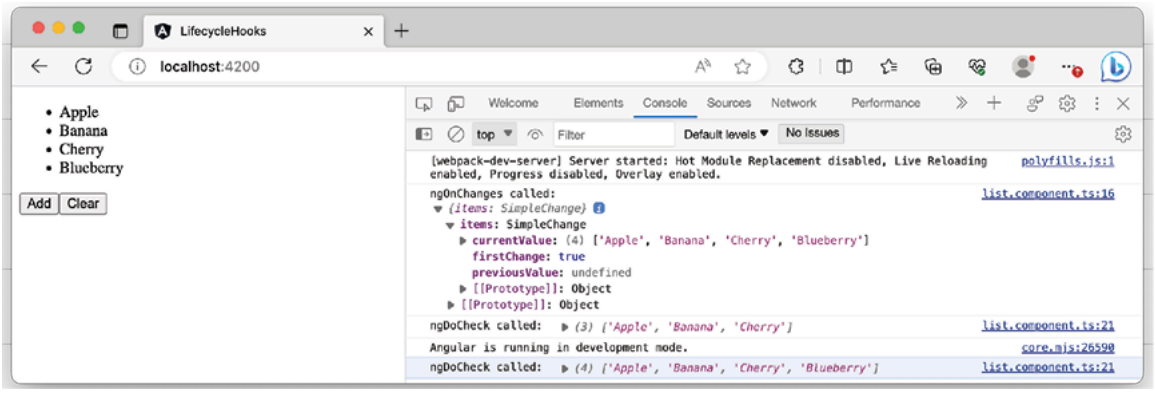


Figure 8.5: *ngDoCheck catches changes to an array of item*

There are a few more subtle differences between these two hooks.

- **ngOnChanges** receives a parameter of type **SimpleChanges**, which contains information on the changed input properties. **ngDoCheck** does not receive any parameter, but it can access the current and previous values of the input properties through the **KeyValueDiffers** and **IterableDiffers** services. An example code is as follows:

```
import { Component, Input, DoCheck, KeyValueDiffers } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `<p>Hello, {{ user?.name }}!</p>`
})
export class UserComponent implements DoCheck {
  @Input() user: { name: string; } | undefined;
  differ: any;

  constructor(private differs: KeyValueDiffers) {
  this.differ = this.differs.find({}).create();
  }

  ngDoCheck() {
  console.log('ngDoCheck called');
  const changes = this.differ.diff(this.user);
  if (changes) {
  changes.forEachChangedItem((item: { key: any; previousValue: any; currentValue: any; }) => {
  console.log('Property', item.key, 'changed from', item.previousValue, 'to', item.currentValue);
  });
  }
  }
}
```

Call this from **app.component.ts** as follows and add a method to change the username:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <app-user [user]="user"></app-user>
    <button (click)="changeName()">Change name</button>
  `
})
export class AppComponent {
  user: { name: string };

  constructor() {
    this.user = { name: 'Veda' };
  }

  changeName() {
    this.user.name = 'Adi';
  }
}
```

The dev **Console** will print the log message when it detects a change in the username on click of the button:

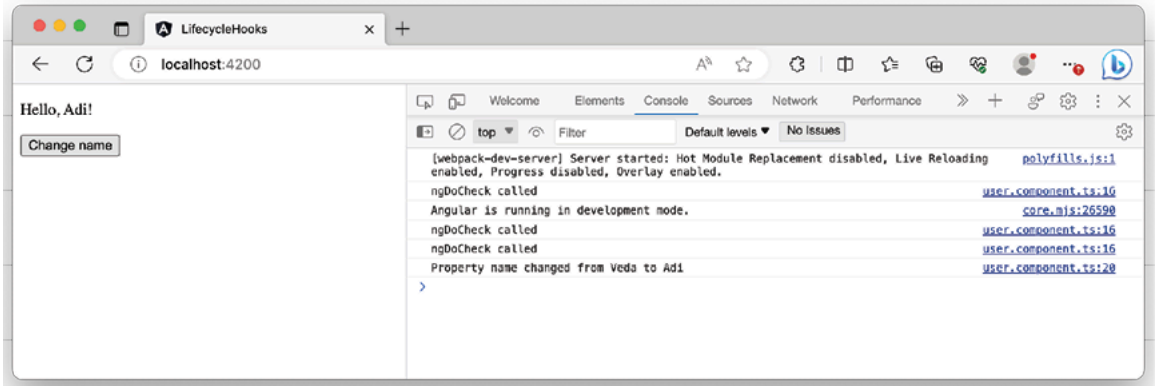


Figure 8.6: *Detecting change using KeyValueDiffers service*

- **ngOnChanges** is a more efficient and optimized way of detecting changes in input properties, as it only fires when there is a change in the bound data. **ngDoCheck** is a more flexible and customizable way of detecting changes in any property, but it can create performance issues when implemented incorrectly, as it fires with every change detection cycle.

ngAfterContentInit

The **ngAfterContentInit** hook is called once after Angular projects the external content into the component's view or into the view that the directive is in. It is useful for performing initialization tasks that depend on the component's content. The external content is the content that is passed from outside of the component's template using **<ng-content>** or **[ngTemplateOutlet]**. You can use this hook to access and manipulate the projected content using **@ContentChild** or **@ContentChildren** queries. Let us create two components to see this in action. Create a **CustomPanelComponent** as follows:

```
import { Component, AfterContentInit, ContentChild, ElementRef } from '@angular/core';

@Component({
  selector: 'app-custom-panel',
  template: `
    <div class="panel">
      <ng-content></ng-content>
    </div>
  `,
  styles: [`
    .panel {
      border: 1px solid #ccc;
      padding: 10px;
    }
  `]
})
export class CustomPanelComponent implements AfterContentInit {
  @ContentChild('header') headerElement: ElementRef | undefined;

  ngAfterContentInit() {
    console.log('ngAfterContentInit called:');
    if (this.headerElement) {
  console.log('Header content found:', this.headerElement.nativeElement.textContent);
    } else {
      console.log('No header content found.');
    }
```

```
  }
}
```

Now, create a parent component to use the above component as follows:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-parent',
  template: `
    <app-custom-panel>
      <div #header>Header Content</div>
      <p>Some other content goes here...</p>
    </app-custom-panel>
  `,
})
export class ParentComponent {}
```

You will notice the messages in the dev **Console** when the hook is called:

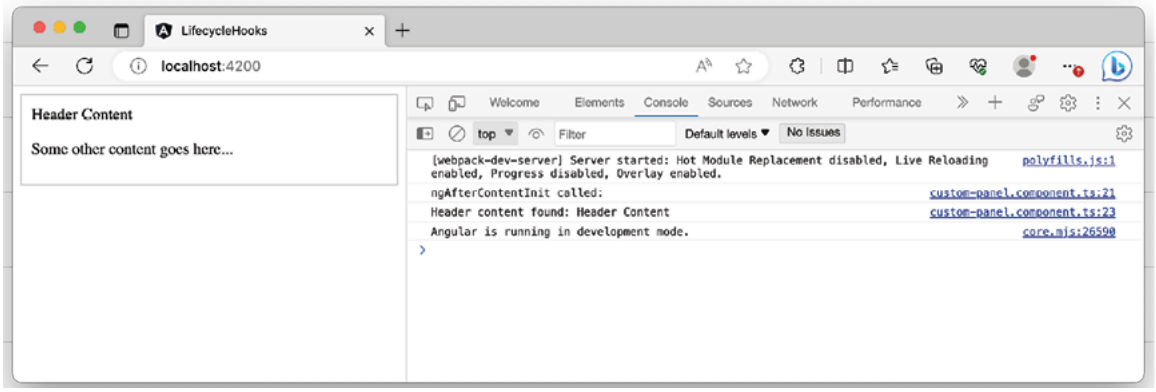


Figure 8.7: *detecting content elements using ngAfterContentInit*

In the **ParentComponent**, we use the **<app-custom-panel>** tag to include our custom panel. Inside the **CustomePanelComponent**, we have a ** ** with a reference name **#header** which corresponds to the **@ContentChild('header')** query. The content within **<ng-content></ng-content>** will be projected into the custom panel. When the **ngAfterContentInit** method of **CustomPanelComponent** runs, it checks if the header content exists and logs its text content.

ngAfterContentChecked

ngAfterContentChecked is a lifecycle hook those Angular calls after it checks the content projected into the component. The content is the part of the template that is enclosed by the **<ng-content>** tag. This hook is useful for performing any additional tasks after the content is checked, such as updating a variable or calling a function.

The purpose of this hook is to allow the component to react to any changes in the content that are caused by the change detection cycle. For example, if the content contains a data-bound property that changes its value, the component can use this hook to update its own state or perform some side effects.

Add the following methods in the above code to handle this lifecycle hook:

```
export class CustomPanelComponent implements AfterContentInit, AfterContentChecked {
  ...

  ngAfterContentChecked() {
    console.log('ngAfterContentChecked called:');
  // You can use this hook to perform any additional tasks after the content is checked
```

```
    // For example, you can update a variable or call a function here
    this.updateHeader();
  }

  updateHeader() {
  // This is a dummy function that changes the header content to uppercase
    if (this.headerElement) {
  this.headerElement.nativeElement.textContent =
  this.headerElement.nativeElement.textContent.toUpperCase();
  console.log('Header content updated:', this.headerElement.nativeElement.textContent);
    }
  }
}
```

The component also has a dummy function that changes the header content to uppercase. The **ngAfterContentChecked** hook is called after Angular checks the content projected into the component. In this case, it means after Angular checks the div element with the id **header** and its text content. The hook then calls the **updateHeader** function, which modifies the header content to uppercase and logs it to the **Console**, as shown in the following figure:

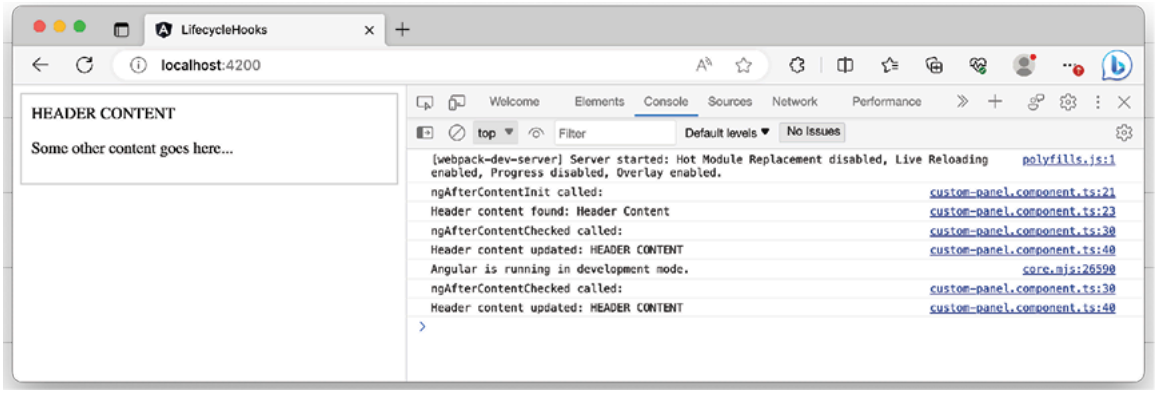


Figure 8.8: *Change header case using ngAfterContentChecked*

Notice in the **Console** logs that **ngAfterContentInit** is called once while **ngAfterContentChecked** is called multiple times thereafter. The purpose of **ngAfterContentInit** is to allow the component or directive to perform any initialization tasks related to the projected content, such as querying for elements or directives using **ContentChild** or **ContentChildren** decorators or accessing the content's data-bound properties. The purpose of **ngAfterContentChecked** is to allow the component or directive to perform any update tasks related to the projected content, such as reacting to changes in the content's data-bound properties or performing some side effects.

ngAfterViewInit and ngAfterViewChecked

Similar to **ngAfterContentInit** and **ngAfterContentChecked** hooks, there are **ngAfterViewInit** and **ngAfterViewChecked** hooks those Angular calls after it initializes and checks the component's views and child views. The component's views are the elements that are defined in the component's template. The child views are the elements that are defined in the templates of the component's children.

The difference between **ngAfterViewInit** and **ngAfterViewChecked** is that **ngAfterViewInit** is called only once after the first initialization of the component's views and child views, while **ngAfterViewChecked** is called after every subsequent check of the component's views and child views.

To illustrate this difference, let us look at a simple example of a parent component and a child component. Create a child component with the following code:

```
import { Component, Input, AfterViewInit, AfterViewChecked } from '@angular/core';

```

```
@Component({
  selector: 'app-child',
  template: `
    <h2>Child Component</h2>
    <p>{{message}}</p>
  `,
})
export class ChildComponent implements AfterViewInit, AfterViewChecked {
  @Input() message?: string;

  ngAfterViewInit() {
    console.log('Child: ngAfterViewInit');
  }

  ngAfterViewChecked() {
    console.log('Child: ngAfterViewChecked');
  }
}
```

Now replace the previously created parent component with the following code:

```
import { Component, AfterViewInit, AfterViewChecked } from '@angular/core';

@Component({
  selector: 'app-parent',
  template: `
    <h1>Parent Component</h1>
    <app-child [message]=»message»></app-child>
    <button (click)="changeMessage()">Change Message</button>
  `,
})
export class ParentComponent implements AfterViewInit, AfterViewChecked {
  message = 'Hello';

  ngAfterViewInit() {
    console.log('Parent: ngAfterViewInit');
  }

  ngAfterViewChecked() {
    console.log('Parent: ngAfterViewChecked');
  }

  changeMessage() {
    this.message = 'Hi';
  }
}
```

In this example, the parent component has a message property that is bound to the child component's message input. The parent component also has a button that changes the message property when clicked. The parent and child components both implement the **ngAfterViewInit** and **ngAfterViewChecked** hooks and log them to the **Console**, as shown in the following figure:

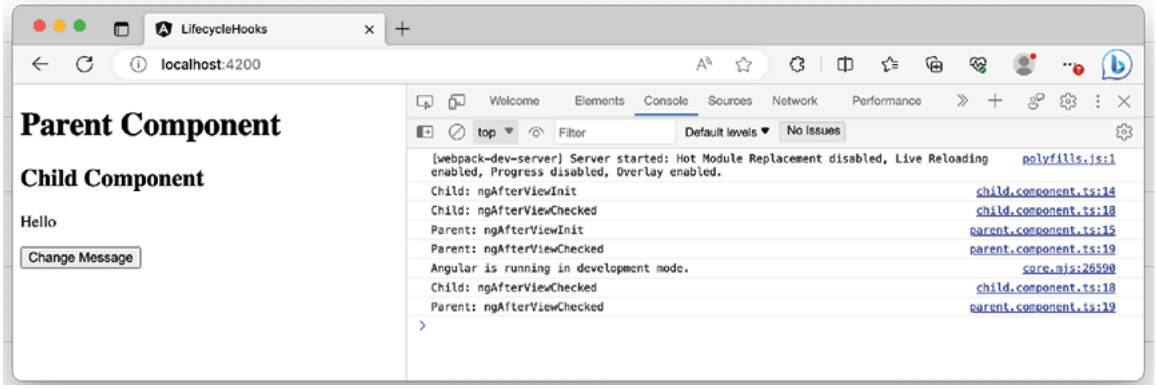


Figure 8.9: *ngAfterViewInit and ngAfterViewChecked log messages*

When the application runs for the first time, Angular will call the **ngAfterViewInit** and **ngAfterViewChecked** hooks in the following order:

1. Child: ngAfterViewInit
2. Child: ngAfterViewChecked
3. Parent: ngAfterViewInit
4. Parent: ngAfterViewChecked

This shows that Angular first initializes and checks the child component's view, then initializes and checks the parent component's view. The **ngAfterViewInit** hooks are called only once for each component.

When we click the button to change the message property, Angular will run change detection again and call the **ngAfterViewChecked** hooks in the following order:

1. Child: ngAfterViewChecked
2. Parent: ngAfterViewChecked

Angular checks the child component's view and then checks the parent component's view. The **ngAfterViewInit** hooks are not called again because the views are already initialized.

The purpose of these hooks is to allow us to perform any additional tasks after the views are initialized or checked. For example, we can use them to query for view elements using **@ViewChild** or **@ViewChildren** decorators or to access the view's data-bound properties. For example, in the above code sample, we can add a **ViewChild** field in the **ParentComponent** to get a reference of the **ChildComponent** and then use that reference to fetch the properties of the child in the **ngAfterViewInit** hook:

```
@ViewChild(ChildComponent) child?: ChildComponent;

ngAfterViewInit() {
  console.log('Parent: ngAfterViewInit');

  // access the child component properties
  console.log('Child message:', this.child?.message);
}
```

ngOnDestroy

The **ngOnDestroy** hook is called just before Angular destroys the component or directive instance. You can use this hook to perform any cleanup tasks that are necessary to avoid memory leaks or unwanted side effects. For example, you can use **ngOnDestroy** to unsubscribe from observables, detach event handlers, cancel timers, or destroy any external resources that were created by the component or directive. A simple example code is as follows that prints the value of a timer every second:

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-timer',
```

```
5.  template: `
6.    <p>Timer: {{timer}}</p>
7.  `,
8. })
9. export class TimerComponent implements OnInit {
10.   timer: number = 0;
11.   intervalId: any;
12.
13.   ngOnInit() {
14.     this.intervalId = setInterval(() => {
15.       this.timer++;
16.       console.log('Timer:', this.timer);
17.     }, 1000);
18.   }
19. }
```

Line 14 in the above code creates the interval function, which will keep running even after the component is destroyed. This can cause memory leaks. To verify this, create a parent component that uses this timer component and has a button to remove the timer component as follows:

```
1. import { Component } from '@angular/core';
2.
3. @Component({
4.   selector: 'app-parent',
5.   template: `
6.     <h1>Parent Component</h1>
7.     <app-timer *ngIf="showTimer" ></app-timer>
8.     <button (click)="removeTimer()">Remove Timer</button>
9.   `,
10. })
11. export class ParentComponent {
12.   showTimer: boolean = true;
13.
14.   removeTimer() {
15.     this.showTimer = false;
16.   }
17. }
```

On render, you can see the timer counter printed on the console log. On click of the button, the component is removed from browser, but the timer keeps logging in the console log, as shown:

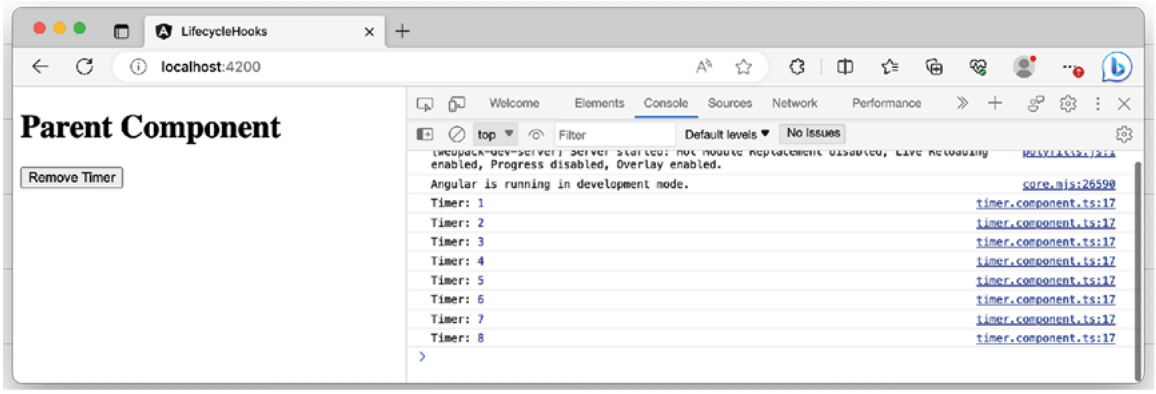


Figure 8.10: *Possible memory leak when resources are not freed up*

Now, implement the **OnDestroy** interface in the timer component and add the following method to it:

```
ngOnDestroy() {
  console.log('ngOnDestroy called')
  // Clear the interval function when the component is destroyed
  clearInterval(this.intervalId);
}
```

Observe that on click of the **Remove Timer** button, the timer does not log any more:

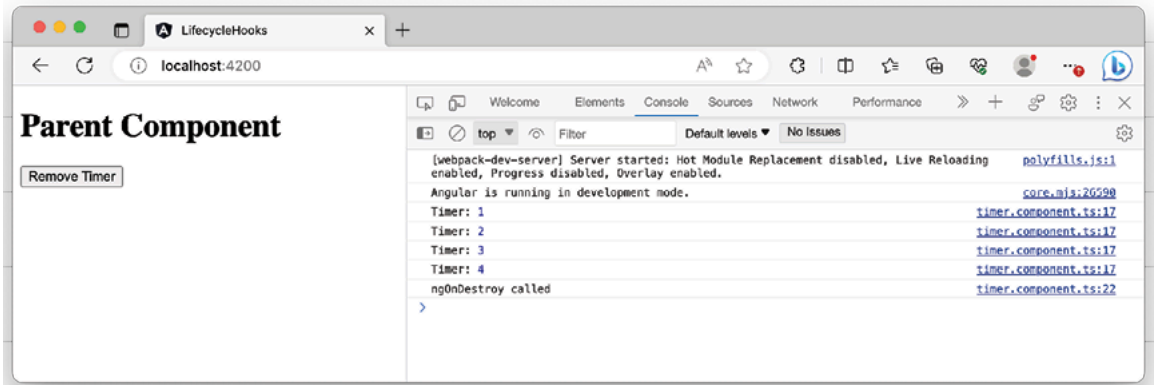


Figure 8.11: *ngOnDestroy clears the interval function*

A more real-world app will need to unsubscribe from observables, as shown in the code below:

```
import { Subscription } from 'rxjs';
import { interval } from 'rxjs';

@Component({ ...})
export class TimerComponent implements OnInit, OnDestroy {
  timer: number = 0;
  subscription: Subscription;

  ngOnInit() {
    const source = interval(1000);
    this.subscription = source.subscribe(val => {
      this.timer = val;
    });
  }

  ngOnDestroy() {
    // Unsubscribe from the observable when the component is destroyed
    this.subscription.unsubscribe();
  }
}
```

We will take a deep dive into observables in *Chapter 12, Observables and Signals*.

DestroyRef

DestroyRef is a new feature in Angular 16 that allows you to register a callback function that will be executed when the component or directive that injects it is destroyed. This can be useful for performing some cleanup logic, such as unsubscribing from observables, removing event listeners, or releasing resources. **DestroyRef** is an injectable class that has an **onDestroy** method that takes a function as an argument. You can use dependency injection to get an instance of **DestroyRef** in your component or directive constructor and then

pass your cleanup function to its **onDestroy** method. To see the code, in our **TimerComponent**, add a constructor as follows that registers a function for **onDestroy** method in **DestroyRef** class. You can import **DestroyRef** from **@angular/core**:

```
constructor(destroyRef: DestroyRef) {
  destroyRef.onDestroy(() => {
    console.log('DestroyRef called')
    clearInterval(this.intervalId);
  });
}
```

This method is called before component destruction. As you can see, using **DestroyRef** is a convenient and elegant way to handle the **ngOnDestroy** lifecycle hook without having to implement the **OnDestroy** interface or write any boilerplate code. You can also use **DestroyRef** in directives, pipes, embedded views, and instances of environment injector. **DestroyRef** runs after **ngOnDestroy** if both are in the same component or directive. This means you can use **ngOnDestroy** to perform some operations before **DestroyRef**, such as notifying other parts of the application that the component or directive is going away.

Ordering of lifecycle hooks

Let us implement all the lifecycle hooks on a directive this time to see how hooks work on a directive as well as to verify the ordering of the lifecycle hooks:

```
@Directive({
  selector: '[appLifecycle]',
})
export class LifecycleComponent implements OnInit, OnChanges, DoCheck, AfterContentInit, AfterConte
  // Define a counter to track the number of times each hook is called
  counter: number = 0;

  constructor() {
    this.counter++;
    console.log('constructor called', this.counter);
  }

  ngOnChanges() {
    this.counter++;
    console.log('ngOnChanges called', this.counter);
  }
  ...
}
```

This can be used in any component as follows:

```
<div appLifecycle></div>
```

When you run it, the ordering will look like this:

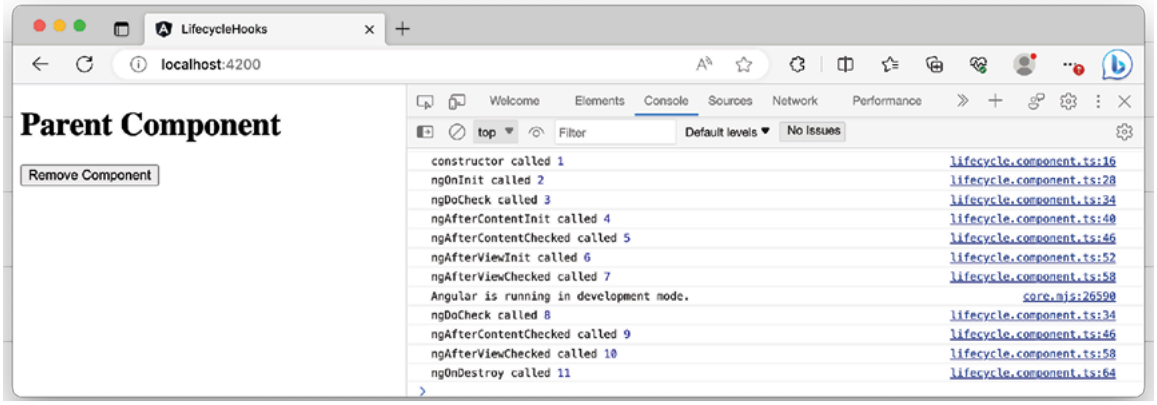


Figure 8.12: *Ordering of lifecycle hooks*

Conclusion

Understanding the lifecycle hooks in Angular is essential for building robust and maintainable applications. By using these hooks, you can interact with your component at different stages of its lifecycle, giving you fine-grained control over how it behaves. From initializing data to detecting changes and cleaning up resources, lifecycle hooks provide developers with a consistent way to manage their components. By mastering these hooks, you can build more efficient and reliable Angular applications that respond appropriately to changes and user interactions.

Points to remember

- Lifecycle hooks allow you to insert code logic at various stages of a component lifecycle.
- **ngOnChanges**: Detects changes in input properties and reacts to them.
- **ngOnInit**: Initializes the component or directive after the first ngOnChanges.
- **ngDoCheck**: Performs custom change detection on the component or directive.
- **ngAfterContentInit**: Queries and initializes projected content after it is rendered.
- **ngAfterContentChecked**: Checks and updates projected content after every change detection.
- **ngAfterViewInit**: Queries and initializes view elements after they are rendered.
- **ngAfterViewChecked**: Checks and updates view elements after every change detection.
- **ngOnDestroy**: Cleans up resources and subscriptions before the component or directive is destroyed.
- It is important to consider performance implications before implementing a lifecycle hook.
- DestroyRef is better than ngOnDestroy in many ways.

Questions

1. Explain why we implement the initialization logic in ngOnInit instead of in the constructor.
2. How are AfterContent* hooks different from AfterView* hooks?
3. Can lifecycle hooks be implemented on a service or a pipe?
4. Explain the differences between DestroyRef and ngOnDestroy.

References

- Angular official doc on lifecycle hooks:
  https://angular.io/guide/lifecycle-hooks

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 9
Data Binding and Events Handling

Introduction

Data binding is a core concept in Angular that allows developers to establish a connection between the component and the view, enabling seamless synchronization of data between the two. With data binding, you can bind the component's data to the view's HTML elements, making it easy to update the user interface in real time as changes are made to the data. In this way, data binding provides a powerful mechanism for creating dynamic, responsive web applications that deliver a rich user experience. Events handling allows developers to respond to user actions in the view, such as clicking a button or typing into a text field and take appropriate actions in response. In this chapter, we will explore the different types of data binding in Angular and how they can be used to build complex web applications. We will also explore the basics of event handling in Angular, including how to define event handlers, how to bind them to HTML elements, and how to respond to user actions in the view.

Structure

The chapter covers the following topics:

- Data binding
- Interpolation
- Property binding
- Event binding
- Two-way data binding

Objectives

By the end of this chapter, you will have a solid understanding of how to use data

binding and event handling in Angular to create rich and interactive web applications.

Data binding

Data binding is a way of connecting data from the component to the view and vice versa. In traditional server-side rendered web apps, the server-side would create an HTML page and send it to the browser. There was never a need to update the data on the page. On any user interaction, the page would send another request back to the server asking for a new page. Even when the same page data was being modified, the server would send back the entire modified page. This concept changed in **single page applications** (**SPAs**), where there was a need to update the page content in the browser without refreshing the entire page. This provided a much better user experience. Data binding is core to achieve this functionality. Through data binding, the view would watch for changes to the underlying data and would auto-update the section within the page whenever the data changes. This allows for a cleaner separation of concerns between the presentation logic (view) and the business logic (data).

In Angular, there are four types of data binding:

- Interpolation
- Property binding
- Event binding
- Two-way binding

Let us dive deep into each type.

Interpolation

Interpolation is a way of embedding expressions into marked-up text, such as HTML. It allows you to bind a component property, method, or template reference variable to a string literal in the view. By default, interpolation uses the double curly braces **{{** and **}}** as delimiters. For example, if you have a component property called **name**, you can use interpolation to display its value in the template:

```
1. <h1>Hello {{ name }}!</h1>
```

Angular evaluates the expression inside the curly braces and replaces it with the string value of the corresponding component property. Suppose the value of the **name** property is **Adi**; Angular will render this as:

```
1. <h1>Hello Adi!</h1>
```

Interpolation is one-way binding, meaning that the data flows from the component to the view but not the other way around. If the component property

changes, the view will be updated automatically.

Interpolation can also be used to invoke methods, perform calculations, concatenate strings, and use template reference variables. For example, you can use interpolation to call a method that returns the current date:

```
<p>Today is {{ getDate() }}</p>
```

You can use interpolation to add two numbers and display the result:

```
<p>The sum of 2 and 3 is {{ 2 + 3 }}</p>
```

You can use it to combine two strings as well:

```
<p>Your full name is {{ firstName + ' ' + lastName }}</p>
```

Interpolation can be used to access a template reference variable and display its value:

```
<input #nameInput type="text" placeholder="Enter your name">
<p>You entered: {{ nameInput.value }}</p>
```

One thing to note is that interpolation works only on properties and not attributes. For example, you cannot use interpolation to set the **id** attribute of an element:

```
<!-- This will not work -->
<div id="{{ myId }}"></div>
```

Instead, you need to use property binding, which is another way of binding data from the component to the view. Property binding uses the square brackets **[]** as delimiters. For example, you can use property binding to set the **id** property of an element:

```
<!-- This will work -->
<div [id]=”myId”></div>
```

Interpolation can also be used with pipes. For instance, you can use the **date** pipe to format the current date in a specific way:

```
<p>Today is {{ getDate() | date:'fullDate' }}</p>
```

It can also be used with the **safe navigation operator** (**?**), which is a way of preventing errors when accessing properties of null or undefined objects. An example of such usage is if you have a component property called **user**, which may or may not have a **name** property, you can use the **?** to display the user’s name only if it exists:

```
<p>Hello {{ user?.name }}</p>
```

Another example is the use of the **non-null assertion operator** (**!**), which is a way of telling the compiler that an expression is not null or undefined. For example, if you have a component property called user, which you know for sure has a name property, you can use the **!** to avoid compiler errors:

```
<p>Hello {{ user!.name }}</p>
```

Interpolation is a powerful and convenient feature of Angular that allows you to

display dynamic data in the view. However, you should avoid using complex or long expressions in interpolation, as they can make your code less readable and harder to debug. Instead, you should use simple expressions or move the logic to the component class or a service.

Property binding

Property binding is a technique of setting the value of a property of an HTML element or a directive to a component's property. Property binding is one-way as well, like interpolation. Property binding uses square brackets **[]** to indicate the target property of the element or directive. We saw one such example in the previous section:

```
1. <div [id]=”myId”></div>
```

This binds the **id** property of a ** ** element to the **myId** property of the component.

Property binding can be used for various purposes, such as:

- Setting the attributes of HTML elements, such as **value**, **disabled**, **colspan**, etc.
- Setting the properties of directives, such as **ngClass**, **ngStyle**, **ngModel**, etc.
- Setting the inputs of child components, such as **@Input() childItem**, etc.

Following are some examples of property binding:

```
1. <input [value]="title">
2. <img [src]="imgUrl">
3. <button [disabled]="bool">Submit</button>
4. <div [ngClass]="currentClasses">This div is styled by currentClasses</div>
```

In the above sample, the **src** attribute of **img** tag is set to **imgUrl** property in the component class. You may also set the attribute to an expression as follows:

```
1. <input [value]="2 + 3">
```

To bind the **@Input() childItem** of a child component to the **parentItem** property of the parent component, use the following code:

```
1. <app-child [childItem]="parentItem"></app-child>
```

Event binding

Event binding is a way of listening and responding to user actions, such as clicks, keystrokes, mouse movements, and touches. Event binding uses the syntax **(target-event)="template-statement"** to bind a target event name within parentheses to a template statement within quotes. The template statement can be a method call, an assignment, or any other expression that Angular can evaluate.

For instance, suppose you have a component **app.component.ts** that has a method called **onSave()** that saves some data:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  onSave() {
    alert('Saved!');
  }
}
```

You can use event binding in **app.component.html** to call this method when a button is clicked, like this:

```
<button (click)="onSave()">Save</button>
```

Angular will execute the **onSave()** method whenever the button is clicked. Refer to the following figure:

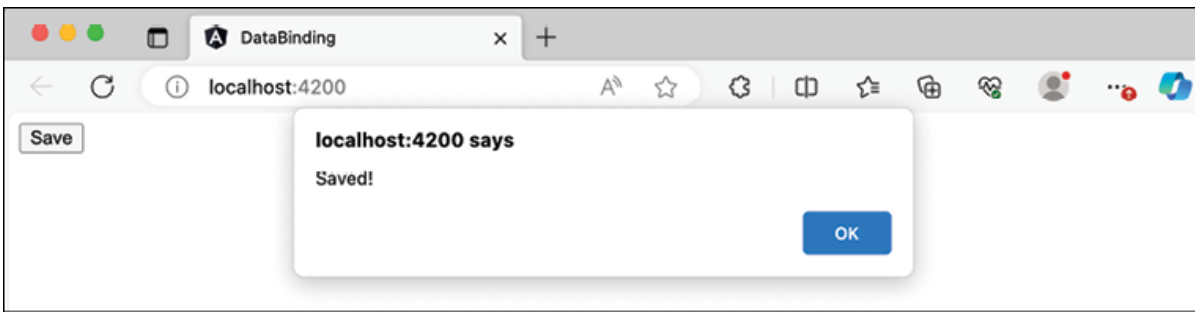


Figure 9.1: *Click event binding*

There are different types of events that you can bind to, depending on the HTML element or directive that you use. For example, you can bind to input events, such as keyup, change, or input, to get the user input from text fields, checkboxes, or radio buttons. You can also bind to keyboard events using the key or code fields of the native browser keyboard event object. We will look at a few sample codes for user input in *Chapter 11, User Input Using Forms*.

You can bind to custom events that you create in your own components or directives using the **@Output()** decorator and the **EventEmitter** class.

To understand this, say you have a child component that emits an event called **ratingChange** whenever the user changes the rating of a product. You can use

event binding to listen to this event in the parent component and update the parent's rating property, like this:

```
<app-rating [rating]="rating" (ratingChange)="rating=$event"></app-rating>
```

Angular will assign the value of the emitted event to the **rating** property of the parent component.

Two-way data binding

All the above code samples we saw were related to one-way data binding. Two-way binding allows you to update the value of a property in both the component and the template simultaneously. If you change the value of an input element in the template, it will also update the value of the bound property in the component class and vice versa.

We need to keep the following points in mind to use two-way binding:

- Import the **FormsModule** in your **app.module.ts** file and add it to the imports array of the **NgModule** decorator. Without this, you will get a compile time error while trying to use two-way binding: **Can't bind to 'ngModel' since it isn't a known property of 'input'**
- Use the **[(ngModel)]** directive in your template to bind a property of your component class to an input element. The **[(ngModel)]** syntax is a combination of property binding and event binding, also known as **banana in a box**.
- Make sure the name of the bound property matches the name of the input element and that the property is decorated with the **@Input()** decorator in the **component** class.

To see this in action, create a new Angular app and add **FormsModule** in **app.module.ts** file as follows:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
```

```
    BrowserModule,
    FormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Now, in the **app.component.ts** file add an **@Input** property to store the name of a person as follows:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  @Input() name: string = 'Adi';
}
```

In the **template** file, show the name property as well as modify it using the input tag as follows:

```
<h1>Hello {{name}}</h1>
<input type="text" [(ngModel)]="name">
```

In this example, the name property of the **AppComponent** class is bound to the input element in the template using the **[(ngModel)]** directive. If you change the value of the input element, the **name** property will also change, and the **h1** element will display the updated value. Similarly, if you change the value of the **name** property in the **component** class, the input element will reflect the new value. Refer to the following figure:

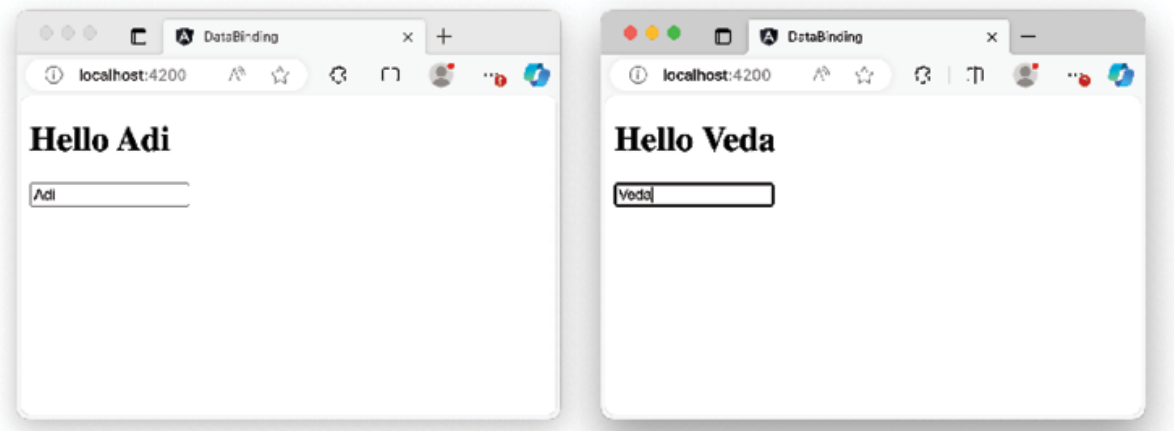


Figure 9.2: *Two-way data binding*

Two-way binding is useful for creating interactive forms and dynamic user interfaces. However, it also comes with some drawbacks, such as performance issues and complexity. Therefore, you should use it with caution and only when necessary. You can also use alternative techniques, such as reactive forms or template-driven forms, to handle user input and validation in Angular. We will look at reactive and template-driven forms in *Chapter 11, User Input Using Forms*.

Conclusion

Binding is a way to connect data from your component class to your template and vice versa. It allows you to create dynamic and interactive user interfaces, as well as share data between components. Angular offers different types of binding, such as interpolation, property binding, event binding, attribute binding, class binding, style binding, and two-way binding. Each type of binding has its own syntax and use cases, and you can combine them to achieve various effects. Binding in Angular is one of the core features of the framework, and it makes development easier and faster.

In the next chapter, we will look at how to deploy an Angular application to a web server.

Points to remember

- Data binding is a core concept of Angular that can improve user experience by updating the view instantly as the component property changes.
- Interpolation allows the coding of small expressions inside the template.
- Property binding allows binding an HTML attribute to the component property.
- Event binding allows an event name to be bound to an event handler method.
- Two-way binding allows the view to update when the component property

changes and vice versa.

Questions

1. Explain the difference between one-way and two-way data binding.
2. Why is it not recommended to have complex expressions in interpolation?
3. Explain the difference between interpolation and property binding.
4. Explain how using two-way binding can cause performance issues.

References

- Angular official document on binding:
  https://angular.io/guide/binding-overview

CHAPTER 10
Deployment

Introduction

Deploying an Angular application is a crucial step in the development process. It allows you to share your work with the world and showcase your skills as a developer. However, the deployment process can be intimidating, especially for beginners. But with the right tools and guidance, deploying an Angular application can be a breeze. In this chapter, we will walk you through the steps to deploy your Angular application and make it available for the world to see.

Structure

The chapter covers the following topics:

- Building your Angular app
- Choosing a hosting service
- Deployment using Angular CLI
- Deploy to Azure Static Web App
- Adding a custom domain

Objectives

By the end of this chapter, you will understand the major hosting services available to deploy an Angular app.

Building your Angular app

During development, you can use the **ng serve** command to serve the application on a local development web server and watch for changes. This is, however, not a safe option to deploy to production. You must run the **ng build** command in your project folder for production deployment. This will create a **dist** folder that contains the production ready files of your application. You can customize the output folder and other build options in the **angular.json** file. If you have production settings in your app, then you can use that as a parameter as follows:

```
ng build --configuration production
```

The **dist** folder is a fully functional Angular app compiled in JavaScript. It can now be deployed using any of the popular hosting services that we will discuss in the rest of the chapter.

You may also choose to build your application with a custom script instead of using **ng build** command. The steps involved are primarily to transpile from TypeScript to JavaScript. This can alternatively be done using the Transcript compiler CLI **tsc**. If you copy the transpiled files to a web server, the application will be served. However, for a production level experience, you may need to take care of other aspects like code minification, concatenation, name mangling, etc. You can achieve this using separate tools or compiler options. This can become overwhelming if you are doing it for the first time. Another option is to use bundling tools like webpack or SystemJS to create a **dist** folder. For most applications it is recommended, however, to use the **ng build** command.

To build for production, Angular uses the **ahead-of-time** (**AOT**) compiler. The AOT compiler in Angular is a feature that allows you to compile your Angular application AOT, before the browser downloads and runs it. This can improve the performance, security, and compatibility of your application. Here are some benefits of using the AOT compiler:

- **Faster rendering**: The browser can render the application immediately without waiting to compile the app first.
- **Smaller bundle size**: The compiler removes unnecessary Angular code and templates that are not used in the app.
- **Fewer errors**: The compiler detects and reports template binding errors during the build step before the users can see them.

- **Better security**: The compiler does not need to evaluate any expressions in the browser, which reduces the risk of injection attacks.

Choosing a hosting service

An Angular app can be deployed into any web server like any other website. If you have an IIS, Apache, or Nginx web server, you can deploy the **dist** folder to it. An easier option is to use a hosting service that provides hassle-free and seamless deployment from source control. There are several hosting options available. Here are a few of the most used ones:

- Firebase
- Vercel
- GitHub pages
- Heroku
- Netlify
- AWS S3
- NPM
- Azure Static Web App

Let us look at a few hosting services in the next sections:

Deployment using Angular CLI

Angular CLI allows to deploy to third-party hosting solutions like Firebase using **ng deploy** command. Firebase is an app development platform from *Google*. Firebase Hosting allows static and dynamic content to be hosted. To deploy to Firebase, first add the builder.

```
ng add @angular/fire
```

Then run the following command each time to deploy the latest code:

```
ng deploy
```

Similarly, other third-party hosting solutions have builders available to deploy using **ng deploy** command. Please check the *References* section for links to the hosting services documentation. You may need to set up an account in the hosting service before you can deploy your app.

Deploy to Azure Static Web App

You can deploy your app to Azure Cloud using the following steps:

1. Go to the Azure portal (**https://portal.azure.com**) and sign into your account. You may have to sign up if you do not have an account.
2. Select **Create a resource**, as shown in the following figure:

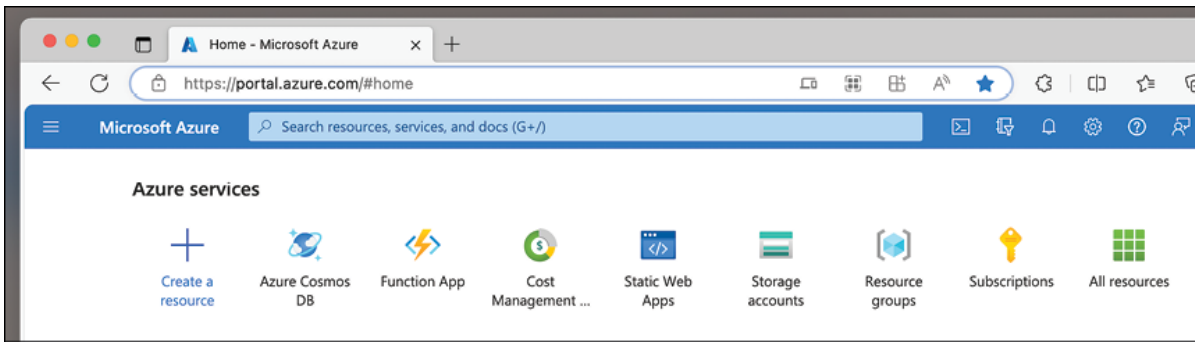


Figure 10.1: *List of resources shown in the Azure portal*

3. Search for **Static Web App** and click on the **Create** option on the box from the result, as shown in the following figure:

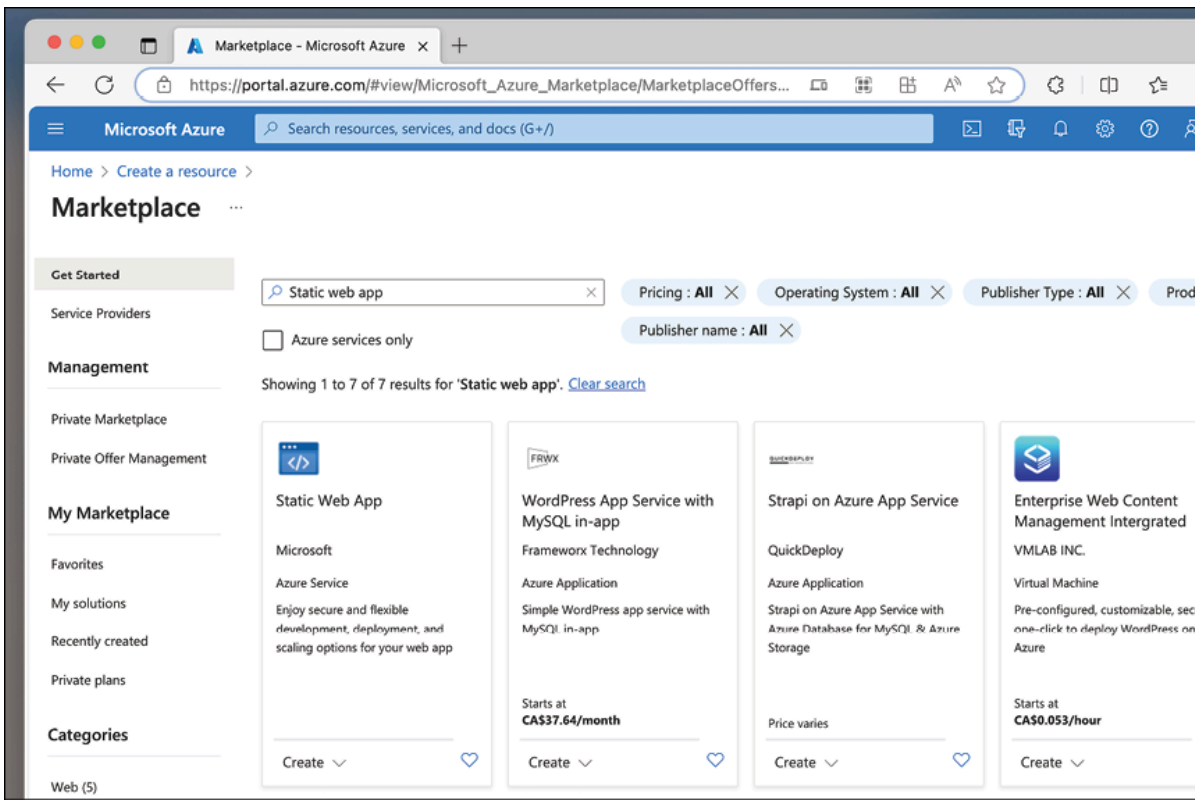


Figure 10.2: *Select Static Web App from the Marketplace page*

4. Follow the steps to create a Static Web App. You can choose to connect to a code repository in **GitHub** or **Azure DevOps**, as shown in the following figure:

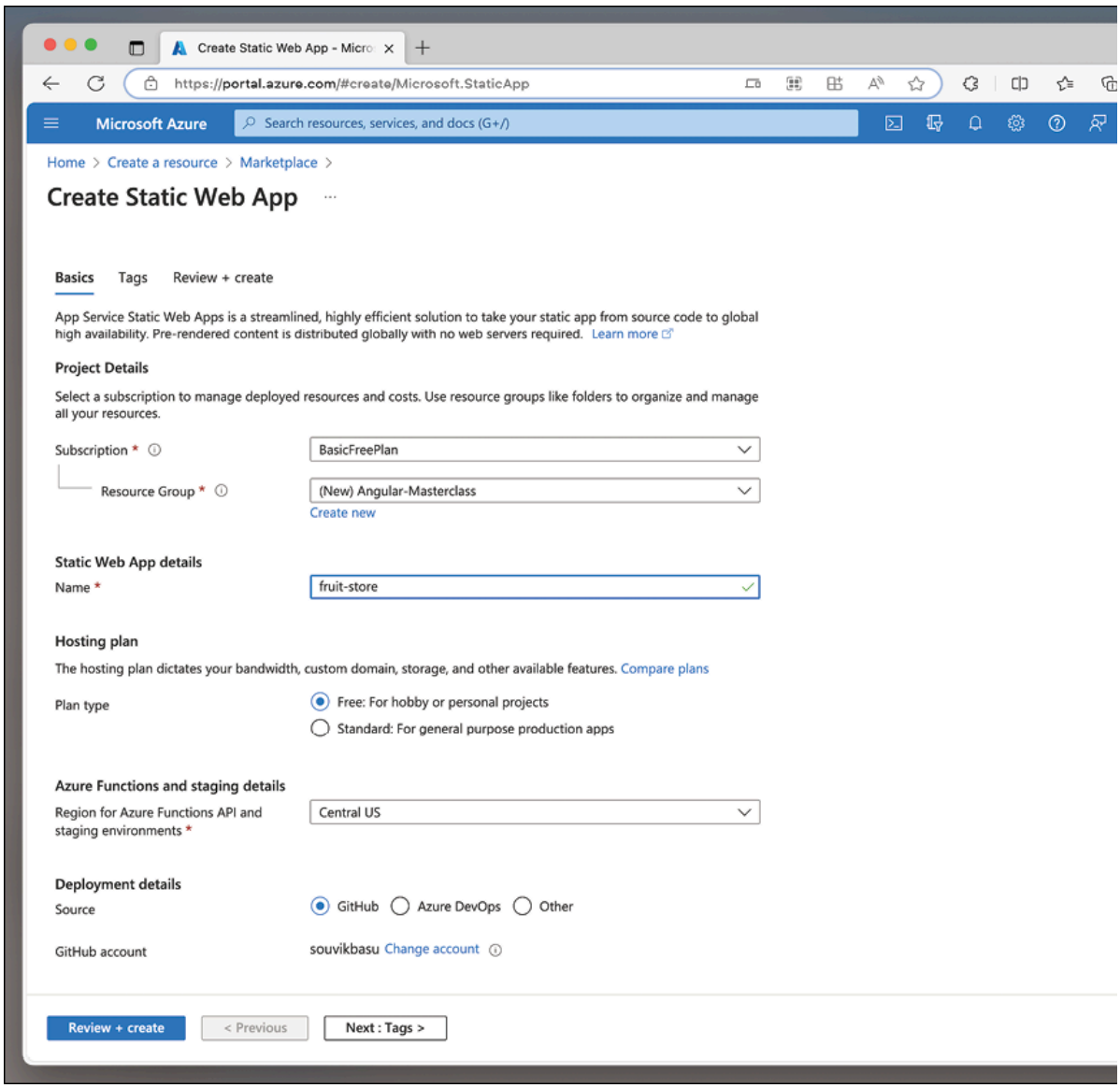


Figure 10.3: *Static Web App creation form*

5. If your code is in GitHub, sign in to GitHub and select the **Organization**, **Repository** name and **Branch** name from the drop down. For instance, we uploaded our **fruit-store** app created in *Chapter 5* to GitHub, that we can now select as follows:

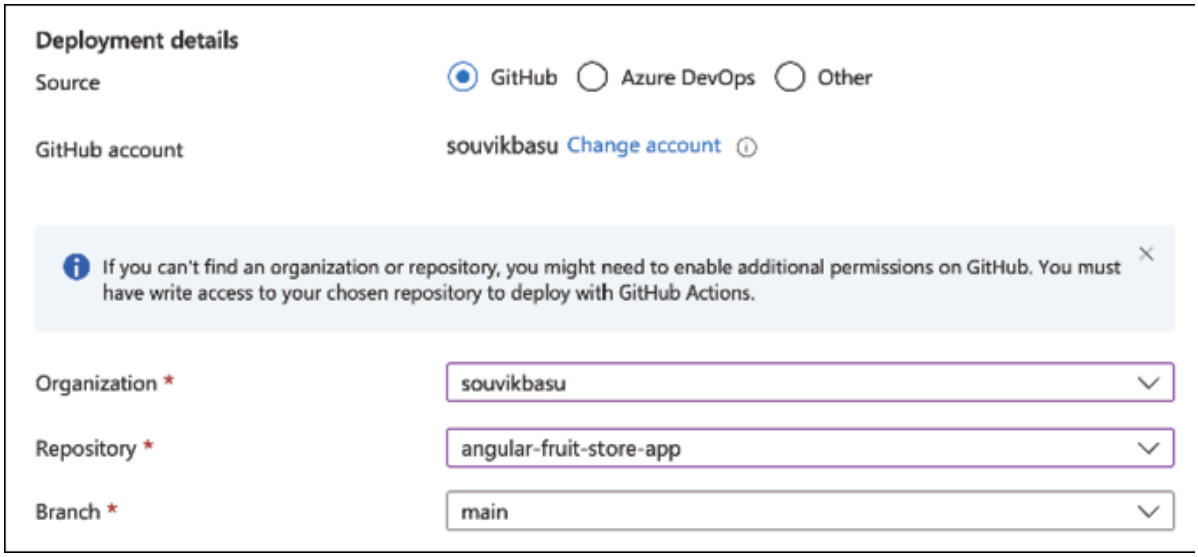


Figure 10.4: *Connect to a GitHub repository*

6. In the **Build Details** section, select **Angular** for **Build Presets**. For **App location**, keep it /, leave the **Api location** box and for **Output location** type **dist/<your app name>**. Refer to the following figure:

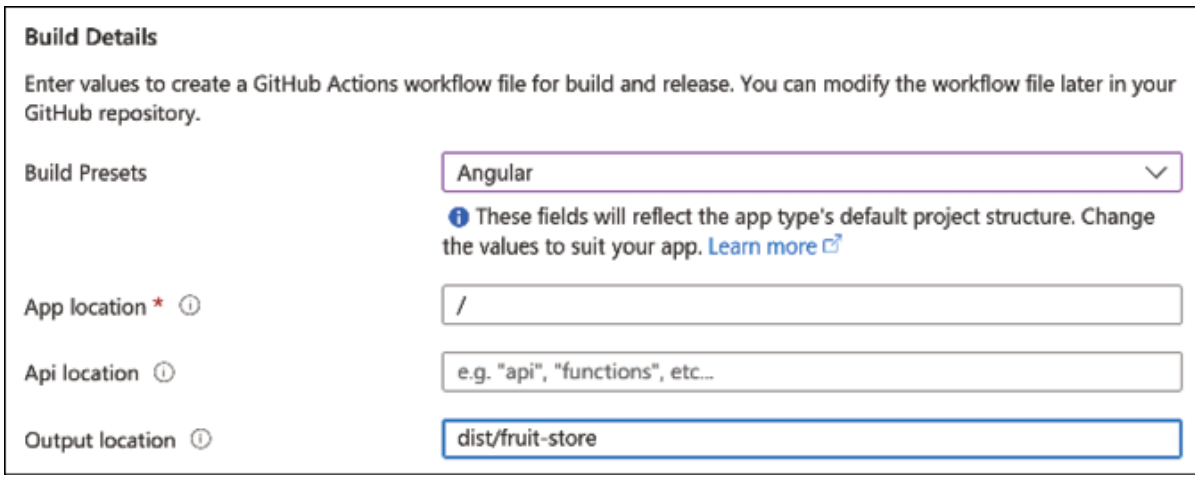


Figure 10.5: *Build Details*

7. Follow rest of the steps. This will deploy your app to Azure Static Web App to a URL like: **https://gray-sand-0a8fc5c10.4.azurestaticapps.net/**.
8. Now every time your push a code change to GitHub, the GitHub Actions workflow will auto deploy to your site. You do not need to deploy manually.

Adding a custom domain

In the previous sections, we saw how to deploy your app to a third-party

hosting solution. For all these services, the app will be deployed to a subdomain within the hosting solution. For instance, for the Azure Static Web App, we saw that it deployed our app to **https://gray-sand-0a8fc5c10.4.azurestaticapps.net.** If you have a custom domain like **www.example.com** that you want to use to show your deployed web app, you can follow the steps given in the hosting solution documentation. For the Azure web app that we created in the last section, we can set the domain by clicking on **Add a custom domain** link as shown below:

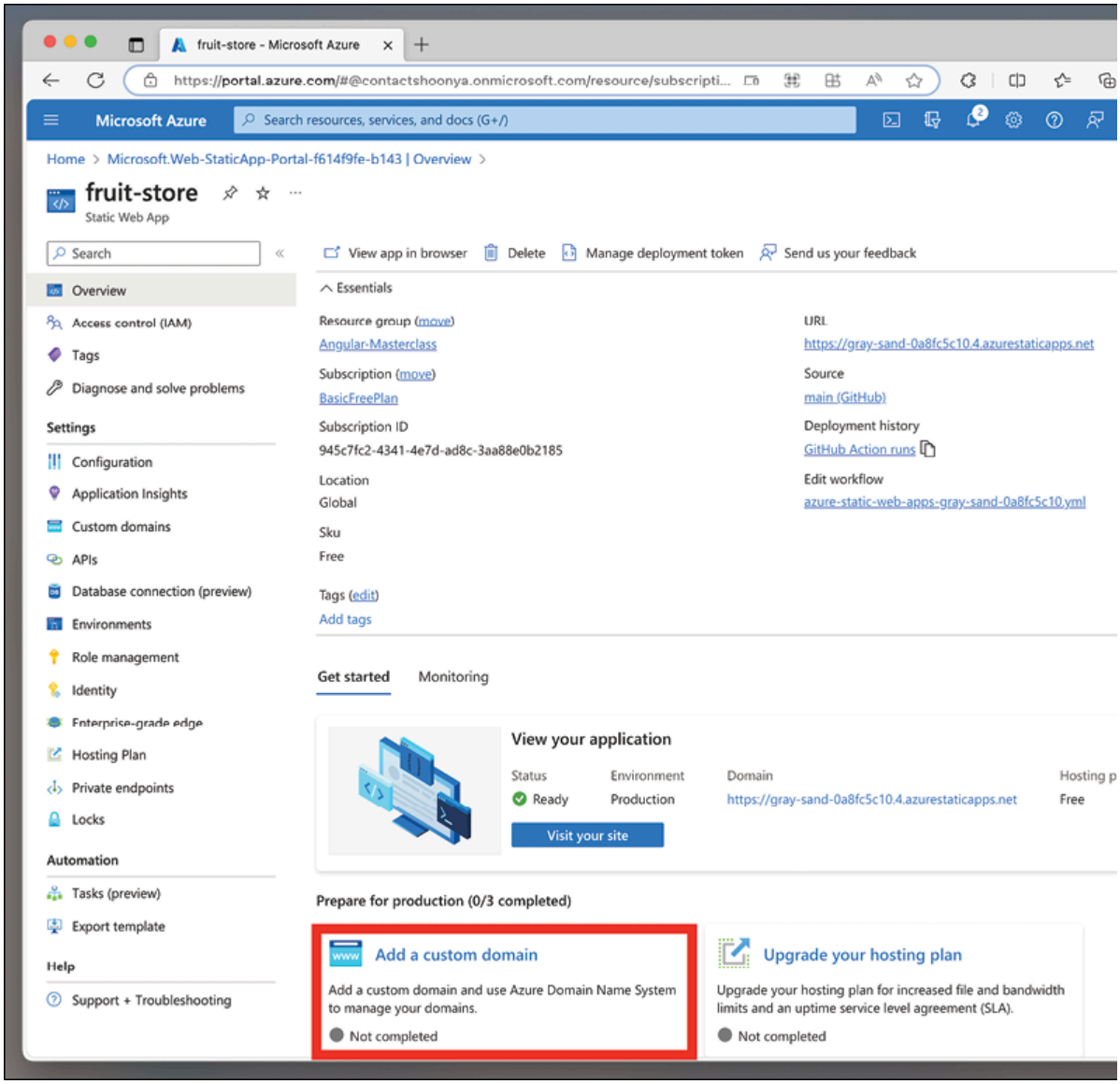


Figure 10.6: *Add a custom domain*

The detailed steps to add custom domain is out of the scope of this book. We have added a link to the Azure documentation in the *References* section on how to set it up.

Conclusion

In conclusion, deploying an Angular application can seem like a daunting task, but it does not have to be. By following the steps outlined in this guide and utilizing the right tools, you can successfully deploy your application and make it available for others to use. If you are doing it for the first time, it is recommended to use a hosting service instead of maintaining your custom web server. Always test your application thoroughly before deployment to ensure it is running smoothly. The next chapter starts a new section of the book on user interactions. In the next chapter, you will learn about handling user inputs.

Points to remember

- Angular CLI can build a production ready folder to deploy to a web server.
- Prod build uses AOT compiler.
- There are many third-party hosting services available to host Angular web apps.
- Angular CLI allows to deploy to a hosting service using a builder package.

Questions

1. Explain what a AOT compiler does.
2. How is a hosting service different from a traditional web server.
3. Explain how Angular can be deployed to Firebase.
4. Explain how Angular can be deployed to Azure Static Web App.

References

- Firebase hosting package:
  https://www.npmjs.com/package/@angular/fire
- Vercel hosting package:
  https://github.com/vercel/vercel/tree/main/examples/angular
- Netlify builder:
  https://github.com/ngx-builders/netlify-builder

- GitHub pages builder:
  https://github.com/angular-schule/angular-cli-ghpages/
- Deploy Angular app on Azure Static Web App:

https://learn.microsoft.com/en-us/azure/static-web-apps/deploy-angular?pivots=github

- Setup Custom domain on Azure Static Web App:

https://learn.microsoft.com/en-us/azure/static-web-apps/custom-domain-external

- Angular advanced deployment options:
  https://angular.io/guide/deployment

CHAPTER 11
User Input Using Forms

Introduction

Forms are an integral part of any web application, as they allow users to interact with the application and provide valuable input data. In Angular, handling user input through forms is a critical aspect of building dynamic and interactive web applications. The framework provides developers with a powerful set of tools and components to handle user input in a seamless and efficient way. In this chapter, we will explore the various methods and best practices for handling user input in Angular through forms. We will cover topics such as template driven forms, reactive forms, form validation, and form submission.

Structure

The chapter covers the following topics:

- User input events handling
- Introduction to forms
- Template driven forms
- Form validation
- Reactive forms
- Nested form groups and form array
- Form builder and validation
- Listening for changes

Objectives

By the end of this chapter, you will understand the various types of forms in Angular to receive user input.

User input events handling

In *Chapter 9, Data Binding and Events Handling,* we saw the basics of event handling in an Angular app. We saw the following example code:

```
1. import { Component, Input } from '@angular/core';
2.
3. @Component({
4.   selector: 'app-root',
5.   templateUrl: './app.component.html',
6.   styleUrls: ['./app.component.css']
7. })
8. export class AppComponent {
9.   onSave() {
10.     alert('Saved!');
11.   }
12. }
```

The **onSave()** method is called when we associate a click event of a button as follows:

```
<button (click)="onSave()">Save</button>
```

Similarly, we can handle changes of text in an **input** control. This section will go over a few examples to show event handling in **input** controls. Let us see an example code:

```
<input (input)="onChange($event)">
```

Handle this event in the component code:

```
onChange(event: any) {
  console.log(event.target.value);
}
```

This will show the input box text in the dev **Console** on every keystroke as follows:

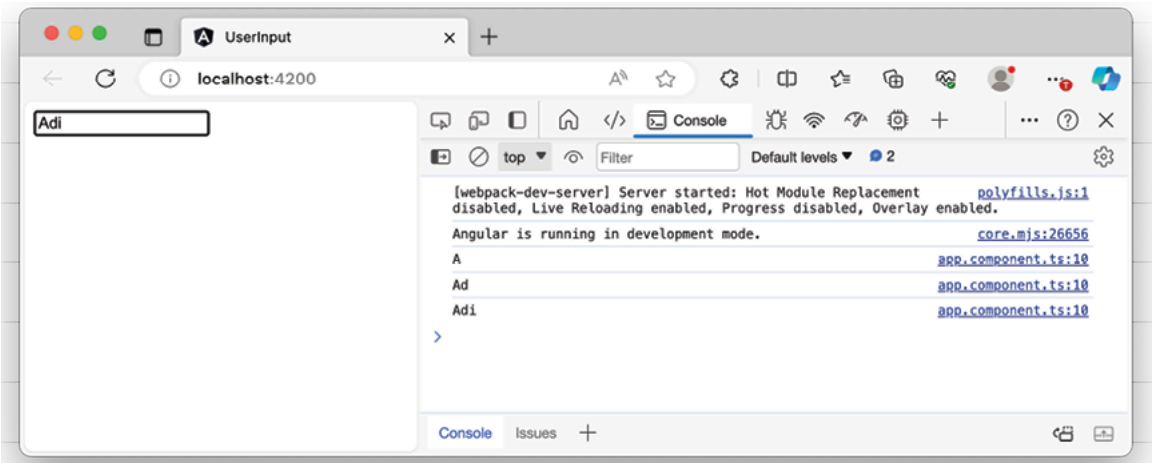


Figure 11.1: *Handling input event*

You can also specify the key code while binding the key event:

```
<input (keyup.enter)="onKeyup($event)" />
```

This event will be triggered only when the *Enter* key is pressed. Add an event handler to confirm this:

```
onKeyup(event: any) {
  console.log(event.target.value);
}
```

Notice in the **Console** now that the input value shows only on press of the *Enter* key:

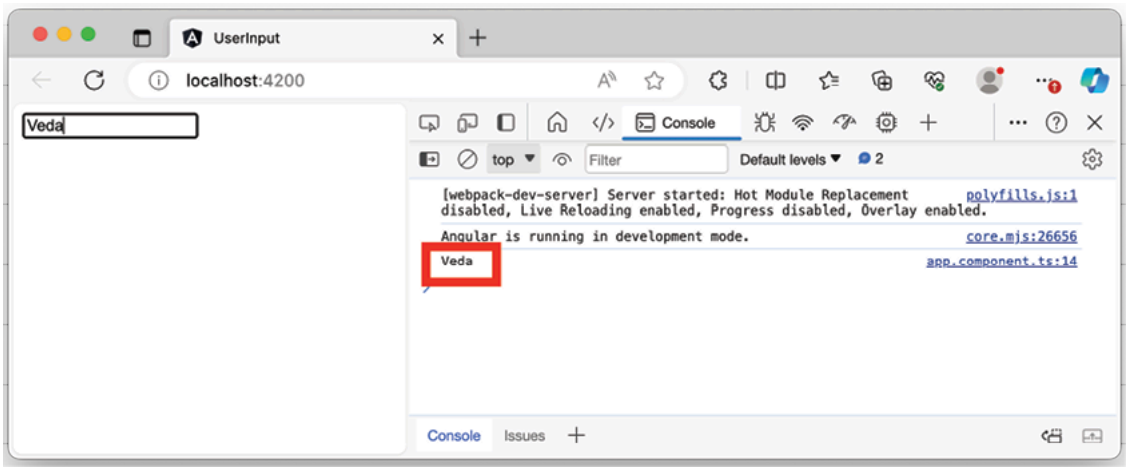


Figure 11.2: *Handling key up event on Enter key press*

Similarly, you can handle *Shift*, *Control*, and *Alt* key combinations as well. For instance, the following **input** control raises a **keydown** event only on the press of *Shift+A* key:

```
<input (keydown.shift.a)="onKeydown($event)" />
```

For a **checkbox** input control, there are two-ways to handle the toggle event. One way is to associate a model to the **checkbox** and handle any change to that model, as follows:

```
checkboxValue: boolean = false;
onCheckboxValueChange() {
  console.log(this.checkboxValue);
}
```

The **template** file will be as follows:

```
<input type="checkbox" [(ngModel)]="checkboxValue" (ngModelChange)="onCheckboxValueChange()"
```

The other way is to handle **change** event for the **input** control as follows:

```
<input type="checkbox" (change)="onCheckboxChange($event)">
<label>
  Checkbox
</label>
```

```
onCheckboxChange(event: any) {
  console.log(event.target.checked);
}
```

Notice that this time, we are using **event.target.checked** and not **event.target.value**.

Introduction to forms

One of the key features of Angular is its powerful form handling capabilities, which allow developers to create complex and responsive forms with ease. In this context, forms are the primary means of collecting data from users, and they play a critical role in the overall user experience of an application. Angular provides a robust set of tools for building forms, including built-in validation, data binding, and error-handling features that make it easy to create forms that are both user friendly and functional. In the next sections, let us go over two types of forms that Angular offers—template driven forms and reactive forms.

Template driven forms

Template driven forms are a way to add simple forms in an Angular app. They rely on directives in the template to create and manipulate the underlying form model. They use two-way data binding to update the data model in the component as changes are made in the template and vice versa. To demo this, let us create a copy of the Angular **fruit-store-app** created in *Chapter 10, Deployment*. We will add a form on the contact page to fill out customer details. The first step is to add **FormsModule** to the **module** file, in this case—**contact.module.ts**. If you only have the **root** module at **app.module.ts**, you may add it there:

```
...
import { FormsModule } from '@angular/forms';

@NgModule({
  declarations: [
    ...
  ],
  imports: [
    ... ,
    FormsModule
  ]
})
export class ContactModule { }
```

We need an underlying model object to hold the form values. Create a class **UserFeedback** in file **app/contact/models/UserFeedback.ts** file as follows:

```
export class UserFeedback {
```

```
    constructor(
      public firstName: string,
      public email: string,
      public description: string,
      public category?: string
    ) { }
}
```

Note that all fields except category are required. Later, we will see how to validate the form for the required fields.

Generate a new component **contact-form** under **app/contact/components** folder. Add the following code to the component class file **contact-form.component.ts**:

```
import { Component } from '@angular/core';
import { UserFeedback } from '../../models/UserFeedback';

@Component({
  selector: 'app-contact-form',
  templateUrl: './contact-form.component.html',
  styleUrls: ['./contact-form.component.css']
})
export class ContactFormComponent {
  model = new UserFeedback('', '', '', '');
  categories = ['Product quality', 'Product availability',
          'Order related', 'General query'];

  onSubmit() {
    console.log(this.model);
  }
}
```

Observe that a model of type **UserFeedback** is created in this class in *line 10* and initialized to empty values for all fields. **categories** are a string array field that holds the possible values to show in a dropdown control to select a category. Also, notice the **onSubmit()** method that logs the model's value to the **Console**.

Paste the following form template in the **contact-form.component.html** file:

```
<form (ngSubmit)="onSubmit()" #contactForm="ngForm">
    <h2>Please submit the form below to contact us</h2>
    <div class="form-control">
      <label for="first_name">Your name</label>
  <input #firstName="ngModel" type="text" required="" id="first_name"
    [(ngModel)]="model.firstName" name="firstName">
    </div>
    <div class="form-control">
      <label for="email">Email</label>
  <input type="email" required id="email" [(ngModel)]="model.email" name="email">
    </div>
    <div class="form-control">
      <label for="category">Issue Category</label>
  <select id="category" [(ngModel)]="model.category" name="category">
  <option *ngFor="let category of categories" [value]="category">{{category}}</option>
      </select>
    </div>
```

```
17.     <div class="form-control">
18.         <label for="description">Description</label>
19.   <textarea type="text" required id="description" [(ngModel)]="model.description"
        name="description"></textarea>
20.     </div>
21.     <button type="submit">Submit</button>
22.  </form>
```

This simple form collects the user's **first_name**, **email**, **category**, and **description**. The top-level element is a **form** tag. When a **form** tag is used, Angular automatically assigns a **ngForm** directive to it, which can be referred to in this case using **contactForm** reference. This gets activated when **FormsModule** is imported. Also, notice that the **ngSubmit** directive is used, which is assigned to the **onSubmit()** method in the component class file. This method will be called when the button with **type="submit"** in *line 21* is clicked within the form. Each combination of **label** and **input** field is wrapped in a **div** with a class **form-control**. This is just for organizing and beautification purposes. For brevity, we have not mentioned the styling of the controls, but you can style it as you please to give it a decent look. Styling of components is discussed in detail in *Chapter 6, Styling and Theming*.

You will notice that the **input**, **textarea**, and **select** controls bind to the **ngModel** directive. This is a notation of a two-way data binding that we discussed in *Chapter 9, Data Binding and Events Handling*. Using this two-way data binding, the model fields are bound to the input control on the template. When the user types or selects a value on the template, the values will automatically get assigned to the model field in the component class. In Angular, when you use **[(ngModel)]** to bind data to an element, it is necessary to define a **name** attribute for that element. The **ngModel** directive creates a **FormControl** instance for each input and binds it to the **name** attribute. For instance, you will notice the **name** attribute set to **firstName** for the first input control and so on. The **name** attribute is used by Angular to register the element with the **NgForm** directive that is attached to the parent **<form>** element. This registration allows the form to track the state and validation of the element and enables you to access and manipulate the form data within your application. Therefore, it is important to ensure that all elements that use **[(ngModel)]** have a unique and meaningful **name** attribute. Also, notice the usage of the **ngFor** directive in the **select** tag on *line 14* to show all possible issue categories.

Render the page, fill in values in the input controls, and click on the **Submit** button. Check that the values are getting logged in the dev **Console**. In a real-world application, this console log will be replaced by some meaningful action, like making an API call. Refer to the following figure:

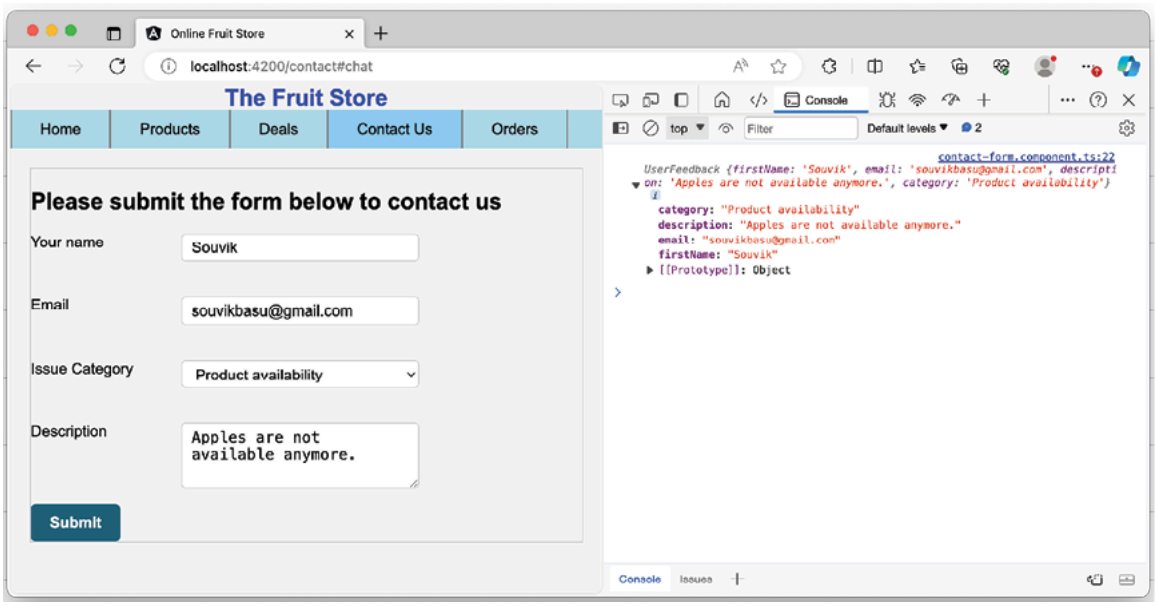


Figure 11.3: *Template driven form submit*

Form validation

The previous section demonstrated a very simple form to allow the user to input their details and submit. Out of these fields, **firstName**, **email**, and **description** fields were marked as required fields. To enforce this, we can validate the input before submitting the form. When we bind the **ngModel** directive to the form controls, it adds **class** names to the control automatically to show the current state. The following table shows the different class names being added to a control based on the current state:

Condition	Class name if true	Class name if false
Control has been visited	**ng-touched**	**ng-untouched**
Control's value has changed	**ng-dirty**	**ng-pristine**
Control's value is valid	**ng-valid**	**ng-invalid**

Table 11.1: *Class names for form control*

You can add style statements based on these class names. For instance, add the following CSS in the **contact-form.component.css** file:

```
.ng-valid {
    border: 1px solid #ccc;
}

.ng-invalid {
    border-color: red;
    border-width: 3px;
}
```

The page refreshes to show the first name, email, and description fields with a red border. This is because the fields were marked as required, and a value has not yet been provided. Open the browser dev tool and notice the class names applied to the control, as shown in the following figure:

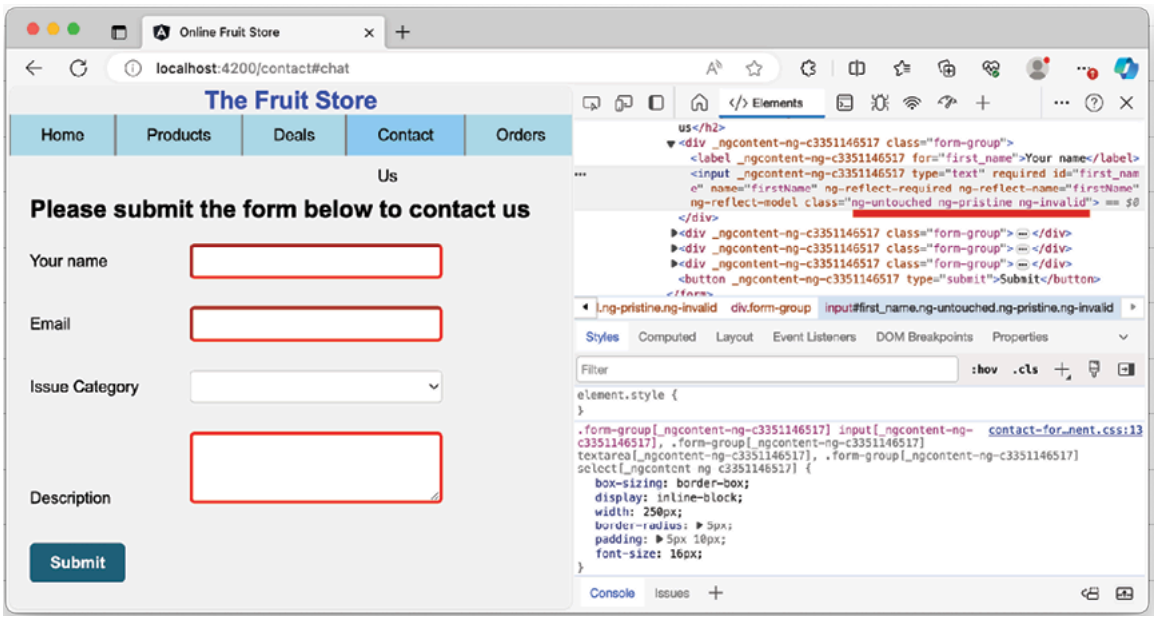


Figure 11.4: *Class names applied to input control within a form*

If you click on, say, the first name input control, observe **ng-untouched** change to **ng-touch**. Now, type something, and **ng-pristine** changes to **ng-dirty** and **ng-invalid** changes to **ng-valid** as all the conditions are now met. This will show the regular grey border again.

Generally, we would want the input controls not to show an error border till the user starts typing. To do this, you can add the combination of **ng-invalid** and **ng-touched** to the CSS:

```
.ng-valid, .ng-untouched {
    border: 1px solid #ccc;
}
```

```
4.
5. .ng-invalid.ng-touched {
6.     border-color: red;
7.     border-width: 3px;
8. }
```

A better user experience is to show the error border when the **Submit** button is clicked. This can be achieved by using another class **ng-submitted** that is applied to the form element once it is submitted:

```
1. .ng-valid, .ng-invalid, .ng-touched, .ng-untouched {
2.     border: 1px solid #ccc;
3. }
4.
5. .ng-submitted .ng-invalid {
6.     border-color: red;
7.     border-width: 3px;
8. }
```

You can also show a custom error message per field when validation fails. To demonstrate this, add the following **div** under the **firstName** input control:

```
1. <input #firstName="ngModel" type="text" required="" id="first_name" [(ngModel)]="model.firstName" r
2. <div [hidden]="firstName.valid || firstName.pristine" class="error-msg">Please enter a name</div>
```

We are showing the **div** conditionally based on the state of the input control. A template reference variable **#firstName** has been added to the input control and assigned to its **ngModel**. This allows us to access the state of the form control using the reference variable. The error message div is conditionally rendered if the input control is dirty and not valid:

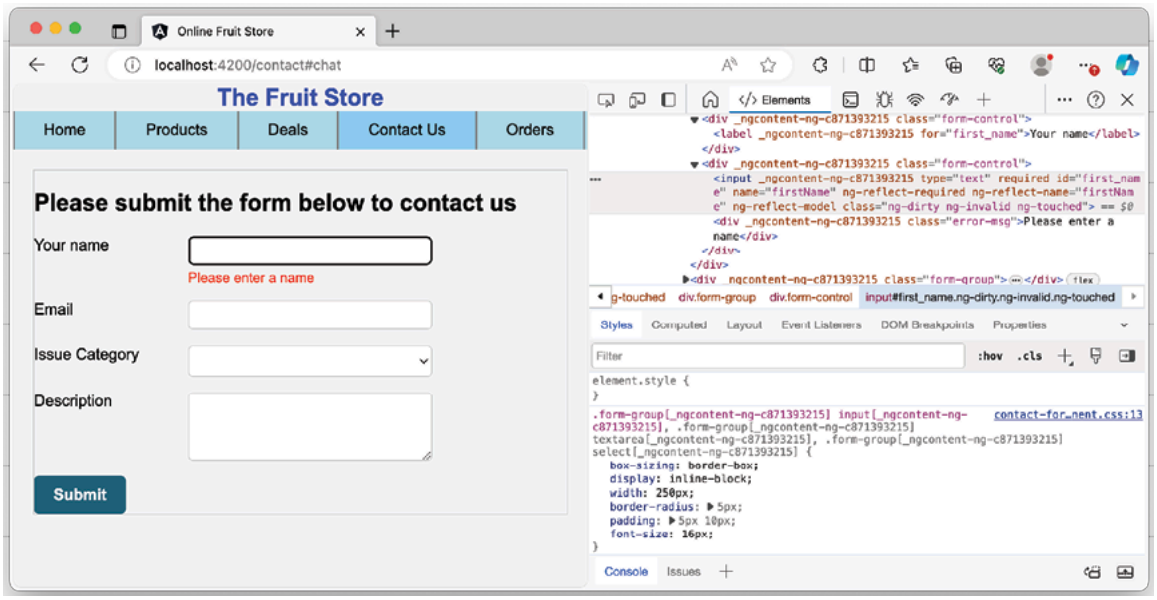


Figure 11.5: *Form validation on submit*

You can also use the template reference variable **#contactForm** that we attached to the **ngModel** of the form tag to detect if the entire form is valid. If you want to disable the **Submit** button until the form is valid, add a disabled attribute conditionally as follows:

```
1. <button type="submit" [disabled]="!contactForm.form.valid">Submit</button>
```

Add CSS statements to style the **disabled** button:

```
1. button[disabled] {
2.     background-color: #ccc;
3.     cursor: not-allowed;
4. }
```

Now, the button will be disabled unless all the fields in the form are valid.

For complex business logic handling, you can write custom logic in the component class file. For that to work, you need to send the form template reference variable to the **onSubmit** method as follows:

```
<form (ngSubmit)="onSubmit(contactForm)" #contactForm="ngForm">
```

Then you can handle the following:

```
onSubmit(form: NgForm) {
  if(form.invalid) {
    alert('Please fill in all the required fields.');
    return;
  }
  // Handle form submission here

  form.reset();
}
```

In this specific code example, note that our **Submit** button will remain disabled unless the form is valid, and hence, the conditional logic within the **onSubmit()** method will not be called. Remove the disabled attribute from the button and let the **onSubmit()** method handle from an invalid case.

Reactive forms

Reactive forms in Angular are a way of creating and managing forms that are based on the model of the data rather than the template of the view. Reactive forms provide more control and flexibility over the form state, validation, and dynamic changes. Reactive forms use the **ReactiveFormsModule** and the **FormControl**, **FormGroup**, and **FormArray** classes to create and manipulate form controls programmatically.

Some of the use cases for reactive forms are:

- Working with complex data structures or nested forms.
- Testing the form logic and validation without depending on the view.
- Creating custom validators or async validators.
- Changing the validation rules or appearance of the form based on user input or other conditions.
- Dynamically adding, removing, or updating form controls or groups.

To understand reactive forms, let us recreate the contact form from the previous sections using a reactive form. Add a new component:

```
ng g c contact/components/contact-reactive-form
```

Use the following form on the contact page, **contact.component.html**, instead of the previous template driven form:

```
<app-contact-reactive-form></app-contact-reactive-form>
```

Open the **contact-reactive-form.component.html** file and add an input control as follows:

```
<label for="first_name">Your name </label>
<input id="first_name" type="text" [formControl]="firstName">
<p>Value: {{ firstName.value }}</p>
```

In the **contact-reactive-form.component.ts** file, add the **firstName** field as follows:

```
import { Component } from '@angular/core';
import { FormControl } from '@angular/forms';

@Component({
  selector: 'app-contact-reactive-form',
  templateUrl: './contact-reactive-form.component.html',
  styleUrls: ['./contact-reactive-form.component.css']
})
export class ContactReactiveFormComponent {
```

```
10.    firstName = new FormControl();
11.  }
```

Ensure that **ReactiveFormsModule** is added to the **module** file as follows:

```
1.  ...
2.  import { FormsModule, ReactiveFormsModule } from '@angular/forms';
3.  import { ContactReactiveFormComponent } from './components/contact-reactive-form/contact-reactive-fo
4.
5.
6.  @NgModule({
7.    declarations: [
8.      ...
9.      ContactReactiveFormComponent
10.   ],
11.   imports: [
12.     ...
13.     ReactiveFormsModule
14.   ]
15. })
16. export class ContactModule { }
```

Now, the page will show the input control. If you change the value in **Your name** field, observe that the **Value** label shows the updated name field as follows:

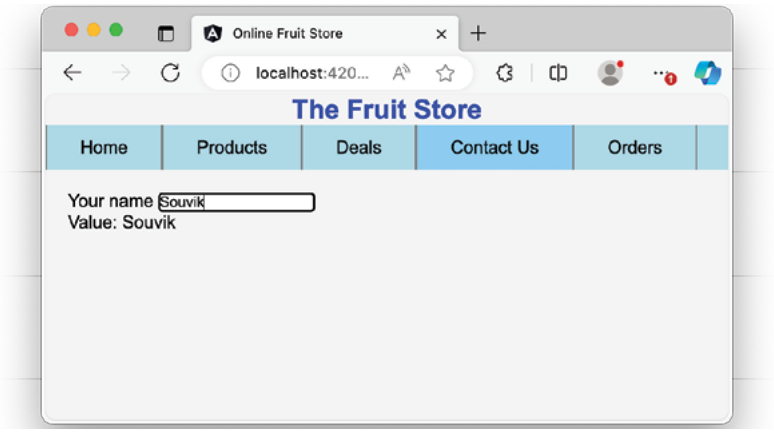


Figure 11.6: *Basic FormControl*

This was a very simple demonstration of how to use **FormControl** directive. The view is bound to the **firstName** field defined in the component file. If the value of the field changes in the component file, it will reflect in the view and vice versa. An attentive reader would notice that we have not used a form yet in the template. We only assigned a **FormControl** directive to the input control. Next, let us wrap this code inside a **form** element and bind **FormGroup** directive to the form, as follows:

```
1.  <form [formGroup]="contactForm" >
2.      <label for="first_name">Your name </label>
3.      <input id="first_name" type="text" formControlName="firstName">
4.  </form>
5.  <p>Value: {{ contactForm.value | json }}</p>
```

Here, we use the directive **formGroup** in the form element and bind it to the field **contactForm**. A **FormGroup** is a way to combine related **FormControl** elements. In this case, it is assigned to the form element. Also, notice now that instead of binding **formControl** to the **firstName** field, we are using the attribute **formControlName** and assigning the value **firstName** to it. This is done because the form is already bound to the field **contactForm** hence, no further binding is needed. Let us define the field **contactForm** in

the component class as follows:

```
import { Component } from '@angular/core';
import { FormControl, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-contact-reactive-form',
  templateUrl: './contact-reactive-form.component.html',
  styleUrls: ['./contact-reactive-form.component.css']
})
export class ContactReactiveFormComponent {
  contactForm: FormGroup = new FormGroup({
    firstName: new FormControl(),
  });
}
```

Here, the **firstName: FormControl** is now wrapped inside **contactForm: FormGroup**. Since the **form** element is bound to the **contactForm**, all **FormControl** instances within the **contactForm** can now be accessed using the **formControlName** attribute. We are displaying the value of the entire form now. The page should look like this:

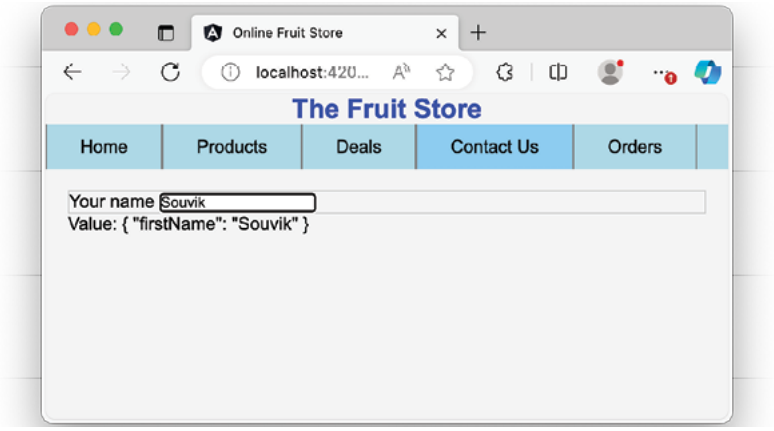


Figure 11.7: *Using FormGroup to wrap FormControl*

Let us now add the remaining fields to complete the **Contact Us** form along with a **Submit** button and the **onSubmit()** handler method:

```
export class ContactReactiveFormComponent {
  categories = ['Product quality', 'Product availability',
  'Order related', 'General query'];

  contactForm: FormGroup = new FormGroup({
    firstName: new FormControl(''),
    email: new FormControl(''),
    category: new FormControl('Product quality'),
    description: new FormControl(''),
  });

  onSubmit() {
    // Handle form submission here

    this.contactForm.reset();
  }
```

```
}
```

Add the respective form controls to **contact-reactive-form.component.html** file as follows:

```
<form [formGroup]="contactForm" (ngSubmit)="onSubmit()">
  <h2>Please submit the form below to contact us</h2>
  <div class="form-control">
    <label for="first_name">Your name </label>
    <input id="first_name" type="text" formControlName="firstName">
  </div>
  <div class="form-control">
    <label for="email">Email</label>
    <input id="first_name" type="text" formControlName="email">
  </div>
  <div class="form-control">
    <label for="category">Issue Category</label>
    <select id="category" formControlName="category">
<option *ngFor="let category of categories" [value]="category">{{category}}</option>
    </select>
  </div>
  <div class="form-control">
    <label for="description">Description</label>
<textarea type="text" id="description" formControlName="description"></textarea>
  </div>
  <button type="submit">Submit</button>
</form>
<p>Value: {{ contactForm.value | json }}</p>
```

This will show the new contact form as below:

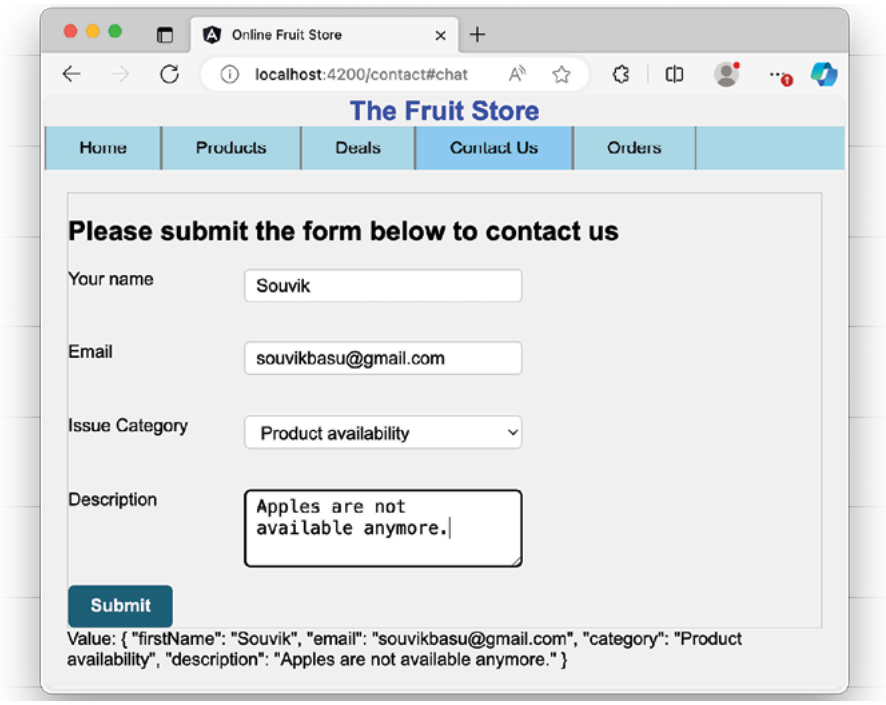


Figure 11.8: *Contact Us form using reactive form*

Observe that the form value printed at the bottom of the page is in sync with the input control values. In this reactive form, the underlying data is defined in the component class, and the view is just a presentation of that

data. This makes adding validations and business logic to the form elements much easier.

Nested form groups and form array

A form group is a logical grouping of form elements. Another way to combine form controls is to create an array using **FormArray**. For complex forms, we can create nested groups. The example below of a user profile form shows how to nest form groups and form array inside another form group:

```
import { Component } from '@angular/core';
import { FormGroup, FormControl, FormArray } from '@angular/forms';

@Component({
  selector: 'app-nested-form',
  template: `
    <form [formGroup]="form">
      <div formGroupName="profile">
        <label>Name: <input formControlName="name"></label>
        <label>Age: <input formControlName="age"></label>
      </div>
      <div formArrayName="hobbies">
        <div *ngFor="let hobby of hobbies.controls; let i = index">
  <label>Hobby {{ i + 1 }}: <input [formControlName]="i"></label>
        </div>
        <button (click)="addHobby()">Add Hobby</button>
      </div>
    </form>
    <p>Value: {{ form.value | json }}</p>
    <p>Status: {{ form.status | json }}</p>
  `
})
export class NestedFormComponent {
  form = new FormGroup({
    profile: new FormGroup({
      name: new FormControl(''),
      age: new FormControl('')
    }),
    hobbies: new FormArray([
      new FormControl('Reading'),
      new FormControl('Writing')
    ])
  });

  get hobbies() {
    return this.form.get('hobbies') as FormArray;
  }

  addHobby() {
    this.hobbies.push(new FormControl(''));
  }
}
```

Using **FormArray**, you can add unnamed controls to the form dynamically as shown in *line 40* above. This makes your form a lot more flexible and extensible than what template driven forms could achieve.

As an exercise, try to create a nested form in the **Contact Us** form from the previous section.

Form builder and validation

Form builders are helper functions to create reactive forms with minimal code. The reactive form we created in the **Contact Us** page can be re-written using a form builder as follows:

```
1.  export class ContactReactiveFormComponent {
2.    categories = ['Product quality', 'Product availability',
3.      'Order related', 'General query'];
4.    contactForm: FormGroup;
5.
6.    constructor(private fb: FormBuilder) {
7.      this.contactForm = fb.group({
8.        firstName: [''],
9.        email: [''],
10.         category: ['Product quality'],
11.         description: [''],
12.       });
13.   }
14. ...
15. }
```

Now, it is very easy to add validations into the form controls from within the definition of **contactForm** in the component class as follows:

```
1.  export class ContactReactiveFormComponent {
2.    categories = ['Product quality', 'Product availability',
3.      'Order related', 'General query'];
4.    contactForm: FormGroup;
5.
6.    constructor(private fb: FormBuilder) {
7.      this.contactForm = fb.group({
8.        firstName: ['', Validators.required],
9.        email: ['', [Validators.required, Validators.email]],
10.         category: ['Product quality'],
11.     description: ['', [Validators.required, Validators.minLength(20)]],
12.       });
13.   }
14.
15.   onSubmit() {
16.     if(this.contactForm.invalid) {
17.       alert('Please fill in all the required fields.');
18.       return;
19.     }
20.     // Handle form submission here
21.
22.     this.contactForm.reset();
23.   }
24. }
```

Here, we are adding a validation for the **firstName** form control to be a required field. **email** form control has two validations—to be required and to be of email format. Notice how the two validations are clubbed inside an array notation. Similarly, the **description** form control has two validations—to be required and to have a minimum length of 20 characters. Like we did in the template driven form, we can check for form validation in the **onSubmit()** method before submitting the form. A **FormGroup** is valid only when all form controls within it are valid. In this case, the form is bound to a **FormGroup**, and hence, it is valid when all its controls pass validation.

In the template driven form example, we added an error message in the template when validation failed for the **firstName** field. We can add a similar error message in the reactive form as well:

```
1.    <div class="form-control">
2.       <label for="first_name">Your name</label>
3.       <div>
4.  <input id="first_name" type="text" formControlName="firstName">
5.  <div *ngIf="contactForm.get('firstName')?.invalid && (contactForm.get('firstName')?.dirty ||
    contactForm.get('firstName')?.touched)"
6.               class="error-msg">Please enter a name</div>
7.       </div>
8.    </div>
```

If you run the app, type some text in the **first_name** input control, and remove it, you will notice an error message as follows:

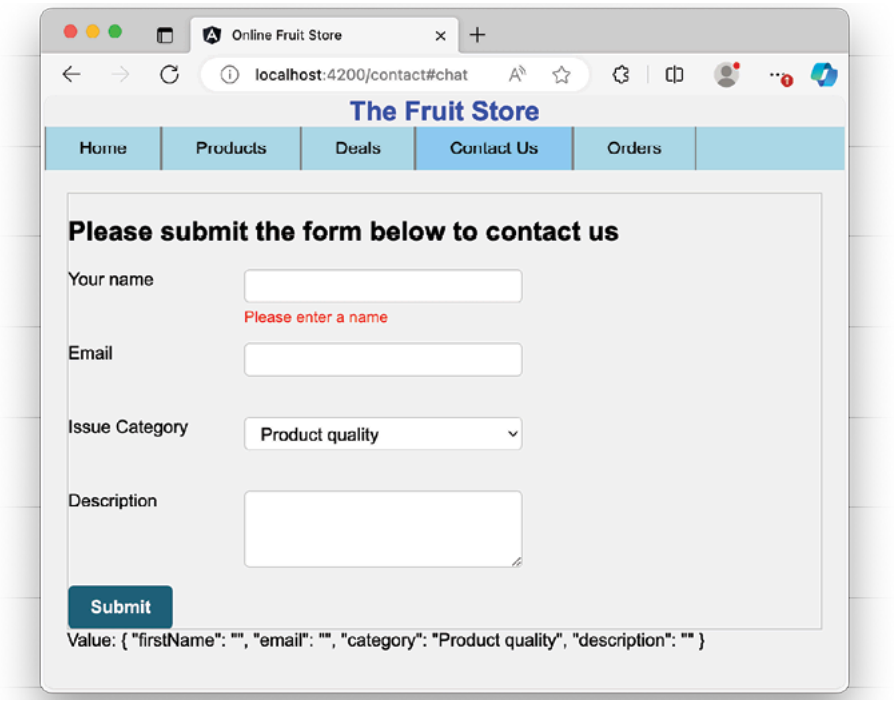


Figure 11.9: *Reactive form validation*

Similarly, if you click the **Submit** button while the input controls are empty, you will notice the red border indicating errors in the input control along with the error message alert as follows:

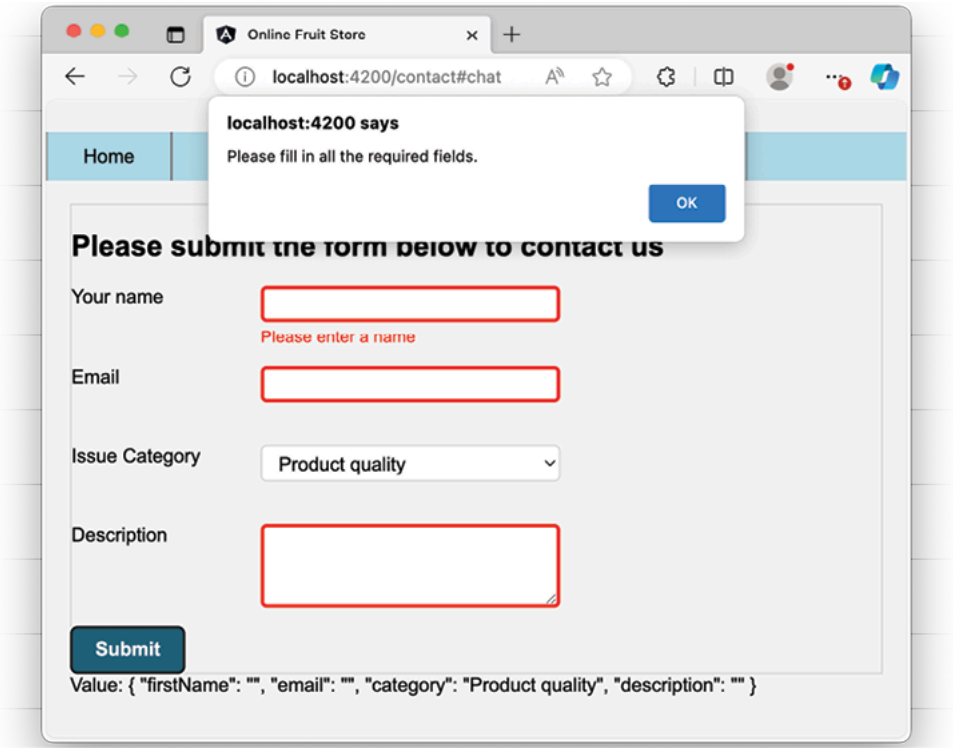


Figure 11.10: *Form validation on Submit*

Listening for changes

One of the major advantages of using reactive forms is the ability to write clean code to listen to form value changes. The following code will print the new values to the console if any input control value changes:

```
1.  ngOnInit() {
2.  this.contactForm.valueChanges.subscribe((value) => console.log("Value Updated ", value));
3.  }
```

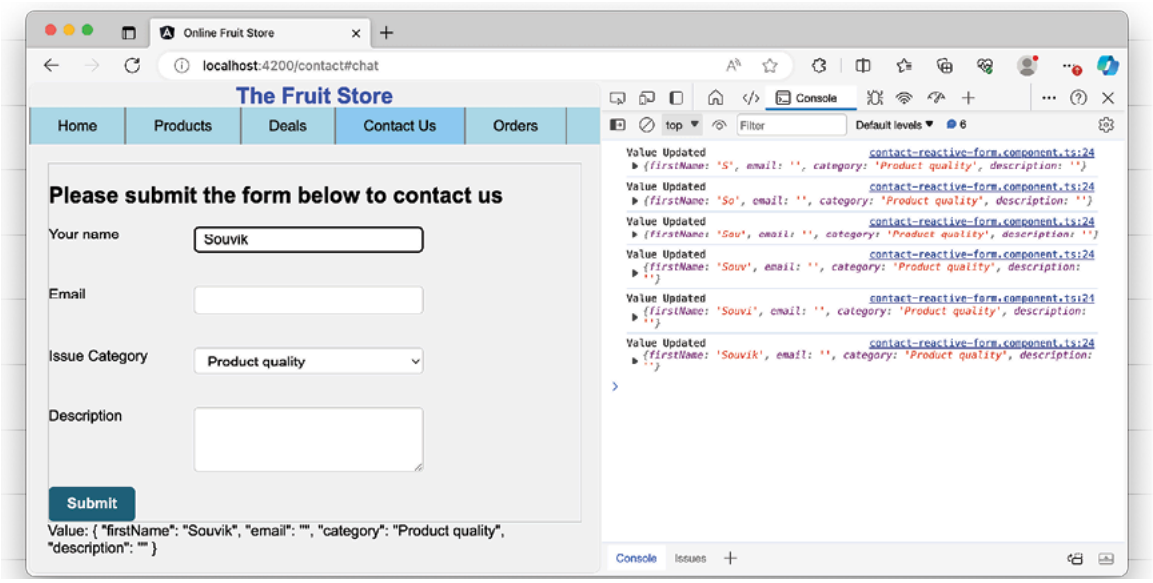


Figure 11.11: *Listening to form value changes*

Conclusion

Forms are an essential aspect of any web application, and Angular offers a powerful set of features to handle

them with ease. With the reactive forms approach, we can build complex forms with dynamic behavior and validation. Alternatively, the template driven forms approach offers a simpler and more intuitive way to handle forms. Regardless of the approach we choose, Angular provides us with numerous options and tools to make form handling a breeze. By leveraging these features effectively, we can create robust, user friendly forms that meet the needs of our application and users. In the next chapter, you will learn about handling reactivity using observables and signals.

Points to remember

- User input on input controls can be listened to using event handling.
- Template driven form relies on directives in the template to create and manipulate the underlying form model.
- Template driven forms are intuitive and work best for simpler forms.
- Reactive forms are driven by the form group and form controls defined in the component class and are useful for complex forms.
- Form builder is a helper function to create reactive forms with ease and allows form validators to be attached to form controls.
- FormArray helps to add or remove FormControls dynamically into a form.

Questions

1. Explain the difference between input and change events on input controls.
2. What are the differences between a template driven form and a reactive form?
3. Explain the scenario when it is better to use reactive forms.
4. How do you add custom validators to reactive forms?

References

- Angular doc on forms:
  https://angular.io/guide/forms-overview
- Angular doc on template driven forms:
  https://angular.io/guide/forms
- Angular doc on reactive form:
  https://angular.io/guide/reactive-forms

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 12
Observables and Signals

Introduction

Reactivity is a powerful paradigm in Angular that allows developers to handle asynchronous data streams and events efficiently. At its core, reactivity revolves around the use of observables and **Reactive Extensions for JavaScript** (**RxJS**), which provide a robust framework for managing data flows and state changes in an application. By leveraging these tools, Angular applications can respond to user interactions, server responses, and other asynchronous events seamlessly and declaratively. This approach enhances the responsiveness and performance of applications and simplifies the complexity of managing asynchronous operations, making the code more readable and maintainable. This chapter will explain two reactivity constructs in Angular: Observables and Signals.

Structure

This chapter covers the following topics:

- Understanding observables and observers
- Reactive extensions for JavaScript
- Operators in RxJS
- Combining multiple observables
- Subject and multicast
- Using observables in Angular

- Signals

Objectives

By the end of this chapter, you will understand the concept of an observable and how events and network calls are handled using observables in Angular. You will also understand the concept of signals that can simplify communication between Angular components and services.

Understanding observables and observers

Observables represent a significant concept in reactive programming, particularly in frameworks like Angular. They are a foundational building block for handling asynchronous data streams and events. At their core, observables are essentially a representation of data that can be passed around within an application. What makes observables powerful is their ability to emit multiple values over time, including potentially infinite sequences of values. This makes them especially suitable for modeling asynchronous data streams such as user inputs, server responses, and other dynamic sources.

Observers, on the other hand, are entities that express interest in the values emitted by observables. They subscribe to observables in order to receive notifications whenever new values are emitted. This subscription creates a channel through which the observer can react to the incoming data, performing operations such as processing, transforming, or consuming the emitted values. Observers can also react to other types of notifications such as errors or the completion of the observable stream.

The relationship between observables and observers is fundamental to the concept of reactive programming. Observables serve as the source of data, emitting values over time, while observers are the consumers that react to these emitted values. This separation of concerns between the data source (observable) and the data consumer (observer) allows for a clear and decoupled handling of asynchronous data streams within an application. It promotes a reactive and event-driven approach to managing data and interactions, enabling developers to write efficient, scalable, and easily maintainable code.

Reactive extensions for JavaScript

RxJS is a library for reactive programming using observables, which makes it easier to compose asynchronous or callback-based code. It is heavily used in Angular for asynchronous data handling. Let us understand observables using RxJS code samples. In the previous section, we defined observable and observer. An observable is created using the observable constructor as shown below:

```
1. import { Observable } from 'rxjs';
2. const messages$ = new Observable(function subscribe =
3. subscriber => {
4.  subscriber.next('This message is emitted by the observable');
5.  subscriber.next('This is the 2nd message');
6.  subscriber.next('This is the last message before it completes');
7.  subscriber.complete();
8. });
9.
10. const subscription = messages$.subscribe(x => console.log(x));
11. subscription.unsubscribe();
```

This will console log the three statements. In the above code sample, **messages$** is the observable that produces a stream of data. By convention, observables are postfixed by a **$** symbol. There are four stages of an observable that we can notice in the above code:

1. **Creating an observable**: *Line 2* shows how to create an observable using the **Observable()** constructor. The constructor takes a **subscribe** function as an input. There are other ways to create an observable by using RxJS operators like **of()**, **from()**, **interval()**, **fromEvent()**, etc. that we will see later in the chapter.
2. **Subscribing to observables**: An observer consumes the values delivered by an observable using a subscription. A subscription represents an execution of an observable. As you see in *line 10*, the **subscribe** method is called by the **subscriber** to receive streams of data. It is internally mapped to the **subscribe** method used to initiate the observable in *line 2*. If there are multiple subscriptions to the same observable, they are run independently of each other.

3. **Executing an observable**: The code inside the **subscribe** method defined in the observable constructor is executed every time an observer subscribes to the observable. Each subscription is independent of the other, meaning that each **messages$.subscribe(...);** statement will execute the statements defined inside the observable independently. For simplicity, you can imagine observables as function calls in many ways. They both are lazily called; that is, the statements are executed only when a caller calls (or subscribes). So, every time you subscribe to an observable, all the statements defined within the observable are executed at that time. Where an observable differs majorly from a function call is that a function returns a single value while an observable can return multiple values. This is very similar to how iterators can return multiple values from a function. The following table shows the subtle differences between these mechanisms:

	Single value returned	**Multiple values returned**
Pull	Function	Iterator
Push	Promise	Observable

Table 12.1: *Examples of pull and push mechanisms*

A **pull** mechanism is where the producer of the data is passive, and the consumer actively determines when to request data. A **push** mechanism is where the producer actively creates data at its own pace while the consumer passively reacts to the received data. Function and iterator are pull mechanism where the caller determines when to request data. While promise and observable are push mechanisms where the caller reacts to the received data. So, to some extent, you can imagine an observable as conceptually similar to a promise that returns multiple values.

Every **.next()** call within an observable definition returns a value to the observer. There are three types of calls that can be made by the subscriber inside observable definition:

- **next**: This sends a number, string, or object to the subscriber.
- **error**: This sends a JavaScript error or exception.
- **complete**: This does not send any value. It only indicates the end of the data stream. Any further **next()** call after a **complete()** call does

not return any data from the observable.

Our initial **subscribe** code can be written as follows to handle the above three conditions:

```
1. messages$.subscribe({
2. next: value => console.log('Next:', value),
3. error: err => console.error('Error:', err),
4. complete: () => console.log('Completed')
5. });
```

By default, an observable executes synchronously. Consider the following simple **Observable** that returns two numbers:

```
1. const numbers$ = new Observable(observer => {
2. observer.next(100);
3. observer.next(200);
4. observer.complete();
5. });
6.
7. console.log("Start");
8. numbers$.subscribe(x => console.log(x));
9. console.log("End");
10.
11. console.log("Start");
12. numbers$.subscribe(x => console.log(x));
13. console.log("End");
```

The output is as follows:

1. Start
2. 100
3. 200
4. End
5. Start
6. 100
7. 200

8. End

You can return asynchronous values from an observable using async constructs like **setTimeout()**:

```
1. const numbers$ = new Observable(subscriber => {
2. subscriber.next(100);
3. subscriber.next(200);
4. setTimeout(() => {
5. subscriber.next(300);
6. subscriber.complete();
7. }, 1000);
8. });
```

This will output asynchronously as follows:

1. Start
2. 100
3. 200
4. End
5. Start
6. 100
7. 200
8. End
9. 300
10. 300

It is a recommended practice to wrap the code within the **subscribe** method in a **try**/**catch** block and return **error** from the **catch** block:

```
1. const numbers$ = new Observable(subscriber => {
2. try {
3. subscriber.next(100);
4. subscriber.next(200);
5. setTimeout(() => {
6. subscriber.next(300);
7. subscriber.complete();
```

```
}, 1000);
} catch (error) {
  subscriber.error(error);
}
});
```

4. **Disposing of an observable**: Since observables can return multiple values, theoretically, they can return infinite values. Hence, it is important to dispose of an observable after use. This can be done using the **unsubscribe** method, similar to how we do for the **setTimeout** function:

```
const subscription = messages$.subscribe(x => console.log(x));
subscription.unsubscribe();
```

Missing to **unsubscribe** an observable may cause memory leaks depending on the definition of the observable.

Operators in RxJS

RxJS has many built-in operators used to create and modify the stream of data within the observable. Operators are functions that allow you to transform, filter, and combine observables. There are two types of operators:

- Creational operators
- Pipeable operators

Creational operators

Apart from the **Observable()** constructor, RxJS provides many utilities to create observables. Following is an example of generating an observable from a fixed set of values using the **of()** operator:

```
import { of } from 'rxjs';

const numbers$ = of(100, 200, 300);
numbers$.subscribe(x => console.log(x));
```

You can create an observable from an array using the **from()** operator, as shown:

```
import { from } from 'rxjs';
```

```
2.
3. const numbers$ = from([100, 200, 300]);
4. numbers$.subscribe(x => console.log(x));
```

Another example is to generate data at fixed interval of time using the **interval()** operator as below:

```
1. import { interval } from 'rxjs';
2.
3. const numbers$ = interval(1000 /* milliseconds */);
```

A fourth example is the **fromEvent()** operator, which is used to generate an observable from a stream of events:

```
1. import { fromEvent } from 'rxjs';
2.
3. const clicks = fromEvent(document, 'click');
4. clicks.subscribe(event => console.log(event));
```

The above code logs **MouseEvent** on every click on the **document**. This operator is heavily used in Angular, as we will see later in the chapter. There are many other creational operators that you can find in the RxJS documentation link from the *References* section at the end of this chapter.

Pipeable operators

Pipeable operators are functions that take an observable as an input and return another observable. The original observable remains unchanged. These operators use the **pipe()** method to receive a data stream from the input observable. An example is a **map()** operator that can be used to modify each value using a function as follows:

```
1. import { map, of } from 'rxjs';
2.
3. of(100, 200, 300)
4.     .pipe(map(x => x/100))
5.     .subscribe(x => console.log(`New value: ${x}`));
```

The output is:

```
1. New value: 1
2. New value: 2
3. New value: 3
```

Similarly **first()** operator returns the first item from the stream:

```
1. import { first, of } from 'rxjs';
2.
3. of(100, 200, 300)
4.    .pipe(first())
5.    .subscribe(x => console.log(x));
```

This outputs:

```
1. 100
```

A **filter()** operator allows only the values passing the predicate condition to be returned:

```
1. import { filter, of } from 'rxjs';
2.
3. of(100, 200, 300)
4.   .pipe(filter(x => x > 150))
5.   .subscribe(x => console.log(x));
```

This outputs the following:

```
1. 200
2. 300
```

Many such operators are defined in the **RxJS** library, which you can discover on their documentation page.

You may arrange multiple operators in a pipe to execute sequentially on the data received from the steam. For instance, the following code will find the square of every value, filter out even values, and take the first three items from it:

```
1.  import { filter, map, of, take } from 'rxjs';
2.
3.  of(1, 2, 3, 4, 5, 6, 7)
4.    .pipe(map(x => x * x),
5.      filter(x => x % 2 === 0),
6.      take(3))
7.    .subscribe(x => console.log(x));
```

This will output the following:

```
1. 4
```

2. 16
3. 36

You can also reduce a stream of data into a scalar value using the **reduce()** operator as below:

```
import { filter, map, of, take } from 'rxjs';

of(1, 2, 3, 4, 5, 6, 7)
  .pipe(map(x => x * x),
    filter(x => x % 2 === 0),
    take(3),
    reduce((n, x) => n + x))
  .subscribe(x => console.log(x));
```

This will console out **56**.

Combining multiple observables

There are operators to combine observables. The **merge()** operator can combine multiple observables into a single observable:

```
import { filter, interval, merge, take } from 'rxjs';

const odd$ = interval(1000).pipe(filter(x => x % 2 === 1), take(3));
const even$ = interval(500).pipe(filter(x => x % 2 === 0), take(3));
const alphabets$ = of('a', 'b', 'c');
const merged = merge$(odd$, even$, alphabets$);
merged$.subscribe(x => console.log(x));
```

This produces the output as follows:

1. a
2. b
3. c
4. 0
5. 2
6. 1
7. 4
8. 3

9. 5

Notice that the values are printed in the order received in the timeline. If, instead, you wanted to print these values sequentially, one observable after the other, you can use the **concat()** operator as follows:

```
1. const concat$ = concat$(odd$, even$, alphabets$);
2. concat$.subscribe(x => console.log(x));
```

This will wait for each observable to complete before concatenating the next one:

1. 1
2. 3
3. 5
4. 0
5. 2
6. 4
7. a
8. b
9. c

A comprehensive discussion of all RxJS operators is beyond the scope of this book. I encourage you to play with all operators defined within RxJS. That will empower you to decide how best to transform the data for your use case.

Subject and multicast

A Subject in RxJS is a special type of observable that acts as both an observer and an observable. This dual nature makes Subjects very versatile and useful in many scenarios.

The characteristics of a Subject are as follows:

- **Observable**: It can be subscribed to, like any other observable.
- **Observer**: It can emit values to its subscribers using the **next()**, **error()**, and **complete()** methods.
- **Multicast**: Unlike regular observables, which are unicast (one separate execution per subscriber), a Subject shares a single execution among multiple subscribers.

Let us take an example of a plain Subject that does not have an initial

value of replay behavior:

```
const subject = new Subject<number>();
subject.subscribe(value => console.log('Subscriber 1:', value));
subject.subscribe(value => console.log('Subscriber 2:', value));

// Emit values
subject.next(1);
subject.next(2);
subject.next(3);
```

This outputs the following:

```
Subscriber 1: 1
Subscriber 2: 1
Subscriber 1: 2
Subscriber 2: 2
Subscriber 1: 3
Subscriber 2: 3
```

There are different other types of Subjects. Let us take a look at them:

- **BehaviorSubject**: It requires an initial value and emits its current value to new subscribers. Refer to the following:

```
const behaviorSubject = new BehaviorSubject<number>(0);
behaviorSubject.subscribe(value => console.log('A:', value));
behaviorSubject.next(1);
behaviorSubject.subscribe(value => console.log('B:', value));
```

This outputs the following:

```
A: 0
A: 1
B: 1
```

This is also referred to as a **hot observable**, as every subscriber receives data that is available after the **subscribe()** method is called. The values emitted by the observable before the **subscribe()** method call are not received by the subscriber. Also, all subscribers receive the same data under a shared execution context.

Also observe that a single **.next()** call on the subject is received by all subscribers. This is a demonstration of Multicast using subjects.

- **AsyncSubject**: Emits only the last value to observers when the sequence is completed. Refer to the following:

```
1. const asyncSubject = new AsyncSubject<number>();
2. asyncSubject.subscribe(value => console.log('Async:', value));
3. asyncSubject.next(1);
4. asyncSubject.next(2);
5. asyncSubject.next(3);
6. asyncSubject.complete();
```

This outputs:

```
1. Async: 3
```

Using observables in Angular

Angular heavily integrates RxJS to handle asynchronous operations. It is used in:

- Event handling
- HTTP requests
- Form handling
- State management
- Component communication

Let us see examples of a few of the usages mentioned above.

Event handling using observables

In *Chapter 9, Data Binding and Events Handling,* we explored how to bind to DOM events in Angular. Template event binding is the simplest form of event handling in Angular. While it does not directly use observables, it is often the starting point for more complex event handling. **EventEmitter** is used to create custom events for child components that parent components can listen to. It extends RxJS Subject, making it an observable. Here is an example of a child component emitting a custom event:

```
1. import { Component, EventEmitter, Output } from '@angular/core';
```

```
2.
3. @Component({
4.   selector: 'app-child',
5.   template: '<button (click)="emitCustomEvent()">Emit</button>'
6. })
7. export class ChildComponent {
8.   @Output() customEvent = new EventEmitter<string>();
9.
10.   emitCustomEvent() {
11.     this.customEvent.emit('Hello from child');
12.   }
13. }
```

The **ParentComponent** can listen to this event as follows:

```
1. @Component({
2.   selector: 'app-parent',
3.   template: `<app-child
   (customEvent)="handleCustomEvent($event)"></app-child>`,
4. })
5. export class ParentComponent {
6.   handleCustomEvent(event: string) {
7.     console.log(event)
8.   }
9. }
```

A click on the button shows the event string **Hello from child** in the browser dev tool console.

For more complex event handling, you can use RxJS **fromEvent()** operator to create an observable from DOM events, as follows:

```
1. import { fromEvent } from 'rxjs';
2. import { debounceTime, map } from 'rxjs/operators';
3.
4. ngOnInit() {
5.   const input = document.getElementById('search-input');
6.   fromEvent(input, 'input').pipe(
```

```
    debounceTime(300),
    map((event: any) => event.target.value)
  ).subscribe(value => {
    this.search(value);
  });
}
```

HTTP request handling using observables

Since observables are good at handling an async stream of data, they are extensively used in handling network calls. The next chapter, *Chapter 13, API Calls,* will delve deep into how to handle HTTP calls in Angular using observables. The built-in **HttpClient** object in Angular returns an observable as a response for HTTP calls. Taking an example from our fruit store, a product service returning the list of products will look as below:

```
import { HttpClient } from '@angular/common/http';

export class ProductService {
  constructor(private http: HttpClient) {}

  getProducts(): Observable<Product[]> {
    return this.http.get<Product[]>(`${this.apiUrl}/products`);
  }
}
```

Then, the component can make a call to the **ProductService** as follows:

```
productService.getProducts().subscribe((data) => {
    this.products = data;
  }, error => console.error('Error:', error)
);
```

We will skip the details for now as the next chapter will cover it.

Reactive forms using observables

Angular reactive forms use observables extensively. You can listen to value changes as follows:

```
import { FormBuilder, FormGroup } from '@angular/forms';
```

```
2.
3.  export class ContactFormComponent {
4.    form: FormGroup;
5.
6.    constructor(private fb: FormBuilder) {
7.      this.form = this.fb.group({
8.        username: [''],
9.        email: ['']
10.     });
11.
12.     this.form.valueChanges.subscribe(value => {
13.       console.log('Form value changed:', value);
14.     });
15.   }
16. }
```

Chapter 11, User Input Using Forms, explored how to use reactive forms.

Router events using observables

Angular router emits events as observables, allowing you to react to navigation changes. *Chapter 5, Routing,* explored how to subscribe to observables in routes as follows:

```
1.  ngOnInit() {
2.     this.activatedRoute.paramMap.subscribe(paramMap => {
3.       this.productName = paramMap.get('name');
4.       if (this.productName) {
5.   this.product = this.productService.getProduct(this.productName);
6.       }
7.     });
8.   }
```

Signals

Signals are a new feature introduced in Angular 16 as part of the framework's reactivity model. They provide a simpler and more efficient

way to handle reactive state management compared to traditional RxJS Observables. Signals are reactive values that notify consumers when they change. They are synchronous, always have a current value, and can be read in templates or other computations. A simple example is as follows:

```
1. import { Component, computed, effect, signal } from '@angular/core';
2.
3. @Component({
4.   selector: 'app-root',
5.   template: `<h1>Click Count: {{ count() }}</h1>
6.   <p>Square of {{ count() }} is {{ square() }}</p>
7.   <button (click)=»increment()»>Increment</button>`,
8. })
9. export class AppComponent {
10.   count = signal(0);
11.   square = computed(() => this.count() * this.count());
12.
13.   constructor() {
14.     effect(() => {
15.       console.log(`The count changed to: ${this.count()}`);
16.     });
17.   }
18.
19.   increment() {
20.     this.count.update(c => c + 1);
21.   }
22. }
```

Notice the following from the above code:

- **count** is a signal initialized to **0**.
- **square** is a computed value that updates every time the **count** changes.
- **effect** is called on every change to **count**. This allows us to react to **count** changes.
- **increment** is a regular component method called on the button click,

and it updates the **count** value using the **update** method. This update triggers changes to the computed **square** field and calls the **effect** method. It also updates the values of **count** and **square** on the template.

You can see the value changes in console messages on button clicks in the following figure:

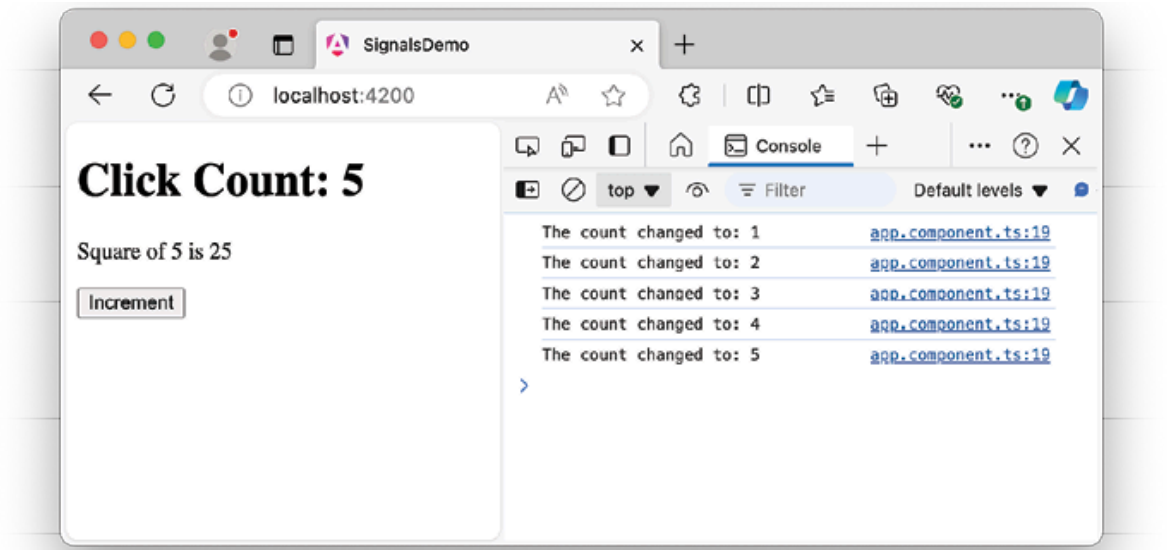


Figure 12.1: *Signals demo*

In the next chapter, we will see how signal can be used in services to update data in the template on API calls.

Conclusion

Observables and signals are crucial concepts in Angular that enable developers to handle asynchronous data streams and events effectively. By leveraging observables and RxJS, developers can create more responsive and efficient applications. Observables allow for the modeling of asynchronous data streams, while signals simplify communication between Angular components and services. Understanding and effectively utilizing observables and signals empowers developers to build highly reactive and scalable Angular applications.

Points to remember

- An observable is a powerful concept in reactive programming that represents a data stream and can emit multiple values over time,

including potentially infinite sequences of values.

- RxJS is a library used extensively in Angular for reactive programming using observables.
- Observers subscribe to observables in order to receive notifications when new values are emitted.
- RxJS provides various operators like of(), from(), interval(), and fromEvent() to create observables.
- Observables can emit multiple values over time, making them suitable for modeling asynchronous data streams such as user inputs and server responses.
- Pipeable operators are a set of functions that allow you to perform operations on the data emitted by an observable, creating a new observable as a result.
- Signals provide a way to broadcast and listen for messages or events, simplifying the communication between components and services.

Questions

1. What are observables, and how do they work?
2. How do observables and observers interact in the context of reactive programming?
3. What is RxJS, and how is it used for reactive programming in Angular?
4. Can you provide an example of creating and subscribing to an observable using RxJS in Angular?
5. What are Subjects, and how are they different from plain observables?
6. What are some ways to create observables in RxJS using operators?
7. How can you merge observables using RxJS operators?
8. Name a few regularly used pipeable operators from RxJS.
9. What are Signals, and how are they useful in Angular?

References

- RxJS operators:
  https://rxjs.dev/guide/operators
- Signals:
  https://v17.angular.io/guide/signals

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 13
API Calls

Introduction

Handling **application programming interface** (**API**) calls in Angular is a crucial aspect of application development. Effectively managing API calls ensures seamless interaction with backend services, allowing for efficient data retrieval and manipulation. Developers can build robust and responsive Angular applications that deliver a seamless user experience by understanding the best practices and techniques for handling API calls. This chapter will dive into the intricacies of managing API calls in Angular, exploring various approaches and methods to effectively integrate and interact with external data sources.

Structure

This chapter covers the following topics:

- Application programming interface
- Handling REST API
- GraphQL calls
- Working with WebSockets

Objectives

By the end of this chapter, you will understand how to make API calls in Angular.

Application programming interface

An API is a way for different software components to communicate with each other. It provides rules or protocols allowing software applications to exchange data, features, and functionality. Essentially, an API acts as a bridge, enabling seamless interaction between different system parts.

Consider the fruit store application we created in previous chapters. We hardcoded the product data in JSON format within the **ProductService** as follows:

```
export class ProductService {
  products: Product[] = [
    new Product('Apple', 6.99, "Canada"),
    new Product('Orange', 5.99, "India"),
    new Product('Banana', 3.99, "Mexico"),
  ];

  getProducts() {
    return this.products;
  }
}
```

In a real-world scenario, an application's data will be dynamic in nature. The system can be represented as a client-server architecture, where the client requests data, and the server is the module that contains the data or resources. A simple diagrammatic representation of storing and serving data in an API server is as follows:

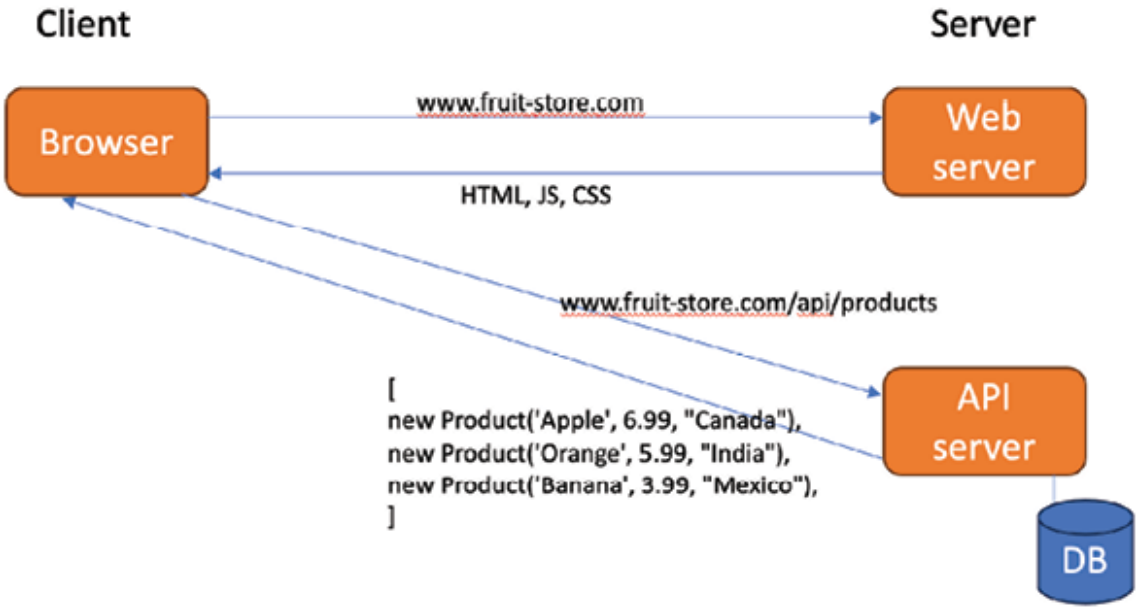


Figure 13.1: *Client-server architecture*

The client (browser, in this case) calls the webserver to load bare-bone HTML. It then loads JS and CSS to make the page interactive and stylish. It makes a call to an API server for data. Once the API request succeeds, the page can show the latest available data to the end user.

Handling REST API

Representational State Transfer (**REST**) is an architectural style for building and interacting with web services. It follows a client-server model and uses standard HTTP methods like GET, POST, PUT, and DELETE to perform operations on resources. They also rely on a stateless communication model, meaning each request from the client to the server must contain all the necessary information for the server to fulfill the request without relying on any stored context on the server-side. This architectural style promotes scalability, performance, and simplicity in web service design. Let us dive into how to make REST API calls in our Angular app.

First, create an API server using Express.js. This is a simple way to create an API server. For a more scalable and cost-effective solution, you may use a serverless application like *AWS Lambda* or *Azure Functions* to host your API endpoints. We will skip that for now.

To create a local API server, run the following on the terminal:

cd src

mkdir api

cd api

npm init

npm install express

touch server.js

Paste the following code in **server.js**:

```
const express = require('express');
const app = express();

const port = process.env.PORT || 3000;
app.listen(port, () => {
    console.log(`Server listening on port ${port}`);
});

const data = [
    { name: 'Apple', price: 1.99, origin: 'USA' },
    { name: 'Banana', price: 0.99, origin: 'Mexico' },
    { name: 'Cherry', price: 2.99, origin: 'Chile' }
];

app.get('/products', (req, res) => {
    res.json(data);
});

app.get('/products/:name', (req, res) => {
  const name = req.params.name;
  const product = data.find((item) => item.name === name);
  if (!product) {
    res.status(404).json({ error: `${name} not found` });
  } else {
    res.json(product);
  }
```

```
27. });
```

Notice that the API server returns a list of products defined in *line 9*. In a real-world case, this data will be fetched from a database or other data sources. Sometimes, an API may even call another API for data. Two endpoints are defined in the above code: One to return the list of products and another to return product details given the product name. For brevity, we have kept the code to the bare minimum.

Now run the following command on the terminal to run the API server:

node server.js

If you access the localhost API server on the browser at **http://localhost:3000/products**, you will see the list of products returned as follows:

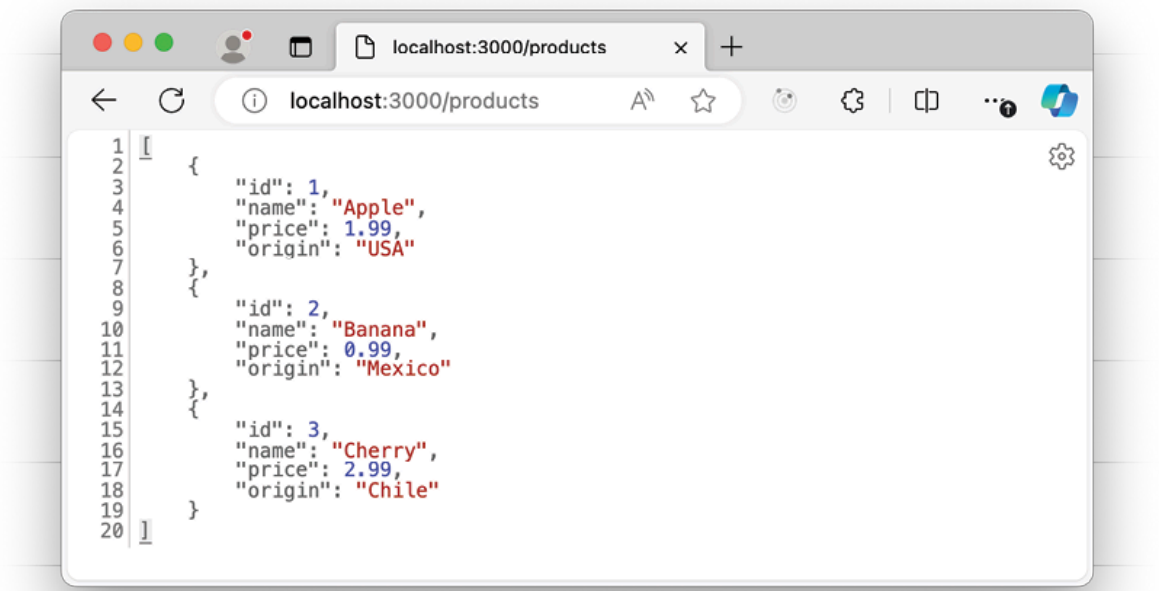


Figure 13.2: *API call for products route*

Angular allows the creation of proxy servers to use an app route to point to any URL. In this case, we will create a proxy for **/api** route to point to our API endpoints at **http://localhost:3000**. To do this, create a file **proxy.conf.json** at the **root** and paste the following:

```
1. {
2.   "/api": {
3.     "target": "http://localhost:3000",
4.     "secure": false,
5.     "changeOrigin": true,
```

```
    "pathRewrite": {
      "^/api": ""
    }
  }
}
```

Now, you can access the same products API endpoint using the path **http://localhost:4200/api/products**.

To create the proxy using this config, rerun **ng serve** with the following key:

ng serve --proxy-config proxy.conf.json

Open the **fruit-store** app created in the previous chapter. Import **HttpClientModule** in **app.module.ts** file as follows:

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [
    ...
    HttpClientModule
  ],
  ...
})
export class AppModule { }
```

Open the **product.service.ts** file and fetch the products using **HttpClient** as follows:

```
import { Injectable } from '@angular/core';
import { Product } from '../models/Product'
import { Observable } from 'rxjs';
import { HttpClient } from '@angular/common/http';

@Injectable()
export class ProductService {
  apiUrl = '/api';
  constructor(private http: HttpClient) { }
```

```
10.
11.    getProducts(): Observable<Product[]> {
12.      return this.http.get<Product[]>(`${this.apiUrl}/products`);
13.    }
14.
15.    getProduct(name: string): Observable<Product> {
16.      return this.http.get<Product>(`${this.apiUrl}/products/${name}`);
17.    }
18.
19.    addProduct(product: Product) {
20.   return this.http.post<Product>(`${this.apiUrl}/products`, product);
21.    }
22.  }
```

Notice that now the **getProducts()** method returns an **Observable** of **Products[]** instead of **Products[]**. This is because the API calls are async in nature, and the return value of **http.get()** call is an **Observable**. Hence, we need to modify the caller code in components to subscribe to the **Observable** and set the products array when a new value arrives. Following is the code change in the **product.component.ts** file:

Figure 13.3: *Changes in product.component.ts file*

Similarly, the **product-details.component.ts** file is modified as follows:

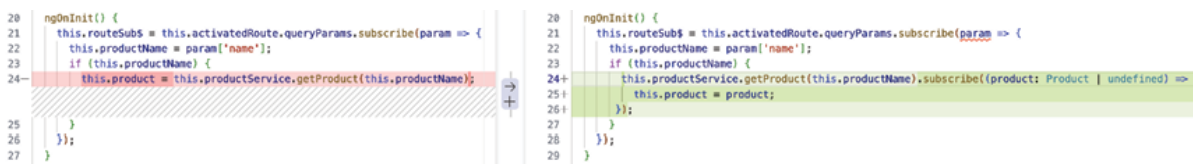


Figure 13.4: *Changes in product-details.component.ts file*

Now, if you run the app and go to the **Products** tab, you will notice the page shows the data retrieved from the API:

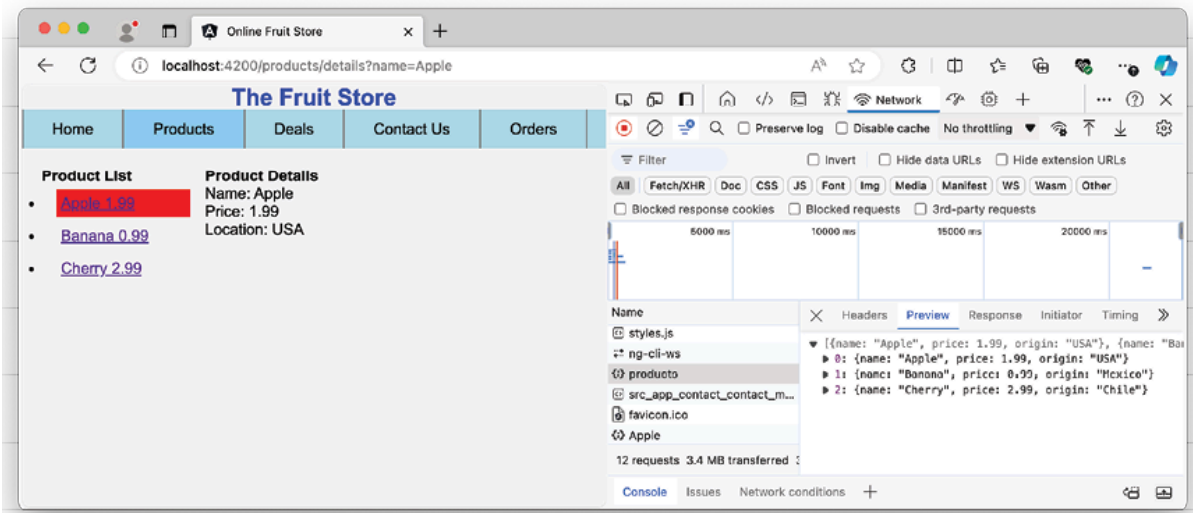


Figure 13.5: *Making API calls in the app*

On clicking on any product, it will show the details under the **Product Details** section. In the **Network** tab of the developer tool, you can see the API calls made to:

- **http://localhost:4200/api/products**
- **http://localhost:4200/api/products/Apple**

We saw how to make RESTful calls using the HTTP GET verb. Similarly, we can add API endpoints for the following:

- **POST**: To create a resource.
- **PUT**: To update a resource.
- **DELETE**: To delete a resource.

These four **Create, Read, Update, and Delete** operations are collectively called **CRUD** operations.

You can add the following methods to the **src/api/server.js** file for CRUD functionality:

```
app.post('/products', (req, res) => {
  const newItem = req.body;
  data.push(newItem);
  res.status(201).json(newItem);
});

app.put('/products/:name', (req, res) => {
  const name = req.params.name;
  const index = data.findIndex((item) => item.name === name);
```

```
  if (index === -1) {
    res.status(404).json({ error: `${name} not found` });
  } else {
    data[index] = req.body;
    res.json(data[index]);
  }
});

app.delete('/products/:name', (req, res) => {
  const name = req.params.name;
  const index = data.findIndex((item) => item.name === name);
  if (index === -1) {
    res.status(404).json({ error: `${name} not found` });
  } else {
    data.splice(index, 1);
    res.json({ message: `${name} deleted` });
  }
});
```

These calls can be called from the **product.service.ts** file as follows:

```
addProduct(product: Product): Observable<Product> {
  return this.http.post<Product>(`${this.apiUrl}/products`, product);
}

updateProduct(product: Product): Observable<Product> {
 return this.http.put<Product>
  (`${this.apiUrl}/products/${product.name}`, product);
}

deleteProduct(name: string): Observable<Object> {
  return this.http.delete(`${this.apiUrl}/products/${name}`);
}
```

GraphQL calls

GraphQL is a query language for APIs and a runtime for executing those queries by using a type system that you define for your data. It provides a more efficient, powerful, and flexible alternative to traditional REST APIs. With GraphQL, clients can request only the data they need, and the server responds with the exact data in a single request, which can help reduce over-fetching and under-fetching of data. It allows clients to specify the structure of the response, making it easier to evolve APIs over time without impacting existing clients. GraphQL calls run using HTTP POST. The body of the request defines the query. A sample query would look as shown:

```
query {
  product {
    name
    price
    origin
  }
}
```

The major advantage of using GraphQL instead of REST is that the caller can decide which fields it needs. For instance, in the above query, if the page only needs to show the names of the products, it need not fetch price and origin and can send a query as follows:

```
query {
  product {
    name
  }
}
```

This provides a lot of flexibility in the contract between the client and the server. The network payload can be reduced significantly, which may be a huge gain in a low-network area. A detailed description of GraphQL is beyond the scope of this book, but we encourage you to delve more into it through the reference links provided at the end of this chapter.

There are many third-party libraries available to make GraphQL calls in Angular apps. First, we need to create a GraphQL server that can understand and resolve a GraphQL request. A popular library is **express-graphql**. Once the server is set up, we need to make GraphQL requests

from the Angular app. **Apollo** is a popular client-side library for making GraphQL calls from Angular components. We have provided a link in the *References* section for code samples if you want to try out GraphQL with your Angular app.

Working with WebSockets

A WebSocket is a communication protocol that provides full-duplex communication channels over a single TCP connection. It allows for real-time data transfer between a client and a server, enabling both parties to send data asynchronously. This makes WebSockets suitable for applications requiring continuous data exchange, such as chat applications, real-time gaming, and financial trading platforms. WebSocket is different from HTTP because it is bidirectional and stateful.

- **Bidirectional**: WebSocket allows data to be sent from the client to the server and vice versa.
- **Stateful**: The connection between client and server will remain alive until either one terminates it.

An Angular app can communicate with a WebSocket server using either the **socket.io-client** package or **RxJS**. Code samples are provided in the references provided at the end of the chapter.

Conclusion

This chapter provided a comprehensive understanding of handling API calls in Angular. By exploring the concepts of REST API, GraphQL calls, and working with WebSockets, developers can now effectively integrate and interact with external data sources in their Angular applications. In the next chapter, we will look at authentication techniques used in API calls to identify and authorize a user.

Points to remember

- API calls are crucial in Angular application development for interacting with backend services.
- REST is an architectural style for building web services, using standard HTTP methods like GET, POST, PUT, and DELETE.
- GraphQL provides a more efficient and flexible alternative to REST

APIs, allowing clients to request only the data they need.

- WebSockets enable bi-directional, real-time communication between a client and a server over a single TCP connection.

Questions

1. What are some best practices for handling API calls in Angular?
2. Can you explain the concept of REST API and its architectural style?
3. What is the purpose of creating a proxy server in Angular for API endpoints?

References

- GraphQL:
  https://graphql.org/
- Making GraphQL calls in Angular:
  https://signoz.io/blog/angular-graphql/
- socket.io-client:
  https://socket.io/docs/v4/client-initialization/
- WebSocket using RxJS:
  https://rxjs.dev/api/webSocket/webSocket

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 14
Authentication and Authorization

Introduction

In the modern web landscape, security is paramount. As Angular developers, we are responsible for creating applications that not only provide seamless user experiences but also safeguard sensitive information and restrict access to authorized users only. Authentication ensures that users are who they claim to be, while authorization determines the actions users are allowed to perform within the application. These processes are essential for maintaining the security and integrity of the application, protecting sensitive data, and providing a personalized experience for each user. In this chapter, we will explore the fundamental concepts of authentication and authorization and then move on to practical implementations.

By the end of this chapter, you will have a comprehensive understanding of how to secure your Angular applications, protect user data, and implement role-based access control.

Structure

This chapter covers the following topics:

- Understanding authentication and authorization
- Sign up and sign in API endpoints
- Setting up an authentication service
- Implementing user registration
- Adding a sign in page
- Securing routes with Auth Guard
- Managing user sessions
- Role-based authorization
- JSON Web Tokens in Angular
- Handling authentication errors

Objectives

By the end of this chapter, you will be equipped with the knowledge and skills to implement a comprehensive authentication and authorization system in your Angular applications. You will understand the underlying concepts, be able to apply best practices and have the tools to maintain robust application security.

Understanding authentication and authorization

Authentication and authorization are two fundamental concepts in application security that, while related, serve distinct purposes. Authentication is the process of verifying a user's identity, typically through credentials such as an email and password. It answers the question, *Who are you?* Authorization, conversely, determines what an authenticated user is allowed to do within the application. It addresses the question, *What are you allowed to access?*

Implementing these concepts in Angular applications involves several components working together. The frontend Angular application interacts with a backend server, which manages user data and performs authentication and authorization checks. Angular's role is to provide a secure interface for users to

authenticate, maintain the authenticated state throughout the user's session, and enforce authorization rules on the client-side.

It is crucial to understand that while client-side authentication and authorization provide a layer of security and improve user experience, they should always be complemented by robust server-side checks. This is because client-side code can be manipulated and, therefore, cannot be solely relied upon for security.

Sign up and sign in API endpoints

We will create a simple Node.js API endpoint to expose the **/api/signup** and **/api/signin** routes. Open **src/api/server.js** file created in the previous chapter and add the following methods:

```
1. const expressApp = require('express');
2. const mongooseDB = require('mongoose');
3. const bcryptjs = require('bcryptjs');
4. const jsonwebtoken = require('jsonwebtoken');
5. ...
6.
7. // Connect to MongoDB
8. mongooseDB.connect('mongodb://localhost/auth_demo', { useNewUrlParser: true, useUnifiedTopology: tr
9.
10. // User model
11. const User = mongooseDB.model('User', new mongooseDB.Schema({
12.   email: { type: String, unique: true, required: true },
13.   password: { type: String, required: true }
14. }));
15.
16. app.post('/api/signup', async (req, res) => {
17.   try {
18.     const { email, password } = req.body;
19.
20.     let newUser = await User.findOne({ email });
21.     if (newUser) {
22.   return res.status(400).json({ message: `User already exists with email ${email}. Please sign in with
    password.` });
23.     }
24.
25.     newUser = new User({ email, password });
26.
27.     // Hash password
28.     const salt = await bcryptjs.genSalt();
29.     newUser.password = await bcryptjs.hash(password, salt);
30.     await newUser.save();
31.
32.   res.status(201).json({ message: `User signuped successfully with email ${email}` });
33.   } catch (error) {
34.     console.error(error);
35.     res.status(500).json({ message: 'Got server error' });
36.   }
37. });
38.
39. app.post('/api/signin', async (req, res) => {
```

```
40.   try {
41.     const { email, password } = req.body;
42.
43.     // Check if user exists
44.     const user = await User.findOne({ email });
45.     if (!user) {
46.   // It's a good practice to not specify if the user exists or not. Just say "Email and password combination
      not found."
47.   return res.status(400).json({ message: 'Email and password combination not found.' });
48.     }
49.
50.     // Validate password
51.     const isMatch = await bcryptjs.compare(password, user.password);
52.     if (!isMatch) {
53.   return res.status(400).json({ message: 'Email and password combination not found.' });
54.     }
55.     res.json(user);
56.
57.   } catch (error) {
58.     console.error(error);
59.     res.status(500).json({ message: 'Got server error' });
60.   }
61. });
```

This is an Express server. You can use any other server as you need. *Line 10* connects to a MongoDB instance. You can choose any other DB based on your project needs. *Line 13* defines a simple user model. In a real-world case, this user model may have more fields than just email and password. A good practice is to keep these account-related fields in a separate model and other user-related fields, like first name, last name, address, etc., in a separate model. Another good practice is to never store passwords in plain text in the DB. Hence, *line 33* creates an encrypted string for the password using the **bcrypt** package.

The **signin** method first makes a DB query to fetch the user with the given email and then matches the password with the one from the request param. If the password matches, it returns the user object in the response.

Install the required packages and run this server using the following command:

```
node src/api/server.js
```

Setting up an authentication service

Let us use our fruit store application from the previous chapter to demonstrate how to authenticate a user. We will create an **AuthenticationService** that will handle user sign up, sign in and sign out and maintain the current authentication state:

```
1.  import { Injectable } from '@angular/core';
2.  import { HttpClient } from '@angular/common/http';
3.  import { BehaviorSubject, Observable } from 'rxjs';
4.  import { map } from 'rxjs/operators';
5.
6.  @Injectable({
7.    providedIn: 'root'
8.  })
9.  export class AuthenticationService {
10.    private currentUserSubject: BehaviorSubject<any>;
```

```
  public currentUser: Observable<any>;

  constructor(private http: HttpClient) {
  this.currentUserSubject = new BehaviorSubject<any>(JSON.parse(localStorage.getItem('currentUser')
    || '{}'));
    this.currentUser = this.currentUserSubject.asObservable();
  }

  public get currentUserValue(): any {
    return this.currentUserSubject.value;
  }

  signup(user: { email: string, password: string }) {
    return this.http.post<any>('/api/signup', user)
      .pipe(map(response => {
        return response;
      }));
  }

  signin(email: string, password: string) {
    return this.http.post<any>('/api/signin', { email, password })
      .pipe(map(user => {
        localStorage.setItem('currentUser', JSON.stringify(user));
        this.currentUserSubject.next(user);
        return user;
      }));
  }

  signout() {
    localStorage.removeItem('currentUser');
    this.currentUserSubject.next(null);
  }
}
```

This service uses RxJS **BehaviorSubject** to maintain the current user's state. The **signin** method sends a POST request to the server and, upon successful authentication, stores the user details in **localStorage** and updates the **currentUserSubject**. The **signout** method clears the stored user data.

Implementing user registration

User registration is often the first step in the authentication process. Let us create a registration component that utilizes Angular's reactive forms for data collection and validation:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { AuthenticationService } from 'src/app/shared/services/auth.service';

@Component({
  selector: 'app-signup',
  template: `
    <div class="signup-container">
```

```
  <form [formGroup]="signupForm" (ngSubmit)="onSubmit()" class="signup-form">
        <label for="email" class="form-label">Email</label>
  <input id="email" formControlName="email" placeholder="Email" class="form-input">

        <label for="password" class="form-label">Password</label>
  <input id="password" formControlName="password" type="password" placeholder="Password"
 class="form-input">

  <button type="submit" [disabled]="signupForm.invalid" class="submit-btn">Sign up</button>
    </form>
   </div>
  `,
  styleUrls: ['./signup.component.css'],
  imports: [
   ReactiveFormsModule
  ],
  standalone: true
})
export class SignUpPageComponent implements OnInit {
  signupForm!: FormGroup;

  constructor(
    private formBuilder: FormBuilder,
    private authService: AuthenticationService
  ) { }

  ngOnInit() {
    this.signupForm = this.formBuilder.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  onSubmit() {
    if (this.signupForm.valid) {
      this.authService.signup(this.signupForm.value).subscribe(
        response => {
          console.log('Registration successful', response);
          // Navigate to signin page or automatically sign in the user
        },
        error => {
          console.error('Registration failed', error);
        }
      );
    }
  }
}
```

We are using **FormBuilder** to create a registration form with field validations. We will skip the CSS styling

for brevity. The **onSubmit()** method calls the **signup()** method in **AuthenticationService**. On successful registration, you can redirect to the sign in page or call the **signin** method in **AuthenticationService** with the same credentials, depending upon the registration flow you want to design. To see this page in action, add the **signup** route as follows:

```
const routes: Routes = [
  ...
  { path: 'signup', component: SignUpPageComponent },
];
```

Run the app and access **http://localhost:4200/signup** route to see the registration page as shown:

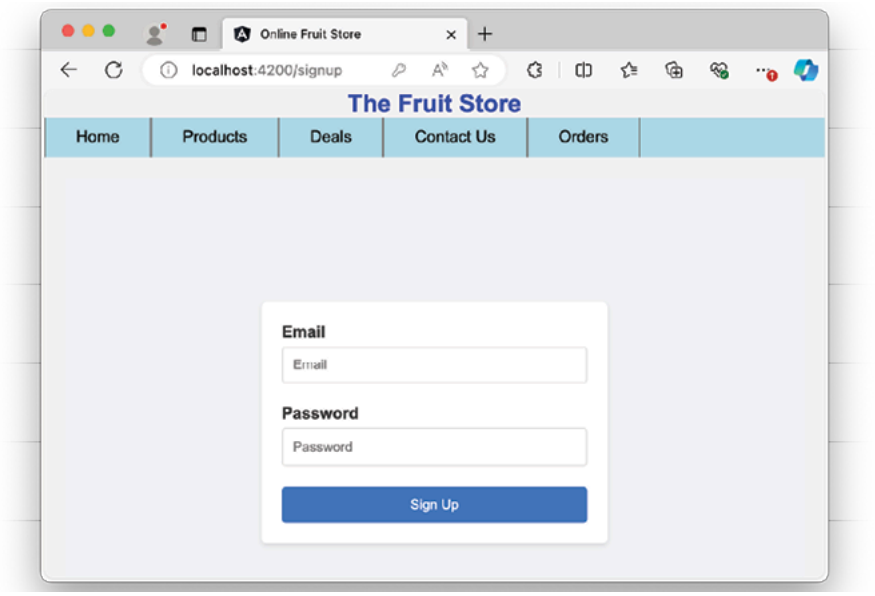


Figure 14.1: *Registration page*

Adding the field values and clicking the **Sign Up** button should make the API call to add a record to the DB.

Adding a sign in page

Similar to the registration page, you can create a sign in page component as below:

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { AuthenticationService } from 'src/app/shared/services/auth.service';
import { FormsModule } from '@angular/forms'; // Import FormsModule

@Component({
  selector: 'app-signin',
  template: `
    <div class="signin-container">
      <form (ngSubmit)="onSubmit()" class="signin-form">
        <label for="email" class="form-label">Email</label>
<input id="email" [(ngModel)]="email" name="email" placeholder="Email" class="form-input">

        <label for="password" class="form-label">Password</label>
        <input id="password" [(ngModel)]="password" name="password" type="password" placeholder="P

        <button type="submit" class="submit-btn">Sign In</button>
      </form>
```

```
19.     </div>
20.   `,
21.   styleUrls: ['./signin.component.css'],
22.   imports: [
23.     FormsModule
24.   ],
25.   standalone: true
26. })
27. export class SignInPageComponent {
28.   email!: string;
29.   password!: string;
30.
31.   constructor(
32.     private authService: AuthenticationService,
33.     private router: Router
34.   ) { }
35.
36.   onSubmit() {
37.     this.authService.signin(this.email, this.password)
38.       .subscribe({
39.         complete: () => {
40.           this.router.navigate(['/orders']);
41.         },
42.         error: error => {
43.           console.error('Sign in failed', error);
44.         }
45.       });
46.   }
47. }
```

This page renders as follows:

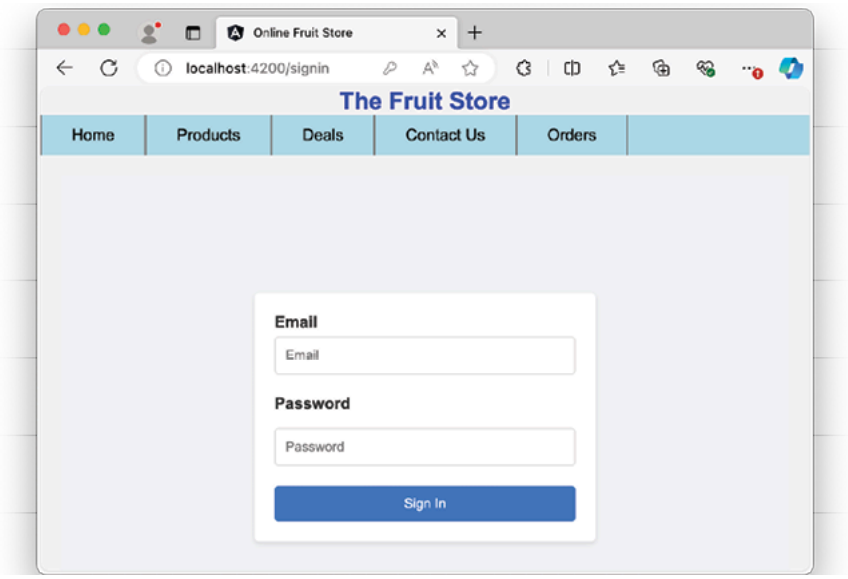


Figure 14.2: *Sign in page*

When clicking the **Sign In** button, it calls the **signin()** method in the **AuthenticationService**. In *line 40*, it

redirects to the **Orders** page on successful sign in. You can refine this logic by taking the redirect URL as a parameter for the page. To complete this flow, let us create a dummy orders page as follows:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-orders',
  template: `<p>This is your personalized orders page</p>`,
  styleUrls: ['./orders.component.css']
})
export class OrdersComponent {
}
```

This page can be refined to make an API call to fetch the user orders. We will skip those details as that is out of the scope of this chapter.

Securing routes with Auth Guard

In *Chapter 5, Routing,* we created an **AuthGuard** class that implemented **CanActivate** interface, as follows:

```
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {}

  canActivate(
    next: ActivatedRouteSnapshot,
  state: RouterStateSnapshot): Observable<boolean | UrlTree> | Promise<boolean | UrlTree> | boolean | UrlTree {
      const currentUser = localStorage.getItem('currentUser');
      if (currentUser) {
        // authorized, return true
          return true;
      }

      // not signed in, redirect to sign in page
      this.router.navigate(['/signin']);
      return false;
  }
}
```

The **canActivate()** method checks whether the **currentUser** object exists in the local storage. If not, it redirects to the sign in page.

This **AuthGuard** can be used in the orders route as follows to ensure the user is authenticated before showing the user orders:

```
const routes: Routes = [
  ...
  { path: 'signin', component: SignInPageComponent },
  { path: 'orders', component: OrdersComponent, canActivate: [AuthGuard] },
];
```

Managing user sessions

Managing user sessions involves keeping track of the authenticated user's state and handling session expiration. We can expand our **AuthenticationService** to include these features:

```
  signin(email: string, password: string) {
    return this.http.post<any>('/api/signin', { email, password })
```

```
3.  .pipe(map(user => {
4.  user.expiresAt = new Date().getTime() + (user.expiresIn * 1000);
5.  localStorage.setItem('currentUser', JSON.stringify(user));
6.  this.currentUserSubject.next(user);
7.  this.autoLogout(user.expiresIn * 1000);
8.  return user;
9.  }));
10.  }
11.
12.  signout() {
13.  localStorage.removeItem('currentUser');
14.  this.currentUserSubject.next(null);
15.  if (this.tokenExpirationTimer) {
16.  clearTimeout(this.tokenExpirationTimer);
17.  }
18.  this.tokenExpirationTimer = null;
19.  }
20.
21.  autoLogout(expirationDuration: number) {
22.  this.tokenExpirationTimer = setTimeout(() => {
23.  this.signout();
24.  }, expirationDuration);
25.  }
26.
27.  refreshToken() {
28.  return this.http.post<any>('/api/refresh-token', {})
29.  .pipe(map(user => {
30.  user.expiresAt = new Date().getTime() + (user.expiresIn * 1000);
31.  localStorage.setItem('currentUser', JSON.stringify(user));
32.  this.currentUserSubject.next(user);
33.  this.autoLogout(user.expiresIn * 1000);
34.  return user;
35.  }));
36.  }
```

This expanded service includes methods for automatic sign out when the token expires and refreshing the token to maintain the session.

Role-based authorization

Authentication is the process of identifying a user as who they claim to be. Once the user has been authenticated, the next step is to authorize the user to access resources based on their roles or identity. Role-based authorization allows you to restrict access to certain parts of your application based on user roles. We can implement this using a combination of route guards and structural directives. First, let us create a directive for role-based access control:

ng g directive /auth/direcives/directive/has-role

Now add the following code into the directive:

```
1.  import { Directive, Input, TemplateRef, ViewContainerRef } from '@angular/core';
2.  import { AuthenticationService } from 'src/app/shared/services/auth.service';
3.
4.  @Directive({
5.  selector: '[appHasRole]'
```

```
})
export class HasRoleDirective {
  @Input() set appHasRole(role: string) {
    const currentUser = this.authService.currentUserValue;
    if (currentUser && currentUser.role === role) {
      this.viewContainer.createEmbeddedView(this.templateRef);
    } else {
      this.viewContainer.clear();
    }
  }

  constructor(
    private templateRef: TemplateRef<any>,
    private viewContainer: ViewContainerRef,
    private authService: AuthenticationService
  ) { }
}
```

You can then use this directive in your templates as follows:

```
<div *appHasRole="'Admin'">
  This content is only visible to admins
</div>
```

JSON Web Tokens in Angular

JSON Web Tokens (JWTs) are a secure way of transmitting information between parties as a JSON object. In the context of authentication, they are often used to securely transmit the user's identity after a successful sign in. To use JWT, install the package using the following command:

```
npm install jwt-decode
```

Let us modify our **AuthenticationService** to work with JWTs. Import the **package jwt-decode** and modify the **signin** and **signout** methods as follows:

```
import { jwtDecode } from 'jwt-decode';
...

  signin(email: string, password: string) {
    return this.http.post<any>('/api/signin', { email, password })
      .pipe(map(response => {
        if (response && response.token) {
          localStorage.setItem('token', response.token);
          const user = this.getUserFromToken();
          this.currentUserSubject.next(user);
        }
        return response;
      }));
  }

  signout() {
    localStorage.removeItem('token');
    this.currentUserSubject.next(null);
  }
```

```
20.
21.   private getUserFromToken(): any {
22.     const token = localStorage.getItem('token');
23.     if (token) {
24.       return jwtDecode(token);
25.     }
26.     return null;
27.   }
```

Notice that the **signin** method now expects a JWT token from the server API call instead of a user object. It also stores the token in local storage instead of the user object. The method **getUserFromToken()** helps retrieve the user data from the token payload.

To make this work, we need to modify the **server** method to return a JWT token, as follows:

```
1.  app.post('/api/signin', async (req, res) => {
2.    try {
3.      const { email, password } = req.body;
4.
5.      // Check if user exists
6.      const user = await User.findOne({ email });
7.      if (!user) {
8.        return res.status(400).json({ message: 'Invalid credentials' });
9.      }
10.
11.     // Validate password
12.     const isMatch = await bcrypt.compare(password, user.password);
13.     if (!isMatch) {
14.       return res.status(400).json({ message: 'Invalid credentials' });
15.     }
16.
17.     // Create and return JWT token
18.     const payload = {
19.       user: {
20.         id: user.id,
21.         email: user.email
22.       }
23.     };
24.
25.     jsonwebtoken.sign(
26.       payload,
27.       'your_jwt_secret', // Replace with your actual secret key
28.       { expiresIn: '1h' },
29.       (err, token) => {
30.         if (err) throw err;
31.         res.json({ token });
32.       }
33.     );
34.   } catch (error) {
35.     console.error(error);
36.     res.status(500).json({ message: 'Got server error' });
```

```
  }
});
```

Handling authentication errors

Proper error handling is crucial for providing a good user experience. Let us create an HTTP interceptor **src/app/auth/interceptors/error.interceptor.ts** to handle authentication errors:

```
import { Injectable } from '@angular/core';
import { HttpRequest, HttpHandler, HttpEvent, HttpInterceptor } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';
import { AuthenticationService } from 'src/app/shared/services/auth.service';

@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
  constructor(private authService: AuthenticationService) {}

  intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    return next.handle(request).pipe(catchError(err => {
      if (err.status === 401) {
        // sign out on 401 api response
        this.authService.signout();
        location.reload();
      }

      const error = err.error.message || err.statusText;
      return throwError(error);
    }))
  }
}
```

To use the interceptor, we need to provide it in the module file:

```
import { HTTP_INTERCEPTORS } from '@angular/common/http';
import { ErrorInterceptor } from './auth/interceptors/error.interceptor';
...
@NgModule({
  // ...
  providers: [
  { provide: HTTP_INTERCEPTORS, useClass: ErrorInterceptor, multi: true },
  ],
  // ...
})
export class AppModule { }
```

This interceptor will catch 401 (Unauthorized) errors and automatically log the user out, providing a seamless experience when their session expires or becomes invalid.

Conclusion

In this chapter, we explored the essential concepts of authentication and authorization in the context of Angular applications. We learned that authentication verifies the identity of a user, while authorization determines the actions the user is allowed to perform within the application. We also delved into practical

implementations, such as creating API endpoints for user sign up and sign in.

We emphasized the importance of maintaining robust application security by implementing both client-side authentication and authorization in Angular applications and complementing them with server-side checks. By combining these measures, we can provide a secure interface for users while protecting sensitive data and ensuring a seamless user experience. By the end of this chapter, we have gained a comprehensive understanding of how to secure Angular applications, protect user data, and implement role-based access control. In the next chapter, we will look at various techniques to test Angular applications.

Points to remember

- Authentication and authorization are fundamental concepts for securing applications by verifying user identity and controlling user access.
- Implementing authentication involves verifying the identity of a user, typically through credentials such as an email and password.
- Authorization determines what an authenticated user is allowed to do within the application, addressing access control.
- Combining client-side authentication with robust server-side checks is essential for comprehensive security, as client-side code can be manipulated.
- Utilize Auth Guards to secure routes and restrict unauthorized access to specific parts of the application based on user authentication and authorization status.

Questions

1. How can we ensure that user passwords are securely hashed and stored?
2. What role does JWT play in Angular applications for maintaining user sessions and ensuring secure communication between the client and server?
3. What are the potential security vulnerabilities associated with client-side authentication and how can they be mitigated?
4. How can role-based authorization be implemented in Angular applications?
5. What are some best practices for handling authentication errors and providing informative error messages to users?
6. How can we prevent unauthorized access to specific routes or components within an Angular application?

References

- Angular JWT Authentication:
  https://blog.angular-university.io/angular-jwt-authentication/

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 15
Testing

Introduction

Testing is a crucial aspect of any application development process, and Angular provides robust tools and frameworks to facilitate efficient testing. Understanding how to effectively manage testing in Angular is essential for ensuring the application's reliability and stability. This includes comprehending the various testing methodologies, tools, and best practices that can be employed to create high-quality, error-free Angular applications.

Structure

This chapter covers the following topics:

- Types of testing
- Unit testing in Angular
- Testing components
- Testing services
- Testing for async code using spy
- Testing directives and pipes
- End-to-end tests
- Code coverage
- Running tests in CI/CD

Objectives

By the end of this chapter, you will understand how to test an Angular application.

Types of testing

Several types of testing can be employed to ensure the reliability and stability of an Angular application. A comprehensive detail about testing is out of the scope of this book, but the following are some of the common types of testing:

- **Unit testing**: This type of testing focuses on testing individual units or components of the application in isolation. In Angular, unit testing is commonly performed using tools like Jasmine and Karma to test components, services, and pipes. In the next section, we will examine a few unit tests.
- **Integration testing**: Integration testing involves testing the interactions and integrations between different parts of the application. In Angular, integration testing can be carried out to test how components, services, and modules work together.
- **End-to-end (e2e) testing**: e2e testing involves testing the application from start to finish to simulate real user scenarios. Until Angular v12, e2e testing was often done using tools like Protractor to test the application's functionality and behavior in a real environment. Protractor has been deprecated since v12. Currently, there is no default e2e test framework in Angular, but Cypress is a popular choice.
- **Component testing**: This type of testing focuses specifically on testing Angular components to ensure their proper rendering, behavior, and data binding.
- **Service testing**: Service testing involves testing Angular services to verify their functionality, data manipulation, and interactions with other parts of the application.

Apart from the above tests, there are other types of testing, such as functional tests, acceptance tests, sanity tests, load tests, and visual parity tests. We will skip these for brevity. We recommend that you familiarize yourself with these different testing techniques and the tools and

frameworks.

Unit testing in Angular

Unit testing focuses on testing small, isolated pieces of code. These tests do not rely on external resources like databases or network calls. In Angular, unit testing is essential to ensure the correctness of individual components, services, and other building blocks.

Angular uses Jasmine as the default testing framework. Karma is the test runner that executes Jasmine tests in a browser environment. When you add a **component/service/pipe** in an Angular app using the Angular CLI, you will also notice that a ***.spec.ts** file is generated. This file is the unit test file and is kept with the component source code in the same folder. When you create a new Angular app using Angular CLI, observe the default tests created in the **app.component.spec.ts** file as follows:

```
1. import { TestBed } from '@angular/core/testing';
2. import { AppComponent } from './app.component';
3.
4. describe('AppComponent', () => {
5.   beforeEach(async () => {
6.     await TestBed.configureTestingModule({
7.       imports: [AppComponent],
8.     }).compileComponents();
9.   });
10.
11.   it('should create the app', () => {
12.     const fixture = TestBed.createComponent(AppComponent);
13.     const app = fixture.componentInstance;
14.     expect(app).toBeTruthy();
15.   });
16.
17.   it(`should have the 'testing-demo' title`, () => {
18.     const fixture = TestBed.createComponent(AppComponent);
19.     const app = fixture.componentInstance;
```

```
    expect(app.title).toEqual('testing-demo');
  });

  it('should render title', () => {
    const fixture = TestBed.createComponent(AppComponent);
    fixture.detectChanges();
    const compiled = fixture.nativeElement as HTMLElement;
    expect(compiled.querySelector('h1')?.textContent).toContain('Hello, testing-demo');
  });
});
```

The test suit is created using a **describe()** method. The parameter for this method is the name of the class under test. Each test is written using **it()** method. The first argument to the **it()** method is a string describing the expectation from the test. So, the first test reads like 'it should create the app' and so on. The second argument is the test method. As a best practice, the test method is divided into three sections—Arrange, Act, Assert. The **Arrange** section initializes the component under test. The **Act** section runs the test. Finally, the **Assert** section matches the actual values with the expected values.

```
it('should create the app', () => {
  // Arrange
  const fixture = TestBed.createComponent(AppComponent);
  // Act
  const app = fixture.componentInstance;
  // Assert
  expect(app).toBeTruthy();
});
```

The above is a simple test, but a more complex test can have elaborate arrangement and assert code blocks. There are three tests shown in the above code block. The first test checks whether the app instance exists using the **toBeTruthy()** method. The second method checks for the value of the string set in **app.title** field to be **'testing-demo'**. And the third test

gets hold of the native DOM element of the component, uses a **querySelector** to find a particular element (h1) inside the component, and compares the element text to contain **'Hello, testing-demo'**.

You may add a **beforeEach()** method to extract the common setup needed to run before each test executes:

```
describe('AppComponent', () => {
  let fixture: ComponentFixture<AppComponent>;
  let component: AppComponent;

  beforeEach(() => {
    TestBed.configureTestingModule({
      declarations: [AppComponent],
    });
    fixture = TestBed.createComponent(AppComponent);
      component = fixture.componentInstance;
  });

  it('should create the app', () => {
    expect(component).toBeTruthy();
  });

  // Add more tests here...
});
```

You can run the unit tests with a simple command as follows:

ng test

This invokes Karma test runner that shows the results in a browser as shown below:

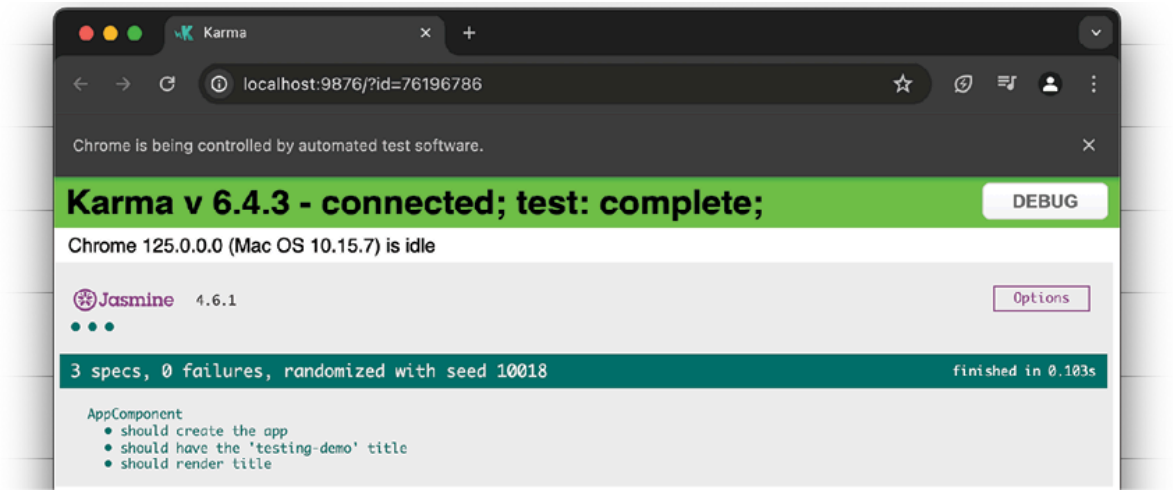


Figure 15.1: *Karma test results*

The **ng test** command watches for changes. The tests will rerun if you keep the test browser window open and save any code changes.

Testing components

When you add a component using the Angular CLI, a **.spec** file is added along with the component **.ts** file. For instance, run the following command:

ng g c product

This will create the **spec** file as follows:

```
1. import { ComponentFixture, TestBed } from '@angular/core/testing';
2.
3. import { ProductComponent } from './product.component';
4.
5. describe('ProductComponent', () => {
6.   let component: ProductComponent;
7.   let fixture: ComponentFixture<ProductComponent>;
8.
9.   beforeEach(async () => {
10.     await TestBed.configureTestingModule({
11.       imports: [ProductComponent]
12.     })
13.     .compileComponents();
14.
```

```
    fixture = TestBed.createComponent(ProductComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
});
```

The default-generated test checks whether the component is created. According to the component logic, you may add further tests to it.

Testing services

Testing a service is similar to testing a component in many ways. If you add a service as follows:

ng g s product

The CLI will generate a spec file **product.service.spec.ts** as follows:

```
import { TestBed } from '@angular/core/testing';

import { ProductService } from './product.service';

describe('ProductService', () => {
  let service: ProductService;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(ProductService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });
```

```
16. });
```

We created a **ProductService** in *Chapter 4, Building Blocks of Angular App*. Let us copy the code here:

```
1. import { Injectable } from '@angular/core';
2. import { Product } from '../models/Product';
3.
4. @Injectable({
5.   providedIn: 'root'
6. })
7. export class ProductService {
8.   products: Product[] = [
9.     new Product('Apple', 6.99),
10.     new Product('Orange', 5.99),
11.     new Product('Banana', 3.99),
12.   ];
13.
14.   getProducts() {
15.     return this.products;
16.   }
17.
18.   constructor() { }
19. }
```

Now, let us add a test for the **getProducts()** method as follows:

```
1.   it('should return the list of products', () => {
2.     expect(service.getProducts()).toEqual([
3.       new Product('Apple', 6.99),
4.       new Product('Orange', 5.99),
5.       new Product('Banana', 3.99),
6.     ]);
7.   });
```

This test will pass when you run **ng test**. Similarly, you can assert other conditions for the data returned by your service.

Testing for async code using spy

In the previous section, the **ProductService** returned an array of data. There may be cases when the data is returned asynchronously. To simulate an asynchronous method, let us add a **setTimeout** method and return a **Promise** as follows:

```
1.  getProducts(): Promise<Product[]> {
2.    return this.fetchProductsAsync();
3.  }
4.
5.  fetchProductsAsync(): Promise<Product[]> {
6.    return new Promise((resolve, reject) => {
7.      setTimeout(() => {
8.        resolve(this.products);
9.      }, 2000);
10.     });
11.   }
```

This code can be tested using a **spyOn()** method that comes built with Jasmine:

```
1.  let expectedProducts: Product[] = [
2.    new Product('Apple', 6.99),
3.    new Product('Orange', 5.99),
4.    new Product('Banana', 3.99),
5.  ];
6.
7.  it('should return the list of products', () => {
8.    let spy = spyOn(service, "fetchProductsAsync").and.returnValue(
9.      Promise.resolve(expectedProducts)
10.   );
11.   service.getProducts().then(products => {
12.     expect(products).toEqual([]);
13.   });
14. });
```

If you run the above test, it should fail as shown below:

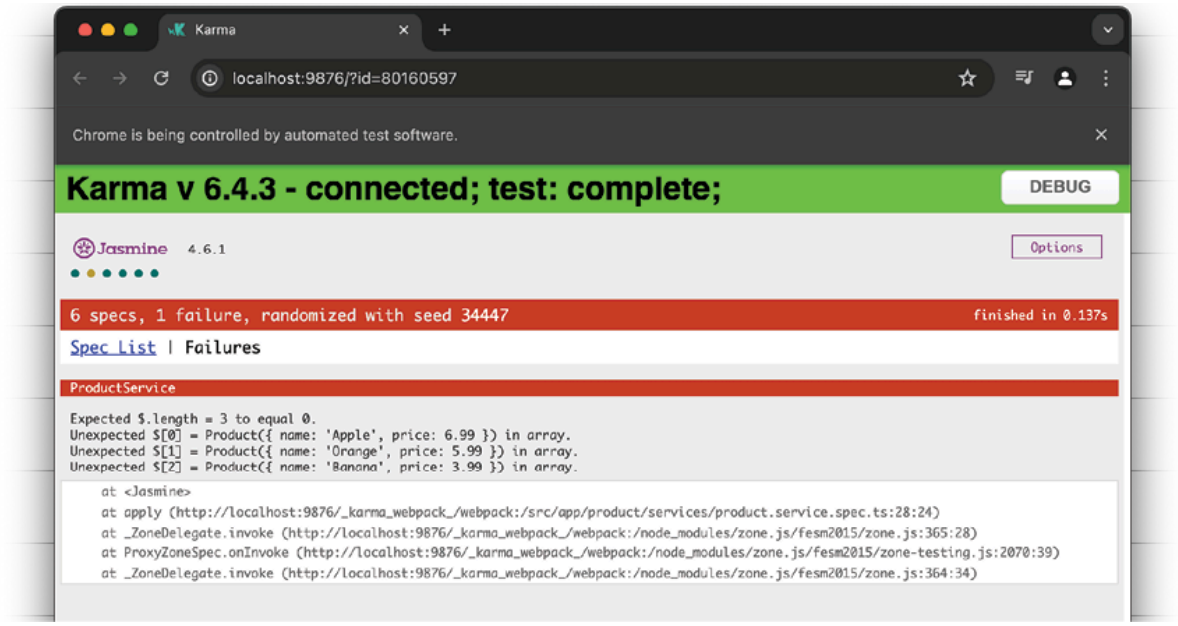


Figure 15.2: *Failed test*

This is because we added the expected result to be an empty array in *line 12*. Now, modify the test to expect the array of product items to pass the test:

```
1. service.getProducts().then(products => {
2.     expect(products).toEqual(expectedProducts);
3.   });
```

The **spyOn()** method is a simple yet powerful tool for spying on any method. In tests where you want to call the original method, you can call the original method as follows:

```
1. spyOn(service, "fetchProductsAsync").and.callThrough();
```

In real cases, most of the time, the data may be fetched through an async network call using an HTTP client. In *Chapter 13, API Calls*, we saw how to make HTTP calls in Angular services. Let us see how to test a service that makes an HTTP call. You have the option to either wrap the HTTP call in a private method or use **spyOn()** to return fake values from the method, as demonstrated above. Otherwise, you can use the **createSpyObj()** method to spy on the **HttpClient**. To see this in action, change the **getProducts()** method in the **ProductService** as follows:

```
1.   getProducts(): Observable<Product[]> {
2.     return this.http.get<Product[]>('/api/products');
```

```
3.   }
4.
5.   constructor(private http: HttpClient) { }
```

Now, we can write a test using **createSpyObj()** as follows:

```
1.  ...
2.  import { HttpClient } from '@angular/common/http';
3.  import { of } from 'rxjs/internal/observable/of';
4.
5.  describe('ProductService', () => {
6.    ...
7.    let httpClientSpy: any;
8.
9.    beforeEach(() => {
10.     TestBed.configureTestingModule({
11.       providers: [
12.         ProductService,
13.   { provide: HttpClient, useValue:
    jasmine.createSpyObj('HttpClient', ['get']) }
14.       ]
15.     });
16.     service = TestBed.inject(ProductService);
17.     httpClientSpy = TestBed.inject<any>(HttpClient);
18.     httpClientSpy.get.and.returnValue(of(expectedProducts));
19.   });
20.
21.   it('should return the list of products using http client', () => {
22.     service.getProducts().subscribe(products => {
23.       expect(products).toEqual(expectedProducts);
24.     });
25.   });
26. });
```

Notice how we injected the **HTTPClient** spy in *line 13* using providers and set the expectation on the spy object to return the desired list of

products on the HTTP get call in *line 18*.

Sometimes, components rely on services or other dependencies. You can mock these dependencies during testing using the abovementioned technique. As a best practice, a unit test should always test just the object under test and mock all other dependencies.

Testing directives and pipes

When you generate code for a directive or a pipe, Angular CLI will add a spec file similar to the one for service, as shown:

```
1.  // my-custom.directive.spec.ts
2.  import { MyCustomDirective } from './my-custom.directive';
3.
4.  describe('MyCustomDirective', () => {
5.    it('should apply custom behavior', () => {
6.      // Test your directive here...
7.    });
8.  });
9.
10.  // my-pipe.pipe.spec.ts
11.  import { MyPipe } from './my-pipe.pipe';
12.
13.  describe('MyPipe', () => {
14.    it('should transform data', () => {
15.      // Test your pipe here...
16.    });
17.  });
```

You may add test code as needed.

End-to-end tests

e2e testing in Angular involves verifying whether the entire application behaves as expected from a user's perspective. Unlike unit testing, which focuses on individual code units, e2e testing ensures that all integrated parts—such as the database, network, and file system—work seamlessly

together. Angular supported Protractor for e2e tests, but that has been deprecated since v12. As of today, if you want to add an e2e test suit, you may use any third-party test framework. For brevity, we will skip any code sample. Following is a few of the popular e2e test frameworks:

- **Cypress**: Cypress has emerged as a popular e2e framework and an alternative to Protractor. Cypress provides a comprehensive solution for e2e testing, allowing developers to simulate real user scenarios and interactions within the application. As against Protractor, which is built on top of Selenium's WebDriverJS, Cypress is a standalone testing framework that runs tests within the browser using a JavaScript-based test runner. It does not use Selenium and has its own architecture, allowing it more control over the browser and providing a faster, more reliable testing experience. It provides a more modern, synchronous, chained command syntax. Cypress also offers built-in support for debugging tests and time-traveling, resulting in faster test execution and more developer-friendly features compared to Protractor.
- **Playwright**: Playwright is another modern e2e testing library that supports multiple browsers (Chromium, Firefox, WebKit). Playwright enables you to perform tasks such as navigating web pages, interacting with elements, taking screenshots, and running end-to-end tests. With Playwright, you can write scripts in various programming languages, including JavaScript, TypeScript, Python, and C#. This flexibility makes it a popular choice for web automation and testing.
- **Jest with Puppeteer**: While not as Angular-specific, Jest combined with Puppeteer can also be used for e2e testing. Jest provides a test runner, assertion library, and mocking capabilities. Puppeteer controls headless Chrome for testing.

We will not discuss the details of e2e tests, but we encourage you to read the *References* section for detailed code samples.

Code coverage

Run the tests with **-code-coverage** switch to generate a coverage report and refer to *Figure 15.3*.

ng test –code-coverage

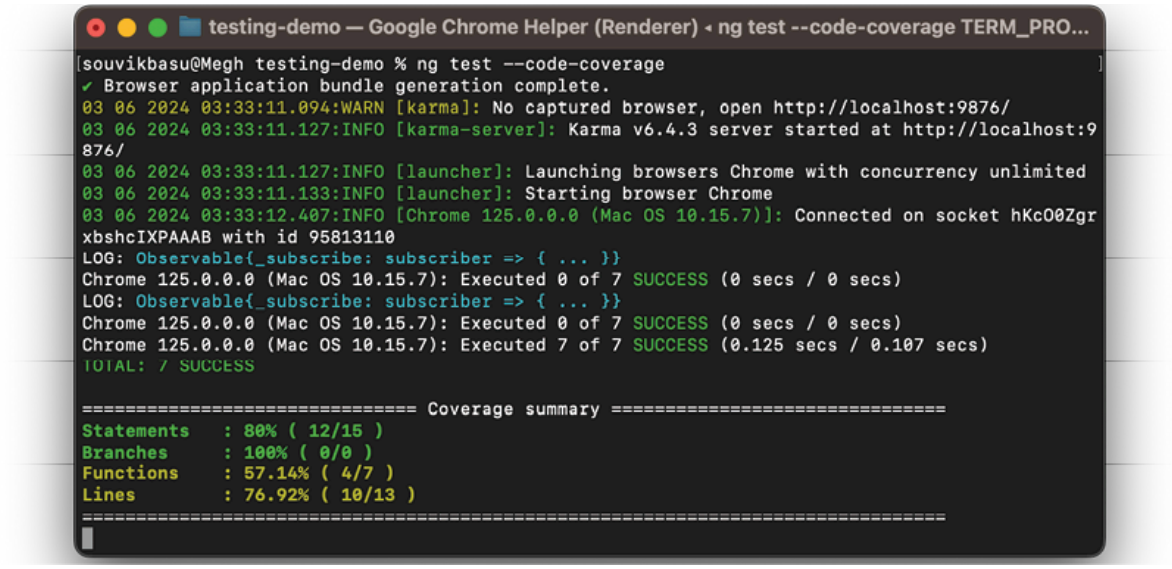


Figure 15.3: *Add code coverage switch to ng test command*

The HTML report is generated inside a **coverage** folder. Opening the **index.html** file inside the **coverage** folder will show the lines of codes that are not covered in the tests. For instance, the **product.service.ts** file report shows as follows:

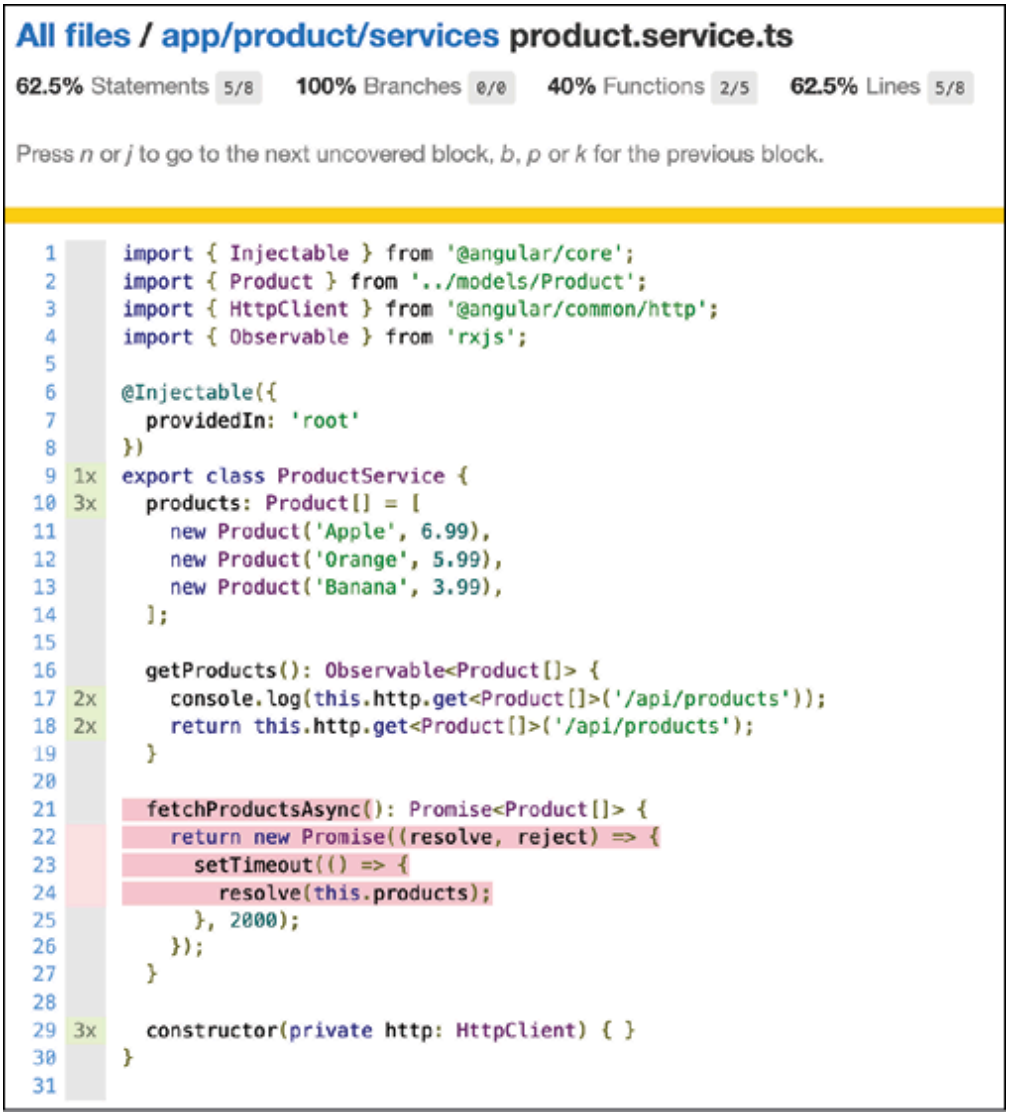


Figure 15.4: *Code coverage report showing code paths not tested*

Although just code coverage is not a good enough indication of code safety, it is one of the basic sanity steps to ensure all code paths are tested well. Aim to maintain at least 90% code coverage as a good practice.

Running tests in CI/CD

By default, **ng test** command watches for changes in your code. If you are running the tests in a CI/CD pipeline, you should run it only once and generate a report. Run the following command in CI/CD pipeline to achieve this:

ng test --no-watch --no-progress --browsers=ChromeHeadless --code-coverage

Conclusion

Mastering the art of testing in Angular is essential for ensuring the reliability and stability of your applications. By understanding the different types of testing, such as unit testing, integration testing, and e2e testing, as well as the tools and frameworks available, you can create high-quality, error-free Angular applications. With proper testing, you can have confidence in the functionality and behavior of your application, leading to a better user experience and overall success.

In the next chapter, we will explore various debugging tools and techniques for your Angular project.

Points to remember

- Angular provides robust tools and frameworks for efficient testing, including Jasmine and Karma.
- Unit testing in Angular focuses on testing small, isolated pieces of code, such as components and services.
- When creating a new Angular app using Angular CLI, default tests are generated alongside the component source code.
- A test suite is created using the describe() method, and individual tests are written using the it() method.
- Best testing practices include following the Arrange, Act, Assert pattern for test methods.

Questions

1. What are the different types of testing that can be employed to ensure the reliability and stability of an Angular application?
2. Which testing framework does Angular use as the default, and what is the role of the test runner in Angular unit testing?
3. Can you explain the structure of a unit test and the best practices for writing unit tests in Angular?
4. Why is end-to-end (e2e) testing important, and what tools are commonly used for e2e testing in Angular?
5. How does Angular facilitate testing for async code using spies, and why is it important in the context of testing Angular applications?
6. What are some best practices for testing components and services in

Angular, and how do they enhance the maintainability of the application?

References

- Official documentation on Testing in Angular:
  https://angular.io/guide/testing
- End to end testing in Angular using Cypress:
  https://testing-angular.com/end-to-end-testing/
- Playwright:
  https://playwright.dev/docs/intro
- Using Jest with Puppeteer:
  https://jestjs.io/docs/puppeteer

CHAPTER 16
Debugging

Introduction

Debugging is a crucial aspect of software development. It involves identifying and resolving errors, issues, and unexpected behaviors in an application. Effective debugging ensures smooth functionality and enhances the overall performance of the application. In this chapter, we will explore various techniques and best practices for debugging in Angular, aiming to equip developers with the necessary skills to manage and troubleshoot Angular applications with confidence.

Structure

This chapter covers the following topics:

- Importance of debugging
- Using breakpoints and logging
- Angular DevTools
- Performance profiling
- Debugging templates and components
- Debug code in IDE using breakpoints
- NgRx DevTools

Objectives

By the end of this chapter, you will understand the various techniques

used to debug an Angular application.

Importance of debugging

Debugging is the process of identifying and resolving issues, errors, or bugs in a computer program. It involves examining the code, tracing the program's execution, and identifying the root cause of unexpected behavior. Techniques such as using breakpoints, stepping through code, and logging are commonly used to debug software and ensure that it runs as intended. An experienced developer would know that more than 80% of the development time goes into debugging an application to ensure correct working. Hence, mastery of debugging tools and techniques can help save development time and find issues in code beforehand efficiently.

Using breakpoints and logging

The crudest (but simplest) form of debugging any JavaScript code is to add the following line:

debugger;

If you open the browser dev tool, the code execution pauses at this line of code. After that, you can use dev tool buttons like step through, step into, etc., to execute each line or block of code and examine the variable values. For instance, in our fruit store app, add a **debugger** statement in the **AppComponent** as follows:

```
1. export class AppComponent {
2.   storeName = 'The Fruit Store';
3.
4.   getStoreName() {
5.     debugger;
6.     return this.storeName;
7.   }
8. }
```

When you run the app on the browser, the code is paused and shows up in the dev tool as follows:

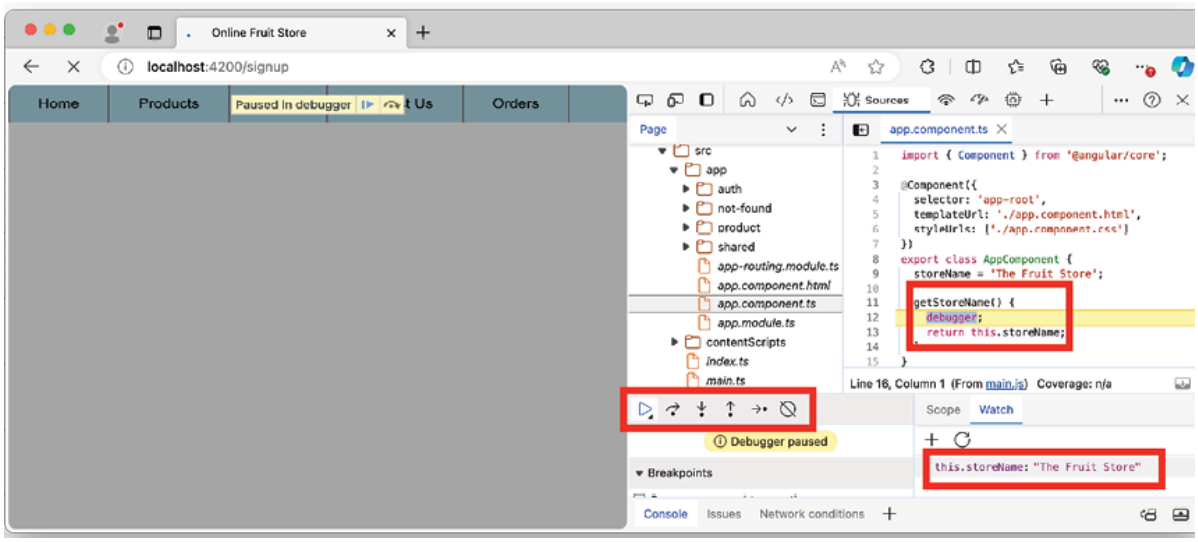


Figure 16.1: *Browser dev tool (on Edge and Chrome)*

There are multiple tabs available in all major browser dev tools, such as:

- **Console**: For logging and executing JavaScript
- **Elements**: For inspecting the DOM for attributes and styles
- **Sources**: For debugging JavaScript and TypeScript
- **Network**: For monitoring HTTP requests

You are encouraged to spend some time understanding each tab in the dev tool.

A common practice is to use **console.log()**, **console.warn()** or **console.error()** methods in the development logs. This can help you to log variable values in the browser dev tool's Console tab. This is useful when you do not want to pause the flow of the application using a **debugger** call and instead log the variable values as the code executes the **console.*** statements. Please make sure not to check in console statements in production code as it might accidentally expose sensitive data on the browser console. The advantage of having **.log()**, **.warn()**, and **.error()** methods is that the dev tool console shows these in different colors, and you can filter the console to show a particular log level. These help to look for the relevant logs quicker if there are too many logs in the console.

Angular DevTools

Angular DevTools is a browser extension that provides powerful debugging capabilities specifically for Angular applications. It offers component tree visualization, change detection profiling, and state

inspection. Search for your browser extension, Angular DevTools, and add it to your browser. Once installed and enabled, you should be able to see an **Angular** tab in the browser dev tool window as shown:

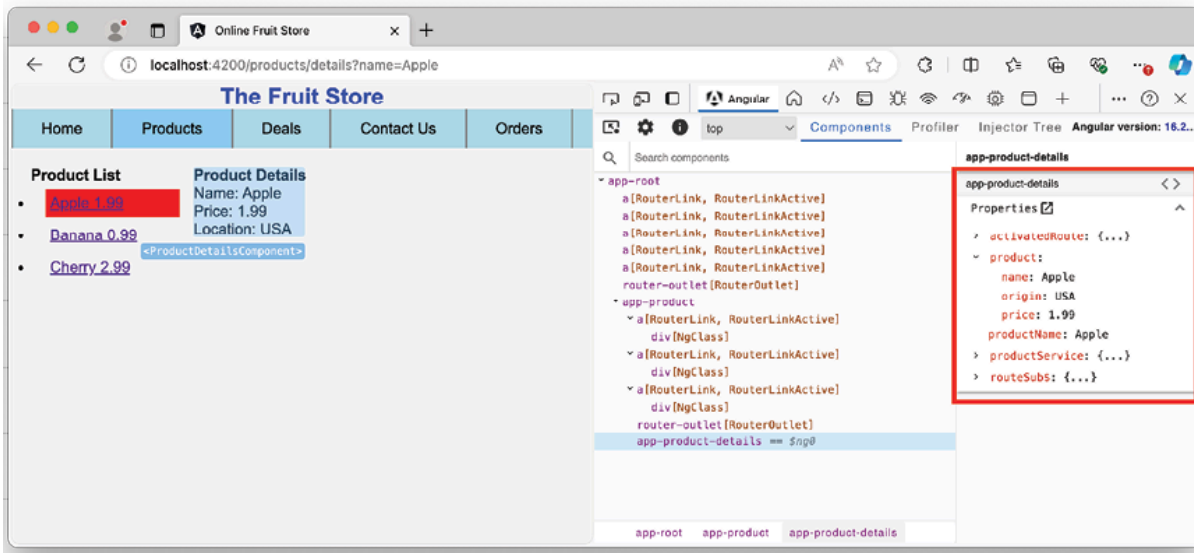


Figure 16.2: *Angular DevTools browser extension*

The Angular DevTools shows the hierarchy of Angular components rendered. When clicking any component, it shows its property values.

Performance profiling

The **Profiler** tab within the Angular DevTools allows you to record a session and show a breakdown of time spent on change detection and other activities. This can help improve the application's render performance.

The browser dev tool also has a Performance tab that can help you record a session and identify long running methods. Since JavaScript is single threaded, if the UI blocks any thread with a long running method or task, the page may freeze and not respond to user actions. It may also create jittery transitions and animations. Hence, it is imperative to identify long running methods and fix them. In *Chapter 23, Performance and User Experience,* we will explore this topic in detail.

Debugging templates and components

Angular provides tools for debugging templates and components. Use the **ng.probe()** and **ng.getComponent()** functions in the console when

DevTools are active.

Type the following on the browser console:

```
const element = document.querySelector('app-product-details');

const component = ng.getComponent(element);

console.log(component);
```

This will show the **component** properties on the **Console** as follows:

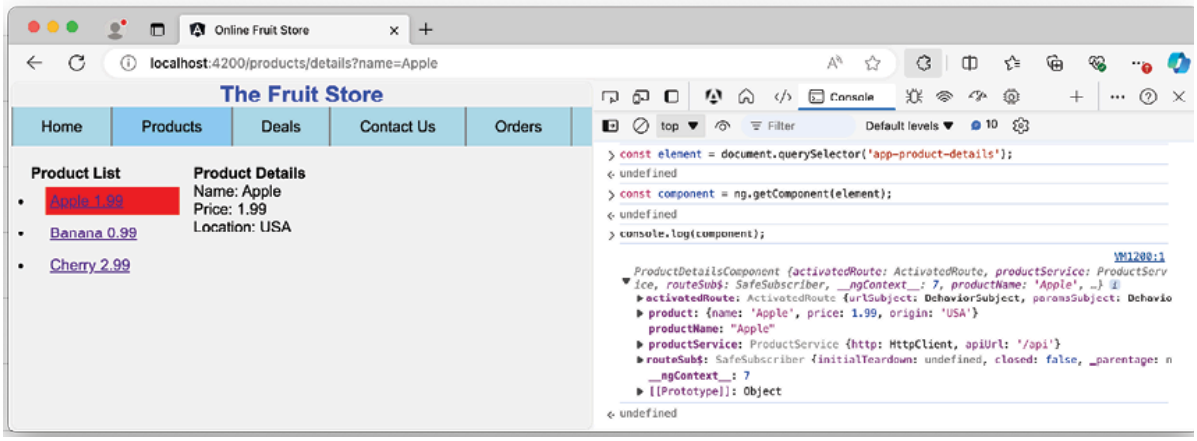


Figure 16.3: *Using ng.getComponent() in console*

Debug code in IDE using breakpoints

Most IDEs allow us to add a breakpoint in code and debug the application. We will demo this using VS Code in this section. Click to the left of the line number on any code that you want to debug. For instance, we will add a breakpoint at *line 22* in the **product-details.component.ts** file as follows:

Figure 16.4: *Adding breakpoint to code in IDE*

Click on the **Run and Debug** button on the action panel in VC Code and select **create a launch.json file** link:

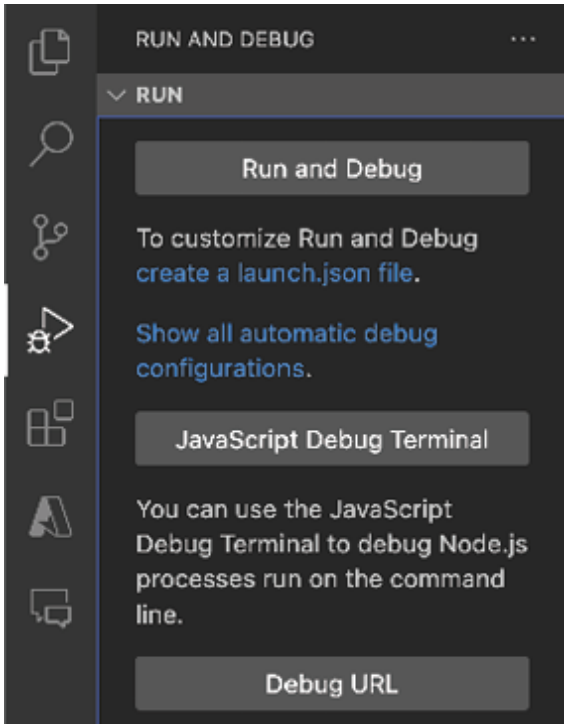


Figure 16.5: *Click on Run and Debug*

Select **Web App (your browser)** link from the top drop down, as shown:

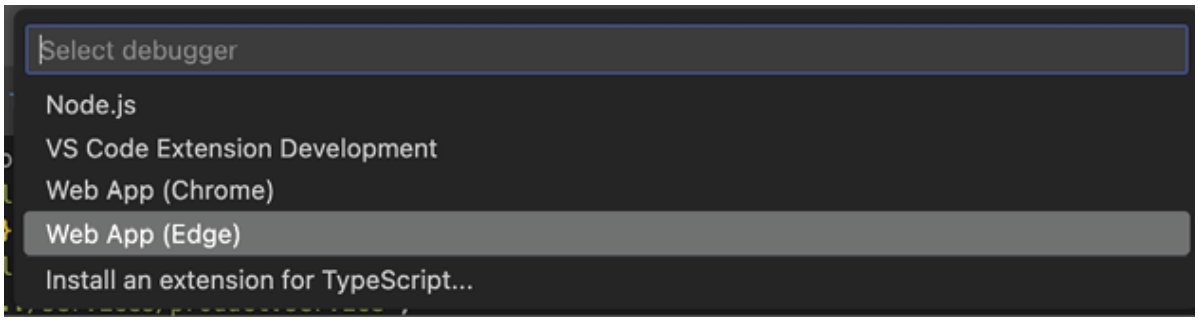


Figure 16.6: *Select Web App from the list of debuggers*

This will generate a **launch.json** file. Modify the **url** field to use port **4200** as follows:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "type": "msedge",
            "request": "launch",
            "name": "Launch Edge against localhost",
            "url": "http://localhost:4200",
```

```
9.            "webRoot": "${workspaceFolder}"
10.       }
11.   ]
12. }
```

Now, run the app using **ng serve** or any other setup that you follow to run the application on localhost. Ensure the newly created config is selected in the **Run** drop-down:

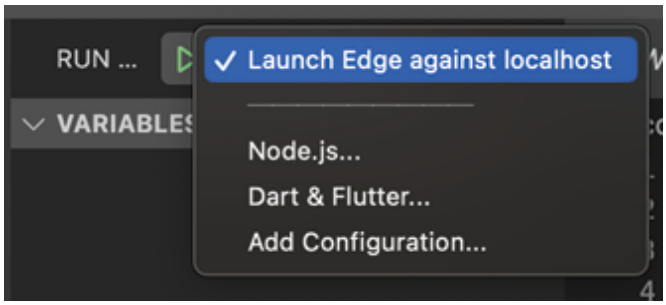


Figure 16.7: *Select the appropriate Run configuration*

Press *F5* or select **Run** | Start Debugging from the top menu.

This will open the app in a new instance of the browser. Since we added a breakpoint to the **Product** details page, going to the products page will pause the code at the line where we added the breakpoint:

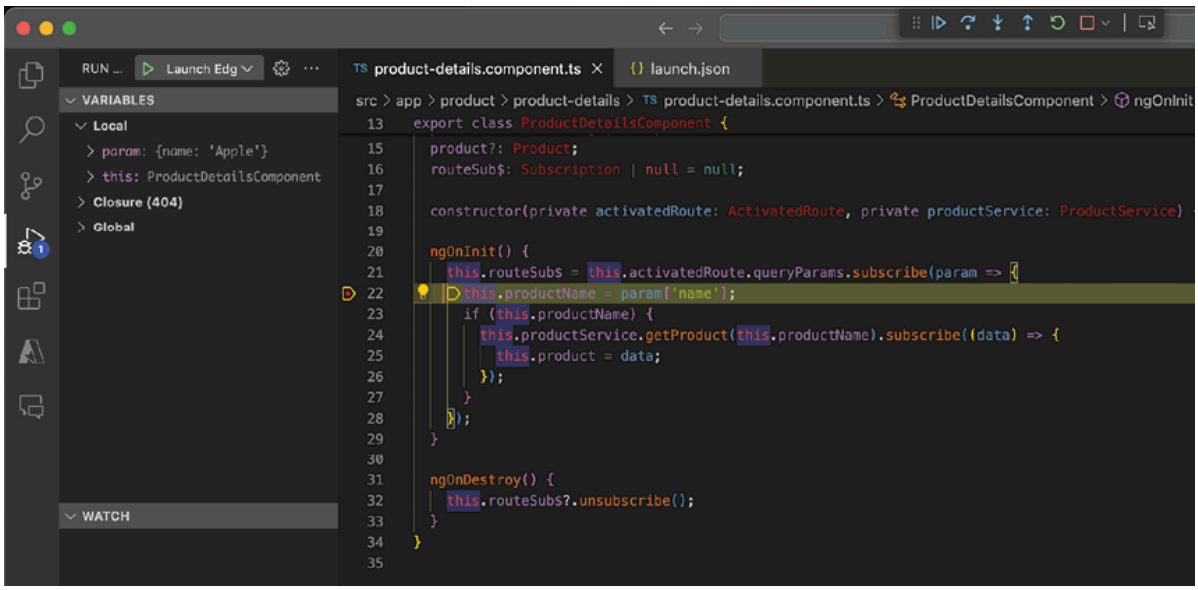


Figure 16.8: *Code pauses at the breakpoint*

You can inspect component properties and navigate through the code in a similar way you did on the browser dev tool.

NgRx DevTools

If you are using NgRx for managing the state, you can use the NgRx DevTools browser extension to observe the state at run time. The installation details and sample code are available on the NgRx site. The references section of this chapter provides a link to the documentation.

Conclusion

In this chapter, we learned about the significance of debugging in software development and explored various techniques and best practices for debugging in Angular. We delved into debugging techniques using breakpoints and logging, Angular DevTools, performance profiling, and debugging templates and components. We also gained insights into the tools and methods for effectively identifying and resolving errors, issues, and unexpected behaviors in an Angular application. This aims to equip developers with the necessary skills to confidently manage and troubleshoot Angular applications.

In the next chapter, we will learn how to handle exceptions in an Angular application.

Points to remember

- Debugging is crucial for identifying and resolving errors, issues, and unexpected behaviors in software development.
- Techniques such as using breakpoints, stepping through code, and logging are commonly used to debug software.
- Using console.log(), console.warn(), or console.error() methods can log variable values in the browser dev tool's Console tab without pausing the flow of the application.
- Angular DevTools is a browser extension that provides powerful debugging capabilities specifically for Angular applications, offering component tree visualization, change detection profiling, and state inspection.
- The browser dev tool's Performance tab can help identify long running methods and tasks that may cause the UI to freeze and not respond to user actions.

- VS Code and other IDEs allow to add breakpoints to the code and debug an Angular Application within the IDE.

Questions

1. Why is debugging important in software development, and how can it help improve the overall performance of an application?
2. Explain a few ways to debug an Angular application.
3. What are some common techniques used for debugging JavaScript code, and how can they be utilized in the browser dev tool?
4. What are the key features and benefits of the Angular DevTools browser extension for debugging Angular applications?
5. Explain how to debug an Angular application using breakpoints within the IDE.

References

- Angular DevTools:
  https://angular.dev/tools/devtools
- NgRx DevTools:
  https://ngrx.io/guide/store-devtools/install
- VS Code JavaScript debug bundler:
  https://github.com/microsoft/vscode-js-debug

CHAPTER 17
Exception Handling

Introduction

In Angular, exception handling is a crucial aspect of ensuring the stability and reliability of your application. This chapter will delve into the various techniques and best practices for effectively managing and responding to exceptions within an Angular application. By the end of this chapter, you will gain comprehensive insights to foster a robust and resilient Angular application.

Structure

This chapter covers the following topics:

- Significance of error handling
- Local error handling
- Global error handling
- Third-party log monitoring
- Handling HTTP errors

Objectives

By the end of this chapter, you will understand how to handle client side and HTTP errors in an Angular application.

Significance of error handling

When JavaScript encounters any error on the browser, the default behavior is to show the error message and stack trace on the browser dev tool console. For instance, let us take a look at this code from the **product.component.ts** file in our fruit store app:

```
1.  productService.getProducts().subscribe((data) => {
2.    this.products = data.sort();
3.  });
```

This code assumes that the data is not null and tries to **sort** the items received from the **productService**. If data is undefined or **null**, the browser will throw an exception as follows:

Figure 17.1: *Default error handler in Angular*

Showing stack traces is helpful to developers but does not serve any purpose to the end user. We need to be able to handle such errors gracefully so that meaningful actions can be taken, such as:

- **Logging the error in a log server**: Since the error happened on the user's browser, the developers would never know about it unless the error is logged into a remote server to which the developer has access.
- **Show a helpful message to the user**: The user will probably not have the dev console open on their browser to see the error messages. Also, we do not want to show the code specific error details to the user. It would be helpful if we could show a more intuitive error message explaining the cause of the error and the intended next steps.

Local error handling

The simplest way to handle errors in JavaScript/TypeScript is to wrap the code in a **try**/**catch** block. The following code snippet would print a

custom error message in the console:

```
productService.getProducts().subscribe((data) => {
  try {
    this.products = data.sort();
  } catch (e) {
    console.log("Products not found");
  }
});
```

This will show in the console as follows:

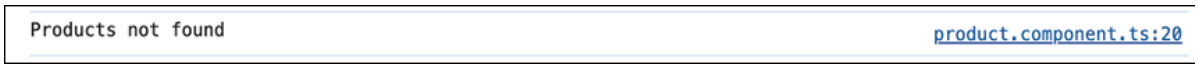


Figure 17.2: *Custom error message using try/catch block*

We can add any custom handling logic in the **catch** block as needed. It is not considered a good practice to console log error messages. Instead, a logger may be used to send log messages to remote servers. Also, some UI components can show meaningful error messages to the user. If you use any state management library, you may store the error as a global state that can be picked up by a UI component that shows an error message to the user based on the error stored in the global state. For brevity, we will skip the code to write such a component.

Global error handling

A good practice is to create a global error handler as a catch-all bucket in case any error is not handled locally. Angular uses the **ErrorHandler** class to catch errors. To see this in action, create a file **GlobalErrorHandler.ts** under the **/app/shared/error-handler** folder in our fruit store app as follows:

```
import { ErrorHandler } from '@angular/core';

export class GlobalErrorHandler implements ErrorHandler {
  handleError(error: any): void {
    console.error('This is a global error handler:', error);
  }
}
```

Replace *line 5* with any custom error handling logic that you want. If you are using the **app.module.ts** file, then you need to provide this error handler in the module file as follows:

```
1. @NgModule({
2.   ...
3.   providers: [
4.     ...
5.     { provide: ErrorHandler, useClass: GlobalErrorHandler }
6.   ],
7. })
8. export class AppModule { }
```

The above code will show the error in the browser dev console as follows:

Figure 17.3: *Global error handler message*

As a rule of thumb, always handle errors locally wherever possible and use global error handlers sparingly for uncaught exceptions.

Third-party log monitoring

Following are a few of the third-party libraries that simplify error handling and logging:

- **Sentry**: A popular error tracking service that supports Angular. It provides detailed error reports and integrates with various platforms.
- **LogRocket**: A session replay and error tracking tool that captures client side errors and user interactions.
- **Rollbar**: Another error tracking service that offers real-time monitoring and notifications.

- **BugSnag**: Provides error monitoring and reporting for web and mobile applications.

A detailed analysis of these tools is beyond the scope of this book. Please refer to their documentation for details on how to use them in your Angular application.

Handling HTTP errors

We have already covered this topic in *Chapter 14, Authentication and Authorization,* under the section *Handling Authentication Error*. In that section, we demonstrated how to intercept a login HTTP call, which can be applied generically to all other HTTP calls as well.

Conclusion

In conclusion, effective exception handling is crucial for maintaining the stability and reliability of an Angular application. This includes handling errors locally using **try**/**catch** blocks, implementing global error handlers for uncaught exceptions, and considering the use of third-party log monitoring services for more comprehensive error tracking. Understanding the importance of handling client-side and HTTP errors equips developers with the necessary tools to create robust and user friendly Angular applications.

Points to remember

- Remember the importance of exception handling in Angular to ensure the stability and reliability of your application.
- The chapter covers local error handling, global error handling, third-party log monitoring, and handling HTTP errors.
- Always handle errors locally wherever possible and use global error handlers sparingly for uncaught exceptions.
- Consider using third-party libraries such as Sentry, LogRocket, Rollbar, or Bugsnag for error handling and logging.

Questions

1. What is the purpose of error handling in an Angular application?

2. How can local error handling be implemented in JavaScript/TypeScript?
3. What is a good practice for creating a global error handler in Angular?
4. Can you name some third-party libraries that simplify error handling and logging for Angular applications?

References

- Sentry:
  https://docs.sentry.io/
- LogRocket:
  https://logrocket.com/
- Rollbar:
  https://rollbar.com/
- BugSnag:
  https://www.bugsnag.com/error-monitoring/

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 18
Internationalization and Localization

Introduction

In today's globalized world, businesses and organizations have a broader reach than ever before. As a result, web applications must cater to users from different parts of the world. To achieve this, web applications must be accessible and usable to users regardless of their language, culture, and location. This is where the concepts of internationalization and localization come in. Internationalization and localization are two key concepts that enable web applications to be adapted and customized based on the user's language, culture, and location. Angular provides a wide range of tools and techniques to achieve internationalization and localization in web applications. In this chapter, we will explore the basics of internationalization and localization, the concept of a locale, how it impacts dates, currency, and number formats, and how to handle bidirectional text.

Structure

The chapter covers the following topics:

- Internationalization and localization
- Performing internationalization
- Locale
- Localizing Angular app
- Best practices for i18n
- Displaying dates, currency, and numbers
- Switching between locales in the UI
- Handling bidirectional text

Objectives

By the end of this chapter, you will know how to internationalize your Angular app so that it can be rendered to a global audience in different languages and locales.

Internationalization and localization

Internationalization and localization are two key concepts that enable web applications to be adapted and customized based on the user's language, culture, and location. **Internationalization** is the process of designing and developing web applications in a way that makes it easy to adapt to different languages and cultures. In other words, it is the process of making an application ready for localization. **Localization**, on the other hand, is the process of adapting a web application to a specific language or culture.

Performing internationalization

Let us create a sample Angular app to demo this:

```
ng new i18n-example
```

Open **app.component.html** and replace it with a simple hello world text as follows:

```
<h1>Welcome to {{ title }}</h1>
```

Run **ng serve** to see the page in English as shown below:

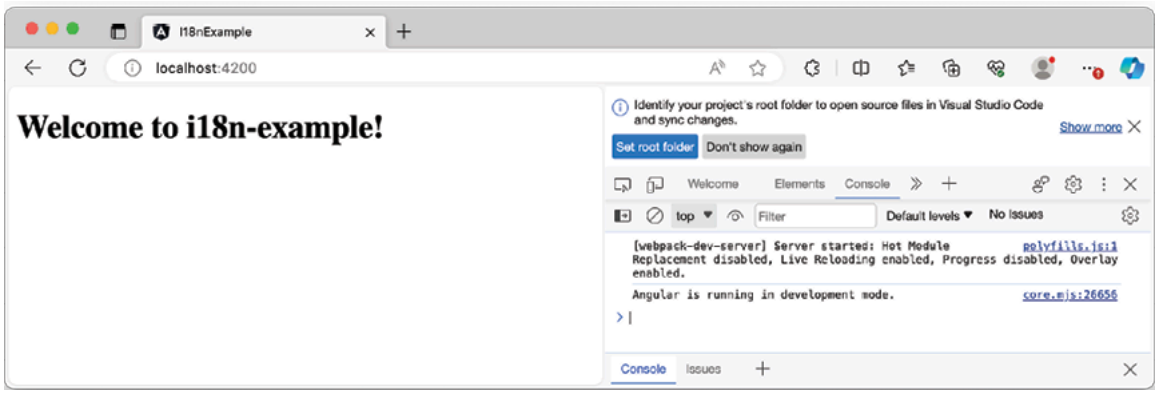


Figure 18.1: *Simple Angular app*

To internationalize your Angular application, you need to use the built-in **i18n** functionality that allows you to mark text in component templates for translation. Start by installing the package as follows:

```
ng add @angular/localize
```

You can use the **i18n** attribute or the **i18n-** prefix to indicate which elements or attributes need to be translated. Let us try out the simplest example by just adding **i18n** attribute as follows:

```
1. <h1 i18n>Welcome to {{ title }}!</h1>
```

Now run the following command to extract the translation text:

```
ng extract-i18n
```

This will generate an XML file in the XLIFF format – **messages.xlf**, as follows:

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2. <xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
3.  <file source-language="en-US" datatype="plaintext" original="ng2.template">
4.   <body>
5.    <trans-unit id="4248521319677765272" datatype="html">
6.     <source>Welcome to <x id="INTERPOLATION" equiv-text="{{ title }}"/>!</source>
7.     <context-group purpose="location">
8.  <context context-type="sourcefile">src/app/app.component.html</context>
9.      <context context-type="linenumber">1</context>
10.     </context-group>
11.    </trans-unit>
12.   </body>
13.  </file>
14. </xliff>
```

This file is the source of translation handed over to translation agents to translate the application texts into different languages. For each translation text in the component template file, a **<trans-unit />** element is created in the **.xlf** file. It indicates the source and location of the text, amongst other things, to assist with translation. We will look at other attributes later in the chapter. Notice that an ID is auto generated in *line 5* above. Do not change that ID in the file as that is used by Angular to determine which element to translate. For now, our application is internationalized and is ready to be localized into any locale. In the next section, we will explore how to translate this simple text into French.

Locale

In simple words, a locale is a dialect of a language used in a specific country or region. For instance, color in American English is spelled as colour in Canada. Similarly, the date format, number format, currency, etc., change as per the country. We need to know the locale to be able to localize an Angular app. A locale is denoted primarily by the language, like **en** for English, **fr** for French. To indicate a region, an optional country

code is suffixed with a hyphen, for example, **en-US** for American English and **en-CA** for Canadian English.

Localizing Angular app

Now that the **.xlf** file is generated, let us create a French translation file. Create a copy of **messages.xlf** file in **src/locale** folder as **messages.fr.xlf**. Add **<target … />** element under each **<source … />**. For instance, our source text will have the following **target** element in French:

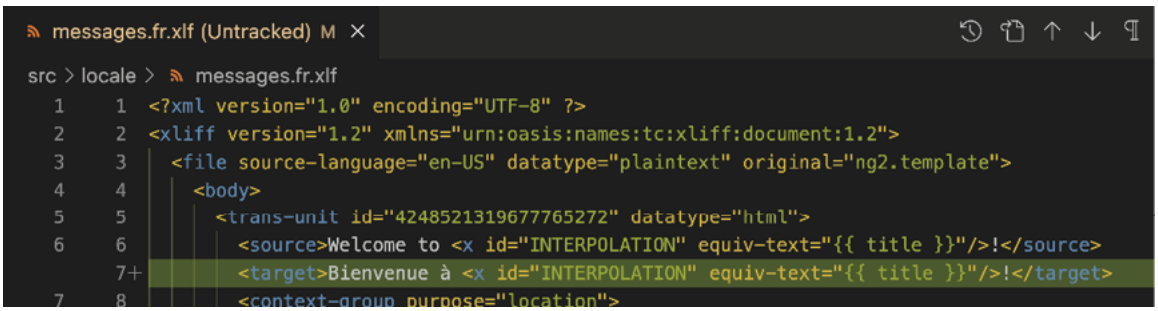


Figure 18.2: *Target element added to translation file*

Now we need to configure the app to build and serve the app in French. Add the following object in **angular.json** file under **project > 18n-example**:

```
"i18n": {
  "sourceLocale": "en-US",
  "locales": {
    "fr": "src/locale/messages.fr.xlf"
  }
},
```

Add the following under **project > 18n-example > architect > build > configurations > production**:

```
"localize": true
```

Run:

ng build --localize

This will generate the folder **dist/i18n-example/fr**. If you deploy this to a web server, it will serve the application in French. Let us see if it works with the development server. Enable **localize** option for development as well. Add the following under **project > 18n-example > architect > build > configurations > development**:

```
"localize": true
```

Now run:

ng serve

You will notice the following error in the terminal:

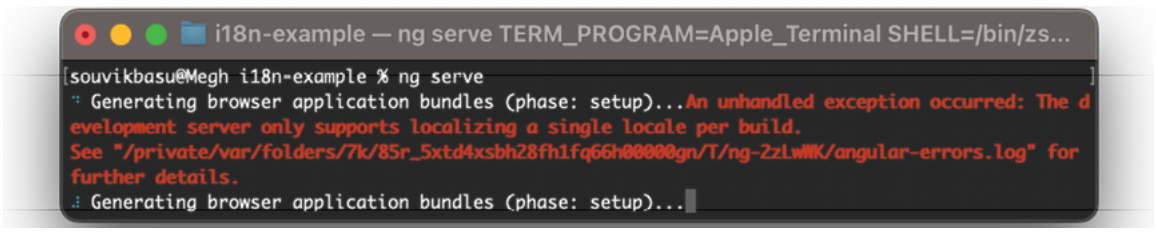


Figure 18.3: *ng serve error*

As mentioned above, the development server only supports one locale at a time. Since we have both **en** and **fr** configured as locales in **angular.json** file, the development server cannot serve this app. For this to work, we need to create separate build configurations for each locale and specify which configuration to use in the development server. Let us do that next. Set **localize** to **false** under development configuration, that is, **project > 18n-example > architect > build > configurations > development**:

```
"localize": false
```

Add the following options under **project > 18n-example > architect > build > configurations**:

```
"en": {
  "localize": ["en"]
},
"fr": {
  "localize": ["fr"]
}
```

Add the following under **project > 18n-example > architect > serve > configurations**:

```
"en": {
  "browserTarget": "i18n-example:build:development,en"
},
"fr": {
  "browserTarget": "i18n-example:build:development,fr"
}
```

Now you can run **ng serve** with the above configuration as an argument:

```
ng serve --configuration=fr
```

This will show the page in French on the browser as follows:

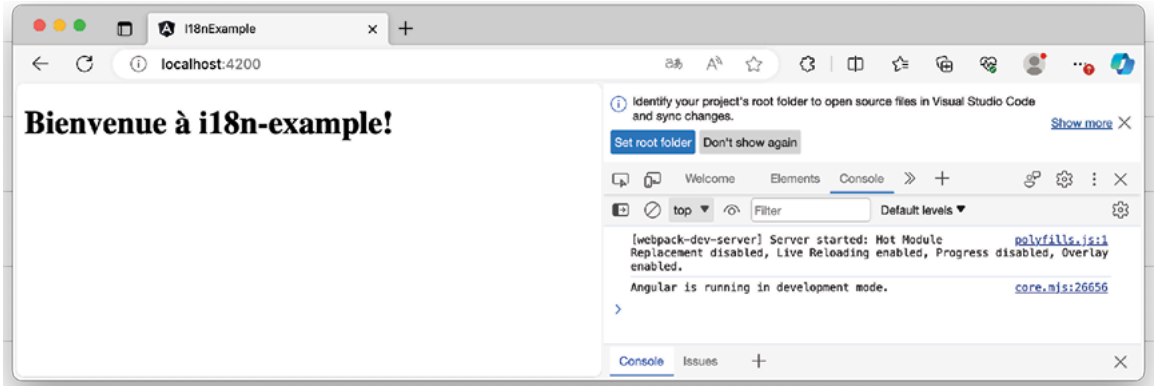


Figure 18.4: *App translated in French*

Best practices for i18n

In the above sections, we saw a simple case of translating a single text from English to French. In a real-world application, it helps to provide more context to translation agents so that they can provide accurate translations. The **@@** prefix is used to assign custom identifiers to the translation units, which are the smallest translatable units of text. In our previous example, we can add an identifier as follows:

```
<h1 i18n="@@app-title">Welcome to {{ title }}!</h1>
```

We can add **meaning** and **description** text to the identifier in the below format:

```
<meaning>|<description>@@<identifier>
```

For example, the above element can be written as:

```
<h1 i18n="Welcome message|This is a message shown at the top of the home page to greet the user@@ap
```

This shows up in the **messages.xlf** file as follows:

```
<?xml version="1.0" encoding="UTF-8" ?>
<xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
  <file source-language="en-US" datatype="plaintext" original="ng2.template">
    <body>
      <trans-unit id="app-title" datatype="html">
        <source>Welcome to <x id="INTERPOLATION" equiv-text="{{ title }}"/>!</source>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">1</context>
        </context-group>
        <note priority="1" from="description">This is a message shown at the top of the home page to greet the user</note>
        <note priority="1" from="meaning">Welcome message</note>
      </trans-unit>
    </body>
  </file>
</xliff>
```

Figure 18.5: *Meaning and description added .xlf file as notes*

Adding **meaning** and **description** to the text element is a helpful way to provide more context to translation agents.

Displaying dates, currency, and numbers

To display dates, numbers, percentages, and currencies in a local format, you can use the built-in pipes provided by Angular, such as **DatePipe**, **DecimalPipe**, **PercentPipe**, and **CurrencyPipe**. These pipes use the locale data registered in your app module. You can import the locale data from the **@angular/common** package and register it using the **registerLocaleData** function. For example, in our app, you could register **fr** locale data as follows:

```
...
import { registerLocaleData } from '@angular/common';
import localeFr from '@angular/common/locales/fr';

import { AppComponent } from './app.component';

registerLocaleData(localeFr, 'fr');

@NgModule({ ... })
export class AppModule { }
```

Now add the following to the HTML template to show current **date**, some number and **currency** values as follows:

```
<p>Today is {{ today | date }}</p>
<p>This is a billion: {{ 1000000000 | currency }}</p>
```

In the **app.component.ts** file define the field **today** as follows:

```
today = new Date();
```

Now, the English and French versions of the page will automatically show the date, number, and currency formats based on the locale, as shown in *Figure 18.6* and *Figure 18.7*:

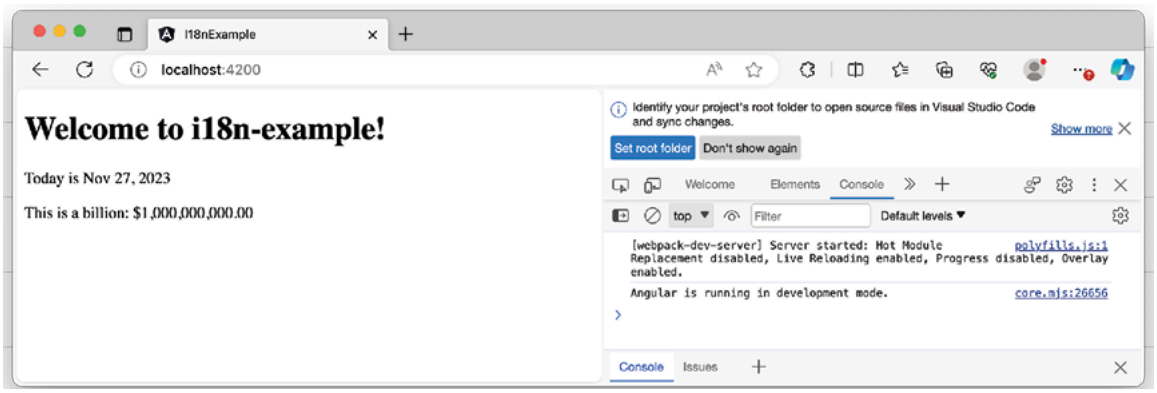


Figure 18.6: *Date, number, and currency in English*

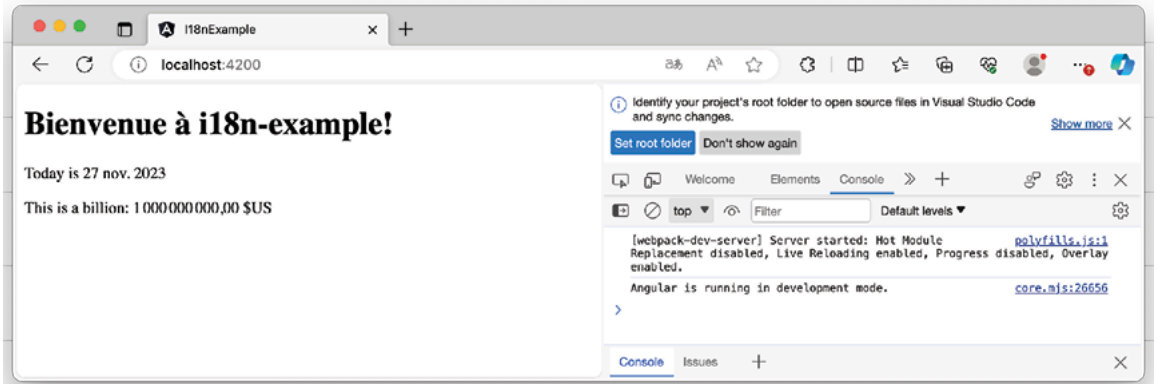


Figure 18.7: *Date, number, and currency in French*

Switching between locales in the UI

To switch between different languages or locales in your Angular application, you can use a language selector component that allows the user to choose their preferred language. You can then use the **APP_INITIALIZER** provider to load the translation file, and the locale data dynamically based on the user's selection. You can also use the **Router** service to navigate to the appropriate version of the app. Following is an example code of a language service:

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { TranslateService } from '@ngx-translate/core';
import { registerLocaleData } from '@angular/common';
import { Router } from '@angular/router';

@Injectable({
  providedIn: 'root'
})
export class LanguageService {

 constructor(private http: HttpClient, private translate: TranslateService, private router: Router) { }

  loadLanguage(): Promise<any> {
    return new Promise((resolve, reject) => {
  this.http.get('assets/config.json').subscribe((config: any) => {
        const language = config.language || 'en';
        this.translate.setDefaultLang(language);
        this.translate.use(language);
        this.loadLocaleData(language).then(() => {
          this.router.navigate([language]);
          resolve();
        }).catch((error) => {
          reject(error);
        });
      });
    });
  }

```

```
  loadLocaleData(language: string): Promise<any> {
    return new Promise((resolve, reject) => {
      switch (language) {
        case 'fr':
          import('@angular/common/locales/fr').then((module) => {
            registerLocaleData(module.default, 'fr');
            resolve();
          }).catch((error) => {
            reject(error);
          });
          break;
        case 'es':
          import('@angular/common/locales/es').then((module) => {
            registerLocaleData(module.default, 'es');
            resolve();
          }).catch((error) => {
            reject(error);
          });
          break;
        default:
          import('@angular/common/locales/en').then((module) => {
            registerLocaleData(module.default, 'en');
            resolve();
          }).catch((error) => {
            reject(error);
          });
          break;
      }
    });
  }
}
```

Handling bidirectional text

Bidirectional text (or bi-directional text) is text that contains both **left-to-right** (**LTR**) and **right-to-left** (**RTL**) scripts, such as Arabic, Hebrew, Persian, Urdu, etc. To handle bidirectional text in the Angular app using **i18n** you need to specify the directionality of your app by using the **dir** attribute on the **<html>** or **<body>** element. You can use the value **ltr** for left-to-right languages, **rtl** for right-to-left languages, or **auto** for automatic detection based on the first strong character in the text. For example:

```
<html dir="rtl">
  <body>
    <!-- Your app content -->
  </body>
</html>
```

You need to use the **bidi** module from the **@angular/cdk** package to provide directionality services and directives for your components. You can use the **Directionality** service to get the current direction of the app, and the **dir** directive to set the direction of a specific element as follows:

```
import { Component } from '@angular/core';
import { Directionality } from '@angular/cdk/bidi';

```

```
@Component({
  selector: 'app-root',
  template: `
    <div [dir]="dir">This is a bi-directional text</div>
    <button (click)=»toggleDir()">Toggle direction</button>
  `
})
export class AppComponent {
  dir = 'ltr';

  constructor(private directionality: Directionality) {
    this.directionality.change.subscribe(dir => {
      console.log('The current direction is ' + dir);
    });
  }

  toggleDir() {
    this.dir = this.dir === 'ltr' ? 'rtl' : 'ltr';
  }
}
```

You need to use the **bidi** pipes from the **@angular/common** package to format dates, numbers, percentages, and currencies in a bi-directional way. These pipes will automatically add Unicode markers to the output to preserve the correct directionality. Here is a sample code:

```
<p>{{ date | date:'fullDate' | bidi }}</p>
<p>{{ number | number:'1.2-2' | bidi }}</p>
<p>{{ percent | percent:'1.0-0' | bidi }}</p>
<p>{{ currency | currency:'USD':'symbol':'1.2-2' | bidi }}</p>
```

Conclusion

This chapter was a brief introduction to handling internationalization and localization in Angular. By following the best practices, developers can ensure that their applications are accessible and usable for users from different parts of the world. With Angular's **i18n** module, developers can easily translate web applications into different languages, making it possible to reach a broader audience and better serve their users. Please refer to the link in the reference section for more advanced options in **i18n**.

In the next chapter, you will learn about how to take care of accessibility in your Angular app.

Points to remember

- Angular provides tools to internationalize and localize an app.
- Design for internationalization and localization from the start.
- Testing and verifying translations are crucial before releasing a localized version of the app.
- Use a translation management tool.
- Considering cultural differences and sensitivities in translations
- Currency, date, number, and percentage formats are dependent on a locale.
- Allow RTL languages to work fine on your page for better user reach.

Questions

1. Explain the difference between internationalization and localization.
2. Is it possible to localize for a language rather than a locale?

3. Localize the fruit store app we created in the previous chapters as an exercise.
4. How do we change the locale based on user selection on the page?
5. How to show the page in the correct locale based on browser settings?

References

- Angular official document on internationalization:
  https://angular.io/guide/i18n-overview

CHAPTER 19
Accessibility

Introduction

Accessibility is a fundamental aspect of web development that ensures that users with disabilities or impairments can access and use web applications with ease. In today's digital landscape, it is critical to provide an inclusive and accessible web experience for all users, regardless of their abilities. A web application that is inaccessible can cause frustration, exclusion, and even legal issues. Angular offers a range of features and tools that make it easier to build accessible web applications. In this chapter, we will explore the basics of accessibility in web development and how to implement them using Angular. We will discuss the importance of accessibility, the guidelines and standards to follow, and the best practices to ensure a fully accessible web application.

Structure

The chapter covers the following topics:

- Understanding disability and impairment
- Guidelines and standards for accessibility
- Adding ARIA attributes
- Using Angular Material Design
- Using Angular Component Dev Kit
- High contrast mode
- Testing and evaluating accessibility

Objectives

By the end of this chapter, you will know the importance of accessibility and how to make a web app accessible using features in Angular. You will also learn a few best practices to achieve accessibility in web development.

Understanding disability and impairment

There are different accessibility issues to consider when developing an Angular app, such as:

- **Visual impairments**, such as low vision, color blindness, or blindness. These users may rely on screen readers, magnifiers, or braille devices to access web content.
- **Motor or mobility impairments**, such as difficulty using a mouse, keyboard, or touch

screen. These users may use alternative input devices, such as voice commands, eye tracking, or switches.

- **Auditory impairments**, such as hearing loss or deafness. These users may need captions, transcripts, or sign language interpretation for audio or video content.
- **Seizures**, such as photosensitive epilepsy. These users may be triggered by flashing or flickering content, such as animations, transitions, or strobe effects.
- **Learning or cognitive impairments**, such as dyslexia, dyscalculia, or attention deficit disorder. These users may need clear and simple language, consistent and predictable layout, or feedback and guidance for completing tasks.

Guidelines and standards for accessibility

To ensure that your web application is accessible, you must follow certain guidelines and standards. The most widely recognized accessibility guidelines are the **Web Content Accessibility Guidelines** (**WCAG**) developed by the *World Wide Web Consortium* (*W3C*). Some of the guidelines and standards to follow are:

- Provide alternative text for images and other non-text content.
- Use semantic HTML elements that convey the structure and meaning of your content, such as headings, lists, tables, forms, etc. This helps screen readers and other assistive technologies to understand and navigate your web app.
- Use **Accessible Rich Internet Applications** (**ARIA**) attributes to enhance accessibility for widgets.
- Provide captions and transcripts for multimedia content.
- Ensure that color is not the only means of conveying information.
- Provide keyboard accessibility for all functionalities.
- Add **alt** attribute to **img** HTML tags. This attribute value is read by a screen reader tool. The text is displayed on the page if the image cannot be displayed for any reason.
- Test your web application with accessibility testing tools, such as **axe-core**, **pa11y**, **codelyzer**, **Lighthouse**, or **BrowserStack,** to ensure that it meets the accessibility guidelines and standards.

These are guidelines for accessibility in general for all web app and not specifically to Angular apps. In the next section, we will look at how to achieve accessibility in an Angular app.

Adding ARIA attributes

ARIA attributes are a set of HTML attributes that can be added to web page elements to make them more accessible to users with disabilities. These attributes provide additional information about the functionality and behavior of the elements, allowing screen readers and other assistive technologies to better understand and navigate the content. ARIA attributes can be used to indicate the purpose of a button, provide labels for form inputs, and describe the state of a dynamic user interface element. Here is an example of using ARIA attributes in a component:

```
<button mat-button aria-label="Delete Item" (click)="deleteItem()">Delete</button>
```

In this example, the **aria-label** attribute is used to provide an accessible label for the button. This is important for users who rely on assistive technologies, such as screen readers, to navigate the web application. A few of the most used ARIA attributes are listed below:

ARIA attribute	Description
aria-label	Defines a string that labels the current element.
aria-labelledby	Identifies the element (or elements) that labels the current element.
aria-describedby	Identifies the element (or elements) that describes the current element.
aria-hidden	Indicates whether the element is exposed to an accessibility API.
aria-invalid	Indicates the validity of the element's value.
aria-required	Indicates that user input is required on the element.
aria-expanded	Indicates whether the element, or another grouping element it controls, is expanded or collapsed.
aria-selected	Indicates the current selected state of the element.
aria-checked	Indicates the current checked state of the element.
aria-pressed	Indicates the current pressed state of the element.
aria-live	Indicates that an element will be updated and describes the types of updates the user agents, assistive technologies, and user can expect from the live region.

Table 19.1: *ARIA attributes*

There are situations when you would want to bind these ARIA attributes to component fields. To bind attributes, you must use the **attr.** prefix. For instance, in the above example, you may bind to a field **deleteLabel** as follows:

```
<button mat-button [attr.aria-label]="deleteLabel " (click)="deleteItem()">Delete</button>
```

Using Angular Material Design

Angular Material Design components are designed to follow the Material Design guidelines and the WCAG standards for accessibility. It is recommended to use this and other such component libraries that have accessibility features pre-built. Following is an example of how to use the Angular Material Design **mat-form-field** component:

```
<mat-form-field>
  <input matInput placeholder="Your name">
</mat-form-field>
```

In this example, the **mat-form-field** component is used to create a form field with a label and input. This component is designed with accessibility in mind and includes ARIA attributes to enhance accessibility.

Using Angular Component Dev Kit

Angular **Component Dev Kit** (**CDK**) is a set of tools and utilities that helps in building complex UI components. Angular CDK also provides a set of accessibility features that can be used to ensure that your application is accessible to everyone, including users with disabilities.

To use Angular CDK for accessibility, you should first ensure that you have installed the CDK package in your Angular application. Once installed, you can use the following CDK features for accessibility:

- **ARIA attributes**: CDK provides a set of ARIA attributes. These attributes can be added to HTML elements using the **@HostBinding** decorator.
- **Focus management**: CDK provides a set of utilities for managing focus in your UI components. This can be especially useful for users who navigate your application using a

keyboard.

- **Keyboard navigation**: CDK provides a set of utilities for implementing keyboard navigation in your UI components. This can be useful for users who are unable to use a mouse or other pointing device.
- **Live announcers**: CDK provides a set of utilities for announcing changes to your UI components. This can be useful for users who are visually impaired and may not be able to see changes to your UI.

High contrast mode

Many operating systems, such as Windows, have a high contrast mode. A high contrast mode helps users with partial visual impairment to work on your website with ease. Angular CDK **cdk/ally** package provides a SASS mixin that allows you to define styles for high contrast mode. You can define your own high-contrast styling inside the **high-contrast** mixin in Angular CDK as follows:

```
@use '@angular/cdk';

button {
  @include cdk.high-contrast() {
    outline: dashed 2px;
  }
}
```

For further details on the topic, refer to the Angular Material documentation on accessibility provided in the reference section at the end of this chapter.

Testing and evaluating accessibility

There are many tools to test the accessibility of your Angular app. For comprehensive test coverage, you can add unit tests using libraries like **axe-core** to catch accessibility issues in your build pipeline, and you can test manually using screen readers. The following is an example to test the **ProductComponent** in our fruit store app for accessibility using the **axe-core** library:

```
import { ComponentFixture } from '@angular/core/testing';
import { axe, toHaveNoViolations } from 'jest-axe';

describe('ProductComponent', () => {
  let component: ProductComponent;
  let fixture: ComponentFixture<ProductComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [ProductComponent],
    }).compileComponents();

    fixture = TestBed.createComponent(ProductComponent);
```

```
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should have no accessibility violations', async () => {
    const html = fixture.nativeElement;
    expect(await axe(html)).toHaveNoViolations();
  });
});
```

In this example, the **jest-axe** library is used to test the HTML generated by the **ProductComponent** for accessibility violations. The **toHaveNoViolations** matcher is used to ensure that there are no violations.

Conclusion

Accessibility is a crucial aspect of web development that cannot be ignored. Angular provides developers with a variety of tools and features that make it easier to create accessible web applications. By following best practices and using the built-in accessibility features of Angular, developers can ensure that their applications are usable by everyone, including people with disabilities. It is essential to keep in mind that accessibility is an ongoing process, and developers should continuously test and evaluate their applications to ensure that they meet the needs of all users. In the next chapter, you will learn about how to create animation in your Angular app.

Points to remember

- There are guidelines and standards for making accessible web pages.
- Angular allows setting ARIA attributes.
- The Angular Material Design component library has accessibility support built in.
- It is a good practice to test accessibility in the build pipeline.

Questions

1. Why is it important to make web applications accessible?
2. Explain the different types of disabilities that a web developer should be aware of.
3. What are ARIA attributes?
4. Provide some best coding practices with respect to accessibility.

References

- Angular official doc on accessibility:
  https://angular.io/guide/accessibility
- Google Web Development (accessibility):
  https://web.dev/articles/accessibility#what_is_accessibility
- Introduction to ARIA:
  https://web.dev/articles/semantics-aria

- Angular Material Components:
  https://material.angular.io/
- Accessibility using Angular CDK:
  https://material.angular.io/cdk/a11y/overview
- Accessibility Design using Angular Material:
  https://m2.material.io/design/usability/accessibility.html

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 20
Animation

Introduction

Using animation, you can add motion and style to your web applications, making them more visually appealing and engaging. Animations in Angular are achieved through a combination of CSS and JavaScript, and by using Angular's powerful animation API, you can create complex animations with ease. In this chapter, we will explore how to use Angular animations to create dynamic and engaging user experiences.

Structure

The chapter covers the following topics:

- Simple animation using CSS
- Animation module in Angular
- Reusing animation
- Note on accessibility

Objectives

By the end of this chapter, you will know how to add animation and transitions to Angular components.

Simple animation using CSS

Animation of DOM elements can be achieved using simple CSS properties like **transition**. Let us create a simple toggle button using plain HTML and CSS. On click, the knob of the button should move smoothly from left to right to turn the state to an ON position. The background should change to green in this state. Similarly, clicking again should switch the button OFF, move the knob to the original position, and background change to gray. To see this in action, create a new angular project and add the following code in **app.component.html** file:

```
<style>
  .button .knob {
    content: '';
    position: absolute;
    top: 4px;
    left: 4px;
    width: 20px;
```

```
    height: 10px;
    padding: 9px 4px;
     background-color: #fff;
     border-radius: 50%;
     transition: 0.3s cubic-bezier(0.18, 0.89, 0.35, 1.15) all;
  box-shadow: 0 20px 25px -5px rgba(0, 0, 0, 0.35), 0 10px 10px -5px rgba(0, 0, 0, 0.20);
  }

  .button .checkbox:checked+.knob {
     left: 32px;
  }

  .button .checkbox:checked~.layer {
     background-color: #018766;
  }

  .knob,
  .layer {
     position: absolute;
     top: 0;
     right: 0;
     bottom: 0;
     left: 0;
  }

  .button {
     position: relative;
     top: 50%;
     width: 65px;
     height: 36px;
     margin: -20px auto 0 auto;
     overflow: hidden;
     margin: 0;
  }

  .button,
  .button .layer {
     border-radius: 100px;
  }

  .checkbox {
     position: relative;
     width: 100%;
```

```
51.     height: 100%;
52.     padding: 0;
53.     margin: 0;
54.     opacity: 0;
55.     cursor: pointer;
56.     z-index: 3;
57.   }
58.
59.   .knob {
60.     z-index: 2;
61.   }
62.
63.   .layer {
64.     width: 100%;
65.     background-color: #ebebeb;
66.     z-index: 1;
67.   }
68. </style>
69.
70. <div class="button">
71.   <input type="checkbox" class="checkbox">
72.   <div class="knob"></div>
73.   <div class="layer"></div>
74. </div>
```

The above code defines a **div** to show a toggle button at *line 70*. It has an input checkbox to hold the toggle state, a **knob** to toggle the button, and a layer to show the background color as green when the button is in the ON state. For brevity, we have added all the styles inside a style tag within the same file. In practice, we will create a separate component for this button and move the styles into a separate **.scss** file. The class **.button .knob** at *line 2* defines the size and look of the rounded **knob**. It sets a shadow to it to give a 3D effect and adds transition property to provide it with a more natural horizontal movement when clicked. *Line 17* sets the horizontal position of the knob to the right (by adding **left: 32px**) when the input is checked. *Line 21* sets the layer background to green when the input is checked. *Lines 24-31* sets the knob and layer to **position: absolute** to be able to move it horizontally within the button. *Lines 33-41* sets the dimension of the button and set **position:relative** to allow free movement of the knob and layer within it. Line 45 gives a rounded look to the button and the green layer. *Lines 48-57* stretch the input checkbox to the entire area but set its opacity to zero. *Line 60* puts the knob behind the checkbox using the **z-index** property so that the checkbox can receive click events. *Lines 63-67* sets the width and initial background color of the layer under the knob. It would show on the browser as follows:

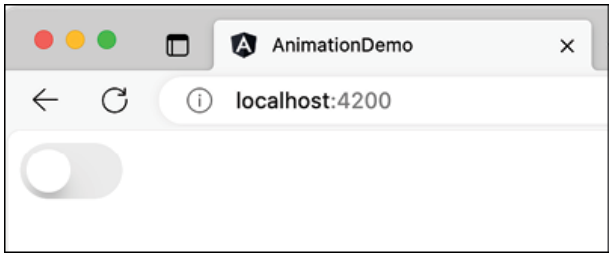


Figure 20.1: *Toggle button in OFF position*

On click, the button changes to the following state with a green background:

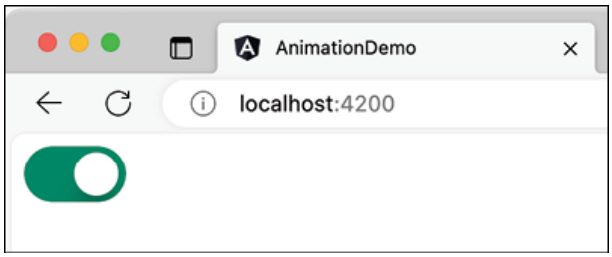


Figure 20.2: *Toggle button in ON position*

In the next section we will create the same animation using Angular's inbuilt animation module.

Animation module in Angular

All animation functions are available in the **@angular/animations** module, which is added as a dependency to the project by default if created using the Angular CLI. Ensure that this is added in the **packages.json** file under dependencies. If not, please add the module using **npm**. Now, add **BrowserAnimationsModule** to **AppModule** in **app.module.ts** file as follows:

```
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';

@NgModule({
  ...
  imports: [
    ...,
    BrowserAnimationsModule
  ]
})
export class AppModule { }
```

Generate a new Angular component for the toggle button as follows:

ng g c toggle-button

To make sure, the previous code works within the component as well, copy over the HTML part from **app.component.html** into **toggle-button.component.html** and CSS part into **toggle-button.component.scss** file. Replace **app.component.html** file with the new **ToggleButtonComponent** as follows:

```
<app-toggle-button></app-toggle-button>
```

You will notice the toggle button works as before. Next, let us implement the same animation using the declarative syntax of Angular. To do this, remove the CSS properties that caused the knob to move from left to right and back smoothly. There are two such properties in our code that enabled the animation using CSS. First is the **left** property on knob when the input checkbox is

checked:

```
.button .checkbox:checked+.knob {
  left: 32px;
}
```

Remove this CSS from **toggle-button.component.scss** file.

The second CSS property that enabled the animation was a transition property on the **.button .knob** selector:

```
.button .knob {
  ...
  transition: 0.3s cubic-bezier(0.18, 0.89, 0.35, 1.15) all;
  ...
}
```

Remove this line as well. You will notice that with a click of the button, the color changes between gray and green, but the knob does not move. This is expected. Next, let us add the animation in the component to set back these CSS properties in a declarative way. Animation is set on an Angular component using the **animations** metadata. Add the following animation metadata to our **ToggleButtonComponent**:

```
import { Component } from '@angular/core';
import { trigger, state, style, transition, animate } from '@angular/animations';

@Component({
  selector: 'app-toggle-button',
  templateUrl: './toggle-button.component.html',
  styleUrls: ['./toggle-button.component.scss'],
  animations: [
    trigger('toggle', [
      state('on', style({
        left: '32px'
      })),
      state('off', style({
        left: '4px'
      })),
      transition('on <=> off', animate('{{duration}} {{easing}}'), {
        params: {
          duration: '0.3s',
          easing: 'cubic-bezier(0.18, 0.89, 0.35, 1.15)'
        }
      })
    ])
  ]
})
export class ToggleButtonComponent {
  checked = false;
```

```
27. }
```

The animations metadata defines a set of triggers. In this case, we defined a trigger **toggle**. When this trigger is invoked in code, the animation will be triggered. The trigger can define a bunch of **states** defined by a string name. In this case, we defined two states - **on** and **off**. Apart from these custom states, we may also define the following states:

- **void**: This refers to the state when the element is created but not yet part of the DOM, or when the element is removed from the DOM.
- **wildcard ***: this refers to all other states not explicitly defined.

Every state defines a set of desired style statements using the **style()** method. In this case, we are setting the **left** CSS property to **32px** in **on** state and **4px** in **off** state. You may add other CSS properties as desired, which would define the styles for the respective states. Apart from the state, a trigger also defines transitions using the **transition()** method. In this case, we defined a transition between **on <=> off** states in *line 16*. The transition function takes an **animate()** method call as the second parameter that defines the duration and easing functions. In this case, we mention that the duration is **0.3s** and the easing function is **cubic-bezier(0.18, 0.89, 0.35, 1.15)**. Notice that these values are the same transition CSS properties that we removed from the component style file. All the methods we discussed above are imported from **@angular/animations** in *line 2*.

So overall, we created an animation declaration that when the state changes from **off** to **on**, change the **left** CSS property from **4px** to **32px** and do this with a smooth transition over 0.3 seconds. Similarly, change the back **left** property from **32px** to **4px** when the state changes from **on** to **off**.

For this animation to take effect, we need to trigger **toggle**. To do this, add a Boolean field **checked** to the component, as shown in *line 26* above. Now, add the following to the component template:

```
1. <div class="button">
2.     <input type="checkbox" class="checkbox" [(ngModel)]="checked" >
3.     <div class="knob" [@toggle]="checked ? 'on' : 'off'"></div>
4.     <div class="layer"></div>
5. </div>
```

You might notice that this is the same template that we previously had, except that the **checked** field is now bound to the **input** using **ngModel** in *line 2*. To enable this, you will need to add the **FormsModule** in **app.module.ts** file. Also, notice that in *line 3*, we are triggering the toggle animation using this **checked** field. Based on the **checked** value, the state **on** or **off** is set on the toggle trigger. If you click on the button now, you will notice that the knob moves back and forth smoothly as before. Thus, we achieved the same animation as we did using CSS before, this time using a declarative syntax in Angular.

This was a simple example to demonstrate how to use animation in Angular. You can find more complex samples in the Angular doc page provided in the references section below.

Animation can also be achieved using **keyframes**. We will skip that for brevity, but you can find example code in the reference links.

There are performance considerations in animation which we will discuss in detail in *Chapter 23, Performance and User Experience.*

Reusing animation

If you want to reuse animations across components, you can refactor the code to export the

trigger method from a common file. For instance, create a file **app/animations/trigger-toggle-animation.ts** and move the **trigger** code into this file as follows:

```
import { trigger, transition, useAnimation, state, style, animate } from '@angular/animations';

export const triggerToggleAnimation = trigger('toggle', [
  state('on', style({
    left: '32px'
  })),
  state('off', style({
    left: '4px'
  })),
  transition('on <=> off', animate('{{duration}} {{easing}}'), {
    params: {
      duration: '0.3s',
      easing: 'cubic-bezier(0.18, 0.89, 0.35, 1.15)'
    }
  })
]);
```

Now, you can use this **animation** function in metadata of **toggle-button.component.ts** as follows:

```
animations: [triggerToggleAnimation]
```

Similarly, if you want to reuse the **style** and **transition** across multiple triggers, the **animation()** function can be used as follows:

```
import { animation, style, animate } from '@angular/animations';

export const setLeftAnimation = animation([
  style({
    left: '{{ left }}'
  }),
  animate('{{duration}} {{easing}}')
]);
```

This can be used in the **trigger** method as follows:

```
import { trigger, transition, useAnimation } from '@angular/animations';
import { setLeftAnimation } from './set-left-animation';

export const triggerToggleAnimation = trigger('toggle', [
  transition('off => on', [
    useAnimation(setLeftAnimation, {
      params: {
        left: '32px',
        duration: '0.3s',
        easing: 'cubic-bezier(0.18, 0.89, 0.35, 1.15)'
      }
```

```
12.     })
13.   ])
14. ]);
```

Note on accessibility

We have covered accessibility in detail in *Chapter 19*. There may be a set of users who might find it difficult to work with a web app that has moving parts. So, whenever we use animation and transitions in a website, it is a good idea to have a version without animation and allow the user to select that option. This ensures a more widespread adoption. This can be achieved by adding a class like **no-transition** at a body level and disable all transitions using CSS as follows:

```
1. body.no-transition * {
2.   transition: none !important;
3. }
```

Conclusion

Animation is a powerful tool that can enhance the user experience and bring life to your web application. With the support of Angular's animation library, you can create smooth and fluid animations that seamlessly integrate with your application's logic. By leveraging the power of Angular's animation features, you can easily create engaging and dynamic user interfaces that are sure to leave a lasting impression on your users. In the next chapter, you will learn about micro frontends and their usages.

Points to remember

- Animation is a useful tool to make an impressive user experience.
- Angular allows declarative way to set animation on component elements.
- Animation can be reused across components using the trigger(), animation() and setAnimation() methods.

Questions

1. What are the advantages of using animation on a website?
2. Does animation cause any performance impact?
3. Explain how Angular allows to set animation in a declarative manner.

References

- Angular official doc on Animation:
  https://angular.io/guide/animations
- In depth examples of Animation:
  https://angularindepth.com/posts/1285/in-depth-guide-into-animations-in-angular

CHAPTER 21
Micro Frontends

Introduction

With the rise of microservices architecture, the concept of micro frontends has become increasingly popular among web developers. Micro frontends allow developers to build modular, scalable, and maintainable web applications by breaking down the user interface into smaller, independent parts. In this chapter, we will explore how Angular can be used to build micro frontends and discuss the benefits and challenges of this approach. We will also provide practical tips and best practices for implementing micro frontends using Angular.

Structure

The chapter covers the following topics:

- Introduction to micro frontend
- Implementing a micro frontend in Angular
- Working with different frameworks

Objectives

By the end of this chapter, you will understand micro frontends and how to implement micro frontends using Angular.

Introduction to micro frontend

A micro frontend is a web development pattern that breaks down a single web application into smaller, independent modules or features. Each module or feature is developed, tested, and deployed by a separate team using their own technology stack and tools. This allows for faster and more flexible development, as well as better scalability and maintainability of the web application.

Following are some of the benefits of using micro frontends:

- **Reduced complexity and increased productivity**: Each team can focus on its own domain and deliver features without worrying about dependencies or coordination with other teams.
- **Increased autonomy and innovation**: Each team can choose the best technology and tools for their module or feature and experiment with new ideas and solutions.
- **Improved performance and reliability**: Each module or feature can be optimized and tested independently and deployed or rolled back without affecting the rest of the web application.

There are some challenges of using micro frontends as well:

- **Increased communication and collaboration**: Each team needs to agree on the interfaces and contracts between their modules or features and ensure they are consistent and compatible.
- **Increased complexity and overhead**: Each module or feature needs to be integrated and composed into a coherent web application and handled by a common router, loader, and orchestrator.
- **Increased testing and monitoring**: One of the benefits we mentioned above is that each module and feature can be tested independently. However, that also brings in the challenge that they need to be tested and monitored not only in isolation but also in combination with other modules or features across

different browsers and devices. This increases the overall testing efforts.

Implementing a micro frontend in Angular

Implementing micro frontends using Angular involves breaking down the user interface into smaller, independent parts, which can be developed and deployed separately. Here are some steps to follow when implementing micro frontends using Angular:

1. **Identify the different modules of your application**: The first step is to identify the different modules of your application and break them down into smaller, independent parts. Each micro frontend should have its own codebase and dependencies.
2. **Use Angular elements**: Angular elements allow you to create standalone web components that can be used in any web application, regardless of the framework. You can use Angular elements to create micro frontends that can be easily integrated into your main application.
3. **Use Angular router**: Angular router is a powerful tool for managing the navigation of your application. You can use it to load different micro frontends based on the URL or route parameters.
4. **Use a shared state management library**: When building micro frontends, it is important to ensure that each micro frontend has access to the same shared state. You can use a shared state management library like NgRx or Akita to manage the state of your application.
5. **Use a common design system**: To ensure consistency across your micro frontends, it is important to use a common design system. This will help to ensure that the user interface looks and feels the same across all micro frontends.
6. **Deploy micro frontends independently**: Once you have developed your micro frontends, you can deploy them independently. This allows you to make changes to one micro frontend without affecting the rest of your application.

Working with different frameworks

Another case of a micro frontend is when you need to integrate a page written using another framework like React into your Angular application or vice versa. Such a case is handled by rendering the page within an **iframe** in an Angular app. To see this in action, let us create a React app and render it inside an Angular app. Run the following command to create a React app:

```
npx create-react-app my-app
```

Now replace **App.js** with the following code:

```
import './App.css';
import { useEffect, useState } from 'react';

function App() {
  const [message, setMessage] = useState("");

  useEffect(() => {
    window.addEventListener(
      "message",
      (event) => {
        const eventData = event.data;
        if (eventData && eventData.type === "SET_MESSAGE") {
          const data = eventData.data;
          setMessage(data.message);
        }
      },
      false,
    );
  }, []);
```

```
20.
21.   const sendMessage = () => {
22.     const message = "Hello from React app!";
23.     window.parent.postMessage(
24.       {
25.         type: "SET_MESSAGE",
26.         data: { message },
27.       },
28.       "*",
29.     );
30.   }
31.
32.   return (
33.     <div className="App">
34.       <h1>This is React app</h1>
35.       <h2>Message received: { message }</h2>
36.       <button onClick={sendMessage}>Send Message</button>
37.     </div>
38.   );
39. }
40.
41. export default App;
```

The above code maintains an internal state in the **message** field in *line 5*. It then listens for a **message** event on the **window** object on *line 8*. This event is raised by the browser when any other browser window sends a post message. When a message is received, this code checks for the message type as **SET_MESSAGE**. If received, it renders the received message on the page. It also defines a **sendMessage()** method on *line 21* to send a message at the click of a button.

If you run **npm start**, it will open the page on the browser at **http://localhost:3000/** as follows:

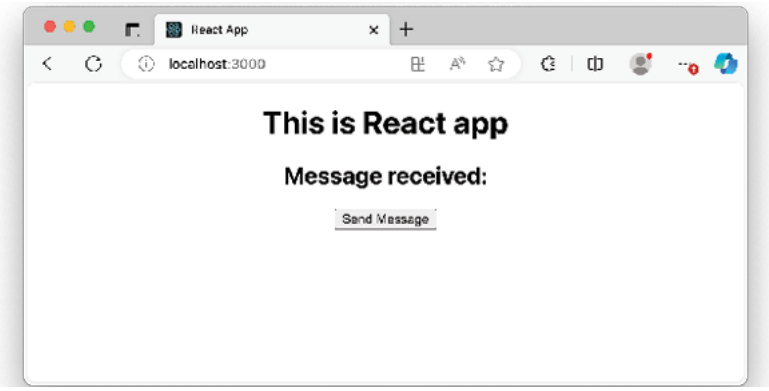


Figure 21.1: *React app*

Now create an Angular app and replace **app.component.html** file with the following code to show the React page inside the Angular app:

```
1. <div>
2.   <h1>This is Angular app</h1>
3.   <h2>Message received: {{ message }}</h2>
4.   <button (click)="sendMessage()">Send Message</button>
5.  <iframe id="react_app" src="http://localhost:3000" width="100%" height="100%" frameborder="1">
    </iframe>
6. </div>
```

Using an **iframe** is a standard way to show one website inside another. Replace **app.component.ts** with the following code:

```
1. import { Component, HostListener } from '@angular/core';
2.
3. @Component({
4.   selector: 'app-root',
5.   templateUrl: './app.component.html',
6.   styleUrls: ['./app.component.css']
7. })
8. export class AppComponent {
9.   title = 'micro-frontend';
10.   message?: string;
11.
12.   @HostListener('window:message', ['$event']) onPostMessage(event: any) {
13.     const eventData = event.data;
14.     if (eventData && eventData.type === "SET_MESSAGE") {
15.       const data = eventData.data;
16.       this.message = data.message;
17.     }
18.   }
19.
20.   sendMessage = () => {
21.     const iframe = document?.getElementById('react_app') as HTMLIFrameElement;
22.     iframe?.contentWindow?.postMessage({ type: 'SET_MESSAGE', data: { message: 'Hi, from Angular a
23.   }
24. }
```

This code also listens to incoming messages in *line 12*. It then validates the message type to be **SET_MESSAGE** and then prints it on the page. Similarly, it has a **sendMessage()** function to send a post message with a click of a button.

This will show in the browser at **http://localhost:4200/** as follows:

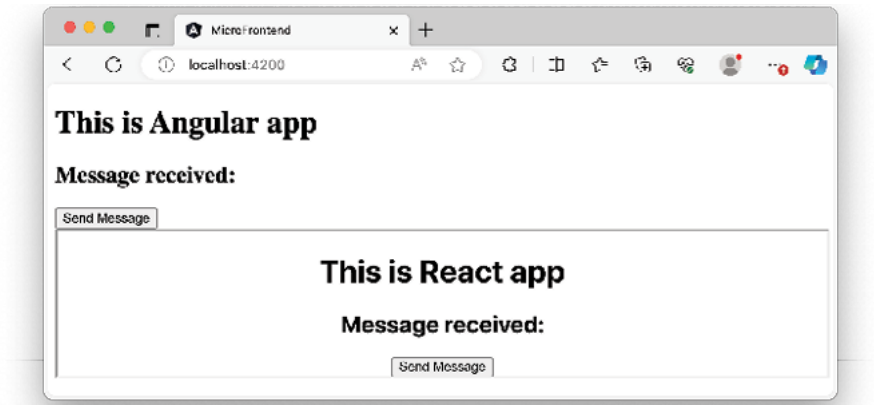


Figure 21.2: *React app embedded within Angular*

If you click on the **Send Message** buttons on both apps, you will notice the messages being received and shown on the other app as follows:

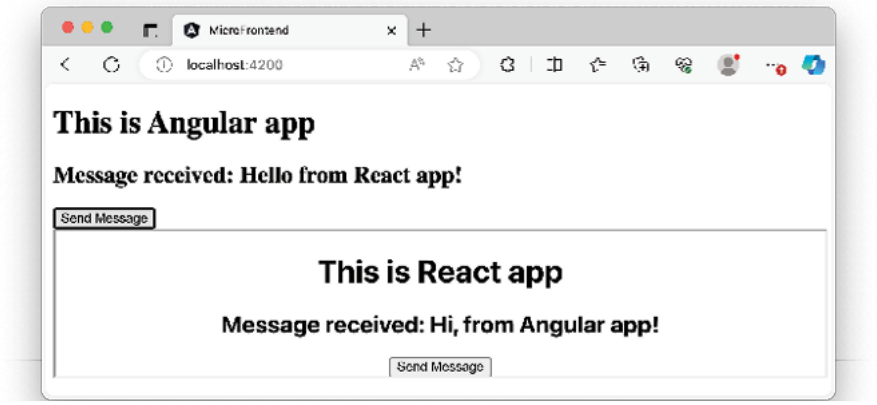


Figure 21.3: *React and Angular apps exchanging messages*

This way, we can embed apps within another app that can communicate with each other seamlessly. There are more options to ensure the security of the origin and message. You can find the parameters in the reference link.

Conclusion

Micro frontends offer a scalable, modular approach to building web applications. By breaking down the user interface into smaller, independent parts, you can develop and deploy each micro frontend separately, which helps to improve the overall performance and maintainability of your application. With Angular, you can take advantage of tools like Angular elements and Angular router to build micro frontends that seamlessly integrate with your main application. By following the steps outlined above and using best practices for micro frontend development, you can build a flexible, robust web application that can adapt and evolve over time. In the next chapter, you will learn about rendering charts in an Angular app.

Points to remember

- Micro frontends help to create different parts of the app by independent teams.
- Each team can decide to have its own tech stack for the front end.
- The teams need to coordinate well to make sure the apps have the same look and feel and are able to share a common state.
- Using window postMessage() API call, an app can communicate with another app embedded within an iframe.

Questions

1. What are the advantages and disadvantages of implementing a micro frontend?
2. Can micro frontends be created using separate JavaScript frameworks?
3. Are there any security concerns of using window postMessage?

References

- Browser message API:
  https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 22
Charts

Introduction

Charts are a crucial part of data visualization and play a significant role in making complex data easy to understand. Angular provides a wide range of tools and libraries to work with charts in our web applications, making it easier to create and use charts to display data in meaningful ways. Whether we need to create simple charts or complex ones with dynamic data, there are several charting libraries available for Angular that we can use to build visually appealing and interactive charts. This chapter will explore some of the popular charting libraries available for Angular and how we can use them to create stunning charts in our Angular applications.

Structure

The chapter covers the following topics:

- Choosing a charting library
- Bar chart using ng2-charts

Objectives

By the end of this chapter, you will understand how to create charts in an Angular app and the various charting libraries available. You will also learn how to select the best charting library for your needs.

Choosing a charting library

A chart helps to visually understand data. Many data driven applications, especially admin consoles and dashboards, use a lot of charts to make it easier to comprehend a data set. Angular does not have built-in components to render charts. However, there are many third-party libraries to choose from. Many of these are open source libraries, while some are proprietary and need a commercial license. The following parameters might help in selecting a library over another:

- **Functionality and chart types**:
  - **Available chart types**: Choose a library that supports the specific chart types you need (for example, bar charts, line charts, pie charts, etc.).
  - **Interactivity**: Consider whether the library provides interactive features like tooltips, zooming, and animations.
- **Ease of integration**:
  - **Angular compatibility**: Ensure the library integrates seamlessly with Angular components and services.
  - **Installation and setup**: Evaluate how straightforward it is to install and configure the library within your project.
- **Customization and styling**:
  - **Customization options**: Look for libraries that allow you to customize colors, labels, fonts, and other visual aspects.
  - **CSS flexibility**: Check if you can easily apply your own CSS styles to the charts.
- **Performance and efficiency**:
  - **Rendering speed**: Consider the library's performance when dealing with large datasets.
  - **Efficient updates**: Libraries that efficiently update charts when data changes are preferable.
- **Community support and documentation**:
  - **Active community**: Libraries with an active community tend to have better support, bug fixes, and updates.

    - **Documentation quality**: Good documentation makes it easier to learn and use the library effectively.
- **License and open source**:
    - **License type**: Ensure the library's license aligns with your project's requirements (for example, MIT, Apache, GPL, etc.).
    - **Open source**: Open source libraries are often preferred due to transparency and community contributions.
- **Accessibility and responsiveness**:
    - **Accessibility features**: Choose a library that adheres to accessibility standards (for example, ARIA attributes).
    - **Responsive design**: Consider how well the charts adapt to different screen sizes.
- **Data binding and real-time updates**:
    - **Data binding**: Evaluate how easily you can bind chart data to your Angular components.
    - **Real-time updates**: If your app requires real-time data updates, check if the library supports it.
- **Dependencies and bundle size**:
    - **Dependency tree**: Analyze the library's dependencies and ensure they don't bloat your app.
    - **Bundle size**: Smaller bundle sizes lead to faster page loads.
- **Additional features**:
    - **Annotations**: Some libraries allow adding annotations (for example, markers, lines) to charts.
    - **Exporting charts**: Consider whether the library supports exporting charts as images or PDFs.

While a side-by-side analysis of all charting libraries is outside the scope of this book, the above points should help you pick a charting library that works best for you. If you want to get started with a library that is open sourced and has good community support, Chart.js is your best option. It is the most popular JavaScript charting library, with over 62k stars on GitHub at the time of writing this book. If you want a library that

provides a variety of charts and good technical support, try Highcharts, which is licensed. We have provided links to all these libraries in the reference section at the end of this chapter. Many libraries create an Angular wrapper to make it easier to create charts, for example, **ng2-charts** and **highcharts-angular**, respectively, for the above two charting libraries we discussed. We will look at a code sample to create a bar chart in the next section using **ng2-charts** library.

Bar chart using ng2-charts

Every charting library has its own custom steps to represent chart data and render a chart in the Angular app. Once you decide on the library to use, please refer to the documentation page of the charting library for sample code to create charts. To give you an introduction to charting, we will provide the steps to create a simple bar chart using **ng2-charts** which is a wrapper over the popular **chart.js** library.

Let us say, we want to create a dashboard for the admin of our fruit store to see the sales figure of Apples vs Oranges year over year. Assuming there is an API endpoint that returns this data in a JSON format.

To create an admin dashboard that will display this data in a bar chart, create a new angular project as follows:

ng new fruit-store-dashboard

Add **ng2-chart** and **chart.js** library as follows:

npm install ng2-charts chart.js

Add a component to show the sales bar chart:

ng g c sales-bar-chart

Open the **sales-bar-chart.component.ts** file and import the **chart.js** library at the top. Then, add code to render the chart. We will assume hard coded sales data for now. Also, add events to handle click and mouse hover events on the charts, as follows:

```
1. import { Component } from '@angular/core';
2. import { ChartConfiguration, ChartEvent } from 'chart.js';
3.
4. @Component({
5.   selector: 'app-sales-bar-chart',
```

```
  templateUrl: './sales-bar-chart.component.html',
  styleUrls: ['./sales-bar-chart.component.css']
})
export class SalesBarChartComponent {
  public barChartLegend = true;
  public barChartPlugins = [];

  public barChartData: ChartConfiguration<'bar'>['data'] = {
    labels: ['2018', '2019', '2020', '2021', '2022', '2023', '2024'],
    datasets: [
      { data: [60, 68, 42, 51, 56, 68, 72], label: 'Apples' },
      { data: [29, 43, 23, 30, 41, 63, 95], label: 'Oranges' }
    ]
  };

  public barChartOptions: ChartConfiguration<'bar'>['options'] = {
    responsive: false,
  };

  public chartClicked({ event, active}: { event?: ChartEvent;
  active?: object[];}): void {
    console.log(event, active);
  }

  public chartHovered({event, active}: { event?: ChartEvent;
  active?: object[];}): void {
    console.log(event, active);
  }
}
```

Now add a canvas element in the **sales-bar-chart.component.html** file to render the chart as follows:

```
<canvas baseChart
        [data]="barChartData"
```

```
3.        [options]="barChartOptions"
4.        [plugins]="barChartPlugins"
5.        [legend]="barChartLegend"
6.        [type]="'bar'"
7.        (chartHover)="chartHovered($event)"
8.        (chartClick)="chartClicked($event)">
9.  </canvas>
```

This will render a nice bar chart, as shown below:

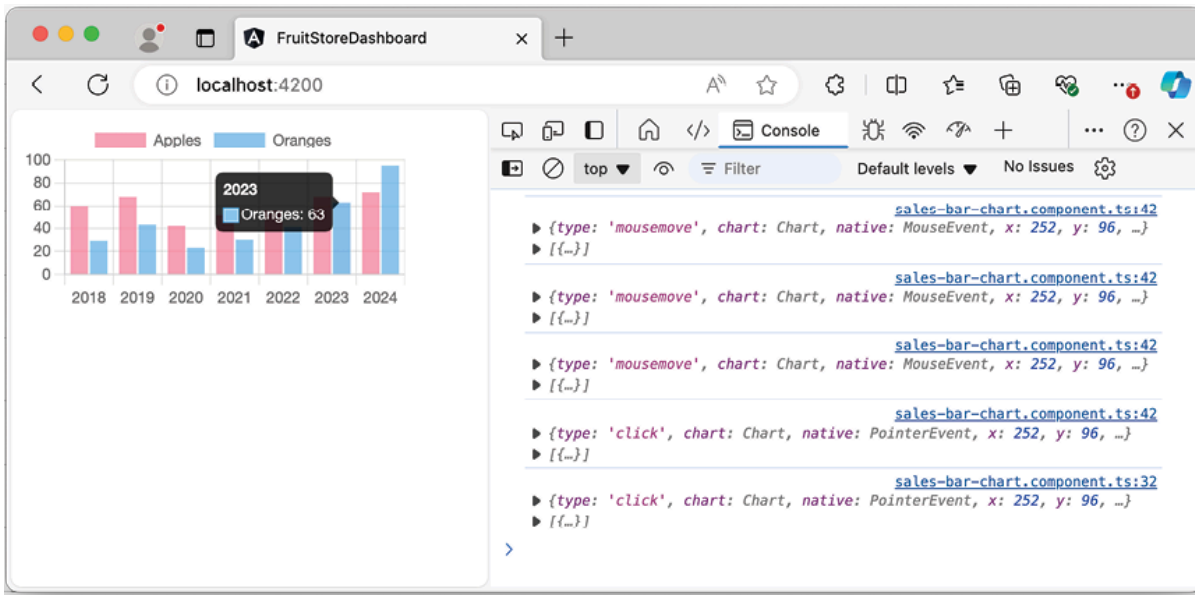


Figure 22.1: *Bar chart example*

The chart will show labels when the user hovers mouse over the bar elements. Open the console and observe the mouse hover and click events logged. We can add interactivity to the chart by handling these events. Similarly, you can add other chart types like line, bar, doughnut, radar, pie, polar area, bubble, scatter charts, etc. Please refer to the **ng2-charts** home page link in the references section for sample code for all supported charts. We only demonstrated using **ng2-charts** library. However, you can select from any other library that works best for your project.

Conclusion

Charts are a powerful tool for data visualization, and Angular provides a wide range of libraries for creating and using them in web applications. By choosing the right library based on the parameters discussed above,

we can create visually appealing and interactive charts that help us understand complex data sets. With the sample code provided for creating a bar chart using **ng2-charts** and **chart.js**, you can get started on creating your own charts in Angular. In the next chapter, you will learn about how to improve the user experience of your app by fine tuning app performance.

Points to remember

- Charts are an essential part of data visualization.
- There are several charting libraries available for Angular, both open source and commercial.
- When choosing a charting library, consider factors such as supported chart types, interactivity, ease of integration, customization and styling options, performance and efficiency, community support, license, accessibility, responsiveness, data binding and real-time updates, dependencies and bundle size, and additional features such as annotations and exporting charts.
- Chart.js is a popular open source library with good community support, while Highcharts is a licensed library with a variety of charts and good technical support.
- Many charting libraries have an Angular wrapper to make it easier to create charts, such as ng2-charts and highcharts-angular.

Questions

1. Why are charts important in data visualization?
2. Does Angular have built-in components to render charts? If not, then what are the options available?
3. What are the parameters to consider while selecting a charting library for an Angular application?
4. What are some of the popular charting libraries available for Angular?
5. What is ng2-charts and how can it be used to create bar charts in an Angular application?

References

- Chart.js:
  https://www.chartjs.org/docs/latest/samples/information.html
- ng2-charts:
  https://valor-software.com/ng2-charts/
- Highcharts for Angular:
  https://www.highcharts.com/blog/tutorials/highcharts-with-angular-v14/
- PrimeNG charts:
  https://primeng.org/chart
- angular-plotly.js:
  https://www.npmjs.com/package/angular-plotly.js
- ngx-charts:
  https://swimlane.github.io/ngx-charts/#/ngx-charts/bar-vertical
- ngx-echarts:
  https://xieziyu.github.io/ngx-echarts/#/basic/basic-usage

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 23
Performance and User Experience

Introduction

In today's competitive digital landscape, ensuring optimal performance and an exceptional user experience are paramount for the success of any web application. This chapter delves into the critical aspects of optimizing Angular applications for peak performance and enhanced user satisfaction. We will explore a wide array of strategies, from fine-tuning Angular's change detection mechanism to implementing lazy loading. Moreover, we will discuss how these performance enhancements directly correlate with improved user experience, lower bounce rates, and higher engagement metrics. The chapter also introduces load testing to ensure the end-to-end performance of an application under high user traffic.

Structure

This chapter covers the following topics:

- Achieving 60fps
- The RAIL model
- Measuring website performance
- Lazy loading and preloading
- Angular change detection optimization
- Using observables with OnPush
- Using pure pipes for heavy computation
- Optimize animations
- Using virtual scrolls
- Optimize *ngFor in template
- Optimize image lazy loading
- Load testing

Objectives

By the end of this chapter, you will learn the various strategies for improving the page load and render performance of an Angular application.

Achieving 60fps

The human eye and brain can process up to 60 distinct images per second. At 60fps, motion appears smooth and fluid to the human eye. This creates a more natural and pleasing visual experience for users, making interactions feel responsive and seamless. Achieving 60 **frames per second** (**fps**) is a crucial goal in web application development. When an application runs at 60fps, there is a new frame every 16.67 milliseconds (1000ms / 60). This means that the delay between a user's action and the screen's response is minimal, creating a sense of immediacy and responsiveness. A lower than 60fps framerate may lead to a degraded user experience where animations and page updates are noticeably janky, and interactions feel slow. For touch-based devices and scrolling interactions, 60fps is particularly important. It ensures that touch movements and scrolling appear smooth and track accurately with the user's finger or mouse movements.

The RAIL model

Google published an article on the **Response, Animation, Idle, Load** (**RAIL**) model, a gold standard in user experience improvement. It shows the level of involvement of a user in a web page load timeline. Refer to the following table:

Time span	User expectation
0 to 16ms	Users are highly sensitive to motion quality in animations. For perceived smoothness, aim for 60fps (approximately 16ms per frame). The browser's rendering time leaves about 10ms for your app to produce each frame.
0 to 100ms	The web page should respond to user actions within 100ms to make it feel responsive.
100 to 1000ms	Page load and view changes should happen within 1000ms or 1 second. Within this window, task progression appears to be natural.
Beyond 1000ms	Users tend to lose focus on the page.
Beyond 10,000ms	User starts to feel frustrated and may not reload or return to the page.

Table 23.1: *User expectations as per page load time*

This model suggests that JavaScript should complete within 10ms for any user interaction, leaving 6ms for the browser to render the page. This will ensure that the page loads within 16ms, or, in other words, achieves 60fps. We have provided a link to the blog to check for detailed analysis.

Measuring website performance

Most browser dev tools have a profiler that detects rendered fps for a given site. The **Performance** tab on the dev tools of Chromium based browsers like Chrome and Edge can allow you to profile a page and show the division of various activities like loading, scripting, rendering, painting, system, and idle. Refer to the following figure:

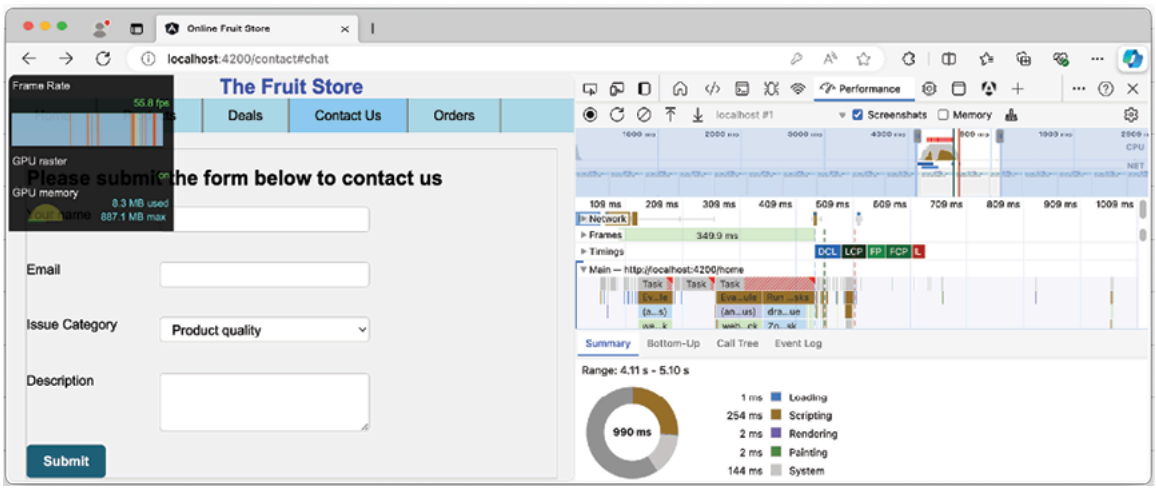


Figure 23.1: *Performance tab in dev tool with FPS monitoring*

The tool shows a flame chart indicating the time consumed by each phase of the render cycle. This helps us look for long-running scripts that we can optimize. Since JavaScript is single threaded, no function must be performed for a long time. That could block the UI or make it janky. You can see an interactive **Frame Rate** Monitor panel by pressing *Ctrl+Shift+P* (or *Cmd+Shift+P* on Mac) and typing **Show FPS meter**. A thorough exploration of the **Performance** tab features is beyond the scope of this book. There are detailed blogs on the subject that we have added to the reference section.

Also, tools like **Lighthouse** can give us a comprehensive report of website performance issues. It is recommended that these reports be run on a staging or production site, if available, instead of on localhost. Localhost runs on development web servers, which may not be that performant, and hence, the report will not be accurate. Refer to the following figure:

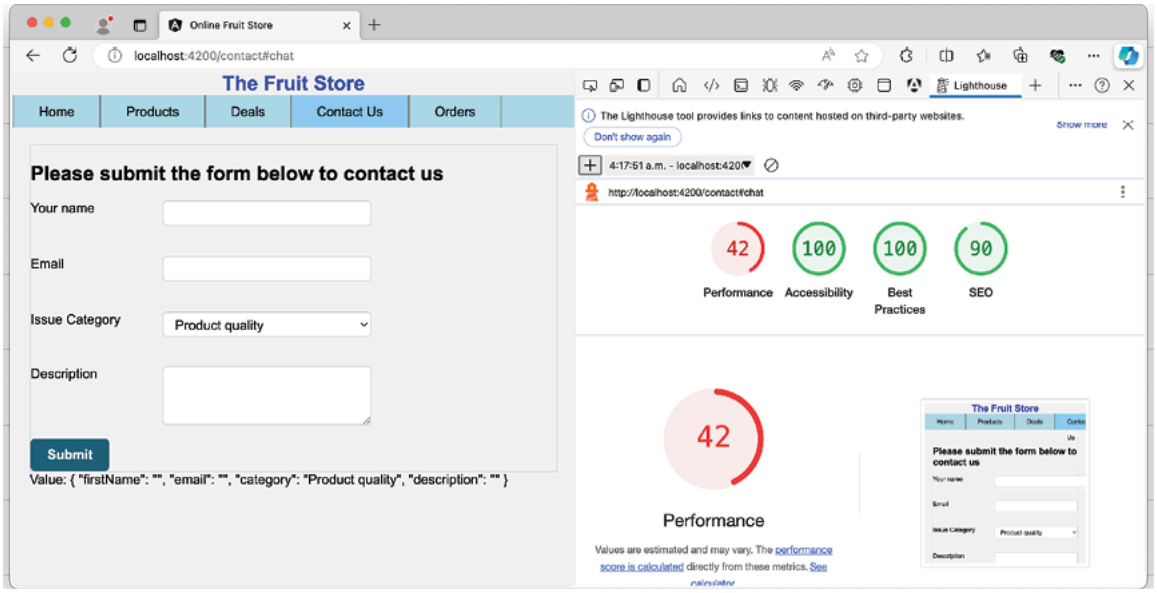


Figure 23.2: *Lighthouse report in Chromium dev tool*

Similarly, there are other tools available, like *WebPageTest*, to measure page performance.

Lazy loading and preloading

By default, all Angular modules are eagerly loaded. That is, the entire application is loaded when the root module loads. This is fine for smaller apps, but for a large scale application, this may add a considerable load time for the initial page. Often, the user may not even visit all the pages in your application. However, an eagerly loaded application will still load all such modules. We covered lazy loading and preloading in *Chapter 5, Routing*. Lazy loading loads a module on a need basis, and preloading fetches the modules while the user interacts with the application. A combination of lazy loading and preloading ensures that the initial page load is fast, and yet the lazily loaded pages are rendered soon on user action.

Angular change detection optimization

Angular's change detection mechanism is at the core of its performance. Understanding and optimizing this process can significantly improve your application's efficiency. Change detection is how Angular determines if the view needs to be updated based on changes in the component's input or computed data. By default, Angular uses a strategy called **Zone.js** to automatically trigger change detection for all components whenever any event occurs (like user interactions, timers, or HTTP requests). There are two change detection strategies in Angular — Default and OnPush. By default, Angular detects changes for the entire component tree whenever changes are made. It traverses the application component hierarchy from top to bottom. If a change occurs, it updates the DOM and renders the corresponding view. So, a change in the parent component model will result in a change detection within the child components. The only way to avoid the change detection in the child component is to detach it as follows:

```
@Component(...)
export class ChildComponent {
  constructor(private cd: ChangeDetectorRef) {
    cd.detach();
  }
}
```

Let us explore how change detection works with an example code. Create a new Angular application and add two components, parent and child, as follows:

```
@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [ChildComponent],
```

```
  template: `
    <button (click)="changeFruitName()">Change fruit name</button>
    <br />
    <app-child [fruit]="fruit"></app-child>
  `,
})
export class ParentComponent {
  fruit:Fruit = { name: 'Apple' };

  changeFruitName() {
    this.fruit.name = 'Orange';
  }
}
@Component({
  selector: 'app-child',
  standalone: true,
  imports: [],
  template: `<span>Fruit name: {{fruit?.name}}</span>`,
})
export class ChildComponent {
  @Input() fruit: Fruit | undefined;
}
```

The **Fruit** interface has a **name** field defined as follows:

```
export interface Fruit {
    name: string | undefined;
}
```

In the above code, the parent sets the **fruit.name** field and sets the **fruit** object as an input on the child component **<app-child>**. When clicking the button, the parent component changes the **name** field of the **fruit**. The reference of the **fruit** object remains the same, and there is no way for Angular to know if the **name** field is modified unless it does a deep scan within the **fruit** object. Hence, by default, Angular will trigger change detection in all children components to update the view as per the changes within the input fields. Run the app and observe that the fruit name shows as **Apple** as initialized by the parent component:

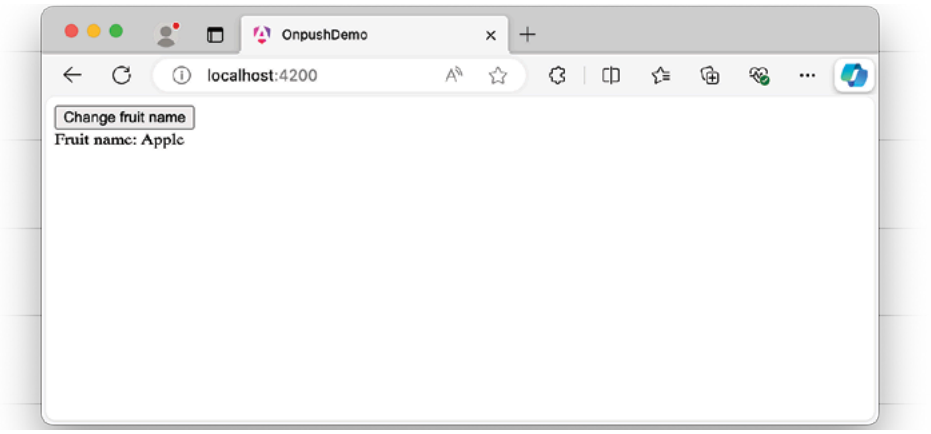


Figure 23.3: *Initial fruit name Apple passed from parent to child component*

Now click the button and observe the name changes to **Orange** in the child component, as shown in the following figure:

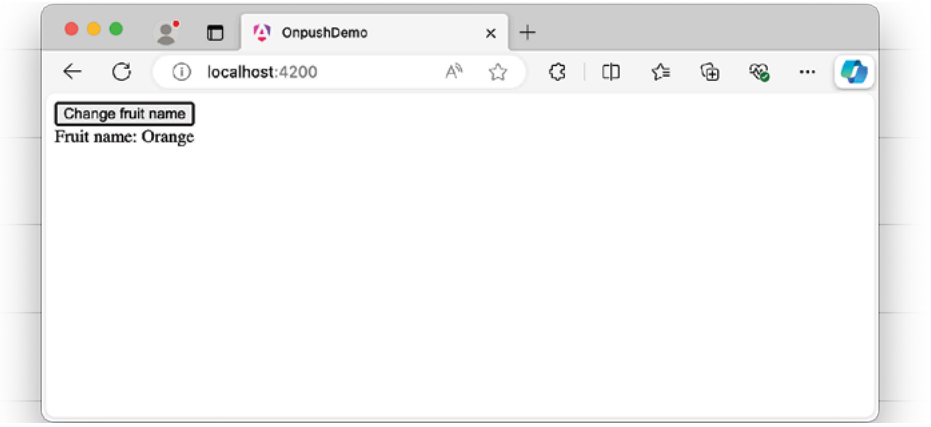


Figure 23.4: *Fruit name changed to Orange by parent and rendered in the child component*

Although the **name** field is within the **fruit** object, by default, Angular change detection figured it out and updated it in the child component. The default change detection mode, internally known as CheckAlways, is costly, especially for complex input objects.

The alternate strategy is **OnPush**, which can be set in the component definition as follows:

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush
})
```

OnPush is internally known as CheckOnce, which implies that change detection is skipped unless the component is marked as **dirty**. Angular has mechanisms to auto set the **dirty** flag on components.

The **OnPush** strategy allows you to optimize change detection by telling Angular to check for changes only when:

- An input property of the component changes.
- An event originates from the component or one of its children.
- An observable linked to the template via the async pipe emits a new value.

Add this strategy to the child component and observe that the fruit name no longer changes when the button is clicked. Using this strategy, Angular will initially mark a component as **dirty** and unset it after the first change detection passes. It will mark it **dirty** again when the reference of the input object changes. To change the object reference, we must create a new **fruit** object every time the **name** field changes. In the parent component, modify the **changeFruitName** method to create a new **fruit** object every time the name is changed, as follows:

```
changeFruitName() {
  this.fruit = {
    ...this.fruit,
    name: 'Orange'
  };
}
```

Now, when the button is clicked, the fruit name changes again within the child component. This is a much faster change detection method, as Angular does not have to do a deep scan anymore to find changes within object properties.

Using immutable data structures complements the **OnPush** strategy. When you use immutable objects, you can quickly determine if a reference has changed, which is what **OnPush** looks for. Libraries like **immer** and **immutable** can create immutable objects so that every time a field value needs a change, a new object is created.

Although using **OnPush** is a performant strategy to detect changes, one has to be careful while using it, as Angular might fail to detect a change in the view model if the object reference is not modified. There might be cases where you want to use **OnPush** and yet not update the object reference on field value change. For such cases, you can manually mark a component as dirty so that Angular will pick it up in the next change detection cycle.

To demo this, let us rollback the parent component method to the following:

```
changeFruitName() {
  this.fruit.name = 'Orange';
}
```

Observe that, as expected, the fruit name does not change anymore with a click of the button. Now add **ngDoCheck** method in the child component as follows:

```
export class ChildComponent {
  @Input() fruit: Fruit | undefined;
  previousName: string | undefined = '';

  constructor(private cd: ChangeDetectorRef) {}

  ngDoCheck() {
    if (this.fruit?.name !== this.previousName) {
      this.cd.markForCheck();
      this.previousName = this.fruit?.name;
    }
  }
}
```

ngDoCheck is called on every change detection cycle. So, when using the **OnPush** change detection strategy, it is an opportunity to mark the component dirty. This tells Angular to run change detection on the component in the next cycle. Notice that **markForCheck** neither triggers nor guarantees a change detection run. Refresh the browser and observe that on clicking the button, the fruit name changes again.

Using observables with OnPush

AsyncPipe internally uses **markForCheck**:

```
export class AsyncPipe {
  transform() {
    if (obj) {
      this._subscribe(obj);
    }
  }

  private _updateLatestValue(async, value) {
    if (async === this._obj) {
      this._latestValue = value;
      this._ref!.markForCheck();
    }
  }
}
```

Hence, when using **OnPush**, it is safer to use observable inputs with async pipes. This ensures that change detection is triggered when the input value changes. Our child component code will look as follows with observable and async pipe:

```
@Component({
  selector: 'app-child',
  template: `<span>Fruit name: {{(fruit$ | async).name}}</span>`,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ChildComponent {
  @Input() fruit$;
```

```
}
```

For this reason, libraries like RxJS work very well with **OnPush**. Similarly, state management libraries like NgRx, which uses observables to track state, are safe and recommended to use with **OnPush**.

Using pure pipes for heavy computation

One way to improve component render performance is to use pure pipes wherever possible. Pure pipes are only re-evaluated when their inputs change, which can significantly reduce computation overhead. Refer to the following:

```
@Pipe({
  name: 'ExpensiveOperation',
  pure: true
})
export class ExpensiveOperationPipe implements PipeTransform {
  transform(value: any): any {
    return heavyComputation(value);
  }
}
```

Optimize animations

Another technique to improve performance is to use Angular's built-in animation wherever needed. This is optimized for performance. An example code for a component that animates on state change is as follows:

```
@Component({
  selector: 'app-slide-in',
  template: `<div [@slideIn]="state">This will slide in</div>`,
  animations: [
    trigger('slideIn', [
      state('in', style({transform: 'translateX(0)'})),
      transition(':enter', [
        style({transform: 'translateX(-100%)'}),
        animate('200ms ease-in')
        ])
    ])
  ]
})
export class SlideInComponent {}
```

Using virtual scrolls

Angular CDK provides a powerful technique called virtual scrolling that renders only visible items. This can render large lists very efficiently. To install CDK, run the following:

```
npm install @angular/cdk
```

Now add the **ScrollingModule** in the **app.module.ts** file as follows:

```
import { ScrollingModule } from '@angular/cdk/scrolling';

@NgModule({
  imports: [
    ...
    ScrollingModule,
  ],
```

```
8. })
9. export class AppModule {}
```

Assuming **largeList** field is a list with several items, say more than 1000, add the following code in the template to render using virtual scroll:

```
1. <cdk-virtual-scroll-viewport itemSize="40">
2.   <div *cdkVirtualFor="let item of largeList">{{item}}</div>
3. </cdk-virtual-scroll-viewport>
```

Optimize *ngFor in template

By default, when you use ***ngFor** to iterate over an array, Angular tracks each item by its object reference. This means that even if the values within the array remain the same, Angular will re-render the entire list if the array reference changes. This can be inefficient, especially when dealing with large lists. To address this, we can use the **trackBy** feature. First, identify a unique field on your array items, say the **name** field in our fruit item. It could be an ID or any other property that uniquely identifies each record. Implement a method in your component that takes an index and an item as parameters and returns the unique identifier for that item:

```
1. getProductId(index: number, fruit: Fruit) {
2.   return fruit.name;
3. }
```

In your template, add the **trackBy** directive to your ***ngFor** loop, passing the name of the method you created:

```
1. <div *ngFor="let product of products; trackBy: getProductId">{{product.name}} - {{product.price}}</
```

Now, when you update the list (even if it is the same list), Angular will only re-render the items that have changed, resulting in better performance. No more unnecessary DOM redraws!

Optimize image lazy loading

A *web.dev* article from *Google* provides a lot of insights into how to best lazy load images on a page. We have added a link to the article in the references section. One of the low-hanging fruits is to specify the **loading** attribute as **lazy** or **eager** on the **img** tag. Also, it is best to provide image dimension as per the render dimension so as not to fetch a big image on smaller screens.

```
1. <img src="image.png" loading="lazy" width="100" height="100">
```

Load testing

Load testing is crucial for ensuring that your Angular application can handle the expected (and unexpected) levels of user traffic and data processing. While Angular itself runs on the client side, load testing typically focuses on how the entire application stack—including the backend services that your Angular app interacts with—performs under various levels of load.

The primary goals of load testing are:

- Determine the maximum operating capacity of the application.
- Identify bottlenecks before the application goes live.
- Ensure the application can handle expected peak load conditions.
- Measure the response times for various operations under different load conditions.

For Angular applications, load testing often involves simulating multiple users interacting with the application simultaneously, which stresses both the frontend and backend components of your system. We will not go into details about how to write load tests, but we encourage you to look at tools like Cypress and Selenium WebDriver for instructions on performing end-to-end load tests on Angular.

Conclusion

In conclusion, this chapter has highlighted the critical aspects of optimizing Angular applications for peak performance and improved user experience. We have explored various strategies, from achieving 60fps to utilizing the RAIL model, measuring website performance, and leveraging techniques such as lazy loading and preloading. Additionally, we examined the importance of Angular's change detection optimization and its

impact on overall performance. By implementing the strategies discussed in this chapter, developers can significantly enhance the page load and render performance of their Angular applications, ultimately resulting in a more satisfying user experience.

In the next chapter, we will explore how to create offline applications using Angular.

Points to remember

- Achieving 60fps is crucial for creating a smooth and fluid visual experience for users.
- The RAIL model sets standards for user interaction and page load timeline.
- Use the Performance tab in browser dev tools to measure website performance and identify areas for optimization.
- Consider lazy loading and preloading to improve the initial page load time and enhance user experience.
- Use OnPush change detection strategy to optimizing Angular's change detection mechanism.

Questions

1. Why is achieving 60 fps important in web application development?
2. What is the RAIL model, and how does it contribute to user experience improvement?
3. How can browser dev tools be used to measure website performance and identify optimization opportunities?
4. Explain how the OnPush change detection strategy works.
5. Explain how to render a large list of items using the virtual scroll.

References

- Measuring performance with RAIL model:
  https://web.dev/articles/rail
- Discover issues with rendering performance:
  https://developer.chrome.com/docs/devtools/rendering/performance/
- Browser-level image lazy loading for the web:
  https://web.dev/articles/browser-level-image-lazy-loading

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 24
Handling Offline Mode with Progressive Web App

Introduction

In today's world, where internet connectivity is almost ubiquitous, creating offline applications may seem like a thing of the past. However, there are still many scenarios where offline capabilities can greatly improve the user experience and provide better reliability. With the help of Angular, it is possible to create robust offline applications that can work seamlessly, even without an internet connection. In this chapter, we will explore the different techniques and strategies that you can use to build offline applications using Angular. We will cover everything from caching data to handling network failures and syncing data when the connection is restored.

Structure

This chapter covers the following topics:

- Understanding offline capabilities
- Offline-first approach
- Service workers
- Progressive Web Apps
- Background Sync and custom service worker

Objectives

By the end of this chapter, you will understand how to create offline apps with Angular.

Understanding offline capabilities

Offline capabilities are essential for applications that need to work seamlessly, even without an internet connection. There are many scenarios where users may not have access to the internet, such as when traveling in remote areas or areas with poor connectivity. In such situations, offline capabilities can significantly enhance the user experience by providing uninterrupted access to the app's features and data. Additionally, offline capabilities can improve the app's reliability by reducing the dependence on network connectivity, which can sometimes be unreliable.

One approach to building offline applications is the offline-first approach. With this approach, the app is designed to work offline by default, and all data and functionality are available even without an internet connection. When the app is online, it syncs data and updates with the server, ensuring the data is always up to date. This approach not only provides a better user experience but also simplifies the app's architecture and reduces the complexity of handling network connectivity issues.

Following are some of the advantages of an offline-first approach:

- **Uninterrupted access**: Imagine a user traveling on a train with spotty network coverage or sitting in an elevator with no Wi-Fi. An offline app remains usable even without a reliable network connection. It presents users with local data immediately, without waiting for network calls to complete.
- **Seamless user experiences**: Users need consistent app experiences, regardless of connectivity. Offline-first apps ensure that essential features work seamlessly even when offline. Users can rely on consistent functionality, enhancing productivity.
- **Enhanced user engagement**: Users want to engage with your app continuously, even during internet downtime. Offline mode facilitates continual engagement, driving user loyalty. Users can access important data anytime, anywhere.

- **Data safety and accessibility**: What if the internet is temporarily unavailable? Offline data storage ensures that critical data remains accessible. Users can work on documents, view stored information, or use app features without internet connectivity.

Offline-first approach

The offline-first approach prioritizes local functionality over network dependency. Design your app to work offline by default. Users should be able to access essential features even without an internet connection. An offline-first app typically has two data sources per repository:

- **Local data source**: The canonical source of truth. Provides data even without network access.
- **Network data source**: Fetches data from external sources when possible.

Start by designing the data layer. Ensure that at least one data source (usually local) does not require network access for critical tasks. This allows the app to start even without a network connection. When online, sync changes are made offline with the network. Prevent data loss by automatically updating once connectivity is restored. To provide a better user experience, it is important to consider battery and data optimization since offline usage is mostly on mobile devices. Fetch data consciously, considering battery status and data availability.

Service workers

Service workers are JavaScript files that run in the background of an application, separate from the website or app. They are responsible for handling network requests, caching data, and managing offline functionality. They function as a network proxy, intercepting all outgoing HTTP requests made by the application and deciding how to respond to them. The following diagram provides a high-level flow of network calls made through service workers:

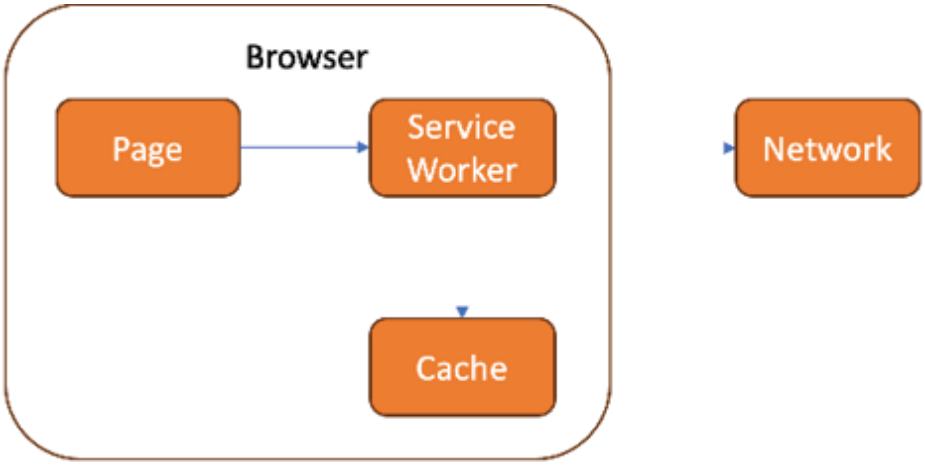


Figure 24.1: *Network call made using a service worker*

Service workers can query a local cache and deliver a cached response if available, even for resources referenced in HTML. Unlike other scripts in your application, the service worker persists after the user closes the tab. When the browser reloads the application, the service worker loads first and can intercept requests for resources, potentially satisfying them without relying on the network. Starting with Angular version 5.0.0, Angular ships with its own service worker implementation. Developers can take advantage of this service worker without needing to code against low-level APIs.

Progressive Web Apps

A **Progressive Web App** (**PWA**) is a type of web application that uses modern web technologies to provide an app-like experience to users. PWAs are designed to be reliable, fast, and engaging, and can be accessed through any web browser, regardless of the device or platform.

PWAs have several attributes that set them apart from traditional web applications:

- **Installable**: PWAs can be installed on a user's device and accessed from the home screen like a native app.
- **Responsive**: PWAs are designed to work on any device or platform, including desktop, mobile, and tablet.
- **Reliable**: PWAs can work offline or with limited connectivity, providing a seamless user experience regardless of network conditions.
- **Fast**: PWAs are designed to load quickly and respond instantly to user

interactions.

- **Engaging**: PWAs provide an immersive user experience with features like push notifications, offline access, and home screen icons.

Let us make our fruit store app that we created in *Chapter 6, Styling*, a PWA. To start with, make a copy of the fruit store folder as **fruit-store-pwa** and serve it on localhost. Open the **Products** page on the browser as follows:

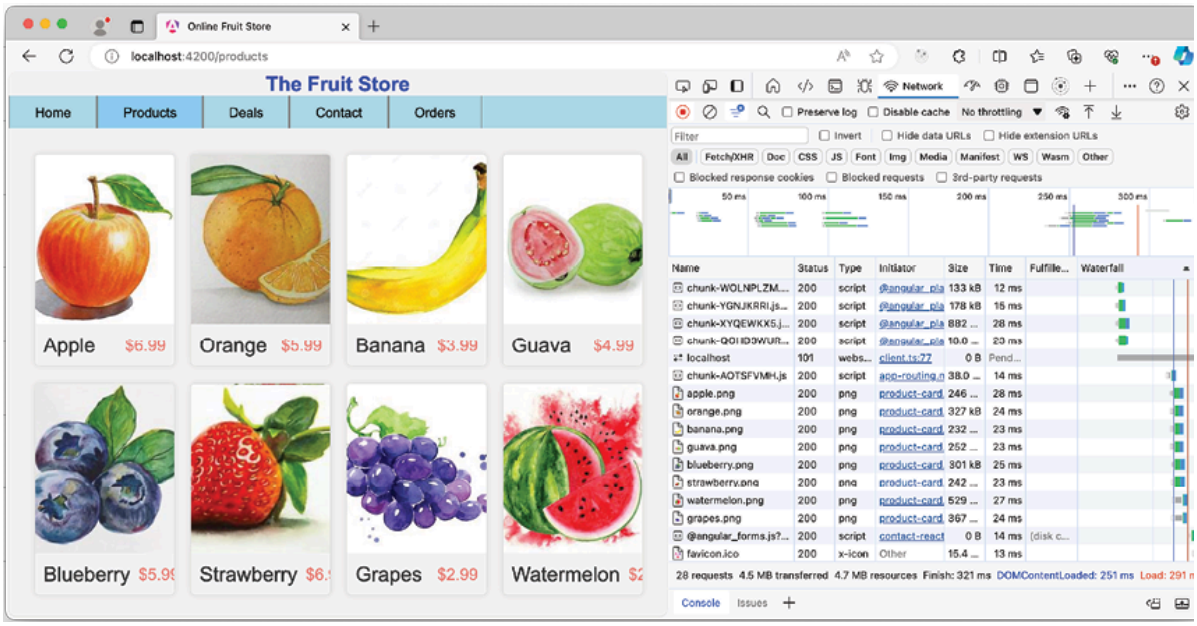


Figure 24.2: *Fruit store app Products page*

Observe that the page, including the images, is fetched from the localhost server. Now, click on the drop-down in the **Network** tab on the dev tool and select **Offline** as follows:

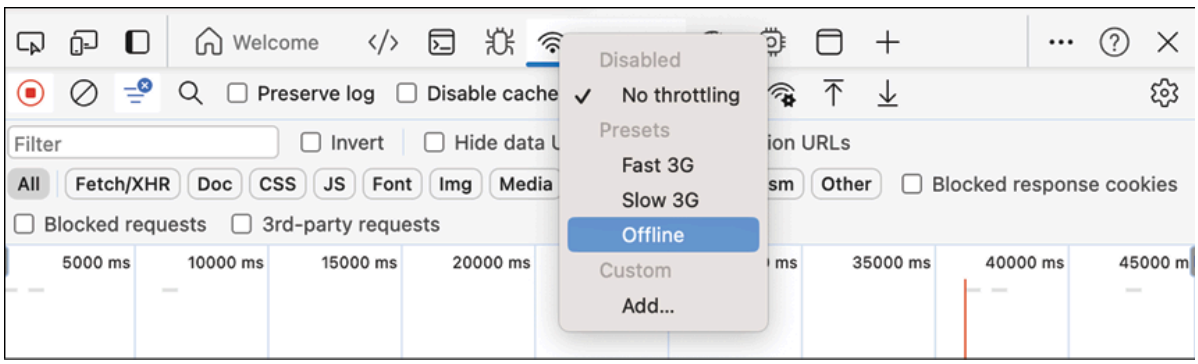


Figure 24.3: *Setting Offline mode in the browser dev tool*

If you try to refresh the page, you should see a Page not available error as follows:

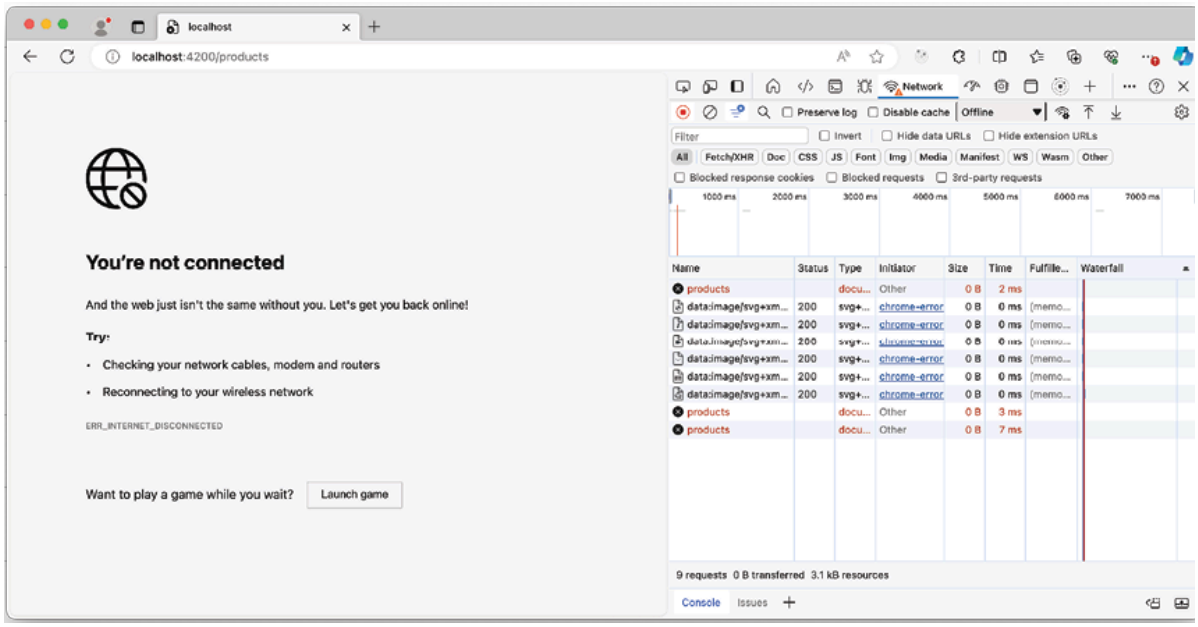


Figure 24.4: *Fruit store does not work in offline mode*

Go to the project folder and run the following command in the terminal:

ng add @angular/pwa --project fruit-store-pwa

This will add and update the following files:

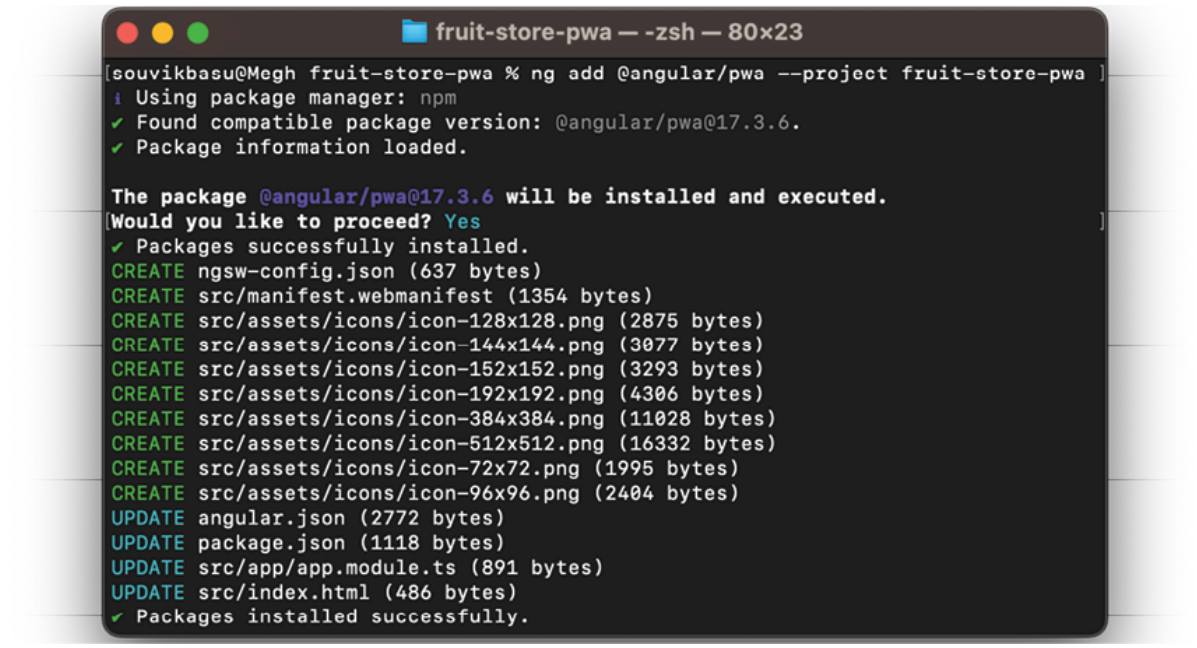


Figure 24.5: *Adding PWA package into the fruit store app*

A new **ngsw-config.json** file is added. This is a config file used by the in-built service worker file in Angular. This file defines all the assets that the app will need to cache:

1. {

```
2.  "$schema": "./node_modules/@angular/service-
    worker/config/schema.json",
3.   "index": "/index.html",
4.   "assetGroups": [
5.    {
6.     "name": "app",
7.     "installMode": "prefetch",
8.     "resources": {
9.      "files": [
10.        "/favicon.ico",
11.        "/index.html",
12.        "/manifest.webmanifest",
13.        "/*.css",
14.        "/*.js"
15.      ]
16.     }
17.    },
18.    {
19.     "name": "assets",
20.     "installMode": "lazy",
21.     "updateMode": "prefetch",
22.     "resources": {
23.      "files": [
24.        "/assets/**",
25.  "/media/*.
    (svg|cur|jpg|jpeg|png|apng|webp|avif|gif|otf|ttf|woff|woff2)"
26.      ]
27.     }
28.    }
29.   ]
30. }
```

The first group of assets is named **app**. It contains the **index.html**, favicon, CSS, and JS files. It also contains a **manifest.webmanifest** file

that is needed to install the app on a device. The second group of **assets** contains all the media and font files.

The **manifest.webmanifest** file looks like the following:

```
{
  "name": "fruit-store-pwa",
  "short_name": "fruit-store-pwa",
  "theme_color": "#1976d2",
  "background_color": "#fafafa",
  "display": "standalone",
  "scope": "./",
  "start_url": "./",
  "icons": [
    {
      "src": "assets/icons/icon-72x72.png",
      "sizes": "72x72",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "assets/icons/icon-96x96.png",
      "sizes": "96x96",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "assets/icons/icon-128x128.png",
      "sizes": "128x128",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "assets/icons/icon-144x144.png",
```

```
      "sizes": "144x144",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "assets/icons/icon-152x152.png",
      "sizes": "152x152",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "assets/icons/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "assets/icons/icon-384x384.png",
      "sizes": "384x384",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "assets/icons/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "maskable any"
    }
  ]
}
```

This has all the details needed to show the app icon on a device's home screen, such as a mobile phone. The list of icons is provided so that, depending on the device and the platform, an appropriate icon file is

shown on the home screen.

The service worker is registered in the **app.module.ts** file with the following lines:

```
1. ServiceWorkerModule.register('ngsw-worker.js', {
2.       enabled: !isDevMode(),
3.       // Register the ServiceWorker as soon as the application is stable
4.       // or after 30 seconds (whichever comes first).
5.       registrationStrategy: 'registerWhenStable:30000'
6.     }
```

Now that the app is modified to have offline capability, let us test it out. Angular supports service workers only in production builds. Build the app for production:

ng build --configuration=production

Run **http-server** on the **dist** folder as follows:

npx http-server dist/fruit-store-pwa/browser/

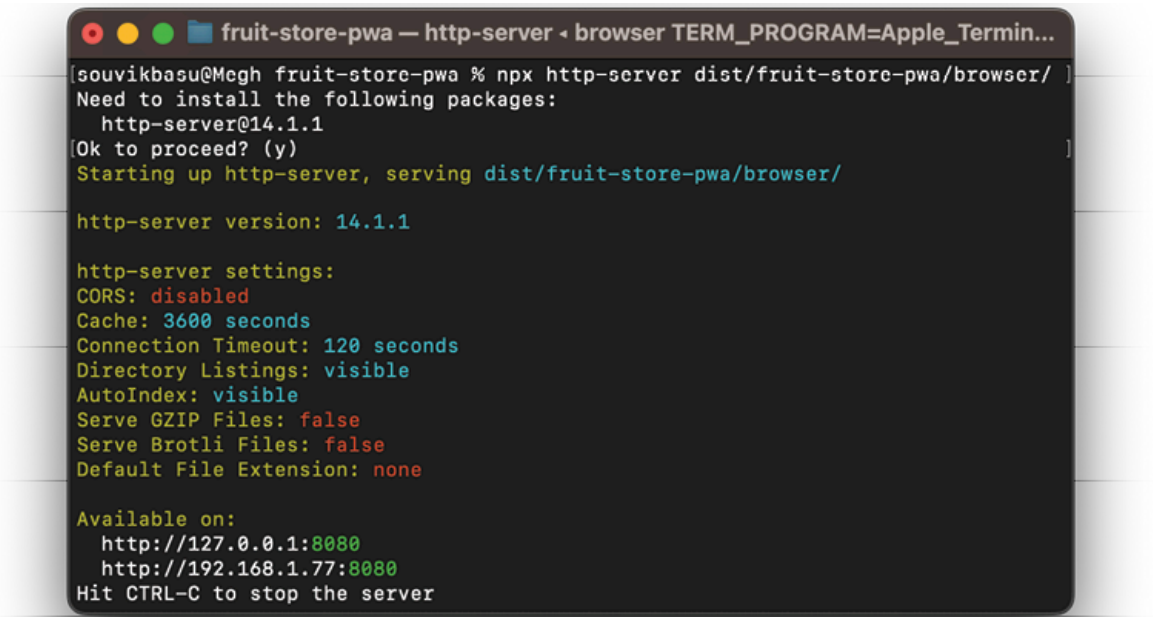


Figure 24.6: *Run http-server*

Now enable online mode in the browser and open the app at **http://127.0.0.1:8080**. Go to the **Product** page and observe the **Network** tab in the dev tool:

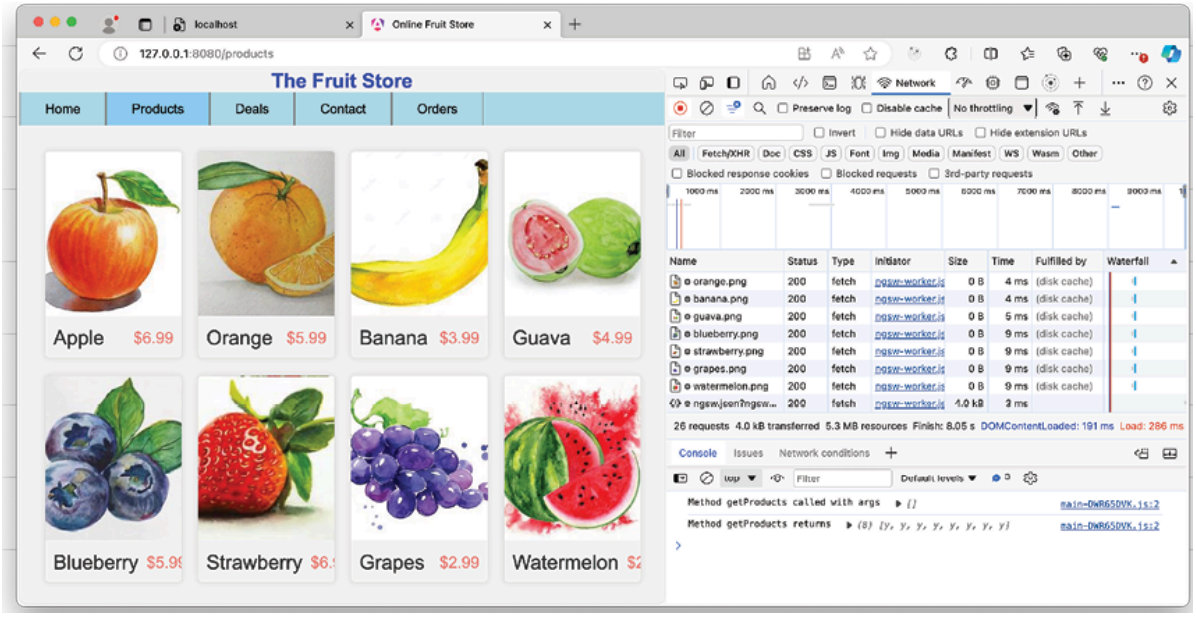


Figure 24.7: *PWA fruit store app in online mode*

Now, set offline mode as before and refresh the page. This time, you will see the page loads even in offline mode:

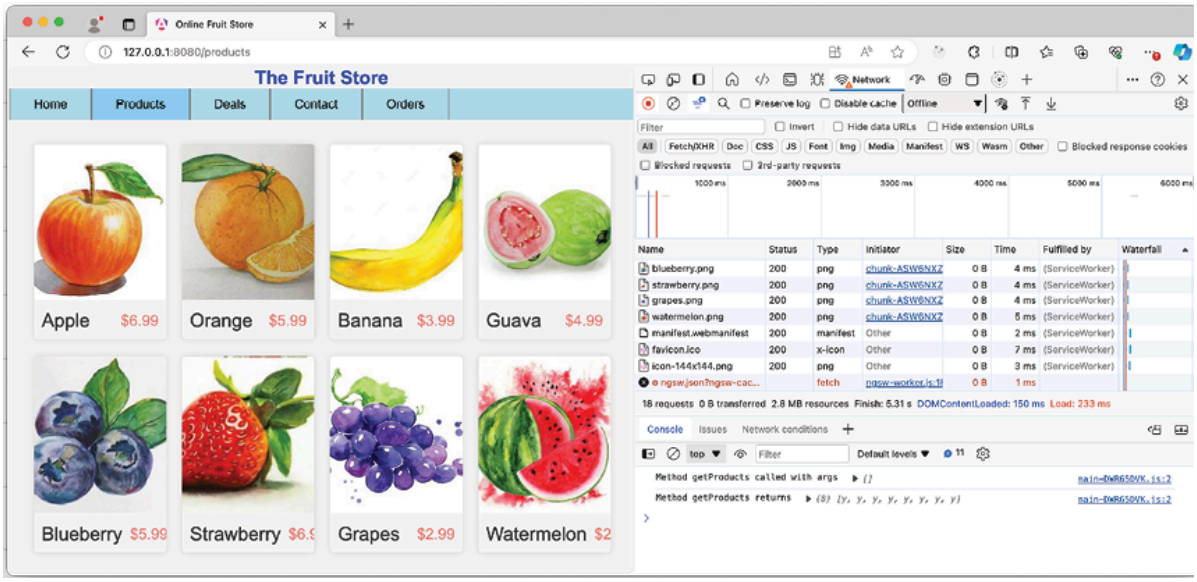


Figure 24.8: *PWA fruit store app in offline mode*

Notice that the images have been fulfilled by the **ServiceWorker** at this time.

Background Sync and custom service worker

Angular comes with its built-in service worker code. If you want to add custom logic, you can create your own script, import Angular's service worker code into it, and register the custom script in the **app.module.ts**

file instead of Angular's built-in service worker file. One such scenario to write a custom service worker script is when you want background sync for your app. **Background Sync** is a new browser API that is not fully supported by all browsers. Using Background Sync API, you can retrieve data and assets for your application when the app is online again. Since this is not an Angular feature, we will skip the details. Please go over the links in the *Reference* section for details.

Conclusion

This chapter provided insights into creating offline applications using Angular. We explored the importance of offline capabilities and the advantages of an offline-first approach. We also delved into the role of service workers in handling network requests and managing offline functionality. These concepts can help developers enhance user experiences by ensuring uninterrupted access to app features and data, regardless of internet connectivity. In the next chapter, we will explore advanced deployment topics like content delivery networks, load balancers, and Jamstack that can greatly improve the page load time for an Angular application.

Points to remember

- Offline capabilities are essential for providing uninterrupted access to the application features and data and also improving reliability.
- The offline-first approach prioritizes local functionality over network dependency and ensures that essential features work seamlessly even when offline.
- Service workers are JavaScript files that run in the background of an application and are responsible for handling network requests, caching data, and managing offline functionality.
- PWAs use modern web capabilities to deliver app-like experiences and can work offline, providing a seamless user experience.
- Background Sync and custom service workers can be used to sync data when the connection is restored, preventing data loss and improving the user experience.

Questions

1. What are some scenarios where offline capabilities can greatly improve the user experience?
2. What is the offline-first approach to building applications?
3. What are the advantages of an offline-first approach?
4. What is a service worker, and what is its role in managing offline functionality?

References

- Angular documentation on service worker:
  https://angular.io/guide/service-worker-intro
- Background Sync API:

https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API

- Using Background Sync API with Angular service worker:
  https://golb.hplar.ch/2018/12/background-sync-ng.html

CHAPTER 25
Advanced Deployment

Introduction

We covered basic deployment topics in *Chapter 10, Deployment*. This chapter covers more advanced deployment topics that can help your Angular app serve faster and reliably to a global user base. This chapter will explore edge servers and load balancers available with cloud services like Google Cloud Provider and Amazon Web Services, guaranteeing speedy delivery of your web app. Furthermore, you can capitalize on benefits such as improved security, scalability, and cost reductions by building your app as a static site and employing the Jamstack architecture. Firebase is a fantastic **backend as a service** (**BaaS**) platform that provides developers with tools for building high-quality mobile and web applications. It is a popular choice for Angular developers because it simplifies the process of setting up a serverless backend. Each of the concepts we will discuss in this chapter requires elaborate exploration and learning. This chapter will introduce these concepts so that you can delve further to fine-tune your Angular app deployment.

Structure

This chapter covers the following topics:

- Content delivery network
- Load balancers
- Jamstack using Scully

- Deploy Angular app using Firebase

Objectives

By the end of this chapter, you will explore many advanced deployment topics that can help speed up page load, improve scalability, and reduce cost.

Content delivery network

A **content delivery network** (**CDN**) is a network of distributed servers that work together to deliver web content, such as images, videos, and HTML pages, to users based on their geographic location. The primary purpose of a CDN is to improve the performance, scalability, and reliability of delivering content by reducing the physical distance between the server and the user. When a user requests content from a website, the CDN identifies the user's location and delivers the content from the nearest server in the network. This reduces latency and accelerates the loading time of the web pages, providing a better user experience. Additionally, CDNs help offload the origin server by distributing the traffic across multiple edge servers, thereby reducing the load on the origin server and improving overall website performance. CDNs work by strategically caching content on edge servers located in various geographic locations and delivering the content to users from the nearest server, thus optimizing content delivery speed and efficiency.

The following diagram gives a visual representation of a web server situated in *North America* with global clients. Client devices across the globe have to connect to this server over the network, causing page load delays:

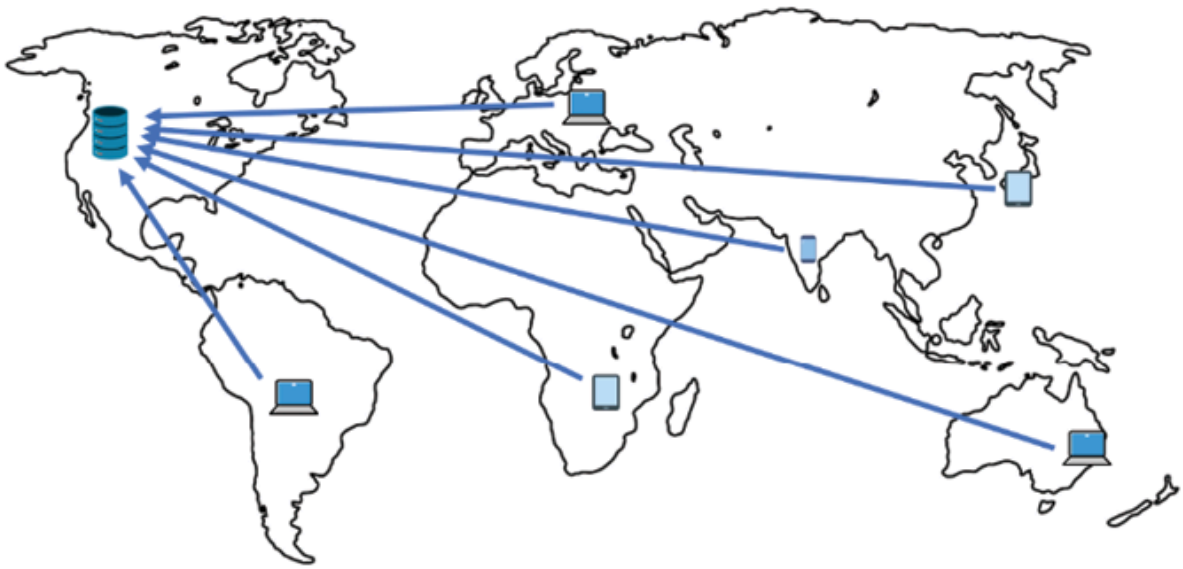


Figure 25.1: *A network topology without CDN*

A CDN drastically reduces this response time by strategically routing the client call to an edge server near the device. Refer to the following figure:

Figure 25.2: *With CDN*

There are many CDN providers like Azure Front Door, Cloudflare, Akamai, etc. A comprehensive exploration of a CDN is out of the scope of the book, but the below points are a few of the major features and benefits of using a CDN:

- **Caching**: CDNs cache static content (like images, videos, and HTML files) in multiple locations around the world. When a user requests content, the CDN serves it from the nearest server, reducing latency

and improving load times.

- **Performance**: CDNs enhance website performance by distributing content closer to visitors. Faster loading times reduce bounce rates and encourage visitors to stay on the site longer.
- **Bandwidth cost reduction**: CDNs optimize data delivery, reducing the amount of data an origin server must provide. This helps lower hosting costs for website owners.
- **Availability and redundancy**: CDNs handle large traffic volumes and withstand hardware failures better than many origin servers. They ensure content availability even during peak usage.
- **Security**: CDNs offer DDoS mitigation, security certificate improvements, and other security optimizations.

Load balancers

A load balancer is a device or software application that efficiently distributes network or application traffic across multiple servers. The primary goal of a load balancer is to optimize resource utilization, maximize throughput, minimize response time, and avoid overload among individual servers. Essentially, it helps to evenly distribute incoming network traffic or application requests across multiple servers, ensuring that no single server becomes overwhelmed with traffic. This not only improves the overall performance and availability of applications but also enhances their reliability and scalability. Following are a few of the features of load balancers:

- **Traffic distribution**: The major purpose of load balancers is to evenly distribute requests among a group of backend servers (such as web servers or application instances).
- **Scalability**: They allow you to scale horizontally by adding more servers to handle increased load.
- **Fault tolerance**: If one server fails, the load balancer redirects traffic to healthy servers.
- **Session persistence**: Some load balancers maintain session affinity, ensuring that a user's requests go to the same backend server.

A load balancer is an essential component if you want to scale up your application and provide fault tolerance. There are two types of load

balancers:

- **Layer 4** (**transport layer**): These operate at the transport layer (TCP/UDP) and distribute traffic based on IP addresses and port numbers.
- **Layer 7** (**application layer**): These work at the application layer (HTTP/HTTPS) and can make routing decisions based on content (for example, URL paths).

Most cloud service providers, like Azure, Google Cloud, and AWS, provide load balancers. *Cloudflare* and *DigitalOcean* also offer load balancing services. Setting these up is essential for a seamless user experience. Depending on the cloud service you selected for your app deployment, please take a look at the load balancing options.

Jamstack using Scully

There is an alternate approach for the deployment of static content sites like blog posts — Jamstack. Jamstack is a modern web development architecture based on client-side JavaScript, reusable APIs, and prebuilt Markup. The term *JAM* stands for JavaScript, APIs, and Markup. In a Jamstack architecture, the entire website is generated at deployment time, and the final HTML files are served to the client. This approach offers several benefits, including improved performance, enhanced security, better scalability, and reduced operational complexity. Jamstack websites are typically served through CDNs, ensuring fast and reliable delivery to users globally. Jamstack has gained popularity in recent years due to its simplicity, flexibility, and ability to leverage the power of modern web development tools and services.

One of the best tools for implementing Jamstack in Angular is Scully. Scully is a static site generator for Angular applications that helps in pre-rendering Angular pages to plain HTML and CSS. This pre-rendering process improves website loading speed and **search engine optimization** (**SEO**) by providing pre-built static pages to the web crawlers. When a user requests a page from the pre-rendered site, the server delivers the pre-built HTML file, which reduces the time required to generate the page on the server. Scully works by analyzing Angular routes, rendering each route to HTML, and then saving the resulting files as static pages. This approach enables faster page loads and better performance, especially for

content-heavy websites.

To add Scully to your Angular app, run the following:

ng add @scullyio/init

This will create a config file **scully.<projectName>.config.ts** with the following content:

```
import { ScullyConfig } from '@scullyio/scully';

export const config: ScullyConfig = {
  projectRoot: './src',
  projectName: '<projectName>',
  outDir: './dist/static',
  routes: {},
};
```

Now you can build the Angular app as usual:

ng build

Then, run Scully:

npx scully

Scully will create all the static pages at **/dist/static**. It will contain one file for every route defined in the Scully config file.

To serve the static files, run the following:

npx scully serve

This will run two servers: one hosting the result of **ng build** and another serving the files built with Scully so that we can see both versions of the app. For more configuration options, please check Scully's documentation provided in the references section.

Deploy Angular app using Firebase

Firebase is a robust BaaS platform provided by *Google*. It offers developers a suite of tools and services to simplify app development, enhance app quality, and grow user bases. Platforms like Firebase serve as a single-stop solution for app deployment, allowing the developers to concentrate on app development. Few are some of the features provided by Firebase:

- **Real-time database**: Firebase provides a real-time NoSQL database that synchronizes data across clients in real-time. It is great for chat apps, collaborative tools, and live updates.
- **Authentication**: Easily integrate user authentication with Firebase. It supports email/password, social logins (like *Google* and *Facebook*), and more.
- **Cloud storage**: Securely store and serve user-generated content (such as images, videos, and files) in the cloud.
- **Hosting**: Firebase Hosting allows you to deploy web apps quickly and securely. It handles HTTPS by default and provides global content delivery.
- **Analytics**: Gain insights into user behavior, engagement, and app performance. Firebase Analytics helps you make data driven decisions.

Please check the documentation for getting started on Firebase provided in the *References* section.

Conclusion

This chapter has provided a comprehensive overview of advanced deployment topics for Angular applications. We have explored the benefits of using CDNs to improve performance, scalability, and reliability by reducing latency and optimizing content delivery. Additionally, the role of load balancers in efficiently distributing network or application traffic across multiple servers has been discussed. These advanced deployment strategies can significantly enhance the user experience, improve website performance, and ensure high availability. By implementing the concepts outlined in this chapter, Angular developers can effectively fine-tune their app deployment to meet the demands of a global user base while optimizing cost and security. The next chapter will explore how to make our Angular apps responsive on different device screens.

Points to remember

- CDNs are networks of distributed servers that deliver web content to users based on their geographic location, improving performance,

scalability, and reliability by reducing physical distance between the server and the user.

- Load balancers efficiently distribute network or application traffic across multiple servers, optimizing resource utilization, maximizing throughput, and enhancing reliability and scalability.
- Implementing Jamstack with Scully can improve the performance and security of your web app by pre-rendering the Angular app and serving it as static files.
- Firebase integration with Angular simplifies the deployment process by providing features such as hosting, database, authentication, and cloud functions, allowing for easy deployment to a global CDN.

Questions

1. What are the benefits of using a CDN for delivering web content, and how does it improve the performance and reliability of delivering content to users?
2. What is the primary purpose of a load balancer, and how does it optimize resource utilization and maximize throughput?
3. In what ways do load balancers improve the overall performance, availability, reliability, and scalability of applications?
4. What is a Jamstack and how does it improve content delivery?

References

- Cloudflare CDN:
  https://www.cloudflare.com/application-services/products/cdn/
- Azure Front Door and Load Balancer:
  https://azure.microsoft.com/en-ca/products/frontdoor/
- Akamai CDN:
  https://www.akamai.com/solutions/content-delivery-network
- Azure Load Balancer:
  https://azure.microsoft.com/en-ca/products/load-balancer
- DigitalOcean Load balancer:
  https://www.digitalocean.com/products/load-balancer
- Scully:

https://scully.io/

- Building Angular app with Firebase:

  https://developers.google.com/codelabs/building-a-web-app-with-angular-and-firebase

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 26
Responsiveness

Introduction

In today's world, users expect web applications to be accessible and functional on a wide range of devices, from desktops to smartphones and tablets. With Angular, creating a responsive web application has become much easier. In this context, responsiveness means that the application should adjust its layout and behavior based on the user's device and screen size. Handling responsiveness in your app helps to improve user experience on a variety of devices and enhances your site's search engine ranking. In this chapter, we will explore how to create responsive designs in an Angular app and the best practices to ensure that your app is accessible and usable across all devices.

Structure

The chapter covers the following topics:

- Using media queries
- Conditional rendering with breakpoints
- Mobile-first approach and performance factors
- Flex layout and grid controls
- Responsiveness using Angular Material
- Responsiveness on server side
- Bootstrap

Objectives

By the end of this chapter, you will understand how to create responsive apps in Angular that can render well on a wide range of device screens.

Using media queries

The most common use case of responsiveness is to shrink or expand the page based on the available width of the device screen. Using media queries in CSS to handle responsiveness is a generic practice irrespective of the web development framework. It can be used in the Angular component's style file as well. For instance, say in our fruit store sample code; we want to show four columns of products in a row. The product page component template (**product.component.html**) shows a list of products as follows:

```
1. <div id="product_page">
2.     <div class="product-list">
3.  <app-product-card *ngFor="let product of products" class="product-card-container col"
   [product]="product"></app-product-card>
4.     </div>
5. </div>
```

Notice the class name **.col** in *line 3*. We can write a CSS to squeeze the width of each product to twenty five percent of the available width so that it shows four products in a row in a desktop view, as follows:

```
1. .col {
```

```
2.     width: 25%;
3. }
```

We are skipping the rest of the styling and template code for brevity. You may check the full code in the code samples of this chapter provided along with the book. The above code will show up as follows on the browser:

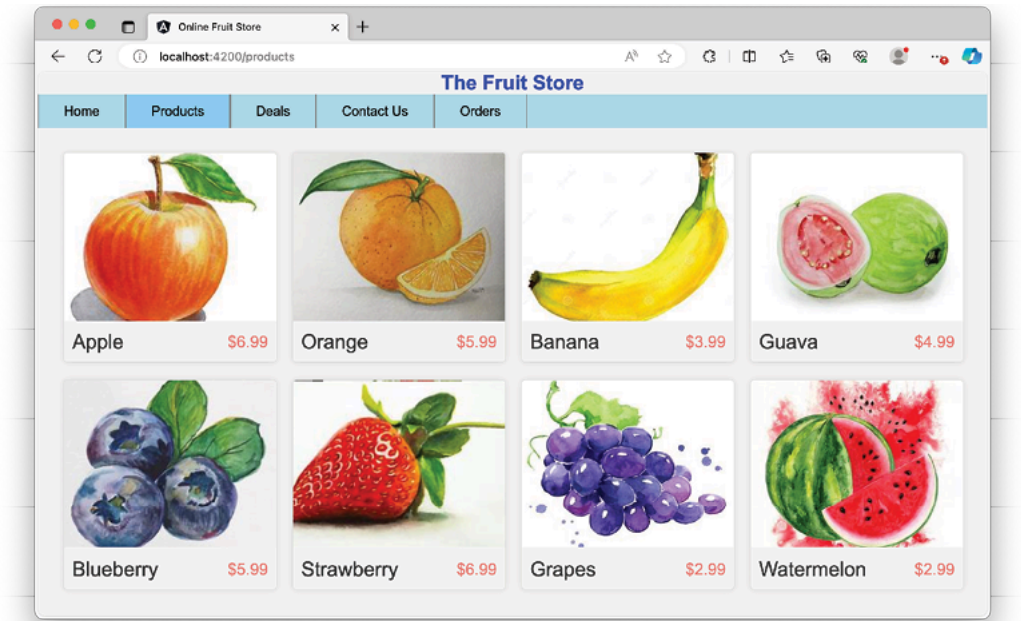


Figure 26.1: *Desktop view with 4 products per row*

Now, if we want to restrict the number of products to just two in a tablet portrait view, and assuming a general tablet screen width is between **600px** and 840px, we can write a media query as follows to achieve this:

```
1. @media screen and (min-width: 600px) and (max-width: 839.98px) and (orientation: portrait) {
2.     .col {
3.         width: 50%;
4.     }
5. }
```

To see this in the browser, open the dev tool and click the device emulation button. Then, select any tablet from the list of devices at the top of the browser. Refer to the following figure:

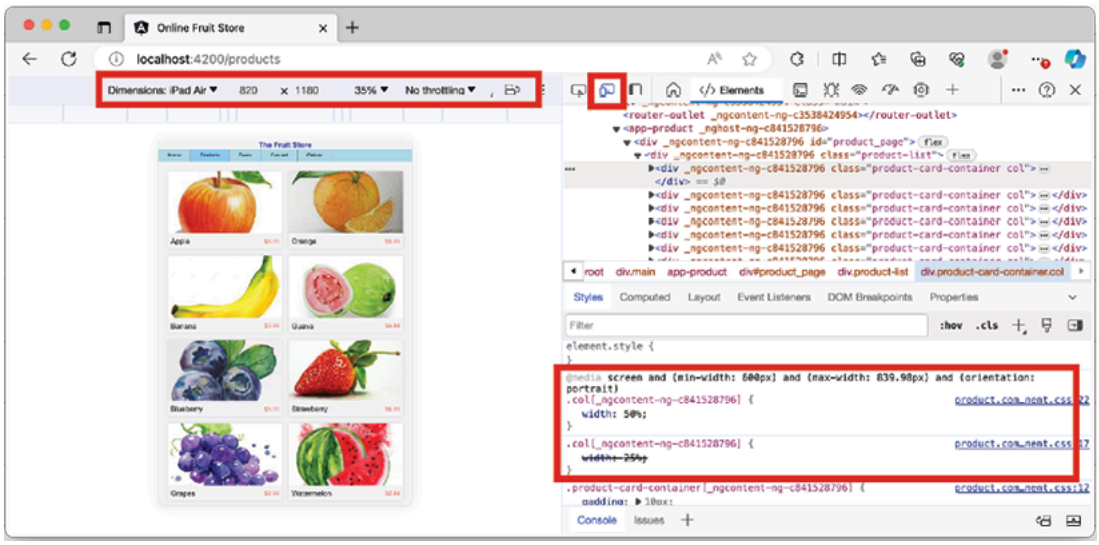


Figure 26.2: *Tablet view with 2 products per row*

Similarly, we can show four items in a row on a mobile screen with a media query as follows:

```
1. @media screen and (max-width: 599.98px) and (orientation: portrait) {
```

```
    .col {
      width: 100%;
    }
}
```

This will render as follows in a mobile view:

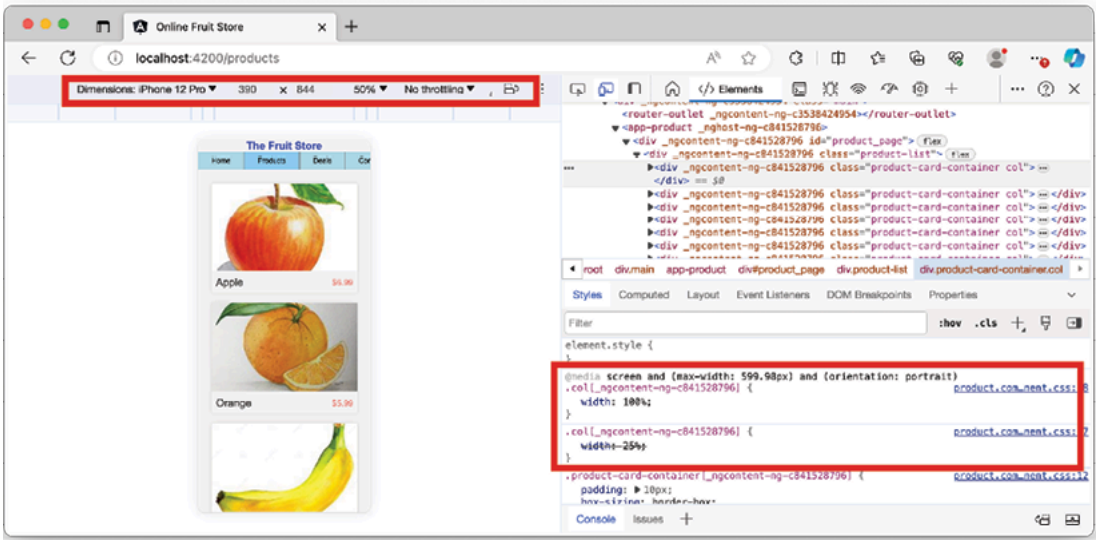


Figure 26.3: *Mobile view with 1 product per row*

Conditional rendering with breakpoints

There is a helpful utility called **BreakpointObserver** service available in Angular CDK that can achieve the same responsiveness that we did using media queries but without using media queries. It allows us to subscribe to listen to UI breakpoints and handle events when breakpoints change. **BreakpointObserver** comes with a list of pre-built CSS breakpoints defined as follows:

```
export declare const Breakpoints: {
    XSmall: string;
    Small: string;
    Medium: string;
    Large: string;
    XLarge: string;
    Handset: string;
    Tablet: string;
    Web: string;
    HandsetPortrait: string;
    TabletPortrait: string;
    WebPortrait: string;
    HandsetLandscape: string;
    TabletLandscape: string;
    WebLandscape: string;
};
```

These are readable breakpoint names that are internally implemented as media queries. These breakpoints are decided based on the heuristics of the available devices in the market. As of now, they are represented with the following media queries:

```
Web:
(min-width: 840px) and (orientation: portrait), (min-width: 1280px) and (orientation: landscape)
```

```
4.  WebLandscape:
5.  (min-width: 1280px) and (orientation: landscape)
6.
7.  WebPortrait:
8.  (min-width: 840px) and (orientation: portrait)
9.
10.  Tablet:
11.  (min-width: 600px) and (max-width: 839.98px) and (orientation: portrait), (min-width: 960px) and (max-
12.
13.  TabletPortrait:
14.  (min-width: 600px) and (max-width: 839.98px) and (orientation: portrait)
15.
16.  TabletLandscape:
17.  (min-width: 960px) and (max-width: 1279.98px) and (orientation: landscape)
18.
19.  Handset:
20.  (max-width: 599.98px) and (orientation: portrait), (max-width: 959.98px) and (orientation: landscape)
21.
22.  HandsetLandscape:
23.  (max-width: 959.98px) and (orientation: landscape)
24.
25.  HandsetPortrait:
26.  (max-width: 599.98px) and (orientation: portrait)
27.
28.  XSmall:
29.  (max-width: 599.98px)
30.
31.  Small:
32.  (min-width: 600px) and (max-width: 959.98px)
33.
34.  Medium:
35.  (min-width: 960px) and (max-width: 1279.98px)
36.
37.  Large:
38.  (min-width: 1280px) and (max-width: 1919.98px)
39.
40.  XLarge:
41.  (min-width: 1920px)
```

To see this in action, install Angular CDK as follows:

```
npm install @angular/cdk
```

Now, we can subscribe to **BreakpointService** for any specific breakpoint. For instance, we can listen to **Breakpoint.TabletPortrait** as follows in the **product.component.ts** file:

```
1.  import { Component } from '@angular/core';
2.  import { ProductService } from '../../services/product.service';
3.  import { Product } from '../../models/Product';
4.  import { BreakpointObserver, Breakpoints } from '@angular/cdk/layout';
5.
6.  @Component({
```

```
  selector: 'app-product',
  templateUrl: './product.component.html',
  styleUrls: ['./product.component.css'],
  providers: [ProductService]
})
export class ProductComponent {
  products: Product[];
  constructor(productService: ProductService, private breakpoint: BreakpointObserver) {
    this.products = productService.getProducts();
  }

  ngOnInit() {
    this.breakpoint.observe(Breakpoints.TabletPortrait)
      .subscribe(result => {
        if (result.matches) {
          console.log("Current screen matches TabletPortrait");
        }
      });
  }
}
```

In the browser, if you switch between the desktop view and responsive view with tablet portrait mode, you will see the console message printed as follows:

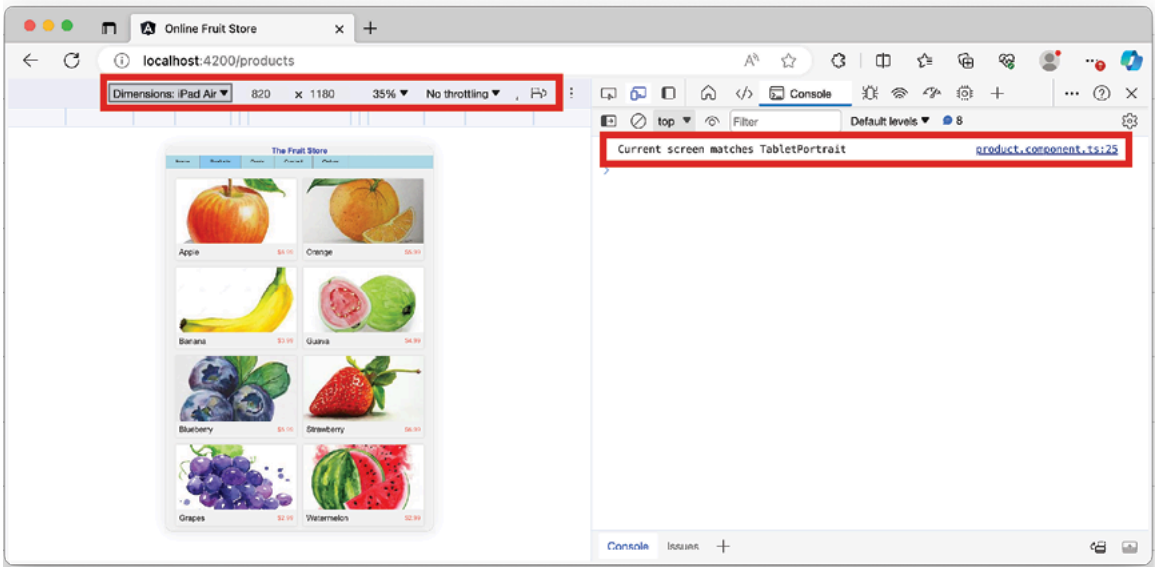


Figure 26.4: *Console message for Tablet Portrait breakpoint*

So, now we can set Boolean fields when **TabletPortrait** and **HandsetPortrait** breakpoints match the current result and use these in our CSS to style the component instead of the complicated media queries. First, declare these fields in the **product.component.ts** file and set them in the breakpoint observer as follows:

```
export class ProductComponent {
  ...
  isTabletPortrait = false;
  isPhonePortrait = false;
  ...
  ngOnInit() {
  this.breakpoint.observe([Breakpoints.TabletPortrait, Breakpoints.HandsetPortrait])
```

```
8.      .subscribe(result => {
9.        this.isTabletPortrait = false;
10.       this.isPhonePortrait = false;
11.
12.       const breakpoints = result.breakpoints;
13.       if (breakpoints[Breakpoints.TabletPortrait]) {
14.         this.isTabletPortrait = true;
15.       }
16.       else if (breakpoints[Breakpoints.HandsetPortrait]) {
17.         this.isPhonePortrait = true;
18.       }
19.     });
20.   }
21. }
```

Notice here that we are listening to two breakpoints now instead of one in *line 7*. Now, we can conditionally set class names in the view template based on these Boolean fields, as follows:

```
1. <app-product-card *ngFor="let product of products" class="product-card-container col" [ngClass]="{'is-tablet-portrait': isTabletPortrait, 'is-phone-portrait': isPhonePortrait}" [product]="product"></app-product-card>
```

Replace the previous media queries with these class names in **product.component.css** file, as follows:

```
1. .col {
2.     width: 25%;
3. }
4.
5. .col.is-tablet-portrait {
6.     width: 50%;
7. }
8.
9. .col.is-phone-portrait {
10.     width: 100%;
11. }
```

Notice how much cleaner this code is than using the actual width dimensions and orientation in the media queries. The above changes will yield the same results as those achieved using media queries in the previous section.

You may also use these breakpoint Boolean fields to conditionally render a portion of a page based on the device and orientation. For instance, if you want to hide a section on the phone in portrait mode due to space limitations, you can achieve that with a code like this:

```
1. <div *ngIf="!isPhonePortrait">
2.   ...
3. </div>
```

Mobile-first approach and performance factors

The mobile-first approach is a technique to start creating your app for the smallest possible device screen and then enhance it for bigger screen sizes. This ensures that unnecessary DOM elements are not fetched and rendered on smaller screens. This dramatically improves the overall user experience on your site. Most search engines give a higher ranking to sites that are mobile friendly and even higher perks for being mobile-first.

In the previous section, we saw how we could render DOM elements conditionally using **BreakpointService**. Consider a page that renders a lot of images and videos. Using conditional rendering based on breakpoints, you can choose different image and video resolutions and file sizes for different device types. This ensures that the page rendered on a mobile screen does not fetch high-resolution images, thereby rendering the page faster

and consuming less network bandwidth. Depending upon your case, you may decide not to show video on mobile screens and show it only on tablets and desktops. **BreakpointService** allows all such possibilities.

Flex layout and grid controls

Flexbox and grid are commonly used CSS solutions to make any web app responsive. Flexbox allows the wrapping of a single dimensional list of items on the page and provides multiple CSS properties to set the alignment of elements on both axes. To use flexbox, use the CSS property **display: flex**.

CSS grid layouts are helpful for more complex scenarios. Grid allows defining layout in a two dimensional space and hence allows fine gained control on the placement of elements on a device screen. A grid can be created by using the CSS property **display: grid**.

It is out of the scope of this book to go over the CSS properties related to flexbox and Grid controls, but we highly recommend that you get acquainted with those to be able to write responsive web applications. If you are new to it, you can get help from the browser dev tool that allows setting flex and grid properties visually as follows:

Figure 26.5: *Flexbox helper in browser dev tool*

Responsiveness using Angular Material

Angular Material is a popular UI library for creating Angular apps. It contains a bundle of standard UI controls that can be used on any Angular web app. One significant advantage of using Angular Material components is that the components are well-tested to be responsive on a variety of devices. Angular Material also contains a **FlexLayoutModule** and grid layout system, which can be used to create responsive apps. We will skip further details on the topic as it is a vast library and out of the scope of this book to cover it. Please refer to the links in the reference section on the Angular Material library.

Responsiveness on server side

One disadvantage of using the **BreakpointService** is that it runs on the client side. If the page renders server side, we have to take our best guess on the client device and orientation and return a page that will not try to re-adjust on the client side. When a server returns a desktop page on the mobile device and that gets rehydrated into a mobile page on the client side, it's not a good experience for the user. The user sees a huge page rendered on the small screen that refreshes after a few seconds to show a mobile-friendly page. This impacts the performance of the page as well. Similarly, if a mobile page is returned by the server for a user trying to open the page on a desktop, that also is a bad user experience. Both of this scenario also causes web vital figures like **Cumulative Layout Shifts (CLS)** to go down as the page needs to render and rearrange the items in the new layout. A good solution to achieve a better server side rendered page that is responsive is by analyzing the user agent header of the request. However, keep in mind that user agents can be spoofed, so this approach is not foolproof. Here is a simplified example of code you can add to your route handler on the server side:

```
1. const userAgent = req.headers['user-agent'] || '';
2. if (userAgent.includes('Mobile')) {
3.   console.log('Rendering on a mobile device.');
```

```
} else {
  console.log('Rendering on a non-mobile device.');
}
```

Bootstrap

Bootstrap is a popular CSS library that makes a page responsive by dividing it into a grid layout. If you prefer to integrate the Bootstrap grid system into your project, **ng-bootstrap** is a descent widget library created using Bootstrap. You can find further documentation details on using the library in the references section.

Conclusion

In conclusion, creating a responsive web application has become a necessity in today's world. With Angular, we can easily handle responsiveness using media queries or the **BreakpointObserver** service. By implementing responsive designs in our Angular apps, we can enhance user experience across multiple devices, improve our site's search engine ranking, and ultimately increase user engagement. By following the best practices discussed in this chapter and staying up to date with the latest trends, we can ensure that our Angular apps are accessible and usable on a wide range of device screens. In the next chapter, we will discuss about design systems and theming. We will also look at a design tool for creating screen mocks.

Points to remember

- Responsiveness means that the application should adjust its layout and behavior based on the user's device and screen size.
- Using media queries in CSS is a generic practice to handle responsiveness, irrespective of the web development framework.
- **BreakpointObserver** service available in Angular CDK is a helpful utility that allows us to subscribe to listen to UI breakpoints and handle events when breakpoints change.
- Using a mobile-first approach and optimizing performance factors can help to improve responsiveness.
- Angular Material provides built-in responsive components that can be used to create responsive designs.

Questions

1. What is responsiveness in the context of web applications?
2. What are the benefits of handling responsiveness in web applications?
3. How can media queries be used to handle responsiveness in Angular?
4. What is the BreakpointObserver service in Angular CDK, and how can it be used to handle responsiveness?

References

- Angular Material:
  https://material.angular.io/
- ng-bootstrap:
  https://ng-bootstrap.github.io/#/home

CHAPTER 27
Design System and Tools

Introduction

A design system is a set of guidelines, rules, and components used to create consistent and cohesive visual and interactive designs for a website or application. It provides a common language and framework for designers and developers to work together, ensuring that the final product is both aesthetically pleasing and user friendly. Design tools help designers create screen mockups and share them with the development team. It is an essential tool for any frontend team to design and collaborate on product designs. There are many design tools available today, and Figma is one of the most popular ones. In this chapter, we will explore a few features of using Figma as a design tool for a frontend team.

Structure

The chapter covers the following topics:

- Design system
- Angular Material
- Figma design tool
- Theming

Objectives

By the end of this chapter, you will understand what design systems are,

how to integrate Angular Material into your project, and how to use Figma to collaborate in design.

Design system

A design system is a comprehensive set of standards that manages and scales design across digital products. Design systems typically include a set of design principles, color palettes, typography guidelines, iconography, and other visual elements that are used to create a cohesive design. They also include a set of reusable design components, such as buttons, forms, and navigation menus, that can be used throughout the application. The goal of a design system is to create a consistent user experience across all parts of the website or application, regardless of who is designing or developing it. This helps to reduce design and development time, eliminate inconsistencies in the visual design, and create a more efficient design process. In addition, design systems can also help to establish brand identity and recognition. By providing a consistent look and feel across all touchpoints, a design system can help to create a strong brand image and increase brand awareness.

A design system majorly encompasses the following three sections:

- **Reusable components**: These are building blocks like buttons, input fields, and other UI elements.
- **Design principles**: Guiding rules that ensure consistency and efficiency.
- **Guidelines**: Instructions on how to achieve a cohesive design language.

The following are the key aspects of any design system:

- **Consistency**: Design systems provide a single source of truth, reducing redundancy. They ensure that all products adhere to the same visual language.
- **Efficiency**: Designers can pull from a library of pre-approved components, saving time and effort.
- **Brand cohesion**: By enforcing consistent design, a design system reinforces the brand's identity.

A few examples of a design system are Material Design from *Google* and Fluent UI from *Microsoft*. There are implementations of these design

systems in multiple web frameworks. For instance, Angular Material is an implementation of the Material Design for Angular.

Angular Material

Angular Material provides a wide range of pre-built components, including buttons, forms, navigation menus, and more, that can be easily customized to match the design of the application. Angular Material also provides a range of design and theming options, making it easy to create a visually appealing and user friendly interface.

One of the key benefits of Angular Material is that it provides a consistent and cohesive user experience across all parts of the application. The pre-built components are designed to work seamlessly together, reducing the risk of inconsistencies in the visual design. Additionally, Angular Material is optimized for performance, ensuring the application runs smoothly and efficiently.

Another benefit of Angular Material is that it is well-documented and supported by the Angular community. This makes it easy to learn and implement, even for developers who are new to Angular development. The library is regularly updated with new features and improvements, ensuring that it stays up to date with the latest design trends and best practices.

Let us explore Angular Material through a code sample. Create a new Angular project as follows:

```
ng new angular-material –style=scss
```

Note that we used SCSS for styling as it makes it easier to integrate the Angular Material theme using SCSS.

Open **app.component.html** file and add a button as follows:

```
<div>
  <input type="checkbox" id="agreement" />
  <label for="agreement">I agree</label>
</div>
<button>Submit</button>
```

This will render in the browser as follows:

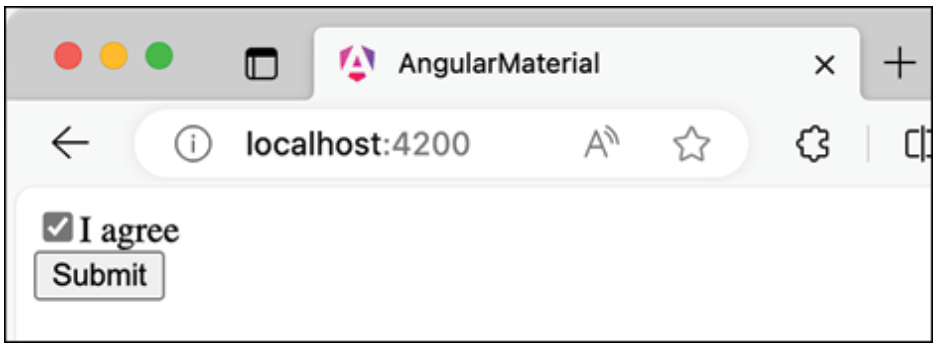


Figure 27.1: *UI controls without any design system*

Notice how bare minimum styling the checkbox, label, and button have. They do not have proper padding, there is no background color and font color, the font family and styling are not defined, there is not much UI response on user action on the button, the controls are not accessible and not localized, they are not responsiveness, there is no common look and feel defined for these controls. All these issues can be fixed by integrating a library that implements a design system. We will demonstrate with Angular Material, but you can choose to use any component library that works for your project.

Now, let us add Angular Material to the project. Angular Material is available in the Angular CLI schema. So, you can add it using the following command:

ng add @angular/material

Import the default color theme of **indigo-pink** to **styles.scss** file as follows:

```
@import '@angular/material/prebuilt-themes/indigo-pink.css';
```

Import **MatButtonModule** and **MatCheckboxModule** modules in the **app.component.ts** file (or **app.modules.ts** if it is not a standalone component) as follows:

```
...
import { MatButtonModule } from '@angular/material/button';

@Component({
  ...
  imports: [MatButtonModule],
})
export class AppComponent {
```

```
9.  ...
10. }
```

Now, change the view template in **app.component.html** to use Material components:

```
1. <div>
2.   <mat-checkbox>I agree</mat-checkbox>
3. </div>
4. <button mat-raised-button color="primary" >Submit</button>
```

This will render the components as shown below:

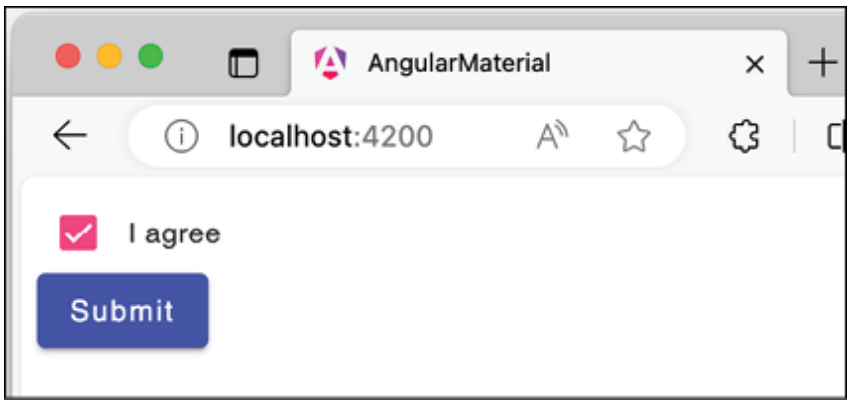


Figure 27.2: *Angular Material UI controls*

This is a lot better than the bare-bone HTML tags in many ways, as described below:

- The look and feel of the controls are nicer.
- The controls use a common theme for colors and fonts.
- There is consistent padding and gaps.
- Hovering over and clicking on the button causes color transitions to provide a response for the user action.
- There is a consistent design across components, and they look more coherent. This reinforces brand identity.
- It is easy to change the theme to a different color scheme.
- The controls are accessible and internationalized.
- It would have taken a lot of effort to create and maintain such controls manually.

You may explore more components from the library by following the link to the documentation page, provided in the reference section at the end of

this chapter.

Figma design tool

A design tool provides a platform for design exchange between the design and the development team. Figma, Sketch, and Adobe XD are a few of the popular design tools. These tools offer a range of features that can greatly streamline the design and frontend development process. Let us look at a few features of Figma:

- **Design components**: Figma makes it easy to create reusable design components that can be used across the project. These components can be easily shared between team members and can help maintain design consistency.
- **Collaboration**: Figma allows teams to collaborate on designs in real time. Multiple team members can work on a design simultaneously, making it easy to share ideas and create a cohesive design.
- **Design handoff**: Figma makes it easy to hand off designs to developers. Developers can inspect the design and extract CSS, images, and other assets directly from Figma.
- **Shared libraries**: It allows developers to create shared libraries of design components. These components can be used across the project, making it easy to maintain design consistency.
- **Responsive design**: It allows developers to create responsive designs that adapt to different screen sizes. This is essential when developing applications for multiple devices.
- **Prototyping**: It allows developers to create interactive prototypes that can be shared with stakeholders. This makes it easy to get feedback on the design and ensure that it meets the requirements of the project.
- **Plugins**: Figma has a rich ecosystem of plugins that can be used to extend its functionality. These plugins can automate repetitive tasks, integrate with other tools, or add new features to Figma.
- **Design versioning**: Figma allows developers to maintain different versions of a design, making it easy to roll back to an earlier version if needed.

The Angular Material design is available in Figma as well. Please check the link in the reference section. A typical design window in Figma looks

as shown:

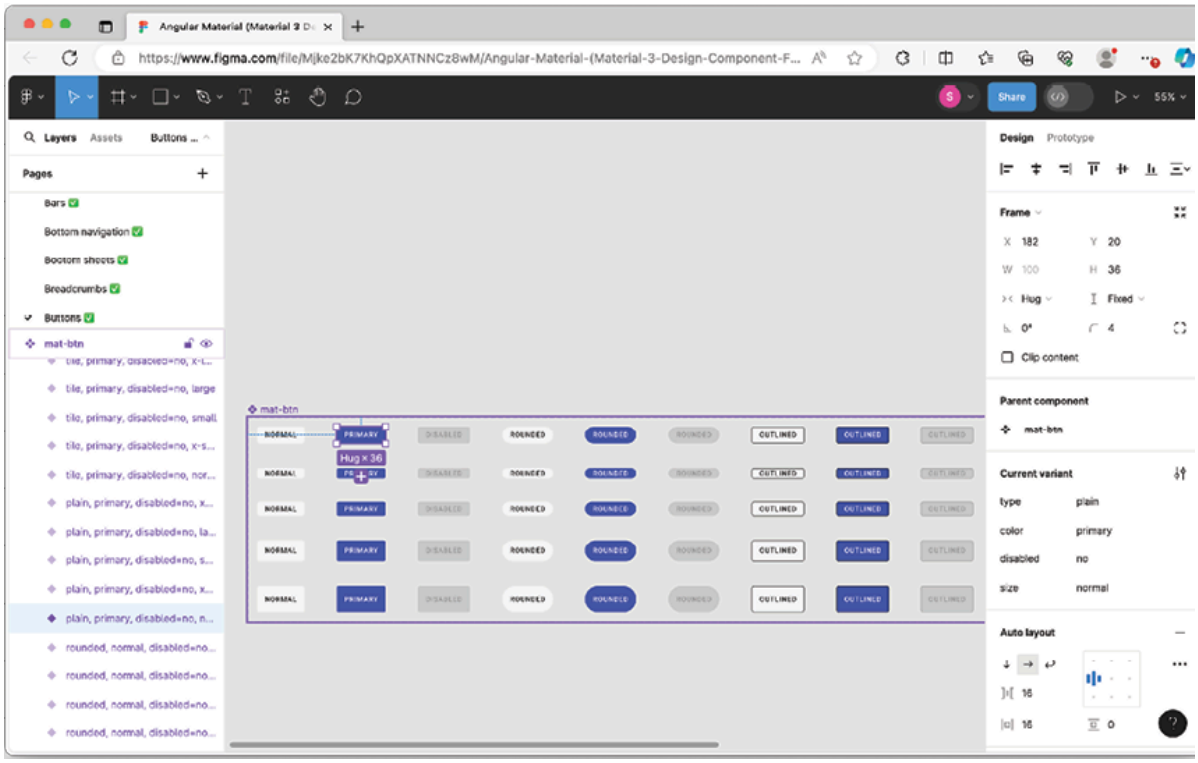


Figure 27.3: *Angular Material design in Figma*

When a component is selected in the middle section of the page, its style details are enumerated on the right-side panel. One can even create CSS output that can be pasted into the component style file. A detailed tutorial of Figma is out of the scope of this book, but you are encouraged to get acquainted with the tool or similar design tools.

Theming

Theming is the process of customizing the appearance of a website or application to match a specific design or brand. This involves modifying the visual elements of the interface, such as the color scheme, typography, and layout, to create a cohesive design that is both aesthetically pleasing and user friendly.

Theming is important because it can greatly impact the user's perception of a website or application. A well-designed theme can improve user engagement and satisfaction, while a poorly designed theme can detract from the user experience and lead to lower user retention rates. Additionally, theming can help to establish brand recognition and create a consistent visual identity across multiple platforms.

In the previous section, we used the **indigo-pink** theme of Angular Material to demonstrate the usage of the components library. There are many other pre-built themes available within Angular Material, and you can explore them or create a custom theme as needed.

If you need to create custom components for your project, it is recommended to declare all colors and font details in a separate **theme.css** file as CSS variables. You can then import this into the global styles file. The advantage of moving themes into CSS variables is that it allows you to switch from one theme to another seamlessly. Most modern websites today allow the user to toggle between light and dark themes. *Chapter 6, Styling and Theming*, discusses theming with code samples. It also demonstrates how to toggle between light and dark themes.

Conclusion

A design system is an important tool for any frontend team to create consistent and cohesive visual and interactive designs for a website or application. It provides a common language and framework for designers and developers to work together, ensuring that the final product is both aesthetically pleasing and user-friendly. Angular Material is one such implementation of a design system that provides a wide range of pre-built components, design and theming options, and consistent user experience across all parts of the application. Figma is a popular design tool that helps designers create screen mockups and collaborate with the development team.

In the next chapter, you will learn about how to create mobile native apps using Angular.

Points to remember

- A design system is a set of guidelines, rules, and components that are used to create consistent and cohesive visual and interactive designs for a website or application.
- Design systems typically include a set of design principles, color palettes, typography guidelines, iconography, and other visual elements that are used to create a cohesive design.
- They also include reusable design components, such as buttons,

forms, and navigation menus, that can be used throughout the application.

- Angular Material is an implementation of the Material Design for Angular. It provides a wide range of pre-built components that can be easily customized to match the design of the application.
- Figma is a popular design tool for frontend teams that provides many features for designers to create screen mockups and collaborate on product designs.

Questions

1. What is a design system, and what are its key components?
2. How can a design system benefit a frontend team?
3. What are the benefits of using Angular Material?
4. What are some common design principles that a design system should follow?
5. How can design tools help designers create and collaborate on product designs?
6. How can a design system help to establish brand identity and recognition?

References

- Material Design:
  https://m3.material.io/
- Fluent UI:
  https://developer.microsoft.com/en-us/fluentui#/
- Angular Material:
  https://material.angular.io/
- What is a design system?
  https://www.figma.com/blog/design-systems-101-what-is-a-design-system/
- Material design system in Figma:
  https://www.figma.com/file/KTiVio4tDwMnRUXNieyaLP/Angular-Material-(Material-3-Design-Component-Figma-Library-For-Angular)-(Community)

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 28
Creating a Mobile App using Angular

Introduction

Creating mobile apps has become an essential part of modern software development. With the rise of mobile devices, businesses and individuals alike need to ensure that their applications are accessible and user friendly on a range of devices. A native mobile app provides many advantages over a responsive web page rendered in a browser on a mobile device. In this chapter, we will explore the various tools and techniques that can be used to create mobile apps using Angular and how you can leverage them to create a seamless and engaging user experience for your users.

Structure

This chapter covers the following topics:

- Advantages of a native app
- Creating a native app
- Making an Angular app native
- Adding native features

Objectives

By the end of this chapter, you will understand how to create native mobile apps using Angular.

Advantages of a native app

There are many advantages of a native mobile app over a web page. They are as follows:

- **Performance**: Native apps deliver superior performance. They are optimized for the specific platform (iOS or Android), resulting in faster load times and smoother interactions with users.

- **Access to device features**: Native apps can directly access device features such as the camera, contacts, and microphone. This allows for more specialized functionality and a seamless user experience.
- **Offline functionality**: Native apps can work offline, which is crucial for scenarios where internet connectivity is intermittent or unavailable. Users can still interact with the app and access cached data.
- **App store support**: Apps receive complete support from app stores (such as the App Store and Google Play). Being listed in app stores enhances discoverability and credibility.
- **Security**: Native apps must pass app store approval processes, ensuring a certain level of security. Users trust apps from official stores more than web apps.

Creating a native app

Many libraries allow us to create a mobile app using Angular. Capacitor and NativeScript are the most popular ones. Let us take a look at them:

- **Capacitor** is a powerful cross-platform app runtime that empowers web developers to build native apps for iOS, Android, Electron, and the web using standard web technology. It includes native SDKs for iOS and Android. These SDKs allow your web app to interact with native device features. Capacitor takes your existing web application (built with any technology stack) and runs it as a native app on each platform. It acts as a bridge between your web codebase and the native platform. It provides hooks into the native platform via JavaScript. These hooks allow you to access native features, such as device sensors, cameras, and file systems. Capacitor runs your web app inside a Web View component on each platform. This Web View acts as a container for your web content. Your Capacitor app has full access to the native platform under the hood. You can display native UI controls over the Web View, making it possible to create seamless user experiences.
- **NativeScript** is an open source framework that allows you to build truly native mobile apps using JavaScript or TypeScript. It provides a native UI and performance on both iOS and Android platforms. NativeScript combines an application framework with core modules, plugins, and NativeScript runtimes. These modules provide access to native APIs, allowing you to interact with device features like camera, geolocation, and more. NativeScript uses the native rendering engine of the target platform to ensure true native performance. When you build your app, NativeScript compiles your JavaScript code into Java (for Android) and Objective-C (for iOS). Instead of relying on virtual DOM diffing, NativeScript surgically updates the native view widgets based on your app's state changes.

In the following sections, we will explore how to use Capacitor to build mobile apps with Angular.

Making an Angular app native

Capacitor can modify an existing Angular app into a mobile app. To see this in action, take any Angular app, say the fruit app we created in *Chapter 26, Responsiveness*. Run it on a web browser in responsiveness mode, as shown:

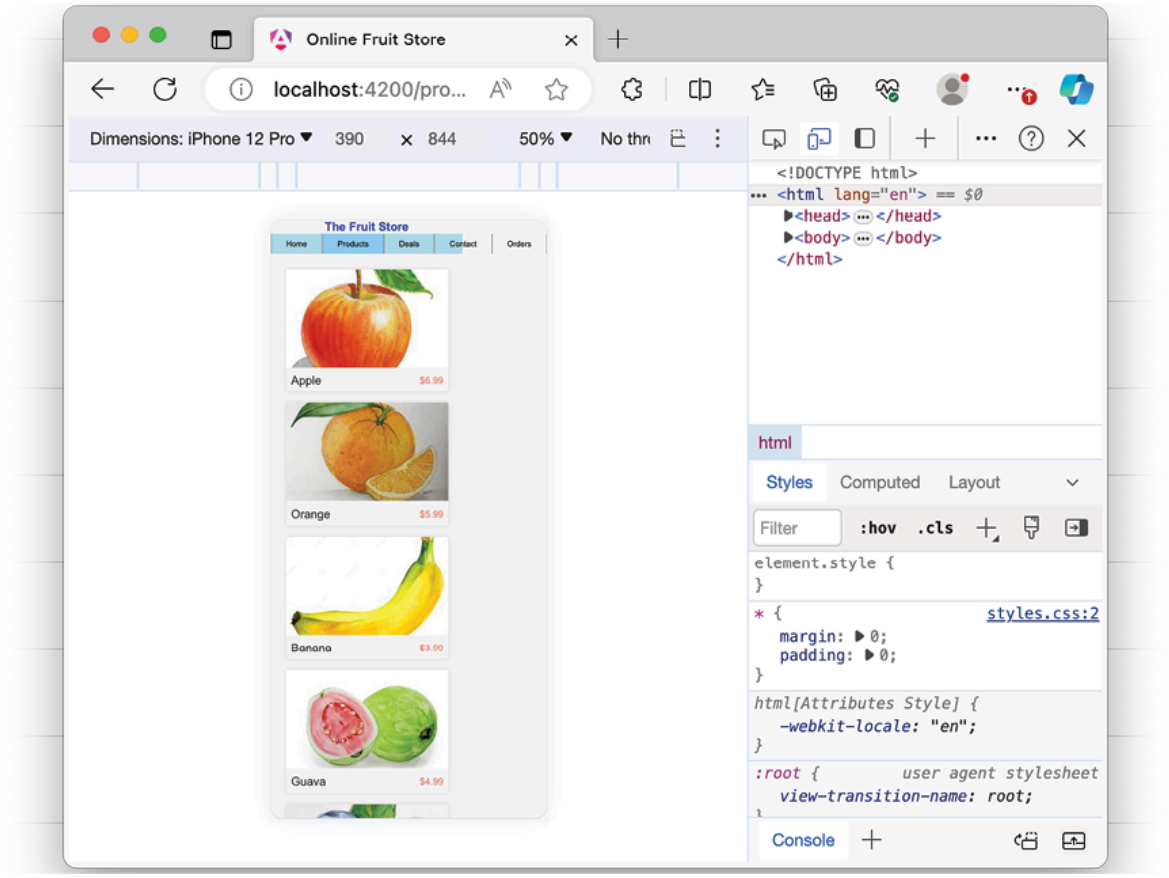


Figure 28.1: *Angular app rendered in a browser*

Now, run the following command in CLI to add Capacitor to the project:

ng add @capacitor/angular

The following figure shows the terminal screen on running the above command:

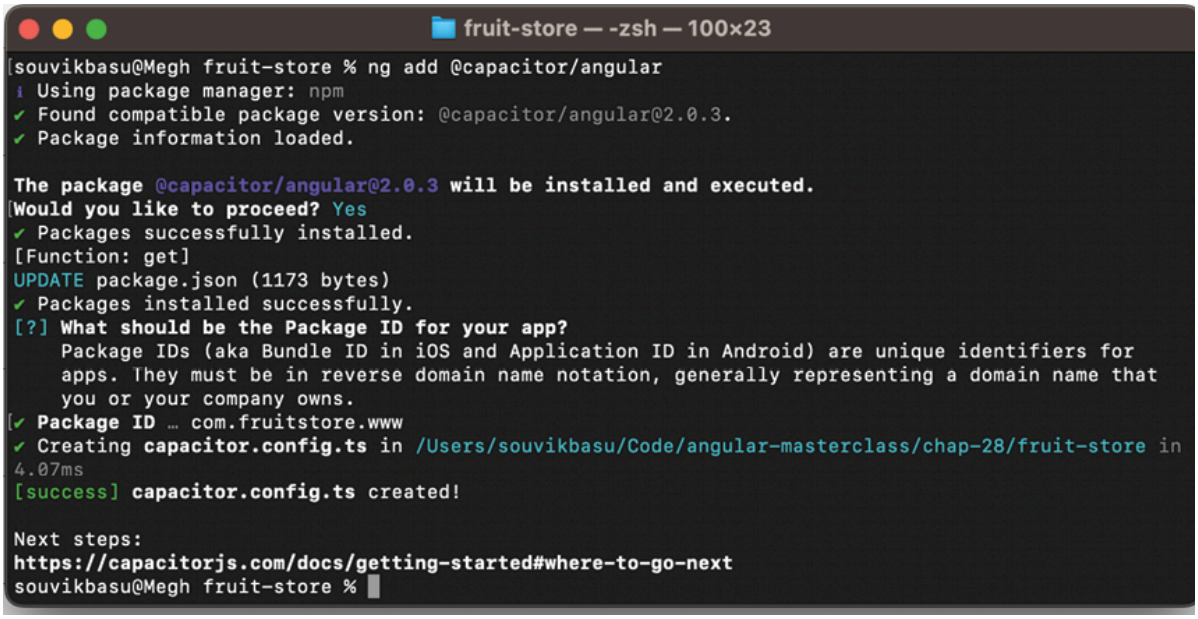


Figure 28.2: *Add capacitor to the project*

Build the app for production as follows:

ng build --configuration=production

Capacitor expects **index.html** to be present at **./dist/fruit-store** folder. If the above command creates the files under the **/browser** subfolder, move the files one level up to the parent folder.

Building an iOS app requires macOS, XCode, and CocoaPods. Whereas Android apps can be built in any OS. So, we will demo developing an Android app. Building an iOS app will follow similar corresponding steps.

Run the following command to install the native platform:

npm i @capacitor/android

npx cap add android

The commands will run on the terminal as follows:

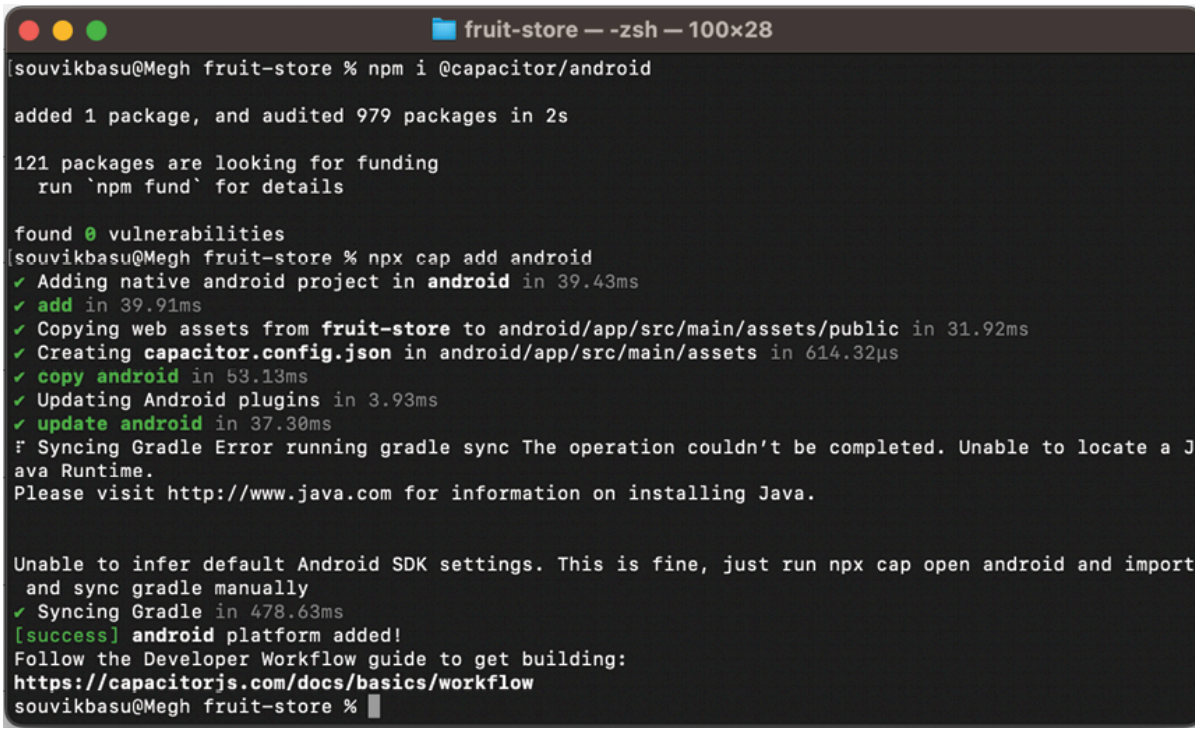


Figure 28.3: *Add native platform packages*

Now run the following to sync files between Angular and Android projects. Refer to *Figure 28.4*:

npx cap sync

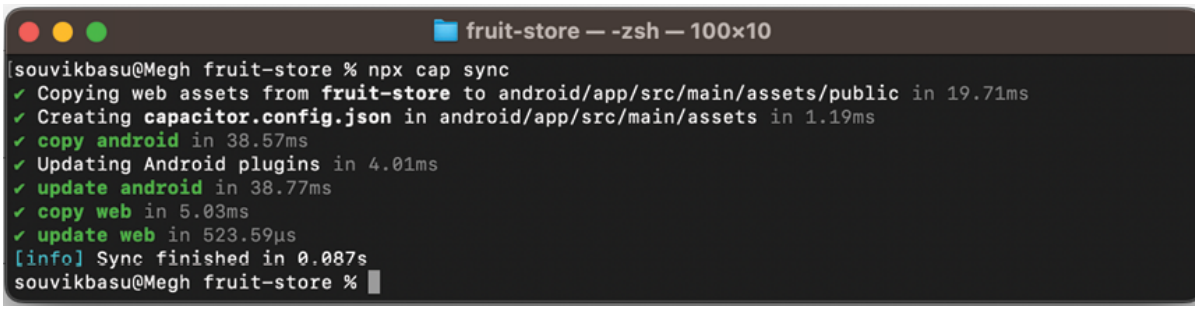


Figure 28.4: *Sync files between Angular and Android project*

Notice that an **android** folder is created at the root folder. This is the native app for Android. You can either run this in an emulator through the command line or in Android Studio. It is recommended to use Android Studio as it has all the necessary toolsets plugged in.

To run the app in CLI, run the following command:

npx cap run android

To open the app in Android Studio, run the following command:

npx cap open android

Now, click on the Run button at the top menu bar in Android Studio as shown:

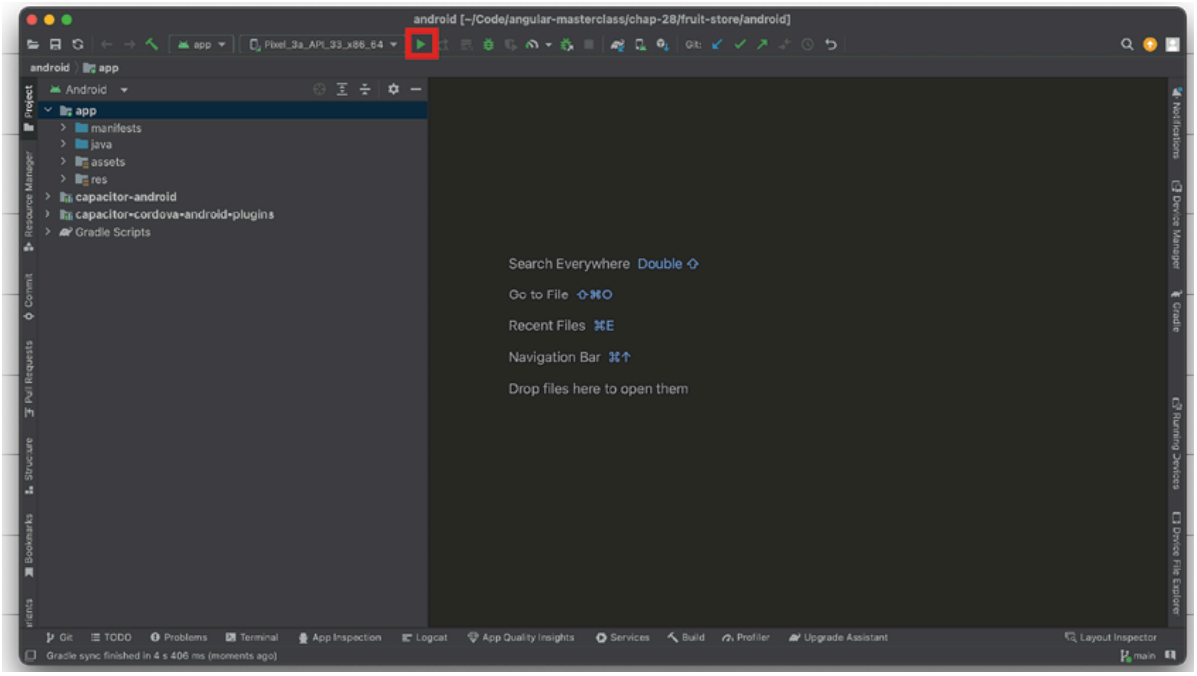


Figure 28.5: *Android studio*

If you have a device emulator setup, the app will load on the device as shown below:

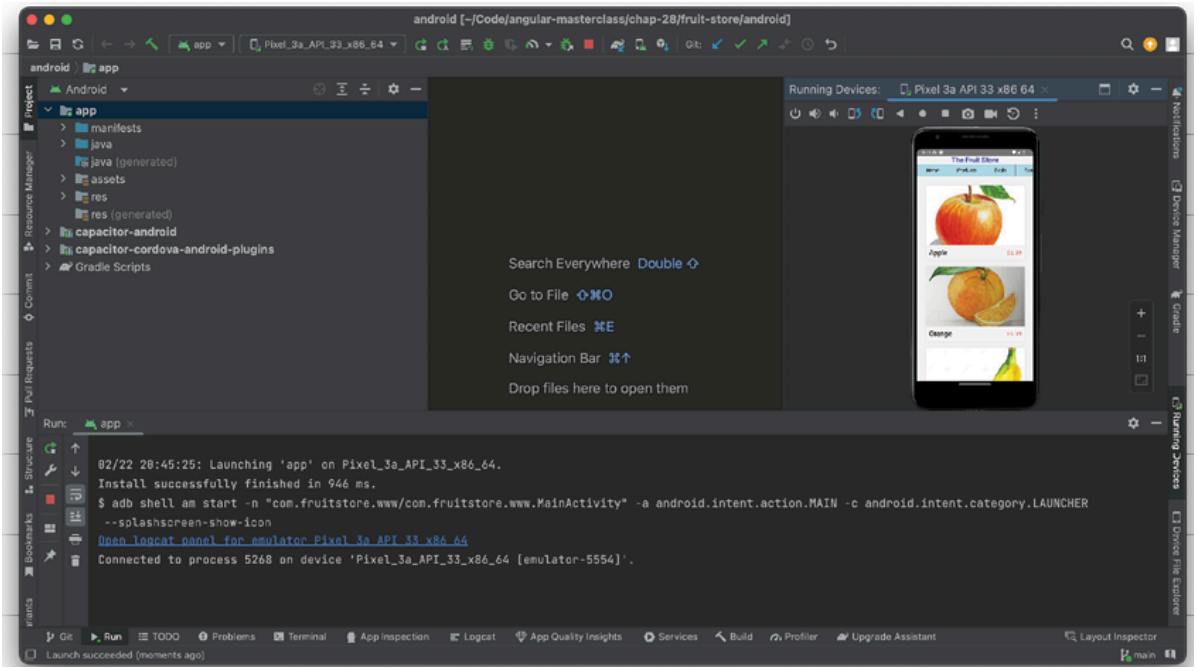


Figure 28.6: *Device emulator in Android Studio*

You can also have a phone connected to the computer in debug mode to see the app running on the phone. It is out of the scope of the book to dive deep into Android app development and debugging techniques. Please refer to the documentation link in the references section for further details.

With Capacitor, it is easy to convert any Angular app into a mobile app. You can even start an app from scratch using Capacitor. Please refer to the link to Capacitor

documentation in the references section.

Adding native features

In the above app, say you want to add camera support. For this, we need to install the camera package as follows:

npm install @capacitor/camera

npx cap sync

The camera feature uses PWA elements from the Ionic framework. Install it as follows:

npm install @ionic/pwa-elements

Open the **main.ts** file and add the following code:

```
import { defineCustomElements } from '@ionic/pwa-elements/loader';
// Call the element loader before the bootstrapModule/bootstrapApplication call
defineCustomElements(window);
```

Replace **app.component.ts** with the following code:

```
import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Camera, CameraResultType } from '@capacitor/camera';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  photo:string | undefined = undefined;

  async takePhoto() {
    const image = await Camera.getPhoto({
      quality: 90,
      allowEditing: true,
      resultType: CameraResultType.Uri
    });

```

```
23.     this.photo = image.webPath;
24.   };
25. }
```

Finally, replace the view template code in **app.component.html** as follows:

```
1. <button (click)="takePhoto()">Take photo</button>
2. <img [src]="photo" *ngIf="photo" class="photo-frame" />
```

Now, build the project for production again and run it in Android Studio. Click on the **Take photo** button. This will open the camera app to take a picture and show the clicked photo on the app. Refer to the following figure:

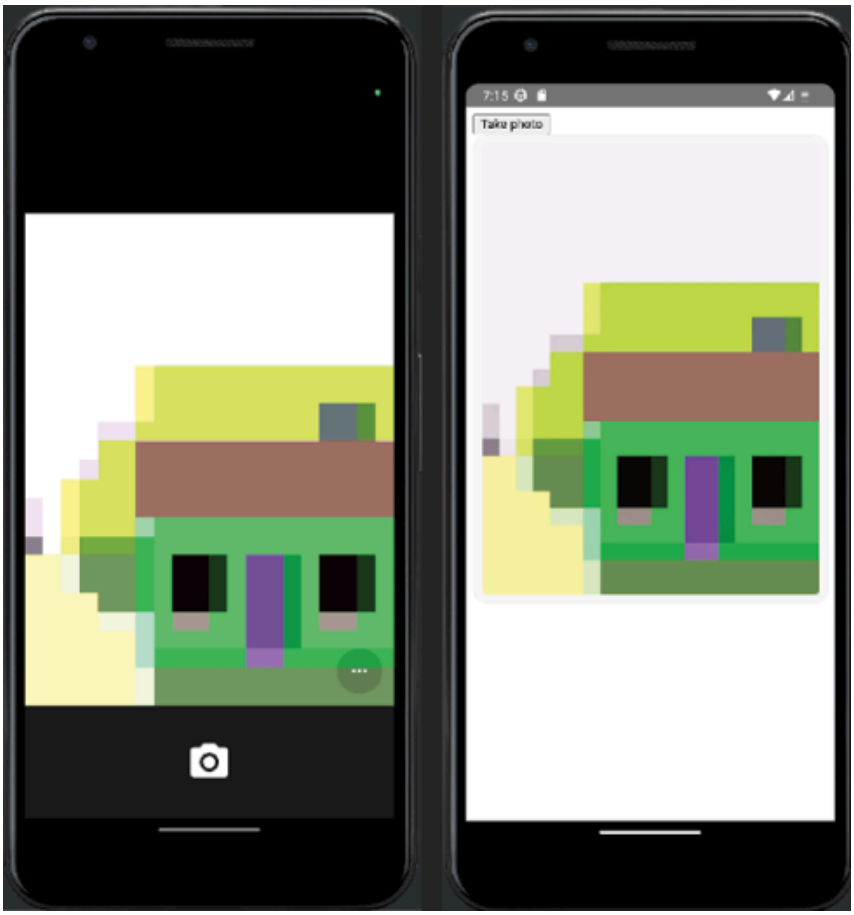


Figure 28.7: *Using camera in the native app*

You can integrate many such native capabilities into your app now. Please check the Capacitor documentation for further details.

Conclusion

Creating a native mobile app using Angular can provide many advantages over a responsive web page on a mobile device. With libraries like Capacitor and NativeScript, building a native app has become easier for developers. Capacitor, with its native SDKs for iOS and Android, allows developers to access native device features like camera, contacts, and microphone via JavaScript. On the other hand,

NativeScript provides a native UI and performance on both iOS and Android platforms. By leveraging these tools and techniques, developers can create mobile apps that deliver superior performance, specialized functionality, offline functionality, and enhanced discoverability and credibility.

In the next chapter, we will explore how to manage the state in an Angular app.

Points to remember

- Native mobile apps provide superior performance, access to device features, offline functionality, and enhanced security compared to web pages.
- Capacitor and NativeScript are two popular libraries that allow us to create mobile apps using Angular.
- Capacitor acts as a bridge between your web codebase and the native platform, allowing you to access native features and display native UI controls over the Web View.
- NativeScript provides a native UI and performance on both iOS and Android platforms by compiling your JavaScript code into Java (for Android) and Objective-C (for iOS).

Questions

1. What are the advantages of a native mobile app over a web page?
2. How can developers create mobile apps using Angular?
3. How does Capacitor modify an existing Angular app into a mobile app?
4. What are some device features that native apps can access directly?
5. What are some advantages of listing an app in app stores like the App Store and Google Play?

References

- Capacitor docs:
  https://capacitorjs.com/docs
- NativeScript with Angular:
  https://blog.angular.io/apps-that-work-natively-on-the-web-and-mobile-9b26852495e7
- Android Studio:
  https://developer.android.com/studio/intro

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 29
State Management

Introduction

State management is an essential aspect of any modern web application, and the Angular community provides developers with a robust set of tools to manage application state efficiently. In Angular, state management involves managing an application's data and UI state that can be shared across different components. This chapter will cover the key concepts of state management in Angular, including how to manage the application state using services, observables, and state management libraries such as NgRx. By the end of this chapter, you will have a solid understanding of how to manage state in your Angular application effectively.

Structure

This chapter covers the following topics:

- The need for a state management
- Approaches to state management
- Debugging state changes

Objectives

By the end of this chapter, you will understand how to share a common application state in Angular.

The need for a state management

State management refers to the process of managing and updating the data and **user interface** (**UI**) state of a web application. It involves maintaining consistency in the application state across different components and ensuring that

all components have access to the most up-to-date state.

State management is essential because it helps improve a web application's overall performance, scalability, and maintainability. By centralizing the management of the application state, developers can avoid duplication of code, reduce the likelihood of errors, and make it easier to debug and test the application. Additionally, state management allows developers to build more complex and dynamic applications that can respond dynamically to user interactions and changes in the data. In short, here are some reasons why you should consider state management:

- **Centralized data**: Storing data in a central location (state) makes it accessible to all components, ensuring consistency and avoiding data duplication.
- **Component communication**: Components can communicate with each other by reading and updating the shared state.
- **Predictable behavior**: A well-managed state ensures predictable behavior across your application.

Approaches to state management

The rest of the chapter will look at how to provide state management in an Angular application. There are three ways to manage the state:

- Local component state
- Service based state
- Third-party state management libraries like NgRx and MobX.

Local component state

Local component state is the simplest case of state management. Here, we manage the state within a component using component properties. Refer to the following:

```
1. export class CounterComponent {
2.   count = 0;
3.
4.   increment() {
5.     this.count++;
6.   }
7. }
```

In the above component, the value of the property **count** is maintained within the component throughout the component's lifespan. If the value is modified by any method, the new value is rendered in the view through the change detection process. The state is local to the component and cannot be shared with other

components or modules within the application.

Service based state

In *Chapter 4, Building Blocks of Angular App*, we looked at how to create services. A service is a class with a narrow, well-defined purpose. It encapsulates the functionality that an application needs, such as fetching data from a server, logging, or validating user input. Services are designed to be reusable, promoting modularity and separation of concerns. Unlike components, services handle tasks that do not directly involve the view or application logic. Hence, services are a preferred way to store application states shared between components. The component state code sample in the previous sample can be rewritten into a service as follows:

```
@Injectable({ providedIn: 'root' })
export class CounterService {
  private count = 0;

  increment() {
    this.count++;
  }

  getCount() {
    return this.count;
  }
}
```

Now, this service can be used in multiple components that can modify the counter and access the shared value. Refer to the following:

```
import { Component } from '@angular/core';
import { StateService } from './state.service';

@Component({
  selector: 'app-counter',
  template: `
    <p>Count: {{ stateService.getCount() }}</p>
    <button (click)=»increment()»>Increment</button>
  `,
})
export class CounterComponent {
```

```
12.   constructor(private stateService: StateService) {}
13.
14.   increment(): void {
15.     this.stateService.increment();
16.   }
17. }
```

NgRx

NgRx is a popular library for managing the application state in an Angular app using the Redux (Flux) pattern. It is a framework for building reactive applications in Angular. It provides tools and patterns to manage application state, handle side effects, and ensure predictable behavior. The following are the core building blocks of NgRx:

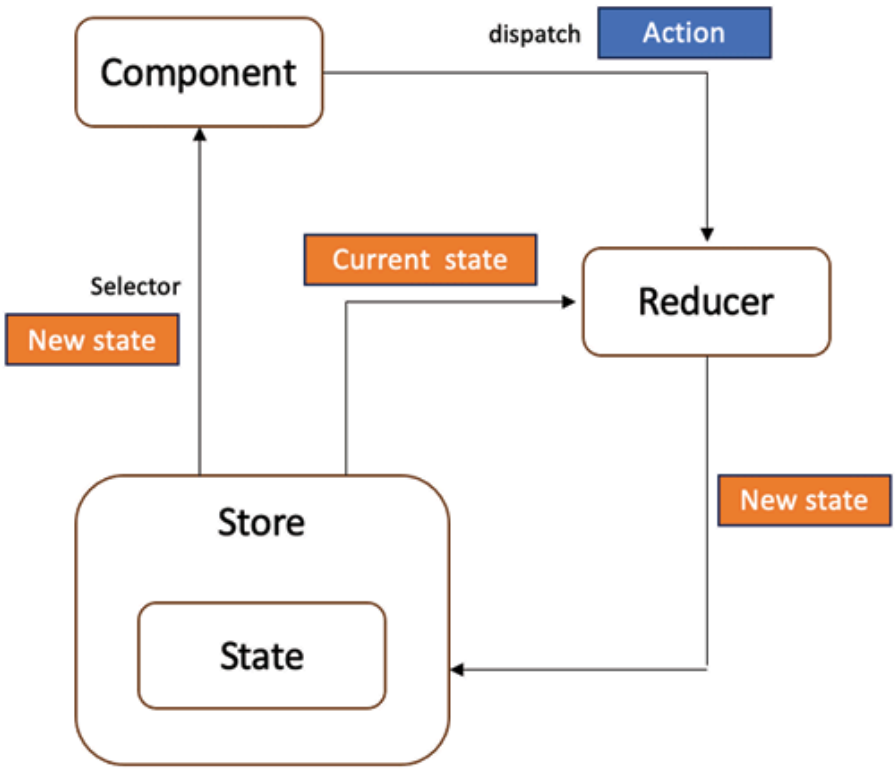


Figure 29.1: *Flow diagram of Redux pattern*

Let us take a look at the components:

- **Store**: This is the central repository for your application's state.
- **Actions**: When a user interacts with the view, like clicking a button, the component triggers an Action.
- **Reducers**: These are pure functions that handle state changes based on actions. A reducer receives the current state from the store, and based on the action, it creates a new state. The new state then replaces the state within the

store.

- **Selectors**: When the store state changes, it notifies all components about the change in state properties. A component subscribes to a store using a selector. A selector allows listening to a specific property within the state, as the component may not be interested in getting the entire state object.
- **Effects**: Manage side effects (for example, API calls, navigation) triggered by actions.

Let us create a demo app to see this in action:

ng new ngrx-demo

Create a component that shows a counter and has buttons to increment the counter, as follows:

ng g c counter

Add the following code in **counter.component.ts**:

```
1. import { Component } from '@angular/core';
2.
3. @Component({
4.   selector: 'app-counter',
5.   standalone: true,
6.   imports: [],
7.   template: `
8.    <p>Count: {{ currentCount }}</p>
9.    <button (click)="incrementCounter()">Increment</button>
10.    <button (click)="incrementCounterBy(2)">Increment by 2</button>
11.   `,
12.   styleUrl: './counter.component.css'
13. })
14. export class CounterComponent {
15.   public currentCount = 0;
16.
17.   public incrementCounter() {
18.     this.currentCount++;
19.   }
20.
21.   public incrementCounterBy(step: number) {
22.     this.currentCount = this.currentCount + step;
23.   }
```

```
}
```

For brevity, the view template is written inline. It is a simple component that shows a counter and has two buttons to increment by a value of 1 and 2, respectively. To demonstrate the need for a state management library, we have created a local **currentCount** property within the component for now.

Similarly, create another component that decrements the counter, as follows:

ng g c countdown

Replace the code in **countdown.component.ts** file as follows:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-countdown',
  standalone: true,
  imports: [],
  template: `
    <p>Count: {{ currentCount }}</p>
    <button (click)="incrementCounter()">Increment</button>
     <button (click)="decrementCounter()">Decrement</button>
  `,
  styleUrl: './countdown.component.css'
})
export class CountdownComponent {
  public currentCount = 0;

  public decrementCounter() {
    this.currentCount--;
  }
}
```

Use these components in the **app.component.ts** file as follows:

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { CounterComponent } from './counter/counter.component';
import { CountdownComponent } from './countdown/countdown.component';

@Component({
  selector: 'app-root',
```

```
8.  standalone: true,
9.  imports: [RouterOutlet, CounterComponent, CountdownComponent],
10.  template: `
11.    <app-counter></app-counter>
12.    <app-countdown></app-countdown>
13.  `,
14.  styleUrl: './app.component.css'
15. })
16. export class AppComponent {
17.  title = 'ngrx-demo';
18. }
```

This will render on the browser as follows:

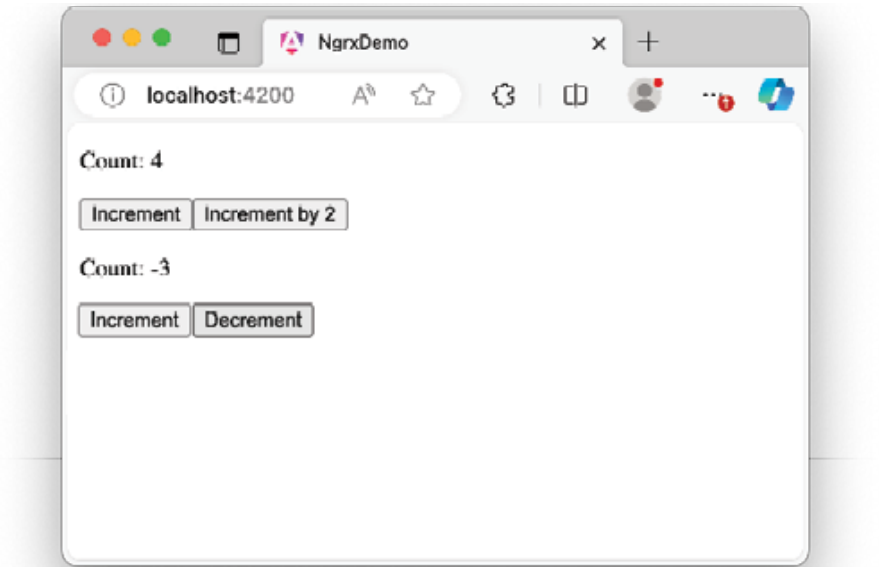


Figure 29.2: *Local state using component properties*

Notice that clicking on the individual buttons shows different values of the respective **Count** as the state is local to each component. The only way to share the value of the **currentCount** property is through the parent **AppComponent** that uses these two components. This is the issue with the local state that you cannot share a common state with other sections of the application. Now, let us use NgRx to share a common count state between these two components.

To use NgRx, add the package as follows:

```
ng add @ngrx/store@latest
```

For now, our state will contain just a numerical value **count**. In a real-world application, the state will be a complex object with multiple properties.

Let us start by defining the structure of the application state. Create a file **/app/store/index.ts** with the following content:

```
export interface CounterState {
    count: number;
}

export interface AppState {
    counter: CounterState;
}
```

It defines a state interface **CounterState** with a numerical **count** field. It also defines a top-level **AppState** that contains **CounterState** as one of its members. This is a helpful convention used in NgRx code to divide the application state into smaller feature-specific states and return specific feature states to individual components using selectors.

Now, create the actions. An action in NgRx is an implementation of the following interface:

```
interface Action {
  type: string;
}
```

An action must have a unique type to differentiate it from others. It is a string in the format **[Source] Event**. **Source** determines where the action is dispatched from (mostly the name of the page or component), and the **Event** determines the category of the action. Additionally, it can have data that may be needed to perform the action. NgRx provides a helper method to create actions. Let us create actions to increment and decrement the counter. As a convention, the actions are defined in a file within the component folder as those are tightly coupled with the user interaction on the view.

Create a file **/app/counter/counter.actions.ts** with the following code:

```
import { createAction, props } from '@ngrx/store';

export const increment = createAction(
    '[Counter] Increment',
);

export const incrementBy = createAction(
    '[Counter] Increment By',
    props<{ step: number }>()
);
```

The above code creates two actions **increment** and **incrementBy**. The **increment** action is simplistic, it does not contain any payload. Hence, to demo an action

with a payload, we have added **incrementBy,** which takes a numerical step param.

Similarly, create action(s) for the other component in the **countdown.actions.ts** file. We will add an **increment** and a **decrement** action for **CountdownComponent**. Notice how the increment actions defined for both components differ in type due to different source values.

```
import { createAction } from '@ngrx/store';

export const increment = createAction(
    '[Countdown] Increment',
);

export const decrement = createAction(
    '[Countdown] Decrement'
);
```

Next, create the reducer methods in file **/app/store/counter.reducer.ts** file as follows:

```
import { createReducer, on } from '@ngrx/store';
import * as CounterActions from '../counter/counter.actions';
import * as CountDownActions from '../countdown/countdown.actions';
import { CounterState } from '.';

export const initialState: CounterState = {
    count: 0
};

export const counterReducer = createReducer(
    initialState,
  on(CounterActions.increment, state => ({ ...state, count: state.count + 1 })),
  on(CounterActions.incrementBy, (state, { step }) => ({ ...state, count: state.count + step })),
  on(CountDownActions.increment, state => ({ ...state, count: state.count + 1 })),
  on(CountDownActions.decrement, state => ({ ...state, count: state.count - 1 })),
);
```

Line 6 defines the initial values of the **CounterState**. *Lines 10-16* modify the state based on the actions we defined previously. Each of these handlers takes the current state and sets a new state object based on the action.

Next, define the selectors in **/app/store/ counter.selectors.ts** that the components can use to display the **CounterState**, as follows:

```
1. import { createSelector } from '@ngrx/store';
2. import { AppState, CounterState } from '.';
3.
4. export const selectCounterFeature = (state: AppState) => state.counter;
5.
6. export const selectCounter = createSelector(
7.   selectCounterFeature,
8.   (state: CounterState) => state.count
9. );
```

Line 4 creates a selector to select the **CounterState** within **AppState,** and *line 6* defines a selector to pick the **count** property within **CounterState**.

Finally, we will use these selectors within our existing components. There are multiple ways to consume the state data. We will show the different ways in these two components.

```
1. ...
2. import { Store } from '@ngrx/store';
3. import { increment, incrementBy } from './counter.actions';
4. import { selectCounter } from '../store/counter.selectors';
5. import { AppState } from '../store';
6.
7. @Component({
8.   ...
9. })
10. export class CounterComponent {
11.   currentCount = 0;
12.
13.   constructor(private store: Store<AppState>) {
14.     store.select(selectCounter).subscribe((count) => {
15.       this.currentCount = count;
16.     });
17.   }
18.
```

```
19.   public incrementCounter() {
20.     this.store.dispatch(increment());
21.   }
22.
23.   public incrementCounterBy(step: number) {
24.     this.store.dispatch(incrementBy({ step: step }));
25.   }
26. }
```

For clarity, we have only shown the lines that changed in this component. The following are the changes:

- The **Store** object is injected into the component constructor now.
- *Line 14* uses the selector created previously that returns an observable.
- The existing **currentCount** field in the component is updated when there is a new value available in the observable stream.
- The methods **incrementCounter()** and **incrementCounterBy(step: number)** now call the store dispatch method and send the appropriate action instead of modifying the local field **currentCount**.

When the user clicks on the Increment or Increment By button, the corresponding action is dispatched. This action is picked up by the reducer, which will modify the state within the store. When the state within the store is modified, the selectors get triggered reactively, and hence, the component gets to know of it and updates the **currentCount** field. This updated value of **currentCount** then gets printed on the view through change detection.

Another cleaner way to write the same component is to use an observable within the component. This is demonstrated in the other component **CountdownCompenent** as follows:

```
1. ...
2. import { Store } from '@ngrx/store';
3. import { selectCounter } from '../store/counter.selectors';
4. import { Observable } from 'rxjs';
5. import { decrement, increment } from './countdown.actions';
6. import { AsyncPipe } from '@angular/common';
7. import { AppState } from '../store';
8.
9. @Component({
10.   ...
11.   imports: [AsyncPipe],
```

```
12.   template: `
13.   <p>Count: {{ count$ | async }}</p>
14.   ...
15.   `,
16.   ...
17. })
18. export class CountdownComponent {
19.   private readonly store: Store<AppState> = inject(Store);
20.   count$: Observable<any> = this.store.select(selectCounter);
21.
22.   public incrementCounter() {
23.     this.store.dispatch(increment());
24.   }
25.
26.   public decrementCounter() {
27.     this.store.dispatch(decrement());
28.   }
29. }
```

Again, for clarity, we have printed only the modified lines. The main change here is the usage of the **count$** observable instead of the numerical field **currentCount** in the previous component.

To connect all these together, we need to provide the store in the **app.config** file as follows:

```
1. ...
2. import { provideStore } from '@ngrx/store';
3. import { counterReducer } from './store/counter.reducer';
4.
5. export const appConfig: ApplicationConfig = {
6.   providers: [provideRouter(routes), provideStore({ counter:
     counterReducer})]
7. };
```

Now, when you run the app, you will observe that the **count** value changes simultaneously if any button is clicked. Refer to the following figure:

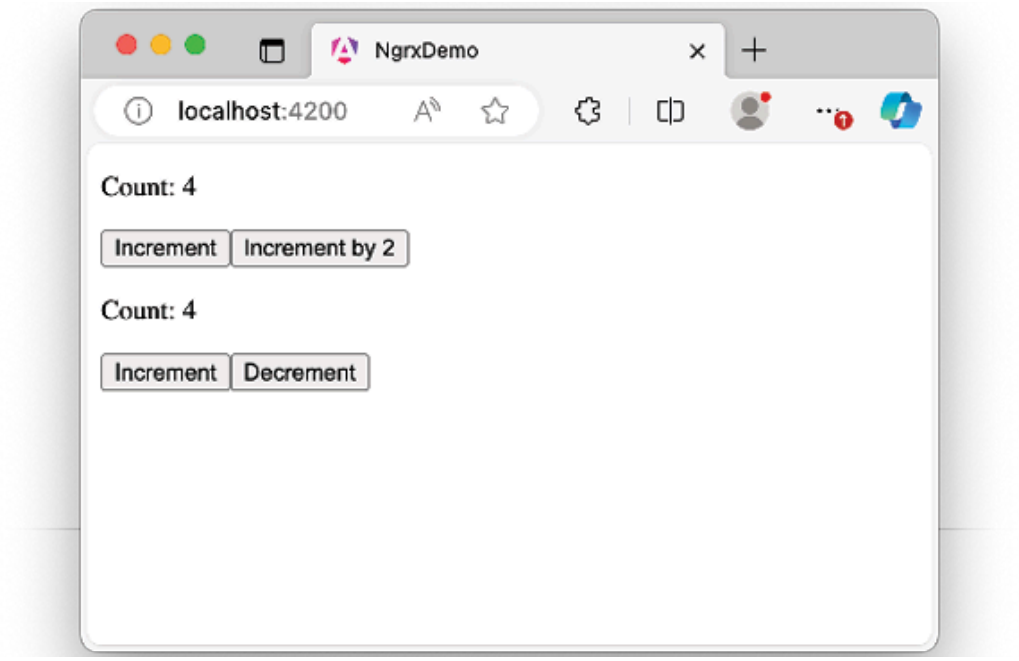


Figure 29.3: *Shared state using NgRx*

Thus, these two components now share the same state. Modification by one is picked up by the other. Even if the two components are defined on separate pages, they will be able to work with the shared state now as long as the pages are rendered on the client side. This way, a state management library like NgRx helps to create a commonly shared application state. We used a very simplistic demo to showcase the usage of the NgRx library. In practice, there will be side effects, like making an API call to update the state values. You can explore more about effects, entities, and other advanced concepts in the NgRx documentation given in the references section.

There are other popular state management libraries, like MobX, Akita, NGXS, etc., that you can explore as well and use the one that suits your project needs. Discussion of all the libraries is beyond the scope of this book and reference links are provided to the other libraries.

Debugging state changes

One major advantage of using NgRx is that the state changes can be monitored in the dev tool. Follow these steps to debug it. Install the dev tool package as follows:

npm install @ngrx/store-devtools

Install the Redux dev tool browser extension. Open the **app.config** file and add the following lines:

```
1. ...
2. import { provideStoreDevtools } from '@ngrx/store-devtools';
3.
```

```
4.  export const appConfig: ApplicationConfig = {
5.   providers: [provideRouter(routes), provideStore({ counter:
    counterReducer}),
6.      provideStoreDevtools({
7.        maxAge: 30,
8.        logOnly: !isDevMode(),
9.        autoPause: true,
10.         trace: false,
11.         traceLimit: 70
12.     }),]
13.  };
```

Now, you can see the state changes in the dev tool by opening the Redux tab (or clicking on the Redux extension in the browser menu), as follows:

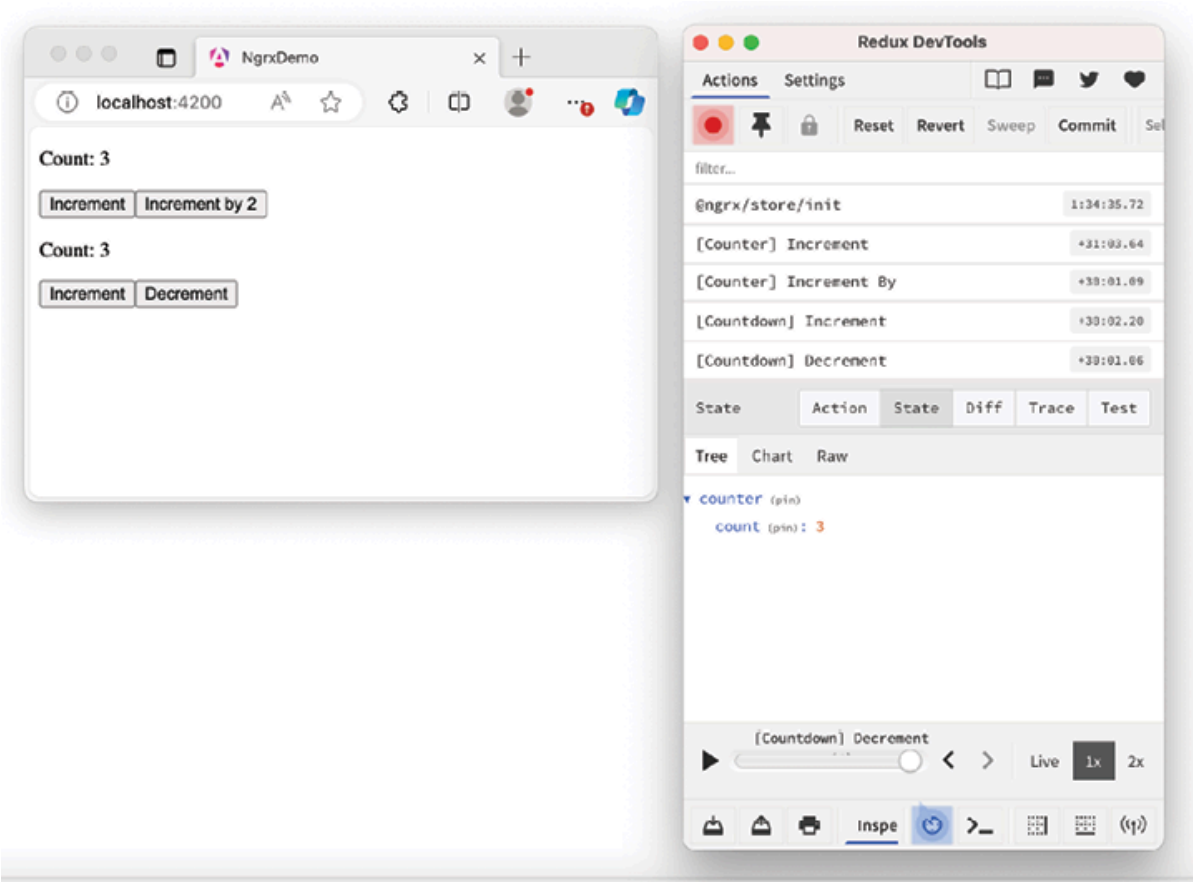


Figure 29.4: *Redux DevTool browser extension*

Conclusion

State management is a crucial aspect of building modern web applications. Angular provides developers with several powerful tools to manage the

application state efficiently, including local component state, service-based state, and third-party state management libraries like NgRx. By centralizing the management of the application state, developers can avoid duplication of code, reduce the likelihood of errors, and make it easier to debug and test the application. In the next chapter, we will explore how to instrument an Angular app to log errors.

Points to remember

- State management involves managing an application's data and UI state that can be shared across different components.
- Local component state is the simplest way of state management, where we manage the state within a component using component properties.
- Service-based state is a preferred way to store an application state that is shared between components.
- NgRx is a popular library for managing the application state in an Angular app using the Redux (Flux) pattern.
- NgRx provides tools and patterns to manage application state, handle side effects, and ensure predictable behavior.
- By centralizing the management of the application state, developers can avoid duplication of code, reduce the likelihood of errors, and make it easier to debug and test the application.

Questions

1. What is state management in Angular?
2. Why is state management important for web applications?
3. What are the three approaches to state management in Angular?
4. What are the core building blocks of NgRx?
5. How does state management improve the maintainability and scalability of a web application?
6. What are some benefits of using a service-based state management approach over local component state?

References

- NgRx doc:
  https://ngrx.io/docs
- MobX:
  https://ngrx.io/docs

- Redux DevTools browser extension (Chrome/Edge):
  https://chromewebstore.google.com/detail/redux-devtools/lmhkpmbekcpmknklioeibfkpmmfibljd

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 30
Telemetry and Instrumentation

Introduction

Telemetry and instrumentation play a crucial role in developing any software application. In the context of Angular, **telemetry** refers to the collection and analysis of data related to the application's performance and usage. **Instrumentation**, on the other hand, involves integrating tools and techniques that enable developers to monitor and diagnose issues in the application. In this chapter, we will explore the concepts of telemetry and instrumentation in the context of Angular and understand how to effectively manage the application using these techniques.

Structure

This chapter covers the following topics:

- Instrumentation of Angular app
- Setup Application Insights

Objectives

By the end of this chapter, you will understand how to instrument an Angular app to collect telemetry data.

Instrumentation of Angular app

Instrumentation refers to the process of adding observability libraries or

dependencies to an application's code. These libraries allow the application to emit logs, traces, and metrics. Instrumentation makes a system observable. In other words, it enables you to collect data about the system's behavior, performance, and internal workings. Telemetry encompasses the automated collection and transmission of data from remote sources. It includes various types of data, such as:

- **Logs**: Text-based records of events, errors, or activities within the system.
- **Metrics**: Quantitative measurements (for example, response time, CPU utilization) that provide insights into system performance.
- **Traces**: Distributed traces that show the flow of requests across different components.

Think of telemetry as the data streams flowing from your system, while observability is the lens through which you analyze and understand that data.

There are a few popular libraries for instrumenting an Angular app, as follows:

- OpenTelemetry from *Cloud Native Computing Foundation* (*CNCF*)
- Azure Application Insights from *Microsoft*
- Sentry

OpenTelemetry is an open source project under CNCF that aims to standardize the generation and collection of telemetry data (logs, metrics, and traces). It allows you to trace Angular applications for performance issues and bugs. You can use OpenTelemetry Angular libraries to instrument your Angular apps and generate traces from your app to downstream services. **SigNoz** is a popular backend analysis tool. It is a full-stack open source **application performance monitoring** (**APM**) tool built natively on OpenTelemetry. It supports **OpenTelemetry Protocol** (**OTLP**) data formats and provides query and visualization capabilities for metrics and traces. You can find setup steps and further details in the documentation link from the references section.

Application Insights is a service provided by *Microsoft* for monitoring and diagnosing live applications. While not specific to Angular, Application Insights is an Azure service designed for monitoring web applications. It allows you to add logging and telemetry to your Angular app. It helps you track user interactions, detect performance anomalies,

and diagnose issues. If your app is deployed on Azure, Application Insights is a preferred monitoring tool.

In the next section, we will examine the setup steps for Application Insights for your Angular app. Once you understand the essential workings of any telemetry tool, choosing one over the other should be straightforward.

Setup Application Insights

Let us create a new Angular app to demo instrumentation using Application Insights:

ng new telemetry

1. You will need an Azure subscription to use Application Insights. If you do not have an account on Azure, you can create a trial account to try this demo. Open **https://portal.azure.com/** and click on **Create a resource** (or the **Application Insights** icon if shown in the suggested list of services). Refer to the following figure:

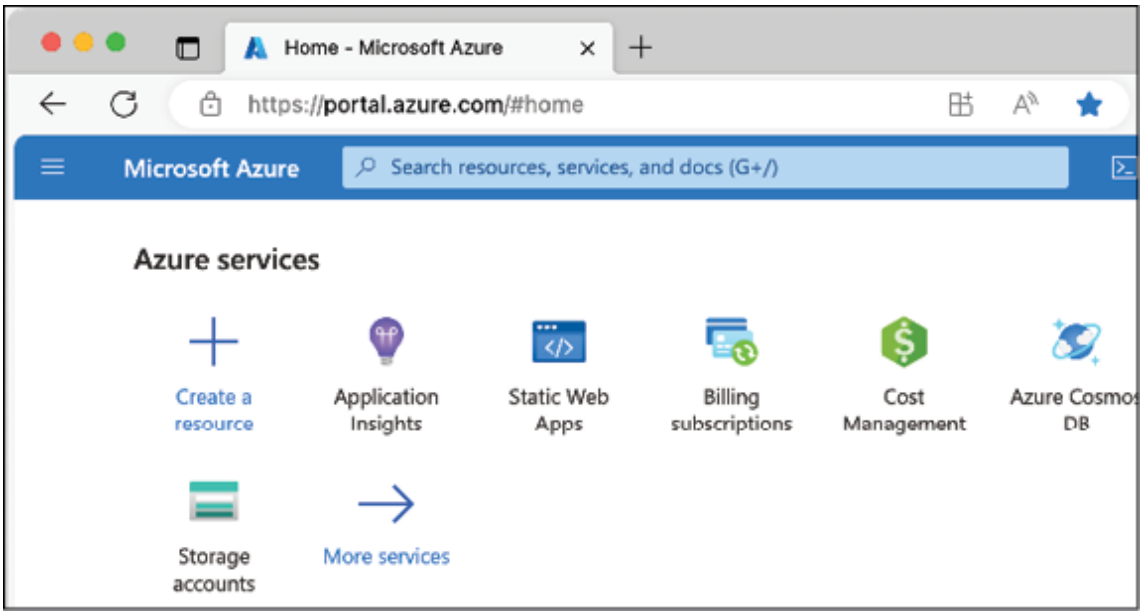


Figure 30.1: *Create a resource in Azure Portal*

2. Search for **Application Insights** from the list and create an instance, as shown in the following figure:

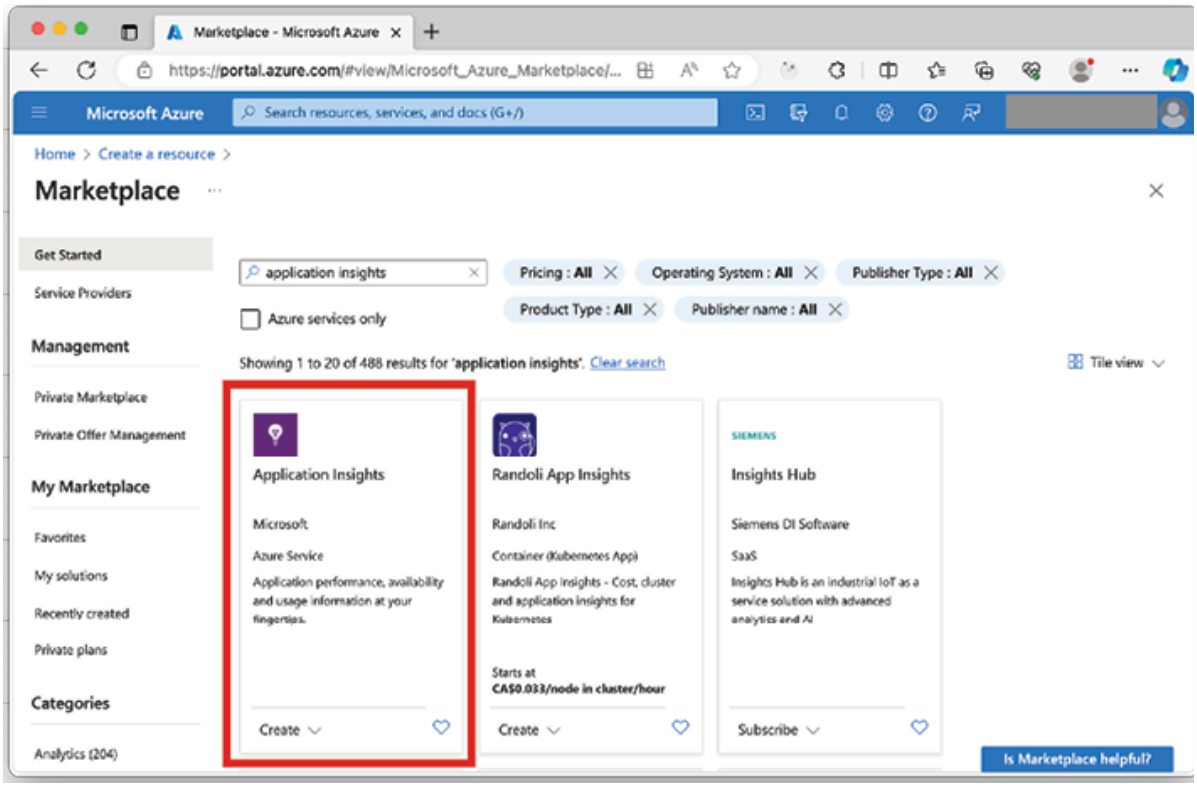


Figure 30.2: *Search for Application Insights*

Refer to the following figure:

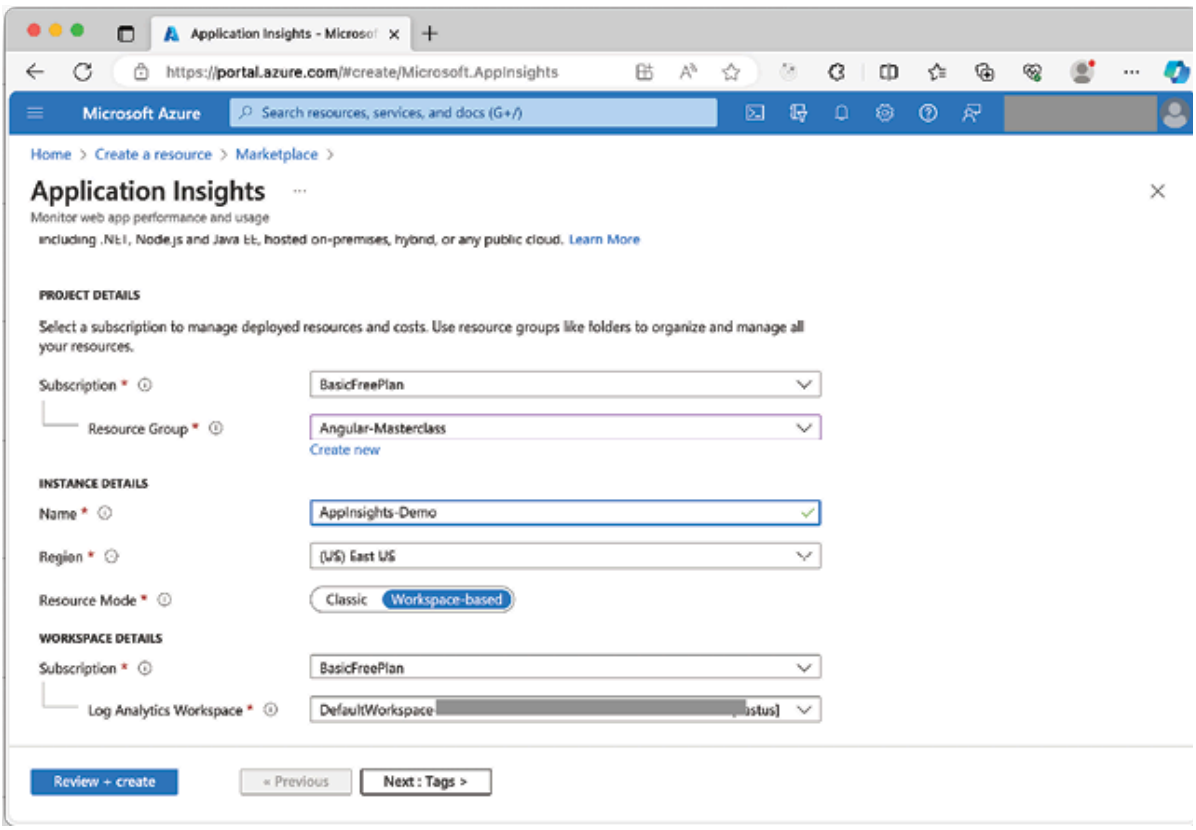


Figure 30.3: *Create an instance of Application Insights*

3. Copy the **Connection String** from the overview page, as shown:

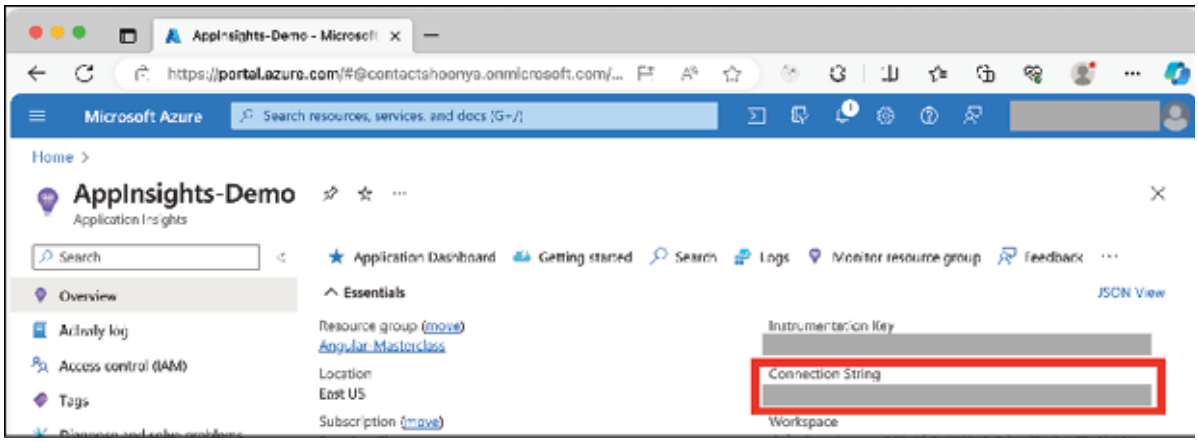


Figure 30.4: *Copy the Connection String*

4. Open the terminal prompt for the Angular project and install Application Insights package to the project:

   npm install @microsoft/applicationinsights-web

5. Create a new service to provide telemetry services:

   ng g s telemetry

6. Copy the following code into the service to create methods for different telemetry options:

```
1. import { Injectable } from '@angular/core';
2. import { ApplicationInsights } from
   '@microsoft/applicationinsights-web';
3.
4. @Injectable({
5.  providedIn: 'root'
6. })
7. export class TelemetryService {
8.  appInsights: ApplicationInsights;
9.  constructor() {
10.  this.appInsights = new ApplicationInsights({
11.  config: {
12.  connectionString: '<your connection string>',
13.  /* ...Other Configuration Options... */
14.  }
15.  });
```

```
    this.appInsights.loadAppInsights();
  }

  logPageView(name?: string, uri?: string) {
  this.appInsights.trackPageView({ name, uri});
  }

  logEvent(name: string, properties?: { [key: string]: any }) {
  this.appInsights.trackEvent({ name }, properties);
  }

  logMetric(name: string, average: number, properties?: { [key:
  string]: any }) {
  this.appInsights.trackMetric({ name , average }, properties);
  }

  logException(exception: Error, severityLevel?: number) {
  this.appInsights.trackException({ exception , severityLevel });
  }

  logTrace(message: string, properties?: { [key: string]: any }) {
  this.appInsights.trackTrace({ message }, properties);
  }
}
```

7. Now open the **app.component.ts** file and call the method to log page views and a **Loaded** event as follows:

```
...
import { TelemetryService } from './telemetry.service';

...
export class AppComponent {
 ...
 constructor(private telemetryService: TelemetryService) {
 this.telemetryService.logPageView('AppComponent', '/');
 this.telemetryService.logEvent('AppComponent', { event:
```

```
    'AppComponent Loaded' });
10.  }
11. }
```

This is a demo code. In a real-world application, you will call appropriate telemetry methods as needed by the application.

8. Now open the Application Insights page on the Azure portal and go to the **Performance** tab. Toggle **Server** to **Browser** to see client-side generated data. You will notice a log recorded for the page visit, as shown:

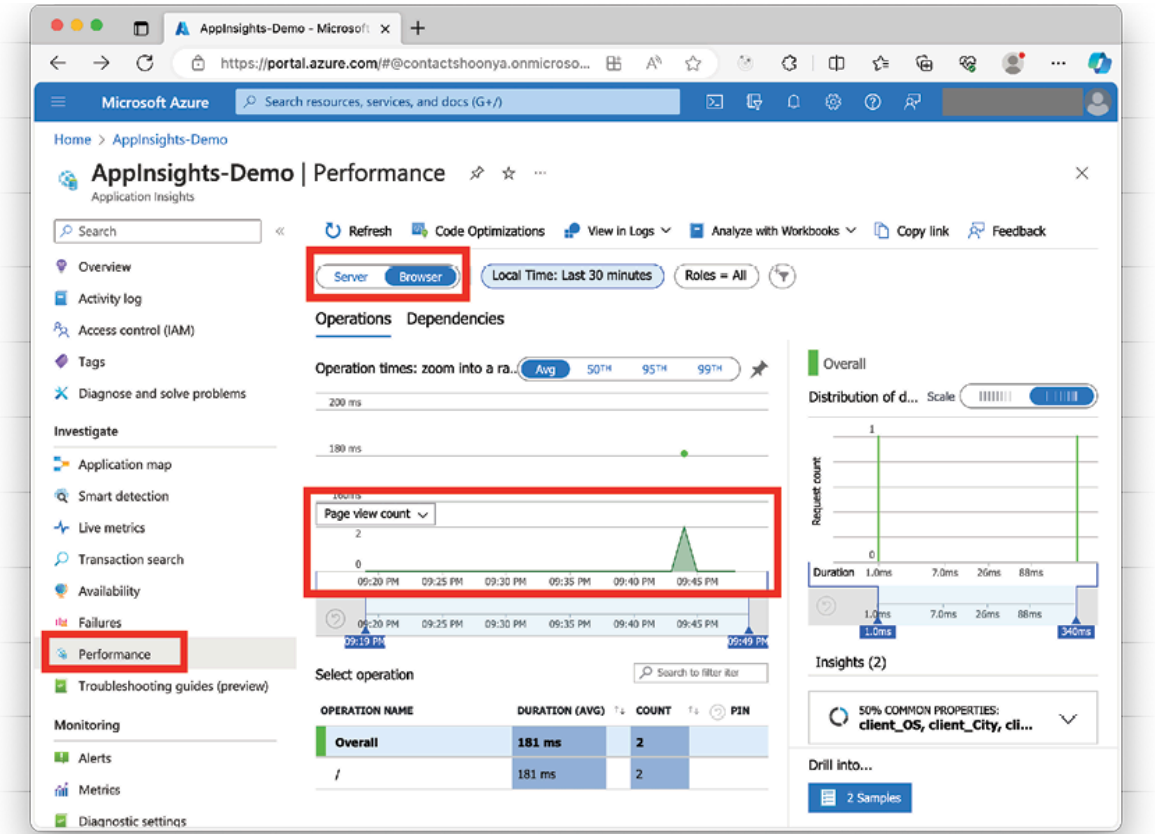


Figure 30.5: *Monitoring Telemetry data in Application Insights*

9. You can drill into the logs for further analysis. You may also fire custom queries in the **Logs** tab as follows:

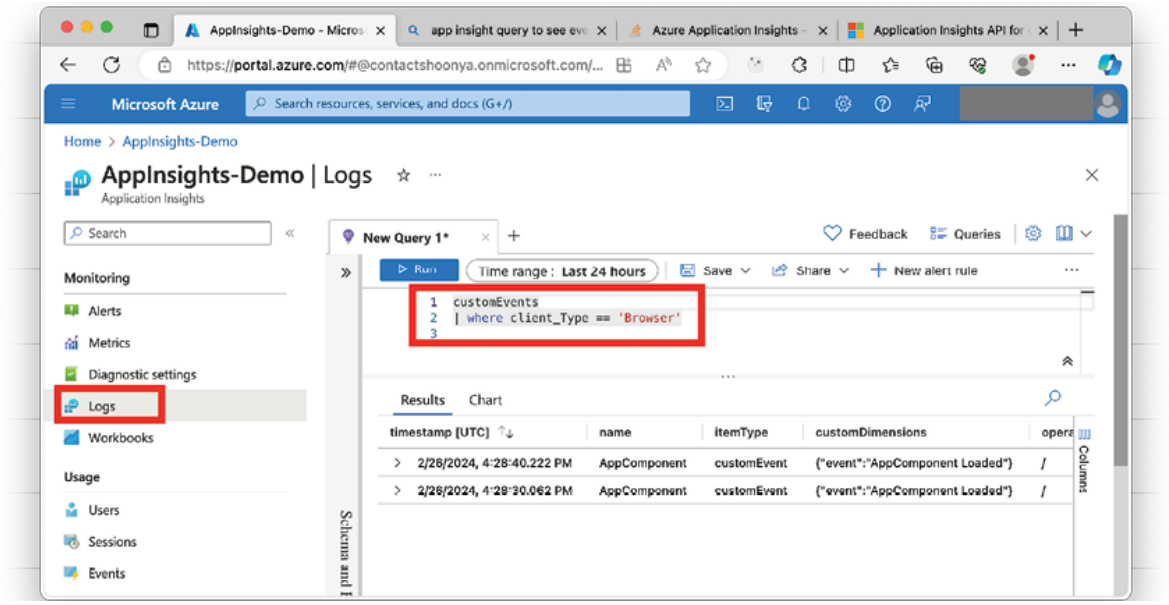


Figure 30.6: *Run custom queries in Application Insights*

Analysis of telemetry data is a vast topic that is beyond the scope of this book. We saw a simple code demo here. We encourage you to explore further by yourself. Please refer to the documentation links in the references section to dive deep into the topic.

Conclusion

Telemetry and instrumentation are essential tools for developers to monitor and diagnose issues in their Angular applications. By instrumenting an Angular app, developers can collect various types of data, such as logs, metrics, and traces, which can help to identify performance issues and bugs. In this chapter, we explored the setup steps for Application Insights, a *Microsoft* service designed for monitoring and diagnosing live applications. With the help of telemetry and instrumentation, developers can effectively manage their Angular applications and ensure a smooth user experience. In the next chapter, we will explore how to improve the page ranking of your website in search engines like *Google* and *Bing*.

Points to remember

- Telemetry refers to the collection and analysis of data related to the application's performance and usage, while instrumentation involves the integration of tools and techniques that enable developers to monitor and diagnose issues in the application.

- Instrumentation makes a system observable by adding observability libraries or dependencies to an application's code.
- OpenTelemetry, Azure Application Insights, and Sentry are popular libraries for instrumenting an Angular app.
- Telemetry and instrumentation help you track user interactions, detect performance anomalies, and diagnose issues, providing valuable insights into the application's behavior, performance, and internal workings.

Questions

1. What is telemetry and instrumentation in the context of software development?
2. What are the types of data that telemetry encompasses?
3. What are some popular libraries for instrumenting an Angular app?
4. What is Application Insights, and how is it used for monitoring and diagnosing live applications?

References

- OpenTelemetry with SigNoz observability:
  https://signoz.io/blog/opentelemetry-angular/
- Microsoft Application Insights:
  https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview
- Add Application Insights into Angular:
  https://devblogs.microsoft.com/premier-developer/angular-how-to-add-application-insights-to-an-angular-spa/
- Sentry:
  https://docs.sentry.io/platforms/javascript/guides/angular/performance/instrumentation/automatic-instrumentation/

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 31
Server-side Rendering and Search Engine Optimization

Introduction

Search engine optimization (**SEO**) is the practice of improving a website's visibility in search engines like *Google* and *Bing*. The goal is to get more organic traffic by fulfilling users' search needs through relevant, high-quality content and an excellent user experience. In essence, SEO helps your site appear on the first page of search results, driving traffic and potentially boosting sales. SEO is an essential aspect of web development, and it can be challenging to implement in Angular apps due to the framework's reliance on JavaScript. This is where **Server-side rendering** (**SSR**) helps. In this chapter, we will cover the various strategies and best practices for optimizing your Angular app for SEO, including server-side rendering, metadata management, and structured data.

Structure

This chapter covers the following topics:

- Server-side rendering
- Hydration and state transfer
- Search engine optimization
- Setting title and metadata
- Schema.org: Structured data markup
- Avoiding hash-based URLs
- Canonical URLs

- Avoid slow or blocked resources
- Make it mobile-first
- Testing SEO
- Skipping a page from crawlers

Objectives

By the end of this chapter, you will have a better understanding of how to implement server-side rendering and how to improve its visibility on search engines.

Server-side rendering

Single page applications like the ones built using Angular are client-side rendered by default. This means that the server sends a bare-bone HTML to the client (browser), and the client then makes API calls to fetch data and populate the page content. Search engines rank a page by crawling its content and then indexing the keywords on the page. If the content received from the server does not contain any useful content, it is difficult for the search engine to access the page. One of the most effective ways to improve SEO for your Angular application is to use SSR. SSR generates HTML on the server, which is then sent to the client. There are many advantages of SSR as follows:

- **Enhanced SEO**: SSR makes it easier for search engines to crawl and index your application's content, improving its SEO.
- **Improved performance**: SSR delivers fully rendered HTML to the client, which can be parsed and displayed even before the application JavaScript is downloaded. This is especially beneficial for users on low-bandwidth connections or mobile devices.
- **Better core web vitals**: SSR leads to performance improvements, resulting in reduced **First Contentful Paint** (**FCP**), **Largest Contentful Paint** (**LCP**), and **Cumulative Layout Shift** (**CLS**). These metrics impact user experience and search engine rankings.

To create an SSR application, add the **–ssr** switch in Angular CLI as follows:

```
ng new –ssr seo-demo
```

If you have an existing Angular project, use the Angular CLI to add SSR as follows:

```
ng add @angular/ssr
```

Now run the app and observe the **Network** tab in the dev tool in the browser. The page returns the entire HTML:

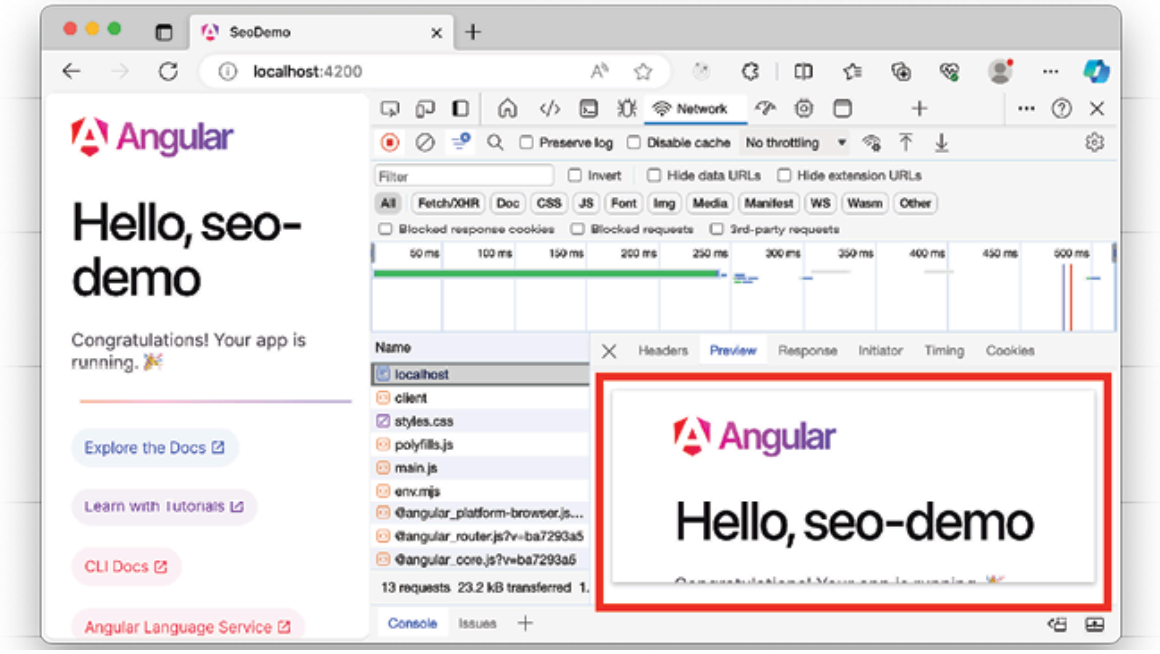


Figure 31.1: *Page content in SSR*

Let us explore how the SSR application code is different from the regular **client-side rendered** (**CSR**) code. In SSR, we need to run a web server that can handle the app routes. Notice that the **server.ts** file is added to the root folder. This is a simple Node.js Express server. It contains a catch-all route definition as follows:

```
server.get('*', (req, res, next) => {
  const { protocol, originalUrl, baseUrl, headers } = req;

  commonEngine
  .render({
  bootstrap,
  documentFilePath: indexHtml,
  url: `${protocol}://${headers.host}${originalUrl}`,
  publicPath: browserDistFolder,
  providers: [{ provide: APP_BASE_HREF, useValue: baseUrl }],
  })
  .then((html) => res.send(html))
  .catch((err) => next(err));
  });
```

The route handler uses **commonEngine** from Angular to generate the HTML on the server side. Also, notice that it imports bootstrap from **main.server.ts** file that has the following definition:

```
import { bootstrapApplication } from '@angular/platform-browser';
```

```
import { AppComponent } from './app/app.component';
import { config } from './app/app.config.server';

const bootstrap = () => bootstrapApplication(AppComponent, config);

export default bootstrap;
```

This is similar to the code in the **main.ts** file, except that the **bootstrap** is exported as default from the module, and the **config** used is defined in the **app.config.server.ts** file, which has the following code:

```
import { mergeApplicationConfig, ApplicationConfig } from '@angular/core';
import { provideServerRendering } from '@angular/platform-server';
import { appConfig } from './app.config';

const serverConfig: ApplicationConfig = {
 providers: [
 provideServerRendering()
 ]
};

export const config = mergeApplicationConfig(appConfig, serverConfig);
```

This **config** file imports the **config** from the **app.config.ts** file and merges **serverConfig** with it, which has a provider for SSR. To build the app, run the following:

ng build

This will generate browser as well as server folders in the **dist** folder. Inside **dist/<app-name>** folder, run the following command to run the app:

node server/server.mjs

Hydration and state transfer

If the page rendered in SSR needs data from an external API endpoint, the server will make that HTTP call and return the page with the appropriate data. Hydration is the process in which the application generated on the server side is restored on the client side. Instead of recreating the DOM structure on the client, hydration ensures that the server side generated HTML is reused on the client side. Also, hydration transfers the existing application state to the client. If an API call is made on the server side to render the page, hydration takes

care to send the state to the client side so that the page does not call the API again when the DOM is rendered on the browser. As a fun example, say we want to show the popular XKCD cartoon of the month on the page. XKCD has a public API endpoint to retrieve the cartoon data in JSON format. Add the following code in the **app.component.ts** file to retrieve the cartoon of the month and show:

```
import { HttpClient } from '@angular/common/http';
import { Component, OnInit } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
 selector: 'app-root',
 standalone: true,
 imports: [RouterOutlet],
 template: `
  <div>{{ cartoonText }}</div>
  <img [src]="cartoonImg" [alt]="cartoonImgAlt" />
 `,
 styleUrl: './app.component.css'
})
export class AppComponent implements OnInit {

 cartoonText = 'Cartoon';
 cartoonImg = undefined;
 cartoonImgAlt = 'Cartoon Image';

 constructor(
 private http: HttpClient,
 ) { }

 ngOnInit() {
 console.log('AppComponent ngOnInit: Fetching xkcd cartoon')
 this.http.get('https://xkcd.com/info.0.json')
 .subscribe({
 next: (cartoon: any) => {
 this.cartoonText = cartoon['safe_title'];
 this.cartoonImg = cartoon['img'];
 this.cartoonImgAlt = cartoon['alt'];
```

```
33.  },
34.  error: (err) => {
35.  console.log(err);
36.  }
37.  });
38.  }
39. }
```

When this page is rendered on the browser, check the page response in the dev tool. You will notice that a **script** tag with **id="ng-state"** is returned inside the HTML body of the page, as shown:

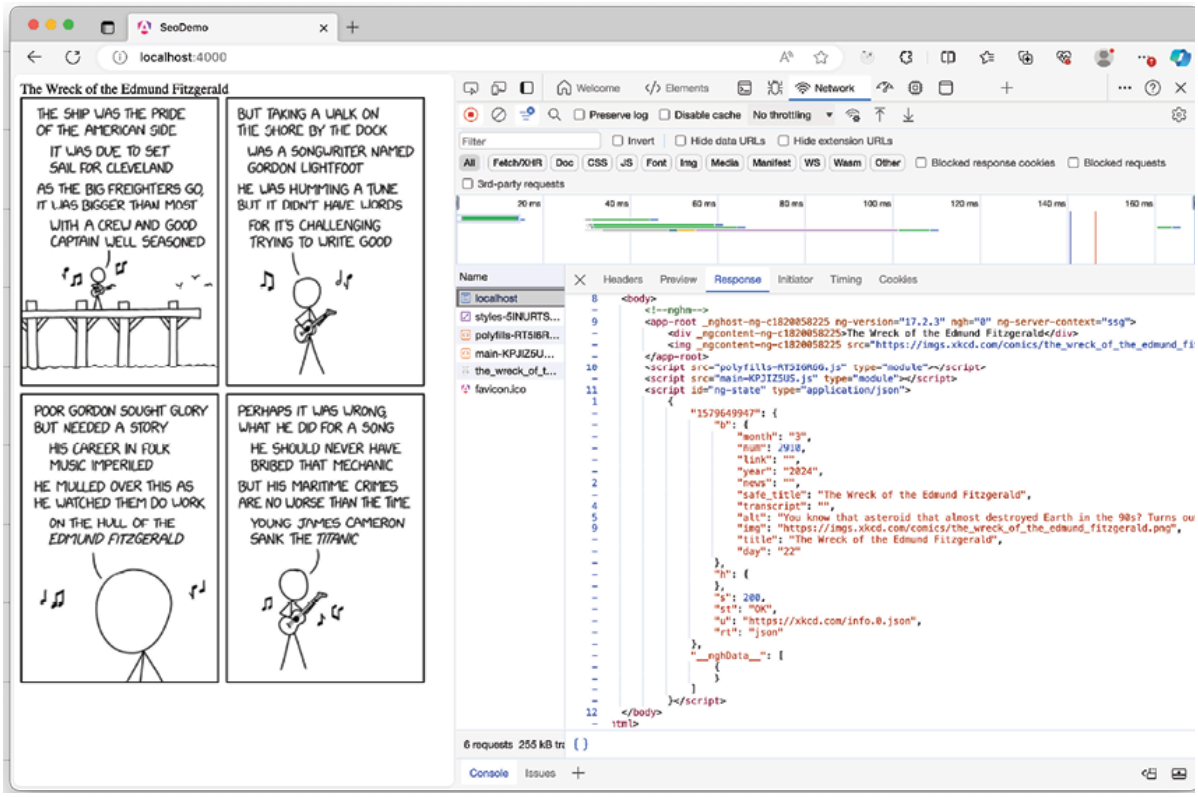


Figure 31.2: *State transfer in hydration*

This script transfers the application state from the server to the client and avoids a redundant API call on the client side. This is a default behavior of hydration in SSR. If you need to send any specific state from SSR to CSR, you can use **TransferState** class defined in Angular. The API reference for **TransferState** is available in the references section.

Search engine optimization

When we create a public-facing website, we want more and more relevant users to visit our site. Most users land on our website by searching for

matching terms in search engines like *Google*, *Bing*, *DuckDuckGo*, etc. Search engines discover our website by crawling through references to our website on the internet and then index our page by scrapping and understanding the page content. Hence, we must maintain the page meta tags and content in ways that a search engine can understand. This process of optimizing our page for a search engine is called SEO. Multiple factors help improve a website's ranking on search engines. The most important consideration is to create an app using SSR. As the page response sent from the server contains content in SSR, that helps search engines index the page content and understand it better. That is the reason why we covered SSR in detail in the previous sections as a prerequisite to improving SEO. Once your page is rendered using SSR, there are multiple other factors that you can choose to fine-tune your app for page ranking. We will cover a few of these methods in the sections below.

Setting title and metadata

The **<head>** section in the **index.html** page contains metadata that is relevant to the search engine to understand the purpose and structure of your website. Properly set **<title>** and **<meta>** tags in your HTML to provide search engines with relevant information about your pages. These tags show as follows:

```
<head>
 <title>Your Page Title</title>
 <meta name="description" content="Your page description">
 <!-- Other meta tags -->
</head>
```

The **title** value appears in the browser tab title and helps search engines use it as a heading for search results. The **description meta** tag allows search engines to get a gist about the page's purpose. This is often displayed in the description section of a search result under the heading.

Schema.org: Structured data markup

Schema.org is a collaborative project by *Google*, *Microsoft*, *Yahoo*, and *Yandex*. Its mission is to create, maintain, and promote schemas for structured data on the internet. Schema.org provides a vocabulary of tags (or microdata) that you can add to your HTML to improve how search engines read and represent your page in **search engine results pages** (**SERPs**). By using Schema.org markup, you enhance the search engine's understanding of your content, which can lead to better relevancy signals and improved visibility in SERPs. There are multiple formats to use structure data on your website:

- **Microdata**: Microdata is an HTML5 extension that allows you to embed structured data directly into your HTML. You define attributes within your HTML tags to describe the content. Following is an example of representing the name of a person using microdata:

```
<div itemscope itemtype="http://schema.org/Person">
 <span itemprop="name">Adi</span>
</div>
```

- **JavaScript Object Notation for Linked Data (JSON-LD)**: It is a lightweight, human-readable format that separates structured data from HTML content, making it easier to manage.

```
{
 "@context": "http://schema.org",
 "@type": "Person",
 "name": "Adi"
}
```

- **Microformats**: Microformats are another way to embed structured data within HTML. They follow specific patterns and classes to mark up content.

```
<span class="h-card">
 <span class="p-name">Adi</span>
</span>
```

JSON-LD is a more preferred way to represent the page structured data for SEO. It is defined inside a **script** tag with the type set to **application/ld+json**. More complex types can be defined as well. For instance, an article with a **headline**, **description**, date of publication, and **author** details can be represented as follows with a custom type - **Person**:

```
<script type="application/ld+json">
 {
 "@context": "http://schema.org",
 "@type": "Article",
 "headline": "Your Article Title",
 "description": "Brief description of the article",
 "datePublished": "2024-03-15",
 "author": {
 "@type": "Person",
  "name": "Your Name"
```

```
}
}
</script>
```

Avoiding hash-based URLs

Use clean, descriptive URLs without hash fragments (#) for better SEO. This enhances page ranking:

```
// app-routing.module.ts
const routes: Routes = [
  { path: 'about', component: AboutComponent },
  // ...
];
```

Canonical URLs

A canonical URL specifies the original source of an article. Search engines penalize a website if the same content is found on another website. More so if the other website is more reputable than yours. Specifying the canonical URL for each page avoids duplicate content issues.

```
<!-- Inside the <head> section -->
<link rel="canonical" href="https://yourdomain.com/your-page">
```

Avoid slow or blocked resources

A request for a resource, such as a script file, must be completed in less than 4 seconds. Otherwise, search engine crawlers mark it as unavailable.

Make it mobile-first

Mobile-first indexing is a change in the way Google indexes web pages, where the mobile version of a site is prioritized over the desktop version. This means having a mobile-friendly site is crucial for SEO since it impacts search engine ranking.

Testing SEO

There are many first-hand and third-party tools to test SEO. *Semrush* is an industry-standard tool to test your page keywords and competitor research. You can use it to fine-tune your page keywords and SEO strategy. *Webmaster* is a tool from *Microsoft* that helps to improve search ranking on *Bing*.

Skipping a page from crawlers

Sometimes, there are pages that are dynamic in nature or require browser permissions that would make it meaningless to be crawled by search engines. To prevent Google from indexing such pages, you can use the **noindex** directive. There are two ways to do this:

- **Using a meta tag in HTML head**: Insert the following meta tag within the **<head>** section of your Angular page:

```
<meta name="robots" content="noindex">
```

- **HTTP response header** (**X-Robots-Tag**): Another method is to add the **X-Robots-Tag: noindex** directive to the HTTP response headers. This approach has broader applications and affects all pages served by your server.

Conclusion

In conclusion, optimizing your Angular application for search engines is crucial for driving organic traffic and boosting sales. Server-side rendering is a powerful technique that can significantly improve your application's visibility on search engines and provide a better user experience. In this chapter, we have covered various strategies and best practices for implementing server-side rendering and optimizing your Angular app for SEO. By following these practices, you can improve your app's performance, achieve better core web vitals, and ultimately enhance your users' experience. In the next chapter, you will explore various AI tools that can improve developer as well as user experience.

Points to remember

- SEO is crucial for improving a website's visibility in search engines like Google and Bing.
- SSR generates HTML on the server, which is then sent to the client, resulting in enhanced SEO, improved performance, and better Core Web Vitals.
- Other best practices for optimizing your Angular app for SEO include metadata management, adding structured data markup, avoiding hash-based URLs, adding canonical URLs, avoiding slow or blocked resources, and making it mobile-first.

Questions

1. What is the goal of SEO?
2. Why is it challenging to implement SEO in Angular apps?
3. What is SSR, and how does it help with SEO?
4. What are some key differences between SSR and CSR code in Angular?
5. How can you build an Angular app with SSR support?

References

- TransferState:
  https://angular.io/api/core/TransferState
- Hydration:
  https://angular.dev/guide/hydration
- Semrush:
  https://www.semrush.com/
- Webmaster:
  https://www.bing.com/webmasters/about

CHAPTER 32
AI Tools and Chatbots

Introduction

In this chapter, we will explore how AI tools can enhance your Angular development experience as well as enhance user experience on your website. With the increasing demand for intelligent applications, integrating AI into your Angular app can give you a competitive edge. We will also delve into the process of building an AI based chatbot using Angular, which can significantly improve user engagement and satisfaction. This chapter can help you leverage the power of AI to create more efficient and effective applications.

Structure

This chapter covers the following topics:

- GitHub Copilot
- Copilot with Angular
- Create a chatbot using OpenAI API

Objectives

By the end of this chapter, you will understand how to integrate AI tools to boost your Angular development and learn to create an AI based chatbot using Angular.

GitHub Copilot

Since most developers use **Visual Studio Code** (**VS Code**) as a preferred IDE for Angular app development, we will look at how AI based tools can enhance your developer experience in VS Code. One of the tools that stand out is the GitHub Copilot, which improves code efficiency and helps reduce code errors. GitHub Copilot uses advanced machine learning models trained on publicly available code from GitHub repositories. As you write code, it provides context-aware code completions, suggestions, and even entire code snippets in real time.

Follow the below steps to get started with Copilot:

1. **Sign up**: Go to **https://github.com/features/copilot**. Sign up for a free trial for your personal account or have a GitHub Copilot seat assigned by your organization.
2. Install the **GitHub Copilot** extension in VS Code. Refer to the following figure:

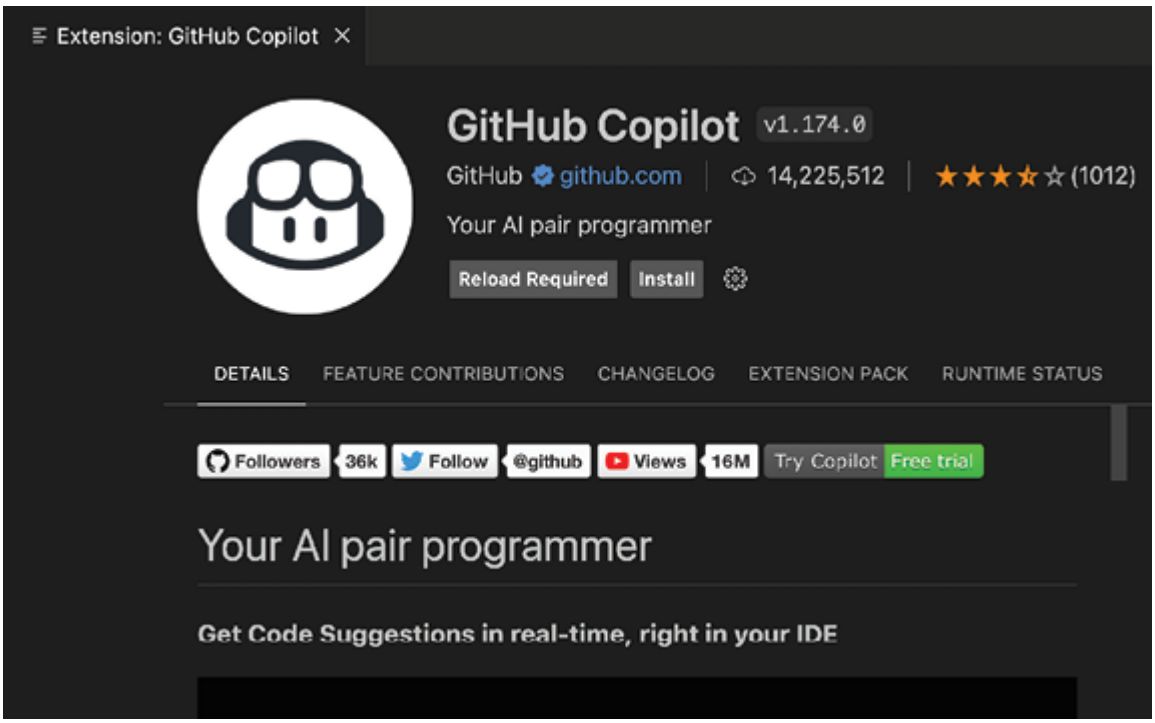


Figure 32.1: *GitHub Copilot extension in VS Code*

3. Follow the steps on the Getting Started guide page. The link is provided in the references section.

For example, let us open the **fruit-store** app we created in *Chapter 26, Responsiveness,* in Visual Studio. Click on the GitHub Copilot icon on

the bottom right corner of the status bar in VS Code:

Figure 32.2: *GitHub Copilot icon in VS Code status bar*

Now, select GitHub Copilot chat from the menu that drops down. You can now ask to fix a code, add tests for your code, or explain a code. Type the following in the chat prompt:

/tests add unit tests for product component

This will generate tests for the **Product Component**, as below:

CHAT

souvikbasu

@.workspace /tests add unit tests for product component

.workspace

> Used 1 reference

Sure, here's a basic set of unit tests for your `ProductComponent`. These tests will check if the component is created, if the `products` array is defined, and if the `isTabletPortrait` and `isPhonePortrait` variables are initially false.

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { ProductComponent } from './product.component';
import { ProductService } from '../../services/product.service';
import { BreakpointObserver } from '@angular/cdk/layout';
import { of } from 'rxjs';

describe('ProductComponent', () => {
  let component: ProductComponent;
  let fixture: ComponentFixture<ProductComponent>;
  let productService: ProductService;
  let breakpointObserver: BreakpointObserver;

  beforeEach(() => {
    const productServiceMock = {
      getProducts: jest.fn().mockReturnValue(of([]))
    };

    const breakpointObserverMock = {
      observe: jest.fn()
    };

    TestBed.configureTestingModule({
      declarations: [ProductComponent],
      providers: [
        { provide: ProductService, useValue: productServiceMock },
        { provide: BreakpointObserver, useValue: breakpointObserverMock }
      ]
    });

    fixture = TestBed.createComponent(ProductComponent);
    component = fixture.componentInstance;
    productService = TestBed.inject(ProductService);
    breakpointObserver = TestBed.inject(BreakpointObserver);
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should have products array defined', () => {
    expect(component.products).toBeDefined();
  });

  it('should have isTabletPortrait initially false', () => {
    expect(component.isTabletPortrait).toBe(false);
  });

  it('should have isPhonePortrait initially false', () => {
    expect(component.isPhonePortrait).toBe(false);
  });
});
```

Please note that this is a basic set of tests. Depending on the complexity of your component and the logic inside it, you might need to add more tests. Also, you might need to adjust the mock services according to your needs.

How can I test a component method that makes an HTTP request?

Figure 32.3: Generate tests using Copilot

A few of the features of Copilot are as follows:

- **Code completions**: As you type, Copilot provides inline code suggestions based on your context. It suggests relevant code snippets, function implementations, and more. For example, when writing a method in an Angular component, Copilot can offer multiple ways to implement a function based on your input.
- **Natural language comments**: You can describe what you want the code to do in plain English, and Copilot will generate the corresponding code. It also supports multiple dialects of languages, such as American and British English.
- **Unit test generation**: Copilot can create unit tests for your methods, saving you time and ensuring code quality.
- **Error fixing**: It helps you fix errors in your code or resolve issues while running commands in the terminal.

The following are the benefits of using Copilot:

- **Efficiency**: Copilot reduces repetitive tasks, allowing you to write code more efficiently.
- **Error reduction**: It minimizes errors by providing accurate suggestions.
- **Productivity**: Spend less time searching for code examples and more time building features.

Copilot with Angular

There are many ways in which Copilot can help with Angular development. Some of them are:

- **Creating an Angular component**: As you start typing an Angular component, Copilot suggests the boilerplate code, imports, and lifecycle hooks.
- **Generating NgRx actions and reducers**: When you define an action or reducer, Copilot can provide the necessary code structure.
- **Creating Angular services**: Copilot can generate service classes, methods, and dependency injection.
- **Routing configuration**: When setting up routes, Copilot suggests

route definitions and guards.

- **Unit testing**: Copilot can create test cases for your Angular components and services.

It is beyond the scope of this book to discuss the details of the above use cases. The above pointers can help you explore further by working with the Copilot VS Code extension.

Create a chatbot using OpenAI API

A common need in most websites today is the ability to interact with customers using a chatbot window. A chatbot can be created using AI-powered services like ChatGPT from *OpenAI*, Gemini**,** and Vertex AI from *Google*, etc. Most of these services provide a JavaScript SDK that can be integrated with the Angular or Express.js code to provide an interface to the **large language model** (**LLM**). A comprehensive exploration is beyond the scope of this book, but we will provide a short sample of integrating with the OpenAI chat assistant.

Go to **https://platform.openai.com/api-keys**, create an account, and generate an API key. Run the following commands to create an Angular app with a chatbot component and a service to fetch data from OpenAI:

```
ng new chatbot-app

cd chatbot-app

ng g c chatbot

ng g s open-ai-api
```

Add the following code to **open-ai.api.service.ts** file:

```
1.  import { Injectable } from '@angular/core';
2.  import OpenAI from "openai";
3.
4.  @Injectable({
5.   providedIn: 'root'
6.  })
7.  export class OpenAiApiService {
8.   openai: OpenAI
9.   assistant: any;
10.
```

```
11.  constructor() {
12.  this.openai = new OpenAI({
13.  apiKey: <your-open-ai-api-key>,
14.  dangerouslyAllowBrowser: true
15.  });
16.
17.  this.initialize();
18.  }
19.
20.  async initialize() {
21.  this.assistant = await this.openai.beta.assistants.create({
22.  name: "Fruits Expert",
23.  instructions: "You know everything about fruits",
24.  model: "gpt-3.5-turbo-0125"
25.  });
26.  }
27.
28.  async generateChatResponse(prompt: string){
29.  const thread = await this.openai.beta.threads.create();
30.
31.  const message = await this.openai.beta.threads.messages.create(
32.  thread.id,
33.  {
34.  role: "user",
35.  content: prompt
36.  }
37.  );
38.
39.  return await
     this.openai.beta.threads.runs.createAndStream(thread.id, {
40.  assistant_id: this.assistant.id
41.  })
42.  }
43.  }
```

The above code uses **openai** npm module to make an API call to OpenAI. It initializes an object of type OpenAI in the constructor with the API key. Then, it creates a **Fruits Expert** assistant that can answer anything about fruits. **generateChatResponse** method takes the user prompt as input and creates a conversation **thread**. A **message** object is created using the **prompt**, and then a run is created in *line 39* that will keep returning responses as and when generated by Open AI.

For security purposes, you should not send API keys from the browser. Hence, in *line 14* above, you will notice that we make an exception to this by setting **dangerouslyAllowBrowser** to **true**. We are making the call to the OpenAI API from the Angular service just to demonstrate the use case. In practice, the Angular service should make a call to a server-side code (for example, an Express.js server) which makes an API call to OpenAI.

To consume this service, add the following code to the **chatbot.component.ts** file:

```
1. import { Component, Inject } from '@angular/core';
2. import { OpenAiApiService } from '../open-ai-api.service';
3. import { FormsModule } from '@angular/forms';
4.
5. @Component({
6.  selector: 'app-chatbot',
7.  standalone: true,
8.  imports: [FormsModule],
9.  template: `
10.  <div class="response">{{ response }}</div>
11.  <div class=»prompt»>
12.  <input type=»text» [(ngModel)]="prompt" />
13.  <button (click)="submit()">Submit</button>
14.  </div>
15.  `,
16.  styleUrl: './chatbot.component.css'
17. })
18. export class ChatbotComponent {
19.  prompt?: string;
```

```
20.  response: string = "";
21.  constructor(@Inject(OpenAiApiService) private
     openAiApiService: OpenAiApiService) { }
22.
23.  async submit() {
24.  if (this.prompt) {
25.  const run = await
     this.openAiApiService.generateChatResponse(this.prompt);
26.  run.on('textDelta', (textDelta, snapshot) => this.response +=
     textDelta.value)
27.  }
28.  }
29. }
```

This is a simple component that takes the user prompt as input, makes a call to the **OpenAiApiService** with the user prompt, and displays the response back. For brevity, we are skipped the styling code. When you run the app, you will see it in action as below:

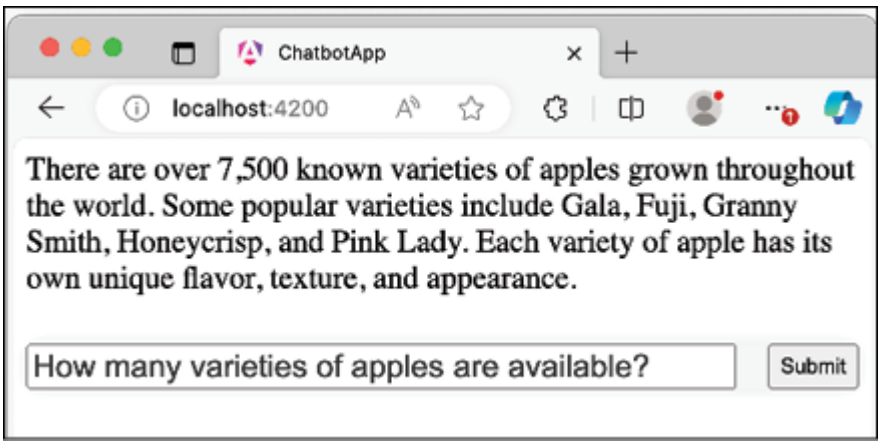


Figure 32.4: *Chatbot app using OpenAI*

The above example was just to give you an introduction to creating chatbots. Similarly, you may explore other API services, such as Gemini, TensorFlow, ML.Net, etc., to achieve different purposes.

Conclusion

Integrating AI into your Angular app can provide a competitive edge and enhance user experience. Tools like GitHub Copilot can significantly

improve code efficiency, reduce errors, and increase productivity. Additionally, building an AI based chatbot using services like OpenAI can improve user engagement and satisfaction. This chapter has provided an overview of how to get started with GitHub Copilot and integrate with the OpenAI chat assistant. This helps you explore the world of AI-powered development and create better applications.

Points to remember

- GitHub Copilot is an AI based tool that provides context-aware code completions, suggestions, and even entire code snippets in real-time.
- Chatbots can be created using AI-powered services like ChatGPT from OpenAI, Gemini, and Vertex AI from Google, etc.
- OpenAI chat assistant provides a JavaScript SDK that can be integrated with the Angular or Express.js code to provide an interface to the LLM.

Questions

1. What are some of the benefits of using GitHub Copilot in Angular development?
2. How can Copilot help with creating Angular components and services?
3. What are some of the AI-powered services that can be integrated with Angular to create a chatbot?

References

- Getting started with Copilot:
  https://code.visualstudio.com/docs/copilot/getting-started
- Gemini AI SDK:
  https://ai.google.dev/
- TensorFlow:
  https://www.tensorflow.org/
- ML.Net:

https://dotnet.microsoft.com/en-us/apps/machinelearning-ai/ml-dotnet

CHAPTER 33
Security

Introduction

Welcome to the last chapter of the book! Security is a crucial aspect of any application, and it is essential to ensure that measures are in place to protect against potential threats. As an Angular developer, it is your responsibility to manage the application's security to remain secure from any unauthorized access. This chapter provides an overview of security considerations that you should keep in mind while developing an Angular application. It covers various topics, including XSS, CSP, CORS, CSRF, etc., to ensure the application's overall security.

Structure

This chapter covers the following topics:

- Cross-site scripting prevention
- Content security policy
- Cross-origin resource sharing
- Cross-site request forgery
- Security best practices

Objectives

By the end of this chapter, you will understand how to make your Angular application more secure.

Cross-site scripting prevention

Cross-site scripting (**XSS**) is a security vulnerability where an attacker injects malicious scripts (usually JavaScript) into a web application. These scripts execute in the context of a victim's browser, potentially compromising user data, session tokens, or other sensitive information. XSS attacks occur when untrusted data is included in a web page without proper validation or escaping.

There are various types of XSS attacks:

- **Reflected XSS**: The injected script is part of the URL or input data, and the server reflects it back to the user. For example, a search query parameter gets echoed in the

search results page.

- **Persistent (stored) XSS**: The malicious script is stored on the server (for example, in a database) and served to users when they visit a specific page. For instance, a comment section where an attacker posts a script that affects all subsequent visitors.
- **DOM based XSS**: The attack occurs entirely on the client-side, manipulating the **Document Object Model (DOM)** directly. In this attack, the DOM is modified using JavaScript to execute malicious code.

Angular provides built-in mechanisms to prevent XSS attacks. Angular automatically escapes and sanitizes untrusted values before rendering them in the DOM. When binding data using interpolation ({{ }}), Angular ensures that the data is treated as text and not executed as code.

To see this in action, create a new project and replace the **app.component.ts** file with the following:

```
@Component({
 selector: 'app-root',
 standalone: true,
 imports: [RouterOutlet, FormsModule],
 template: `
 <label for="username">Name:</label>
 <input id="username" [(ngModel)]="username" placeholder="Name">
 <p>{{ username }}</p>
 `,
 })
 export class AppComponent {
  username = '';
 }
```

Now run the app and enter a malicious **script** in the input control on the browser, like:

```
<script> alert("XSS attack!"); </script>
```

If the input is not sanitized, the **script** will execute, showing an alert and potentially stealing sensitive data. However, Angular automatically escapes the user input, preventing **script** execution. No additional code changes are needed. Observe that the text gets printed as is and the **script** does not execute, in the following figure:

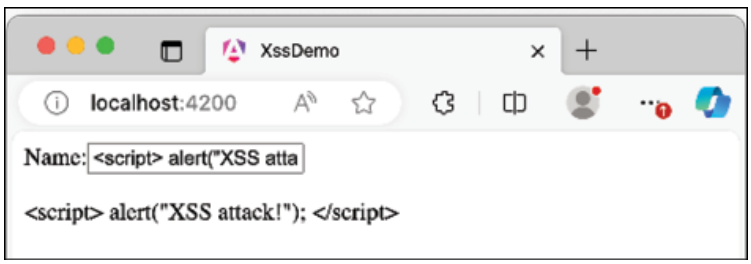


Figure 33.1: *Sanitizing user input*

There are a few cases when Angular may not be able to sanitize the input. If you explicitly want to sanitize your input, you can do so using **DomSanitizer** as follows:

```
import { DomSanitizer } from '@angular/platform-browser';

sanitizedInput = this.sanitizer.bypassSecurityTrustHtml(userInput);
```

Then bind the sanitized input in the template:

```
<p [innerHTML]="sanitizedInput"></p>
```

Content security policy

Content security policy (**CSP**) is a security feature that helps prevent XSS attacks by controlling which resources (such as scripts, styles, and fonts) a web page can load. It defines a set of rules that restrict the origins from which certain types of content can be loaded. By enforcing these rules, CSP reduces the risk of malicious code execution.

To configure CSP, you can add a **meta** tag in the **index.html** file as follows:

```
<meta http-equiv="Content-Security-Policy" content="your-CSP-rules-here">
```

An example CSP rule is as follows:

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'">
```

The following are the common directive rules:

- **default-src**: Specifies the default source for all content.
- **script-src**: Defines allowed sources for JavaScript.
- **style-src**: Specifies allowed sources for styles (CSS).
- **img-src**: Defines allowed sources for images.
- **connect-src**: Specifies allowed sources for network requests (e.g., API calls).
- This book does not cover a detailed discussion of CSP. For more details, please refer to the Angular security documentation link provided in the references section.

Cross-origin resource sharing

Cross-origin resource sharing (**CORS**) is a security mechanism that governs how web browsers allow or restrict web pages from making requests to a domain (origin) different from the one that served the web page. It ensures that only trusted origins can access resources (such as APIs, fonts, or images) hosted on another domain. CORS prevents unauthorized cross-origin requests and protects users from potential security risks. By default, web browsers enforce the **same-origin policy**, which restricts web pages from making requests to a different origin. When a web page tries to make an HTTP request to a different origin (for example, from **myapp.com** to **api.example.com**), the browser checks if the target server allows such requests by making a preflight call. If the server permits cross-origin requests (via CORS headers), the browser allows the communication. Otherwise, it blocks the request. If your Angular web server and API server are hosted on separate domains, you can add CORS headers in your API responses to allow calls from specific domains. Servers can specify CORS policies by sending specific HTTP headers in their responses. Common CORS headers include:

- **Access-Control-Allow-Origin**: Specifies which origins are allowed to access the resource.
- **Access-Control-Allow-Methods**: Lists the HTTP methods (e.g., **GET**, **POST**) allowed for cross-origin requests.
- **Access-Control-Allow-Headers**: Specifies which custom headers are allowed.
- **Access-Control-Allow-Credentials**: Indicates whether credentials (for example, cookies, HTTP authentication) can be included in the request.

- **Access-Control-Expose-Headers**: Lists additional headers exposed to the client.

Cross-site request forgery

Cross-site request forgery (**CSRF/XSRF**) is an attack where an unauthorized user tricks a victim into performing actions on a web application without their knowledge or consent. The attacker exploits the victim's authenticated session to execute malicious requests. CSRF attacks are particularly dangerous in applications where state changes can have irreversible consequences, such as financial transactions.

To understand the attack, imagine this scenario: You logged into your bank account. The browser stores the session cookie, allowing you to open another window on the bank site without authenticating again. Now, through social engineering, an attacker convinces you to click on a link in an email. This takes you to the attacker's website, say **evil.com**, in a new browser tab. Now, evil.com makes a call to the same bank site. Since the browser already has your session cookie, it will allow the attacker to open the page with your session and perform any transaction as desired, all without your notice.

The way to prevent this attack is by validating that the call is generated by the user and not the attacker. For this purpose, when the user visits the bank site, the bank server will generate a random token and set it as an HTTP-only cookie on the page. This token is called the **CSRF token**. The bank site must send the same token back in a header X-CSRF-Token. On receiving the request, the bank server will match the value in the cookie with the one from the HTTP header. Since the attacker will not have the token, they cannot run any request on the bank site.

Avoiding this attack requires coordination between the client and server sides. Angular's **HTTPClient** has built-in support for the client side. You can read about the implementation details from the link shared in the references section. You need to implement the logic on the server side to fetch the tokens from the cookie and HTTP header and match them.

Security best practices

The Angular community offers some guidance for security best practices. A few of these are as follows:

- **Maintain the latest version of Angular for your project**: The latest versions are more secure due to security patches that are identified and tested by the community.
- **Do not modify Angular code**: Although Angular is open source, it is recommended that you do not modify the framework code yourself. Merging the latest Angular code into your framework version often leads to maintenance hassles and introduces the possibility of security holes. Instead, raise a pull request with the necessary changes and let it merge with the community version.
- **Trusting safe values**: Angular APIs marked as security risk should be used with caution. For example, avoid using **innerHTML** directly unless you are sure the content is safe. Always sanitize user input before rendering it in the DOM.
- **Server-side template engines**: Avoid using template engines on server-side templates. If not handled properly, they can introduce security risks.
- **Reporting vulnerabilities**: If you discover security vulnerabilities in Angular, report them through the *Google Open Source Software Vulnerability Reward Program*.

- **Authentication and authorization**: Implement robust authentication and authorization mechanisms to control access to your app's resources. We discussed user authentication in *Chapter 14, Authentication and Authorization*.
- Stay informed about security best practices, keep your app up-to-date, and follow the guidelines provided by the **Open Web Application Security Project** (**OWASP**) to ensure a secure Angular application.

Conclusion

In conclusion, security is of utmost importance when developing any application, including Angular applications. It is essential to understand the various security threats and implement measures to prevent them. Angular provides built-in mechanisms to prevent XSS attacks, and CSP and CORS can be used to further enhance the security of the application. By following the best practices outlined in this chapter and being vigilant about security, you can make your Angular application more secure and protect against potential threats. This brings us to the end of the book. The knowledge you acquired in this book will help you create real-world Angular apps with ease. Happy coding!

Points to remember

- Angular provides built-in mechanisms to prevent XSS attacks by escaping and sanitizing untrusted values before rendering them in the DOM.
- CSP is a security feature that helps prevent XSS attacks by controlling which resources a web page can load.
- CORS is a security mechanism that governs how web browsers allow or restrict web pages from making requests to a domain different from the one that served the web page.

Questions

1. What is XSS, and how can it be prevented in Angular?
2. What is a CSP, and how does it help prevent XSS attacks?
3. What is CORS, and why is it important for web security?
4. What are some best practices for ensuring the security of an Angular application?

References

- Security:
  https://angular.io/guide/security
- HTTP client - security: Cross-site request forgery (XSRF) protection:
  https://angular.io/guide/http-security-xsrf-protection
- Google Open Source Software Vulnerability Reward Program:
  https://bughunters.google.com/
- Open Web Application Security Project (OWASP):
  https://owasp.org/

Index

Symbols

60fps
  achieving 338

A

accessibility 303
  evaluating 307, 308
  guidelines and standards 304
  testing 307
active routes
  showing 98-101
AI tools 435
Angular 2
  evolution 2
  working 10, 11
Angular app 40-43
  building 182
  code organizing 76, 77
  deploying 181
  deploying, with Firebase 367, 368
  instrumentation 418
  localizing 294-296
Angular change detection optimization 340-343
Angular CLI 27
  exploring 34
  for deployment 183
  installing 28
  ng e2e 37
  ng lint 37
  ng new 34
  ng serve 34-36
  ng test 36
Angular Component Dev Kit (CDK)
  using 306, 307
Angular Console 22
Angular DevTools 279, 280
Angular Language Service 21
Angular Material 381, 384
  exploring 385-387
Angular Material Design 306
Angular Snippets 21
Angular workspace
  configuration files 32, 33

e2e tests 33
node modules 33
source files 31, 32
animation module 315-318
animations
CSS, using 312-315
optimizing 346
reusing 318
API endpoints
sign up and sign in 244-247
Application Insights 418
setting up 419-423
application performance monitoring (APM) tool 418
application programming interface (API) 232
ARIA attributes 305
adding 305, 306
Asynchronous JavaScript and XML (AJAX) 2
AsyncSubject 224
attribute directives 62
authentication 243, 244, 256
authentication errors
handling 259, 260
authentication service
setting up 247, 248
Auth Guard
routes, securing with 254
authorization 243, 244
Azure Static Web App
deploying to 183-186

B

backend as a service (BaaS) 363
Background Sync 361
bar chart
creating, with ng2-charts 332-334
base href 85
BehaviorSubject 223
bidirectional text
handling 300, 301
Bootstrap 381
breakpoints
conditional rendering with 375-379
browser developer tools 25
browsers
working 3-5
bundling 19

C

CanActivate 105
CanActivateChild 106
CanDeactivate 106

canonical URLs 433
Capacitor 392
Change Case 22
charting library
  selecting 330, 331
charts 329
chatbot
  creating, with OpenAI API 438-441
classes, TypeScript 137-139
client-side rendering (CSR) 2, 12
code coverage report
  generating 274, 275
Code Spell Checker 22
components 44-49
  multiple components, working with 49-51
component testing 264
compressing 19
content delivery network (CDN) 32, 364, 365
content security policy (CSP) 445
creational operators 219
cross-origin resource sharing (CORS) 446
cross-site request forgery (CSRF/XSRF) 446, 447
cross-site scripting (XSS)
  prevention 444, 445
CSRF token 447
CSS frameworks
  using 116
CSS grid layouts 380
CSS Object Model (CSSOM) 4
CSS preprocessors 123
CSS variables 125
Cumulative Layout Shift (CLS) 426
currency
  displaying 297
currencyCode 68
custom domain
  adding 186, 187
custom service worker 361
Cypress 273

D

data binding 173, 174
dates
  displaying 297
Debugger for Chrome 21
debugging 277
  breakpoints and logging, using 278, 279
  code in IDE, using breakpoints 281-283
  importance 278
  templates and components 280
declarations 6

decorators 72, 73
dependency injection 52
design system 383, 384
DestroyRef 169
development environment 15
DevTools 25
digitInfo 68
directives 62-67
disability 304
Document Object Model (DOM) 3, 444
DOM based XSS 444

E

e2e test frameworks
  Cypress 273, 274
  Jest with Puppeteer 274
  Playwright 274
e2e testing 264
elements 5
Emmet 23
end-to-end (e2e) tests 31, 273
enums, TypeScript 143
error handling
  global error handling 287, 288
  local error handling 287
  significance 286
ESLint 22
event binding 176, 177
event handling 173, 190

F

Figma 383
Figma design tool 387, 388
filter() operator 220
First Contentful Paint (FCP) 426
Flexbox 380
form array 207
form builder 207
form group 205
forms 189, 192
form validation 196-200, 208
fragments 106, 107
Frame Rate Monitor panel 339
frontend development
  history 2

G

GitHub Copilot 436, 437
  benefits 438
  features 437, 438

with Angular 438
GitHub Copilot Chat extension 24
GitLens 22
global styles 122
GraphQL 239
GraphQL calls 239, 240

H

hash-based URLs
avoiding 432
Hello World app
creating 28-30
high contrast mode 307
hosting service
selecting 183
HTTP errors
handling 289
hydration 428-430
Hypertext Markup Language (HTML) 2

I

image lazy loading
optimizing 348
impairment 304
Injectable decorator 52
instrumentation 417
integrated development environment (IDE) 18, 19
integration testing 264
IntelliSense 22
interface, TypeScript 140, 142
internationalization 292
best practices 296, 297
performing 292, 293
interpolation 174, 175

J

Jamstack 366
Jasmine 264
Jasmine Test Explorer 21
JavaScript Object Notation for Linked Data (JSON-LD) 431, 432
Jest with Puppeteer 274
JSON Web Tokens (JWTs) 257, 258

L

large language model (LLM) 439
Largest Contentful Paint (LCP) 426
layout 4
lazy loading 102-104, 340
library packages

importing 117
lifecycle hooks 149, 150
ngAfterContentChecked 161-163
ngAfterContentInit hook 159
ngAfterViewChecked 163-165
ngAfterViewInit 163-165
ngDoCheck hook 154-159
ngOnChanges hook 150-153
ngOnDestroy 166-169
ngOnInit hook 154
ordering of 169, 170
load balancer 365
features 366
load testing 348
local component state 402, 403
locale 294
switching between in UI 298
localization 292
long-time support (LTS) version 17

M

media queries
using 372-374
metadata
setting 431
microdata 431
micro frontend 322
different frameworks, working with 323-327
implementing 322, 323
minifying 19
mobile app
creating 391
mobile-first approach 380
module 43, 44
importing 73-76
multiple observables
combining 221, 222
Multipurpose Internet Mail Extensions (MIME) type 3

N

native mobile app
advantages 392
creating 392-396
native features, adding 397, 398
NativeScript 392
nested form groups 205
nested routes 92-95
ng2-charts 332
ngAfterContentChecked hook 161-163
ngAfterContentInit hook 159-161
ngAfterViewChecked hook 163-165

ngAfterViewInit hook 163-165
ngDoCheck hook 154-159
*ngFor
  optimizing, in template 347, 348
ngOnChanges hook 150-153
ngOnDestroy hook 166-169
ngOnInit hook 154
ngOnInit lifecycle hook 65
ngrok 22
NgRx 404-408
  using 408-413
NgRx DevTools 283
Node.js 16
  installing 16, 17
node package manager (npm) 16
  installing 16, 17
node version manager (NVM) 17
non-null assertion operator (!) 175
numbers
  displaying 297

O

observables 214
observables, using in Angular 224
  event handling 224, 225
  HTTP request handling 226
  reactive forms 226
  router events 227
observers 214
offline capabilities 352
offline-first approach 352, 353
  advantages 352
OnPush 343, 344
  observables, using 345, 346
OpenAI API
  chatbot, creating with 438-441
OpenTelemetry 418
OpenTelemetry Protocol (OTLP) data formats 418
Open Web Application Security Project (OWASP) 448
operators in RxJS 218
  creational operators 219
  pipeable operators 219-221

P

page not found
  handling 98
pages
  skipping, from crawlers 433
paint 4
Path Intellisense 22
Peacock 22

performance profiling 280
Persistent (stored) XSS 444
pipeable operators 219-221
pipes 67-71
Playwright 274
preloading 104, 105, 340
Prettier 22
Progressive Web App (PWA) 353
  attributes 354
  styling 354-361
property binding 66, 176
  examples 176
pull mechanism 216
pure pipes
  using, for heavy computation 346
push mechanism 216

Q

query parameter
  passing 101, 102

R

rasterization 4
Reactive Extensions for JavaScript (RxJS) 213-216
  operators 218
  Subject 222-224
reactive forms 200
  advantages 210
  use cases 200-205
reactivity 213
redirects
  setting up 97
Reflected XSS 444
reflow 4
render tree 4
Representational State Transfer (REST) 233
Response, Animation, Idle, Load (RAIL) model 338, 339
responsiveness
  handling 371
  on server side 381
  using Angular Material 381
REST API
  handling 233-239
role-based authorization 256, 257
router data 107
routerLink 85, 86
router state 107, 108
routes
  adding, to existing app 86-88
  debugging 108
  nested routes 92-95

securing, with Auth Guard 254
routing
in Angular 82-84
in SPA 81, 82
in traditional websites 80, 81
with TypeScript 102

S

safe navigation operator (?) 175
same-origin policy 446
Schema.org 431
SCSS mixins 123
using 123, 124
Scully 367
search engine optimization (SEO) 425, 430, 431
testing 433
search engine results pages (SERPs) 431
security 443
best practices 447
selectors 6, 124, 125
server-side rendering (SSR) 2, 12, 425-428
service based state 403
services 52-59
model level, versus component level services 59-62
service testing 264
service workers 353
signals 227, 228
sign in page component
creating 251-253
SigNoz 418
single page application (SPA) 2, 9, 10, 174
snapshot
versus observables 95-97
spyOn() method 271
state changes
debugging 413, 414
state management 401
approaches 402
need for 402
state transfer 428
static site generation (SSG) 12
structural directives 62
style changes
authoring 125, 126
styleUrls 46
styling 111
best practices 130
inline style in template 115
link tag in template 115, 116
style, setting with TypeScript 128
styles metadata, using 114, 115

styleUrls metadata, using 112-114
Subject in RxJS 222
AsyncSubject 224
BehaviorSubject 223
characteristics 222, 223
symbolDisplay 68

T

Tailwind CSS 128
using, with Angular 128-130
telemetry 417
template driven form example 208, 209
template driven forms 193-196
templateUrl 46
testing 263
component testing 264, 267, 268
directives and pipes, testing 273
e2e testing 264
for async code using spy 270-272
integration testing 264
service testing 264, 268, 269
types 264
unit testing 264
tests
running, in CI/CD 275
theming 126, 127, 388, 389
third-party log monitoring 288
title setting 431
TODO Highlight 22
transpiling 19
tsconfig.json file 144-146
two-way data binding 178-180
type annotation 135
examples 135-137
TypeScript 10, 133, 134
benefits 134, 135
classes 137
config file 144
enums 143, 144
features 10
interface 140
modules 142, 143

U

Uniform Resource Locator (URL) 3
unit testing 264
in Angular 264-267
URL parameters
passing 88-92
user input events handling 190-192
user registration 248

implementing 248-251
user sessions
managing 255, 256

V

view encapsulation 117-122
ViewEncapsulation.Emulated 120
ViewEncapsulation.None 120
ViewEncapsulation.ShadowDom 120
virtual scrolling 347
Visual Studio Code (VS Code) 19, 20, 436
AI assisted tools 24, 25
extensions 21, 22
features 22-24
installing 20, 21

W

Web Content Accessibility Guidelines (WCAG) 304
web page 5
building blocks 5-8
website
versus, web app 8, 9
website performance
measuring 339, 340
WebSocket 240
WebStorm 19

Z

Zone.js 341